The Fabius Function in Lean

A Guided Walkthrough of the `ProveIt` Formalization

Proof architecture, exact computation, asymptotics, and effective approximation

Human-readable guide to Vladimir Reshetnikov's `ProveIt` repository

Merged repository edition: 4 September 2026

historical baseline: source snapshot 25 August 2026, base commit `eaea1b9afe2b7ff0b7dd880bd1710e303dec6d80`; Volterra addenda `3da579387`, `e109088ed`; activation-series and finite-Taylor addendum `a345425d21d90e680bf15e34093af42c69c08a83`; inverse-modulus equality/rigidity addendum `f05c2fc7d`; fixed-nome q-Pochhammer entire addendum, 1 September 2026

Abstract

The Fabius function is a classical example of a smooth function with an elementary-looking self-similarity law and unexpectedly rich arithmetic, Fourier-analytic, probabilistic, and asymptotic structure. Vladimir Reshetnikov's `ProveIt` repository contains a large Lean 4 development that formalizes both of Juan Arias de Reyna's 2017 papers on the function, proves the existence and uniqueness of a canonical bounded solution, constructs its signed global extension, verifies exact evaluation algorithms at dyadic rationals, develops moment and denominator theory, reconstructs Rvachev's Fourier characterization, and goes substantially beyond the papers through q-binomial identities, discrete-limit theorems, sharp shape and inverse theory, finite activation Taylor jets, square-summable activation asymptotics, computability, non-elementarity results, corrected saddle-point asymptotics, and Legendre expansions.

This document is a proof-oriented walkthrough of that codebase. Its aim is not to restate every declaration in source order. Instead, it explains why the development is split into its present modules, which dependencies are mathematically essential, which are merely packaging dependencies, and how the principal proofs are made acceptable to Lean. Particular attention is paid to the two parallel foundations of the project—exact rational arithmetic and analysis under an abstract `IsFabius` specification—and to the bridge that identifies them. The discussion also records source corrections that become unavoidable in a formal proof, recurring proof-engineering idioms, recommended reading paths, and practical advice for extending the library.

Reader's map. A mathematically oriented reader should begin with Sections 2, 5, 7, 8, 10, and 15. A Lean developer should read Sections 1, 3, 4, 21, and 22. Readers interested in the two source papers can use the theorem map in Appendix B; readers trying to locate a particular implementation should use the annotated module guide in Appendix D; and readers reproducing trust checks should use Appendix H.

Artifact status (4 September 2026). The authoritative live lexical audit is computed by `scripts/doc_audit.py` and pinned in `docs/doc_audit_baseline.json`. The earlier incoming 985-module/12,199-declaration checkpoint had no missing module headers or public declaration comments. The merged source also retains `exists_eq_in_residual_interval` in `MeanValueBracket.lean`. The historical 923/11,611 snapshot comprised the merge target's 11,610 declarations and the retained unconditional public Pochhammer bridge. Later branch inventories, including 952/11,884, 967/12,001, 970/12,051, 976/12,116, and 977/12,133, are historical and overlap. The historical 671/8,859 local checkpoint was the 649/8,698 checkpoint plus 22 modules and 161 declarations. The Pochhammer ownership census remains `RvachevPochhammerFactorization.lean` (1+10), including `complexQPochhammerInf_eq_qPochhammerInfIn`; `QPochhammerEntire.lean` (0+5); and `QPochhammerInfinite.lean` (1+29), while `QMultinomial.lean` remains 1+9 and `GaussianBinomialPolynomialStructure.lean` remains 0+5. The historical 665/8,819 checkpoint includes `CyclotomicDivisibility.lean` (0+3), `PrimitiveRootBlock.lean` (0+3), `QLucas.lean` (0+7), `QCatalan.lean` (1+11), `NewtonInterpolation.lean` (2+13), and `QBetaIntegral.lean` (1+8). Newton's definitions are `newtonCoeff` and `newtonInterpolant`; the latter preserves the established scalar-sequence `Fabius.newtonPoly` API. The current Newton module is expanded to 3+19 by the node-qualified family and its compatibility aliases. The retained six-module tranche comprises `GaussianBinomialInteger.lean` (1+10), `GaussianBinomialComplexOrder.lean` (1+5), `QPfaffSaalschutz.lean` (0+3), `QuantumMultinomial.lean` (0+5), `RvachevSuperconvergentSynthesis.lean` (1+8), and `GaussianBinomialBounds.lean` (0+6): three definitions and 37 theorems, hence six modules and 40 unique declarations. That historical union additionally contained the exhaustive `TwoPhiOneReversal.lean` (2+12) and `QChuVandermonde.lean` (0+10) surfaces and the `GeometricRichardsonGenerating.lean` (3+7) generating-function surface, together with the three-theorem extension that brings `GaussianBinomialCumulants.lean` from 2+21 to 2+24 and the zero-definition/five-theorem `LambertWBranchGapBernoulli.lean` leaf, while retaining the public `complexQPochhammerInf_eq_qPochhammerInfIn` bridge. The one-definition/eight-theorem `GeometricUniformMomentPolynomial.lean` leaf supplies the algebraic part of `p7:thm:Pn`. Together with the real and complex coefficient bridges, the sharp-degree theorem, and the global q-factorial-cleared `GeometricUniformMomentRatFunc.lean` coefficient, it makes the moment-polynomial row Exact on its stated algebraic and safe-evaluation domains. `GeometricUniformMomentReciprocity.lean` likewise makes the germ-reciprocity row Exact under $q \neq 0$ and $\|q\| \neq 1$, without asserting a unit-circle or global analytic identity. The retained five-module polynomial q-calculus increment is exactly `PolynomialQDerivative.lean` (2+17), `PolynomialQLeibniz.lean` (0+4), `QPochhammerDerivative.lean` (0+3), `LambertSeriesLog.lean` (0+4), and `QGamma.lean` (2+10): four definitions and thirty-eight theorems, forty-two public declarations in all, with the exact algebraic, analytic, nonvanishing, and boundary hypotheses recorded in the module guide below. The rigorous forward q-monograph ledger is 181 Exact, 79 Partial, 14 None, and 8 interface rows; its source concordance is 103 Lean, 375 human, 60 N/A, and 9 conjecture rows. The bound leaf changes `prop:gaussian-bound` from None to Exact, witnessed by `one_le_gaussianBinomial` and `gaussianBinomial_le_inv_qPochhammerInfIn`. `thm:q-lucas` remains Partial: Lean proves the evaluated primitive-root identity, while the manuscript's polynomial congruence modulo Φ_d still needs a minimal-polynomial lift. `cor:babbage-derivative` also remains Partial because its value, but not its derivative, is formalized. The retained companion PDF is a historical receipt for its named source checkpoint; this source-only merge requires a fresh uninterrupted three-pass Libertinus rebuild before parity can again be claimed. Exact historical source/PDF validation receipts are kept externally in the repository documentation so that neither artifact embeds a self-invalidating fingerprint. Package-local `SHA256SUMS*` ledgers remain abolished and do not participate in validation.

Scope and conventions

The repository is active source code, and module boundaries can evolve. The historical baseline of this walkthrough is commit `eaea1b9afe2b7ff0b7dd880bd1710e303dec6d80` on the `codex/fabius-both-papers` branch, inspected on 25 August 2026. The present merged edition incorporates the later closures described in its post-snapshot passages and in Appendix D; their individual source checkpoints or current-tree inspection dates remain recorded with each addendum.

The walkthrough quotes identifiers and very short fragments where useful, but it paraphrases proof bodies rather than reproducing them. The exact Lean source remains the authoritative statement of hypotheses, namespaces, coercions, and imported lemmas. Claims described as questions, conjectures, conditional results, audits, or counterexamples are not silently promoted here to proved positive theorems.

Mathematically, two functions must be kept distinct throughout:

- the bounded cumulative-distribution form $F\colon \mathbb{R} \to [0,1]$, equal to 0 on the left of the unit interval and to 1 on the right; and
- the compactly supported even Rvachev up-function up, obtained by folding or differentiating the bounded form and naturally living near $[-1,1]$.

The code also defines a signed global extension of the bounded function. It is this extension, rather than the clamped bounded map, that satisfies certain translation and dyadic identities for arbitrary real arguments. Several apparent contradictions in informal accounts disappear once these three objects are not conflated.

Lean names are displayed in a monospaced face, for example `Fabius.IsFabius`, `Fabius.fabiusDyadicValue`, and `FabiusSharpAsymptotic.lean`. Mathematical equations are written in conventional notation even when the formal theorem uses a more elaborated type, a subtype coercion, or a filter formulation.

Unless stated otherwise, empty finite sums are 0, empty finite products are 1, and Lean's natural-power convention makes $0^0 = 1$. The Fourier transform convention used in this walkthrough is

$$\widehat{f}_{2\pi}(\xi) = \int_{\mathbb{R}} f(x) e^{-2\pi i x\xi}\, \mathrm{d}x.$$

These conventions matter at base indices and in every Poisson- or Fourier-normalization check.

Contents

1 Building and entering the development

1.1 Repository location and toolchain

The relevant source tree is

`Analysis/FabiusFunction/Lean/`

and the umbrella import is `FabiusFunction.lean`. The Lake package declares a Lean library named `FabiusFunction` whose source directory is the path above. At the inspected snapshot, `lean-toolchain` selects `leanprover/lean4:v4.32.0`; `lake-manifest.json` pins a corresponding mathlib revision and the transitive packages used by mathlib, including Aesop, Batteries, Qq, proof widgets, and the import-graph tooling.

The normal build command from the project root is:

```
LAKE_JOBS=1 lake build +FabiusFunction
```

The source claims in this walkthrough are pinned to commit `eaea1b9afe2b7ff0b7dd880bd1710e303dec6d80`. Before reproducing a declaration or module count, check the checkout with `git rev-parse HEAD`; a moving `main` branch is not a reproducible specification. After dependencies have been installed, a focused edit loop can build one module at a time with a Lake target such as `LAKE_JOBS=1 lake build +FabiusFunction.DyadicAnalytic`. The root-module build remains the release check.

A useful first test file is:

```
import FabiusFunction

open Fabius

#check IsFabius
#check fabius_spec
#check rvachevUp
#check fabiusDyadicValue
#check extendedFabius
```

Importing the umbrella module is convenient for exploration. During development, importing a narrow leaf module is preferable: it shortens rebuilds, makes accidental dependencies visible, and helps preserve the intended layering.

1.2 What the umbrella module exposes

The top-level `FabiusFunction.lean` does not contain the development itself. It is an import façade. Its principal imports collect six broad products:

1. the formalizations of the two 2017 papers, through `Paper05442.lean` and `Paper06487.lean`;
2. the independent asymptotic project and the k-fold Thue–Morse audit, through `PaperFabiusAsymptotic.lean` and `PaperKFoldThueMorse.lean`;
3. negative-Laplace, Mellin, periodic-correction, and probability-transform interfaces;
4. translated Legendre expansions, least-squares optimality, q-binomial binary reduction, and uniform spline approximation;
5. the sharp shape API—monotonicity, convexity, optimal Lipschitz and derivative bounds, exact analytic loci, and endpoint flatness; and

6. the totalized inverse, non-elementarity and inverse-branch obstructions, Grzegorczyk-style computability, closed saddle jets, and the explicit second saddle correction.

This is a deliberate public surface. The umbrella import answers the question "what has the repository established?" It is not the best answer to "where is this theorem proved?" For the latter, one should descend into the paper aggregators or the subject-specific module families.

1.3 Four kinds of source module

A useful architectural distinction is that a source file usually has one of four roles:

Reusable infrastructure.
Files such as `LegendrePolynomial.lean`, `LegendreSeriesConvergence.lean`, `SaddleAllOrders.lean`, and the Gaussian-polynomial family develop machinery not tied to one canonical Fabius term.

Fabius-specific bridges.
Files such as `DyadicAnalytic.lean`, `AnalyticMoments.lean`, `GlobalExtension.lean`, and `EndpointLaplaceComparison.lean` identify two representations of the same object. These are often the most informative files to read.

Paper-facing façades.
The four `Paper*.lean` aggregators expose source claims in source vocabulary while delegating proofs to stronger internal declarations.

Audit and correction modules.
Files such as `DraftCounterexamples.lean`, `PoissonSummation.lean`, and `FabiusWikipediaObstruction.lean` isolate a false printed claim or a missing hypothesis and expose the checked correction or obstruction.

1.4 Naming conventions that encode intent

Several recurring name patterns help locate the right abstraction:

Generic versus canonical.
A theorem parameterized by `(F:BoundedFabius)(hF:IsFabiusF)` applies to every implementation; a `canonical_` or bare `fabius_` wrapper specializes it to `fabius` and `fabius_spec`.

Exact-to-analytic casts.
Names ending in `_cast` are the load-bearing bridges from rational computation to real analysis.

Convergence forms.
A `HasSum` theorem records a convergence witness; a companion theorem phrased with `tsum` usually provides the convenient equality. Filter and asymptotic APIs use a similar strong-core, wrapper pattern.

Integer scales.
Names containing `_zpow` use an integer exponent, which matters when a dyadic or asymptotic scale can cross 1.

Specifications.
Predicates such as `IsFabius` and `IsOriginalFabius` separate a characterization from a construction and permit independent existence and uniqueness routes.

1.5 A practical exploration workflow

A productive workflow in an editor with the Lean language server is:

1. open the theorem used by a paper aggregator;
2. inspect its type before its proof;
3. use "go to definition" on the first nontrivial helper;
4. note whether the helper belongs to the exact-arithmetic or analytic side of the project;
5. follow imports upward only until the mathematical mechanism becomes clear; and
6. return to the wrapper theorem to see how casts, side conditions, and notation are discharged.

The wrapper theorem is often intentionally short. The real content may be a sequence of modules: one establishing a recurrence over $\mathbb{N}_0$ or $\mathbb{Q}$, one proving an analytic recurrence for an arbitrary object satisfying `IsFabius`, and one proving that both recurrences share the same initial values.

Architectural point. The development is easier to understand as a directed acyclic graph than as a directory listing. Exact arithmetic starts without the canonical analytic function. Generic analysis starts from an abstract specification rather than from a constructed term. Only after both sides are mature does the code identify exact rational expressions with analytic values and package a canonical function.

---

2 The mathematical objects and their Lean interfaces

2.1 The bounded function

The foundational module `Basic.lean` introduces the closed unit interval as a subtype and uses it as the codomain of a bounded map. Schematically, the first layer has the shape

```
abbrev UnitInterval := Set.Icc (0 : Real) 1
abbrev BoundedFabius := Real -> UnitInterval
```

The subtype codomain records the range bound in the type itself. The projection to $\mathbb{R}$, which the code exposes through a convenience definition such as `fabiusReal`, is used for calculus and integration. This division of labor avoids reproving $0 \le F(x) \le 1$ every time a measure-theoretic bound is required, while still giving the differentiable real-valued function expected by mathlib's analysis APIs.

The abstract predicate `IsFabius` packages the characteristic properties of a candidate. Its fields express, in essence:

$$\begin{aligned} F(x) &= 0 && (x \le 0),\\ F(x) &= 1 && (1 \le x),\\ F(x) + F(1-x) &= 1 && (0 \le x \le 1),\\ F'(x) &= 2F(2x) && (0 \le x \le \tfrac{1}{2}), \end{aligned}$$

together with global smoothness. The precise formal structure uses pointwise derivative predicates and interval membership. Later lemmas extend the symmetry law from the core interval to a statement valid for all real x, with tail cases handled separately.

2.1.1 Why specify before constructing?

Most analytic theorems are stated for a variable F with hypothesis `hF : IsFabiusF`. This is an important architectural choice. It allows derivative identities, Taylor formulas, moment recurrences,

and dyadic uniqueness to be proved without unfolding the fixed-point construction. The existence module can then instantiate this generic API, while uniqueness is proved once at the level of the specification.

This organization resembles an interface-and-implementation pattern:

- `Basic.lean` supplies the interface;
- `Differential.lean`, `DyadicAnalytic.lean`, and related modules prove generic consequences of the interface;
- `Existence.lean` constructs `Existence.boundedCandidate` and proves unique existence through `existsUnique_fabius` and `IsFabius.eq`; and
- `PaperStatements.lean` selects the canonical term `fabius` from that witness and exports `fabius_spec`, letting later files forget the implementation again.

2.2 The Rvachev up-function

The compactly supported function up is the symmetric "twin" of the bounded cumulative form. The code defines `rvachevUp` by folding the bounded function around the center of the unit interval. Up to the chosen normalization and translation, the relation is the familiar one

$$\mathrm{up}(x) = F(x+1) \quad (-1 \le x \le 0), \qquad \mathrm{up}(x) = F(1-x) \quad (0 \le x \le 1),$$

with zero outside the support. Symmetry gives the equivalent positive-branch expression $1 - F(x)$ on the unit interval. The exact Lean definition is arranged to make evenness and support proofs tractable and to match the Fourier normalization used in the original paper.

The up-function is the more natural object for Fourier analysis because it is even, smooth, compactly supported, and normalized to have integral one. The bounded form is the more natural object for dyadic arithmetic because its derivative equation and boundary values generate finite Taylor recurrences.

2.3 The signed global extension

The bounded function cannot satisfy an unrestricted translation law: it is constant on both tails. The repository therefore defines `extendedFabius`, a signed global extension assembled as a locally finite sum of odd translates of up, weighted by the Thue–Morse sign. In the repository's normalization,

$$F_{\mathrm{gl}}(x) = \sum_{n \ge 0} (-1)^{\mathrm{wt}_2(n)}\ \mathrm{up}(x - 2n - 1).$$

Only finitely many summands are nonzero at any given x. Lean nevertheless treats the expression through a `tsum`, so the library must prove local finiteness, summability, and the reduction of the infinite sum to a finite set before elementary rearrangements are legal.

On the unit interval the extension agrees with the bounded function. Away from that interval it carries the Thue–Morse block signs needed for global dyadic formulas. This distinction is used pervasively in `GlobalExtension.lean`, `GlobalDyadic.lean`, and the q-binomial modules.

Formalization-sensitive point. When an informal statement writes $F(x)$ for all real x, inspect the theorem's target. It may refer to the bounded map, the real projection of that map, the compactly supported up-function, or the signed extension. Several corrected statements in the repository amount precisely to choosing the right one.

2.4 Auxiliary analytic objects

`Basic.lean` also reserves names for the analytic transforms that later modules identify:

- a complex sinc function with a removable value at zero;
- the Fourier transform and infinite sinc product associated with up;
- moment and half-moment generating series;
- real and complex versions of $(\exp z - 1)/z$, again with the singularity removed; and
- integral definitions of moments over the support.

Defining these objects early stabilizes names and normalizations. Proofs of convergence, equality, and differentiability are deferred to modules with the necessary imports. This keeps `Basic.lean` conceptually small even though it determines the vocabulary of the whole project.

3 The dependency architecture

3.1 A layered view

The following table suppresses many helper modules but captures the principal flow. Arrows point from prerequisites to consumers.

Layer	Principal inputs	Flow	Principal consumers
Exact arithmetic	Arithmetic; parity; normalized moments; denominator bounds	$\longrightarrow$	dyadic closed forms and the exact evaluator
Generic analysis	Basic and `IsFabius`; differential, scale, and translation laws	$\longrightarrow$	`DyadicAnalytic`, the exact–analytic bridge
Canonical object	the bridge plus contraction-based existence and uniqueness	$\longrightarrow$	analytic moments, the signed global extension, Fourier analysis, and probability
Advanced layers	exact evaluation, moments, and the global extension	$\longrightarrow$	q-binomial limits, negative-Laplace asymptotics, paper façades, and Legendre theory

Table 1: Conceptual dependency flow. The source contains finer-grained helper modules between nearly every displayed pair.

3.2 Why existence is not the first analytic file

A newcomer might expect `Existence.lean` to precede every theorem about the function. Instead, the code first proves a great deal about an arbitrary `IsFabius` object. This is not circular. The direction is:

1. assume a candidate satisfies the abstract laws;
2. derive its dyadic values and prove that any two candidates agree on a dense set;
3. construct one candidate by a contraction mapping; and
4. invoke the already-proved generic uniqueness theorem.

This order isolates the hard analytic construction from the algebraic uniqueness mechanism. It also means that the dyadic evaluator is not merely a computational afterthought: it participates in the proof that the canonical object is unique.

3.3 Two independent uniqueness arguments

The repository contains two conceptually different uniqueness proofs.

The first is adapted to the bounded specification. The derivative and symmetry equations determine exact values at every dyadic rational. Dyadics are dense, and the candidates are continuous, so agreement on dyadics implies agreement everywhere.

The second is adapted to Rvachev's original compactly supported characterization. The derivative/refinement law is transformed into a Fourier-domain sinc equation. Iteration forces the entire Fourier transform to equal an infinite sinc product. Fourier injectivity on an appropriate smooth rapidly decreasing class then forces the original functions to coincide.

Keeping both arguments is valuable. The dyadic proof is computational and feeds exact arithmetic; the Fourier proof mirrors the historical characterization and explains the infinite product.

3.4 Paper aggregators versus theorem-producing modules

Files named `Paper05442.lean`, `Paper06487.lean`, and `PaperFabiusAsymptotic.lean` are indexes and audit records. They import theorem-producing modules and expose a source-facing collection of results. For example, `PaperStatements.lean` gives declarations whose numbering and prose correspond to the arithmetic paper, but the actual proof of a denominator theorem may be distributed across `NormalizedMoments.lean`, `Parity.lean`, `TwoAdic.lean`, and `DenominatorBound.lean`.

This separation has three advantages:

- source numbering does not dictate internal dependency boundaries;
- corrected hypotheses can be documented at the aggregator without contaminating low-level lemmas; and
- later developments can reuse strong internal theorems rather than wrappers tailored to a paper's notation.

4 The exact-arithmetic foundation

4.1 Why the development begins over $\mathbb{N}_0$ and $\mathbb{Q}$

The module `Arithmetic.lean` is deliberately independent of the canonical analytic function. It defines the integer and rational sequences that later turn out to be moments, dyadic values, normalizing factors, and denominator bounds. This has a profound practical advantage: exact identities can be proved using induction, finite sums, divisibility, and polynomial normalization without carrying measurability, differentiability, or convergence hypotheses.

The exact side is not just a transcription of closed formulas. It supplies executable definitions. One can evaluate a dyadic rational by normalization and recursion, and Lean's kernel can reduce the resulting rational expression. The same definitions then serve as specifications for correctness theorems.

4.2 Binary weight and the Thue–Morse sign

The smallest combinatorial ingredients are the binary digit sum and its sign:

$$\mathrm{wt}_2(n) = \text{number of ones in the binary expansion of } n, \qquad \varepsilon(n) = (-1)^{\mathrm{wt}_2(n)}.$$

In the code these appear under names such as `binaryWeight` and `thueMorseSign`. Elementary identities under doubling and successor are proved early because they drive both the dyadic evaluator

and the q-binomial development:

$$\mathrm{wt}_2(2n) = \mathrm{wt}_2(n), \qquad \mathrm{wt}_2(2n+1) = \mathrm{wt}_2(n) + 1,$$

so that $\varepsilon(2n) = \varepsilon(n)$ and $\varepsilon(2n+1) = -\varepsilon(n)$.

Lean's natural-number division and remainder operations require explicit control of positivity. Proofs that are a one-line observation on binary expansions often become a short cluster of lemmas about `Nat.div`, `Nat.mod`, powers of two, and bit constructors. Establishing this API once prevents the higher-level files from repeatedly descending to bit arithmetic.

4.3 Prime-power Pascal-row valuations

The exhaustive public surface of `PrimePowerBinomialValuation.lean` consists of six theorems and no definitions. For a prime p, a natural exponent m, and $0 < j \le p^m$,

$$\nu_p\binom{p^m}{j} + \nu_p(j) = m, \qquad \nu_p\binom{p^m}{j} = m - \nu_p(j).$$

These are `primePowerChoose_padicValNat_add` and `primePowerChoose_padicValNat`; they include $j = p^m$ and the boundary $m = 0$. The strict dyadic specialization $0 < j < 2^m$ is `twoPowChoose_padicValNat`. The companion declarations are `primePowerSubOneChoose_padicValNat`, `primePowerSubTwoChoose_padicValNat`, and `twoPowSubTwoChoose_padicValNat`. They state

$$\nu_p\binom{p^m-1}{j} = 0 \quad (0 \le j < p^m), \qquad \nu_p\binom{p^m-2}{j-1} = \nu_p(j) \quad (0 < j < p^m),$$

and specialize the second identity to $p = 2$. The strict upper bound is essential because at $j = p^m$ the companion binomial coefficient is zero. Only valuation-histogram counts remain outside this module.

4.4 Product normalizations

Several finite products recur in moment denominators. The arithmetic module names them rather than expanding them at every use. Principal examples include variants of

$$\prod_{j=1}^{n}(2^j - 1), \qquad \prod_{j=1}^{n}(2^{2j} - 1), \qquad (2n-1)!!.$$

The source uses names such as `mersenneProduct`, `evenMersenneProduct`, and `oddDoubleFactorial`. Their basic positivity, nonvanishing, splitting, and divisibility properties are the denominator-management toolkit for later proofs.

A formal proof benefits from keeping two versions of a formula:

- a mathematically transparent quotient in $\mathbb{Q}$; and
- a division-free equality in $\mathbb{N}_0$, obtained after multiplying by a common product.

The quotient is convenient for stating a moment recurrence. The division-free equality is the robust object for induction, parity, and divisibility, because it avoids repeated side goals asserting that denominators are nonzero.

4.5 Moment and half-moment recurrences

The exact sequences `moment` and `halfMoment` are defined recursively as rational numbers. The first models the even moments $\int_{\mathbb{R}} x^{2n}\,\mathrm{up}(x)\,\mathrm{d}x$. The second has normalized base value $d_0 = 1$ and, for $n \ge 0$, models

$$d_{n+1} = (n+1)\int_0^1 t^n\,\mathrm{up}(t)\,\mathrm{d}t.$$

Its connection to the bounded function's inverse-power values is derived afterward rather than built into the definition. Both exact definitions mirror recurrences obtained analytically by integration by parts and scaling.

Alongside them, `momentNumerator` and `halfMomentNumerator` are natural-number recurrences with the common denominators cleared. Their role is easy to underestimate. They provide:

1. a finite, executable representation of exact values;
2. a target for strong-induction proofs of closed forms;
3. a clean setting for parity arguments; and
4. a bridge to reduced rational numerators and denominators.

A recurring proof pattern is to start from the rational recurrence, multiply by the anticipated global normalizing product, distribute that product through a finite sum, and use factorial or Mersenne product splitting to identify every term with an earlier natural numerator. Once the normalizations are designed correctly, the final goal is a polynomial identity in natural numbers and powers of two.

Lean pattern. For exact rational identities, the development usually proves all denominator factors positive or nonzero near the beginning of the proof, rewrites divisions into a common denominator, and invokes `field_simp` only after the relevant nonzero facts are available. The remaining equality is discharged by a combination of finite-sum rewrites, `ring_nf`, and induction hypotheses. Calling `field_simp` too early tends to produce enormous and poorly shaped goals.

4.6 Closed dyadic formulas

The exact values at inverse powers of two are represented by a sequence such as `halfMoment FabiusValue`. The source-facing closed formula `fabiusDyadic` is total on natural numerators and is a finite sum built from Thue–Morse signs, even binomial coefficients, and the exact moment sequence. After casting, it equals the bounded Fabius value at $a/2^n$ when $a \leq 2^n$, and the signed global extension at that argument for arbitrary natural a. This closed formula is distinct from the inverse-power table used by the faster Horner evaluator.

The formalization maintains a distinction between:

- a closed formula suited to mathematical theorems;
- a Horner form suited to evaluation;
- a unit-interval evaluator that clamps and removes highest set bits; and
- a global evaluator that applies triangular reduction on two-unit cells and a Thue–Morse block sign.

This separation makes correctness modular. One first proves that the Horner program computes the finite polynomial, then that recursive bit reduction computes the closed formula, then that the closed formula equals the analytic function.

4.7 The inverse-power table

The function `fabiusInversePowTwoTable` computes a prefix of exact values

$$F(1), F(2^{-1}), F(2^{-2}), \ldots, F(2^{-n}),$$

so the array has size $n + 1$. A table-based implementation avoids recomputing the same recurrence at every recursive step. Correctness is expressed through prefix stability: extending a table preserves all

previously computed entries. This is essential because the evaluator may request values in an order dictated by the bits of the numerator rather than by a single monotone loop.

Formal prefix stability is expressed through array size and optional `Array.getElem?` lookup, together with compatibility of `Array.push` and the defining recurrence. Such lemmas look implementation-specific, but they prevent computational code from leaking into the mathematical correctness theorem.

4.8 Horner evaluation

The local Taylor polynomial is evaluated by `fabiusTaylorHorner`. Horner's rule is not only computationally efficient; it is proof-friendly. The correctness theorem is an induction over the coefficient list, and the induction invariant is a suffix polynomial. In contrast, a direct power-sum implementation forces repeated reasoning about exponents, list indices, and truncated ranges.

The coefficients are rational and involve powers of two coming from iterated differentiation. The proof therefore includes cast-normalization lemmas ensuring that a natural exponent or binomial coefficient has the same interpretation in $\mathbb{N}_0$, $\mathbb{Q}$, and $\mathbb{R}$. The code frequently uses explicit type annotations at precisely these interfaces; their apparent verbosity documents where coercion inference would otherwise be ambiguous.

4.9 Recursive removal of the highest bit

At the center of the evaluator is a structurally terminating function such as `fabiusDyadicUnitAux`. Given a dyadic numerator a and exponent n, it removes the highest set bit of a. Write

$$a = 2^b + r, \qquad 0 \le r < 2^b.$$

A Taylor identity expresses the value at $a/2^n$ using the value and inverse-power data associated with the smaller remainder r. Since $r < a$ whenever $a > 0$, the recursive call decreases in a well-founded order.

This is the computational counterpart of traversing the ones in the binary expansion of a. The number of recursive Taylor reductions is therefore controlled by $\mathrm{wt}_2(a)$, not by the magnitude of a itself. Together with precomputation of the inverse-power table, the design yields the approximate complexity described in the repository documentation: a quadratic table phase in the exponent and a bit-weight-dependent evaluation phase.

Lean does not accept "the highest bit was removed" as a termination argument. The source proves a chain of inequalities connecting the bit logarithm, the largest power of two not exceeding a, and the remainder. This chain is packaged so the recursive definition can use a measure. The correctness proof later reuses the very same decomposition, an example of an implementation lemma doing double duty as a mathematical induction lemma.

4.10 Clamping, reflection, and the global evaluator

The unit-interval wrapper `fabiusDyadicUnit` clamps natural numerators at the right endpoint and delegates interior points to highest-bit recursion; it does not perform reflection. The rational-valued `fabiusDyadicValue` accepts an integer numerator and power-of-two denominator, returning 0 for nonpositive inputs and 1 at or beyond the right endpoint. Separate refinement theorems prove that equivalent unreduced representations give the same result.

For the signed extension, `extendedFabiusDyadicValue` reduces the numerator modulo a two-unit grid length, reflects the residue into the unit cell, and applies the Thue–Morse sign of the block index. This is triangular cell reduction, not global periodicity. Multiplying numerator and denominator by a common power of two leaves the output unchanged. Refinement invariance is proved explicitly

rather than delegated to quotient types, because the evaluator's input is computational data, not an abstract element of a dyadic localization.

Proof architecture. There are three distinct notions of normalization:

1. arithmetic normalization of a fraction by canceling powers of two;
2. bounded normalization into $[0, 1]$ using the constant tails; and
3. global triangular cell reduction, with a Thue–Morse block sign.

The code proves each normalization preserves the intended mathematical value before composing them. □

4.11 Recognizing dyadic rationals

The definitions `IsDyadicRational` and `dyadicExponent?` support a user-facing exact evaluator for rational inputs. A rational is dyadic when its reduced denominator is a power of two. The decision procedure must connect:

- the internal normalized numerator and positive denominator of $\mathbb{Q}$;
- a Boolean or optional power-of-two test; and
- a proposition asserting the existence of an integer numerator and natural exponent.

The proof that recognition is sound is separate from the proof that the returned exponent and numerator evaluate correctly. This makes failure meaningful: a non-dyadic rational is rejected because no representation with a power-of-two denominator exists, not merely because a search limit was reached.

4.12 Arithmetic API design lessons

The exact core illustrates several design principles that recur throughout the repository.

First, executable functions and theorem-friendly formulas need not be identical definitions. They are connected by named correctness lemmas. Second, normalization data should be made explicit, especially when a mathematical quotient hides representation choices. Third, the strongest useful invariant is often not the final equality but a stability statement about prefixes, refinements, or common denominators. Finally, natural-number recurrences are frequently the right interface for arithmetic theorems even when the motivating quantities are rational.

5 Constructing the canonical bounded function

5.1 The ambient complete metric space

The construction in `Existence.lean` uses Banach's fixed-point theorem. Let

$$I = [0, 1], \qquad C = C(I, \mathbb{R})$$

with the uniform metric. Since the codomain is complete and I is compact, the continuous-map space is complete. The code then cuts out a closed subset of C consisting of functions satisfying two elementary conditions:

$$0 \le f(x) \le 1,$$
$$f(x) + f(1 - x) = 1.$$

The source calls the reflection operator and the admissibility predicate by local names and proves that the admissible set is closed. A linear function, essentially $x \mapsto x$, witnesses nonemptiness.

Using a closed subspace rather than the entire continuous-function space is decisive. Symmetry is not recovered after the fixed point; it is an invariant of the contraction. This makes the integral normalization automatic and improves the contraction constant from the naive value one to one half.

5.2 Extending an interval function

An element of $C(I, \mathbb{R})$ accepts a subtype argument. To integrate it over an ordinary real interval, the code defines an extension by projecting an arbitrary real number to I. This projection clamps values below zero to zero and values above one to one. Continuity follows from continuity of the metric projection to a closed convex interval.

For an admissible f, define the cumulative integral

$$A_f(y) = \int_0^y \widehat{f}(t)\ \mathrm{d}t,$$

where $\widehat{f}$ is the clamped extension. Symmetry implies

$$\int_0^1 f(t)\ \mathrm{d}t = \frac{1}{2}.$$

The Lean proof reflects the integral by the substitution $t \mapsto 1 - t$, adds the two copies, and uses the pointwise symmetry equation. The subtype coercions and interval maps are handled in helper lemmas so the final normalization proof resembles the paper calculation.

5.3 The nonlinear-looking transform that is actually affine

The fixed-point transform is defined piecewise at the midpoint:

$$(Tf)(x) = \begin{cases} A_f(2x), & 0 \le x \le \frac{1}{2}, \\ 1 - A_f(2 - 2x), & \frac{1}{2} \le x \le 1. \end{cases}$$

The two branches agree at $x = 1/2$ because $A_f(1) = 1/2$. Each branch is continuous, and the gluing lemma supplies continuity of Tf on the whole interval. Bounds follow because f is bounded between zero and one; symmetry follows by exchanging the two branches.

The transform is affine on the admissible set. The constants in the right branch cancel when two transformed functions are subtracted. This is why the contraction estimate can be proved through integrals of $f - g$.

5.4 The contraction estimate

Let $h = f - g$ for two admissible functions. Symmetry gives

$$\int_0^1 h(t)\ \mathrm{d}t = 0.$$

For $0 \le y \le 1$, the prefix integral may be estimated in two ways:

$$\left|\int_0^y h\right| \le y\|h\|_\infty, \qquad \left|\int_0^y h\right| = \left|\int_y^1 h\right| \le (1 - y)\|h\|_\infty.$$

Therefore

$$\left|\int_0^y h\right| \le \min(y, 1 - y)\|h\|_\infty \le \frac{1}{2}\|h\|_\infty.$$

This is the key insight of the construction. A crude estimate would give constant one and no contraction. The zero-total-integral identity lets the proof select the shorter of the prefix and complementary suffix.

Lean pattern. The formal estimate is assembled from interval-integral norm bounds rather than from an informal "length times supremum" principle. The code proves integrability of the extended difference, obtains a pointwise uniform bound from the distance in the continuous-map space, applies the interval integral inequality on both subintervals, and rewrites the total integral as zero. A final case split on $y \le 1/2$ selects the useful bound.

Banach's theorem now yields a unique fixed point in the admissible subspace. The source extracts its underlying continuous map and extends it to a bounded real function with constant tails.

5.5 From an integral fixed point to the differential law

A fixed point satisfies an integral equation. On the left half,

$$F(x) = \int_0^{2x} F(t)\ \mathrm{d}t.$$

For the probability CDF this identity is actually valid for every $x \le 1/2$, not only for $x \in [0, 1/2]$: below zero both the CDF and the oriented integral vanish. Lean exposes this stronger form as `weightedSumCDF_eq_intervalIntegral_of_le_half` and retains `weightedSumCDF_left_formula` as the interval-shaped compatibility wrapper. The fundamental theorem of calculus and the chain rule give

$$F'(x) = 2F(2x).$$

On the right half, differentiation of the reflected branch yields the corresponding equation through symmetry. The proof handles the midpoint and endpoints separately because standard derivative rules apply most directly on open neighborhoods, while the desired structure requests derivatives at boundary points as well.

The source uses `HasDerivAt` statements whenever possible. Such statements compose cleanly through affine maps and integral primitives; a theorem about `deriv` alone would require differentiability side conditions each time it is rewritten. Endpoint derivatives are obtained by showing that the formulas from adjacent pieces agree, using the tail values and symmetry.

5.6 Bootstrapping smoothness

The fixed-point theorem initially gives only continuity. The integral equation gives one derivative. The derivative is a scaled composition of the original continuous function, so it is continuous. Iterating this observation upgrades the function to every finite differentiability order.

The formal proof is organized around reusable scale-and-derivative lemmas rather than a single enormous induction on `ContDiff`. At stage $n+1$, the derivative formula expresses the new derivative in terms of a scaled version of the n-th derivative. Boundary compatibility follows from vanishing on the tails and symmetry. The final theorem packages all finite stages as C^∞.

5.7 Uniqueness by dyadic density

The contraction theorem already gives uniqueness inside the admissible continuous space. The exported specification, however, is phrased for real functions satisfying `IsFabius`; it is useful to prove uniqueness at this abstract level. The later dyadic analysis shows that every such function has the same exact value at every dyadic point. Since dyadics are dense in $\mathbb{R}$ and both candidates are continuous, the functions coincide.

The proof has the standard topological shape: approximate an arbitrary x by a sequence of dyadics, rewrite equality at each sequence term, and pass to the limit by continuity. The nontrivial work lies earlier, in proving exact dyadic equality without assuming the fixed-point implementation.

Architectural point. The fixed-point construction proves that the interface is inhabited. The dyadic theory proves that the interface is subsingleton. This yields a canonical object whose public properties are independent of the construction details.

6 Differential identities and the signed global extension

6.1 The role of `Differential.lean`

`Differential.lean` is the calculus engine for an arbitrary `IsFabius` candidate. Starting from the first derivative law on the left half and the globalized symmetry law, it derives corresponding formulas on other intervals and iterates the derivative relation.

The characteristic coefficient after n differentiations is a triangular power of two. In a typical region where all scaled arguments remain in the appropriate branch, one obtains

$$F^{(n)}(x) = 2^{1+2+\cdots+n} F(2^n x) = 2^{n(n+1)/2} F(2^n x).$$

The exact theorem includes domain conditions ensuring that each intermediate scaled point lies in the branch where the derivative law is valid. Reflection introduces signs and translated arguments on the other half.

These iterated formulas explain three later phenomena at once:

- finite Taylor formulas at dyadic points;
- flatness at the support boundary; and
- extremely rapid decay near zero.

6.2 Domain bookkeeping

An informal derivation often differentiates the relation $F'(x) = 2F(2x)$ without restating its domain. Lean forces the domain to be propagated. If a theorem is iterated n times, the proof must establish that $2^j x$ belongs to the relevant interval for every $j < n$. The source packages this as inequalities involving powers of two and dyadic cells.

This bookkeeping is not bureaucratic. One of the asymptotic draft corrections later in the repository arises because a transformed delay-differential identity is valid only beyond a threshold. The formal derivative theorem makes the threshold impossible to omit.

6.3 Scale and translation lemmas

`ScaleTranslation.lean` collects chain-rule identities for functions of the form

$$x \longmapsto c\,F(ax+b), \qquad x \longmapsto c\ \mathrm{up}(ax+b).$$

These are used by approximation arguments, global extension formulas, Fourier transforms, and splines. The value of a dedicated module is that every later proof can invoke a theorem with all factors of a, signs, and differentiability hypotheses already correct.

A common Lean mistake is to rewrite an iterated derivative of a composition as though the affine map contributed only one factor of a. The library's lemmas state the full a^n factor. Specialized powers-of-two corollaries then combine this with the triangular coefficient from the Fabius differential law.

6.4 Local finiteness of the translate sum

`GlobalExtension.lean` proves that the signed translate series defining F_{gl} is harmless. Because up has compact support, only finitely many natural indices n can satisfy $x - 2n - 1 \in [-1, 1]$ for fixed x. More strongly, on a compact neighborhood of x, a single finite set of odd translates contains every nonzero summand.

That stronger local statement is what permits termwise continuity and differentiation. The proof turns support inequalities into natural-number bounds and proves every index outside a finite range contributes zero. The `tsum` can then be rewritten as a finite sum using a summability-by-finite-support lemma.

6.5 Agreement and global identities

Once local finiteness is available, the signed extension is shown to agree with the bounded function on $[0, 1]$. On every cell $2b \le x \le 2b + 2$, exactly one translate survives and

$$F_{\mathrm{gl}}(x) = (-1)^{\mathrm{wt}_2(b)}\ \mathrm{up}(x - 2b - 1).$$

The case $b = 0$, together with the defining fold relation, gives unit-interval agreement.

The extension is therefore a sequence of two-unit cells, not a globally periodic function: the sign of cell b is the Thue–Morse value $(-1)^{\mathrm{wt}_2(b)}$. Scale-and-translation theorems record the precise identities on their valid blocks. `GlobalDyadic.lean` combines this cell structure with exact values to remove assumptions that a numerator be reduced or lie between zero and its denominator.

6.6 Global differential calculus

Local finiteness also permits termwise differentiation and even–odd reindexing. For every $x \in \mathbb{R}$, the theorems `extendedFabius_hasDerivAt` and `deriv_extendedFabius` give

$$F'_{\mathrm{gl}}(x) = 2F_{\mathrm{gl}}(2x).$$

Iterating this identity yields `iteratedDeriv_extendedFabius`: for every $k \in \mathbb{N}_0$ and $x \in \mathbb{R}$,

$$F^{(k)}_{\mathrm{gl}}(x) = 2^{\binom{k+1}{2}} F_{\mathrm{gl}}(2^k x).$$

Unlike the bounded derivative formula above, these identities require no branch-domain hypothesis.

6.7 Exact Taylor reduction

On its initial block, `extendedFabius_add_pow_two` says that

$$F_{\mathrm{gl}}(x + 2^r) = -F_{\mathrm{gl}}(x) \qquad (r \ge 1,\ 0 \le x \le 2^r).$$

A derivative version extends this sign change to an integer scale s whenever the derivative order m is high enough that $0 \le s + m$. Under that condition, `taylorRemainder_translate` packages the exact cancellation of the two adjacent integral Taylor remainders. Write $R_m(y)$ for `fabiusReductionSummy`. If $n \in \mathbb{Z}$, $x > 0$,

$$2^{-n} \le x < 2^{-n+1}, \qquad y = x - 2^{-n},$$

then `extendedFabius_reduction`, exported as `proposition_ten`, states

$$F_{\mathrm{gl}}(x) = -F_{\mathrm{gl}}(y) + \begin{cases} R_{\mathrm{toNat}(n)}(y), & n \ge 0, \\ 0, & n < 0. \end{cases}$$

Thus the finite correction for $n \ge 0$ is an exact Taylor polynomial, not a truncated approximation with an estimated remainder; for $n < 0$, the formula reduces directly to the power-of-two sign-translation law.

6.8 Why use a `tsum` for a locally finite expression?

A hand proof would select the unique active odd translate on each nonnegative two-unit cell. The repository instead uses a conceptually uniform sum over all natural indices. This pays off in translation and differentiation arguments: the same summand family can be manipulated with standard infinite-sum lemmas. The initial cost is a local-finiteness API; the later algebra becomes much cleaner.

7 Sharp shape, inverse branches, and effective computation

This part of the public API gathers consequences that were once scattered among paper wrappers into a reusable shape theory, then uses that theory to construct the inverse and a certified computable evaluator.

7.1 One global derivative equation

The key simplification is the identity

$$F'(x) = 2\ \mathrm{up}(2x-1) \qquad (x \in \mathbb{R}),$$

proved for every `IsFabius` candidate. Since $\mathrm{up} \geq 0$, the bounded function is globally monotone. The strict theory in `Monotonicity.lean` proves that F is strictly increasing on $[0,1]$ and maps that interval bijectively to itself. On the folded side, up is positive exactly on $(-1,1)$, increases strictly to its unique maximum 1 at 0, and then decreases strictly.

`Convexity.lean` feeds this unimodality back through the derivative equation. The derivative of F increases up to $x = 1/2$ and decreases afterward, with exact range $[0,2]$ and unique maximum 2 at the midpoint. Consequently F is convex on the left half-line and concave on the right, strictly so on the two open halves of its transition interval. The same module now identifies the curvature pointwise:

$$F''(x) = 8\big(\mathrm{up}(4x-1) - \mathrm{up}(4x-3)\big).$$

The declarations `deriv_deriv_fabiusReal_pos_iff`, `deriv_deriv_fabiusReal_neg_iff`, and `deriv_deriv_fabiusReal_eq_zero_iff` say respectively that this is positive exactly on $(0,1/2)$, negative exactly on $(1/2,1)$, and zero off $(0,1)$ or at the midpoint.

7.2 Sharp regularity and flatness

`Regularity.lean` proves that 2 is not merely a convenient Lipschitz constant: it is the least global Lipschitz constant for both F and up. The scaling recurrence then yields, for every $k \geq 0$,

$$\left|F^{(k)}(x)\right| \leq 2^{\binom{k+1}{2}},$$

with equality attained at $x = 2^{-k}$ for positive order. The signed extension and the up-function have parallel sharp bounds in `GlobalBounds.lean`; `BoundedDerivatives.lean` transfers them to the clamped function without confusing its constant right tail with the signed extension.

The effective endpoint bounds are stronger than the qualitative assertion that every positive derivative vanishes. If $x \geq 0$ and $2^n x \leq 1$, `SharpFlatness.lean` proves

$$F(x) \leq \frac{2^{\binom{n+1}{2}}}{n!}\, x^n.$$

Reflection gives the analogous estimate for $1 - F(x)$ at the right endpoint. The scale condition is part of the theorem; the family is not a single bound uniform in n.

7.3 The totalized inverse

Strict monotonicity packages the restriction of F to $[0,1]$ as the order isomorphism `fabiusOrderIso`. The definition `fabiusInv` uses the inverse order isomorphism on that interval and `IccExtend` to clamp all real inputs: it is 0 on the left tail and 1 on the right. The module `FabiusInverse.lean` proves both inverse identities on the unit interval, global continuity and monotonicity, endpoint reflection, and exact inversion of every value produced by the bounded dyadic evaluator.

The same module now packages the first-order interior calculus and the exact global smoothness locus. On $0 < y < 1$, `deriv_fabiusInv_eq_inv_two_mul_rvachevUp`, together with `deriv_fabiusInv_pos`, gives

$$(F^{-1})'(y) = \frac{1}{F'(F^{-1}(y))} = \frac{1}{2\,\mathrm{up}(2F^{-1}(y)-1)} > 0,$$

and `fabiusInv_contDiffOn_Ioo` proves C^∞ smoothness on the open unit interval. Because the totalization is locally constant on both open tails, `fabiusInv_contDiffAt_infty_iff` sharpens this to an exact global statement: the inverse is C^∞ at y if and only if $y \neq 0$ and $y \neq 1$. Likewise, `fabiusInv_differentiableAt_iff` says that it is differentiable exactly away from the two clamping endpoints, with `fabiusInv_not_differentiableAt_zero` and `fabiusInv_not_differentiableAt_one` providing the endpoint obstructions. More generally, `fabiusInv_contDiffAt_iff` classifies every order bounded by C^∞: order zero, namely continuity, holds globally, whereas every positive finite order and C^∞ hold exactly off $\{0,1\}$. Its order hypothesis deliberately excludes the stronger analytic order. The divergent one-sided difference quotients quantify the endpoint failure.

The next derivative is also packaged exactly. The declarations `deriv_fabiusInv_hasDerivAt` and `deriv_deriv_fabiusInv` prove

$$(F^{-1})''(y) = -\frac{F''(F^{-1}(y))}{F'(F^{-1}(y))^3} \qquad (0 < y < 1).$$

At the midpoint the exact jet is

$$F^{-1}(1/2) = 1/2, \qquad (F^{-1})'(1/2) = 1/2, \qquad (F^{-1})''(1/2) = 0,$$

by `fabiusInv_half`, `deriv_fabiusInv_half`, and `deriv_deriv_fabiusInv_half`. On $0 < y < 1$, the declarations `deriv_deriv_fabiusInv_neg_iff`, `deriv_deriv_fabiusInv_pos_iff`, and `deriv_deriv_fabiusInv_eq_zero_iff` identify the negative, positive, and zero loci as $(0,1/2)$, $(1/2,1)$, and $\{1/2\}$. The reciprocal reverses the monotonicity of F'. Consequently `strictConcaveOn_fabiusInv_firstHalf` proves strict concavity on the closed interval $[0,1/2]$, while `strictConvexOn_fabiusInv_secondHalf` proves strict convexity on $[1/2,1]$. These closed-interval shape theorems use differentiability only on the interiors, so they are compatible with the singular endpoint slopes.

The downstream module `InverseModulus.lean` packages the sharp global modulus. Its increment `fabiusIntervalMass` is reflection-invariant and globally unimodal for every nonnegative length, by `fabiusIntervalMass_reflect`, `fabiusIntervalMass_eq_zero_of_add_nonpos`, and `fabiusIntervalMass_eq_zero_of_one_le`; the strict rise/fall on the nondegenerate support window is `strictMonoOn_fabiusIntervalMass_firstHalf` and `strictAntiOn_fabiusIntervalMass_secondHalf`. Hence the endpoint comparison has both strict and equality classifications: `fabiusReal_lt_fabiusIntervalMass_of_mem_Ioo`, `fabiusIntervalMass_eq_fabiusReal_iff`, `fabiusReal_sub_lt_sub`, and `fabiusReal_sub_eq_sub_iff`; the additive equality form is `fabiusReal_add_eq_iff`. The global and unit-interval inverse equality loci are `fabiusInv_sub_eq_sub_iff_of_mem_Icc` and `abs_fabiusInv_sub_eq_iff_of_mem_Icc`, respectively. These refinements preserve the clamping caveat: the complete equality classification is

stated on the unit interval, while the global statements retain only the sharp inequalities and exact attained moduli. The same shape results also give `monotoneOn_fabiusIntervalMass_firstHalf`, and `antitoneOn_fabiusIntervalMass_secondHalf`. Hence `fabiusReal_sub_le_sub` and `fabiusReal_add_le` prove the least endpoint mass and constrained forward superadditivity. The three ordered/order-free inverse-gap forms are `fabiusInv_sub_le_sub_of_mem_Icc`, `fabiusInv_sub_le_sub_of_le`, and `fabiusInv_sub_le_sub`; `fabiusInv_add_le` gives global inverse subadditivity. In particular,

$$|F^{-1}(u) - F^{-1}(v)| \le F^{-1}(|u - v|)$$

for arbitrary real inputs, exactly `abs_fabiusInv_sub_le` (with metric spelling `dist_fabiusInv_le`). Saturation is `fabiusInv_min_one`. The attained global and unit-interval maxima and their supremum equalities are `isGreatest_abs_fabiusInv_sub`, `sSup_abs_fabiusInv_sub_eq`, `isGreatest_abs_fabiusInv_sub_Icc`, and `sSup_abs_fabiusInv_sub_Icc_eq`. Finally, `abs_fabiusInv_sub_lt_of_abs_sub_lt_fabiusReal`, `abs_fabiusInv_sub_le_of_abs_sub_le_fabiusReal`, `fabiusReal_le_abs_sub_of_le_abs_fabiusInv_sub`, and `forall_abs_fabiusInv_sub_lt_iff` give the strict, closed, contrapositive, and exact-threshold effective-injectivity forms. This module does not supply recursive-modulus packaging or an inversion algorithm.

`FabiusInverseEffectiveContinuity.lean` supplies the explicit primitive-recursive specialization of that modulus. Its two denominators are

$$D_{\mathrm{fac}}(r) = 2^{\binom{r+1}{2}}(r+1)!, \qquad D_{\Delta}(r) = 2^{\binom{r+1}{2}}(r+1)^{r+1}.$$

They are `inverseFabiusFactorialDenominator`, with closed form `inverseFabiusFactorialDenominator_eq`, and `inverseFabiusDeltaDenominator`; their primitive-recursive proofs are `inverseFabiusFactorialDenominator_primrec` and `inverseFabiusDeltaDenominator_primrec`. Retaining the positive zeroth recurrence term gives `fabiusReal_inverse_two_pow_one_term_lower_bound`. Its weakened factorial form, the comparison $D_{\mathrm{fac}} \le D_{\Delta}$, and the resulting elementary lower bound are respectively `inv_inverseFabiusFactorialDenominator_le_fabiusReal`, `inverseFabiusFactorialDenominator_le_deltaDenominator`, and `inv_inverseFabiusDeltaDenominator_le_fabiusReal`.

The strict and closed inverse moduli for the factorial denominator are `abs_fabiusInv_sub_lt_inverse_two_pow_of_lt_factorialDenominator` and `abs_fabiusInv_sub_le_inverse_two_pow_of_le_factorialDenominator`; the corresponding elementary-denominator forms are `abs_fabiusInv_sub_lt_inverse_two_pow_of_lt_deltaDenominator` and `abs_fabiusInv_sub_le_inverse_two_pow_of_le_deltaDenominator`. Finally `fabiusInv_effectivelyUniformContinuous` packages the reciprocal-precision condition as `EffectivelyUniformContinuous`. These two definitions and twelve theorems are the module's complete public surface. This leaf itself does not provide a tolerant-bisection realizer, sequential computability, or a combined `IsComputableRealFunction` theorem; the downstream effective-inversion modules below now supply all three.

The downstream `FabiusInverseLogarithmicModulus.lean` replaces the coarse choice $r = n$ by

$$r_{\log}(n) = \mathrm{Nat.log2}(n) + 1.$$

The definition is `inverseFabiusLogarithmicOrder`. Its exact binary-length equation, primitive-recursive certificate, least-order theorem for $n > 0$, and comparison $r_{\log}(n) \le n$ are `inverseFabiusLogarithmicOrder_eq_succ_log2`, `inverseFabiusLogarithmicOrder_primrec`, `inverseFabiusLogarithmicOrder_isLeast`, and `inverseFabiusLogarithmicOrder_le_self`. Thus, at positive n, it is exactly the least natural r with $n < 2^r$.

The other two definitions are `inverseFabiusLogarithmicFactorialDenominator` and `inverseFabiusLogarithmicDeltaDenominator`. They take value one at zero and, at positive

n, evaluate the corresponding dyadic denominator at $r_{\log}(n)$. Those positive-input equations are `inverseFabiusLogarithmicFactorialDenominator_of_pos` and `inverseFabiusLogarithmicDeltaDenominator_of_pos`; primitive recursiveness is proved by `inverseFabiusLogarithmicFactorialDenominator_primrec` and `inverseFabiusLogarithmicDeltaDenominator_primrec`; and the pointwise factorial-to-Delta comparison is `inverseFabiusLogarithmicFactorialDenominator_le_deltaDenominator`.

For either logarithmic denominator d, both threshold conventions give a strict reciprocal output estimate:

$$|u-v| < d(n)^{-1} \Longrightarrow |I(u)-I(v)| < \frac{1}{n}, \qquad |u-v| \le d(n)^{-1} \Longrightarrow |I(u)-I(v)| < \frac{1}{n} \quad (n>0).$$

The factorial declarations are `abs_fabiusInv_sub_lt_inv_nat_of_lt_logarithmicFactorialDenominator` and `abs_fabiusInv_sub_lt_inv_nat_of_le_logarithmicFactorialDenominator`; the Delta declarations are `abs_fabiusInv_sub_lt_inv_nat_of_lt_logarithmicDeltaDenominator` and `abs_fabiusInv_sub_lt_inv_nat_of_le_logarithmicDeltaDenominator`. The closed input threshold still has a strict conclusion because $2^{-r_{\log}(n)} < 1/n$. Finally, `fabiusInv_effectivelyUniformContinuous_logarithmic` and `fabiusInv_effectivelyUniformContinuous_logarithmicDelta` separately package the factorial and report-exact Delta witnesses for `EffectivelyUniformContinuous`.

These three definitions and fifteen theorems are the complete public surface. This module itself does not construct the exact endpoint-mass ceiling denominator or a tolerant-bisection realizer. The downstream modules below prove `SequentiallyComputable` and the combined `IsComputableRealFunction` statement; no input-bit asymptotic is claimed.

The adjacent `FabiusInverseExactDyadicModulus.lean` closes the fixed-target ceiling problem without replacing those computable witnesses. For a dyadic order r, `inverseFabiusExactDyadicDenominator` is the natural ceiling of the reciprocal of the exact rational evaluator at 2^{-r}. `inverseFabiusExactDyadicDenominator_pos`, `inv_inverseFabiusExactDyadicDenominator_le_fabiusAtInverseTwoPow`, and `inverseFabiusExactDyadicDenominator_isLeast` prove positivity, the reciprocal endpoint bound, and leastness among positive integer denominators satisfying that bound. Casting the exact evaluator to the real Fabius value gives `abs_fabiusInv_sub_lt_inverse_two_pow_of_lt_exactDyadicDenominator`. If a positive d is smaller, the endpoint pair $(0, F(2^{-r}))$ has input gap below d^{-1} and inverse-output gap exactly 2^{-r}; this is `exists_fabiusInv_gap_of_lt_exactDyadicDenominator`. Combining the two directions gives `inverseFabiusExactDyadicDenominator_isLeast_strictModulus`, whose leastness is only for that fixed dyadic output target.

The second definition, `inverseFabiusExactLogarithmicDenominator`, is 1 at zero and is the fixed-target ceiling at $r_{\log}(n)$ when $n > 0$. The latter equation is `inverseFabiusExactLogarithmicDenominator_of_pos`; composing its strict dyadic modulus with $2^{-r_{\log}(n)} < 1/n$ gives `abs_fabiusInv_sub_lt_inv_nat_of_lt_exactLogarithmicDenominator`. This reciprocal-target theorem is a witness only, not leastness for $1/n$, and no conclusion is asserted at the convention-only input $n = 0$. The the two exact denominator maps are primitive recursive by `inverseFabiusExactDyadicDenominator_primrec` and `inverseFabiusExactLogarithmicDenominator_primrec`; the proof uses natural recurrence codes and does not assume normalized-rational computability. These two definitions and ten theorems are its complete public surface. This surface first entered as a source-only overlay whose preceding walkthrough PDF did not include it. It is included in this rendered publication; the current source/PDF publication receipt is kept externally in the repository documentation.

The generic `EffectiveMonotoneInverse.lean` layer has exactly two public definitions and six theorems. Its definitions are `SequentiallyComputableOn` and `unitClamp`; its theorems are `unitClamp_sequentiallyComputable`, `tolerantDifference_error`, `tolerantDifference_safe_updates`, `tolerantDifference_inconclusive`, `tolerantBisection_correct`,

and `effectiveInversionOn_Icc`. For a computably dyadically approximable strict monotone bijection of $[0,1]$, a computable positive reciprocal inverse modulus makes its inverse sequentially computable on that interval. The comparison is genuinely three-way: a certified large signed difference updates the bracket, while the inconclusive branch already proves the requested inverse approximation, so equality of computable reals is never decided.

The specialization `FabiusInverseComputable.lean` has no public definitions and exactly one theorem, `fabiusInv_isComputableRealFunction`. Tolerant bisection supplies the sequential clause on $[0,1]$, input clamping extends it to the total inverse on $\mathbb{R}$, and the logarithmic Delta modulus supplies effective uniform continuity. This is a computability certificate for every bounded Fabius witness, not an exact least endpoint-mass denominator or an input-bit running-time theorem.

The next two modules close the realizer gap. The exhaustive public surface of `EffectiveMonotoneInverse.lean` is exactly two definitions, `SequentiallyComputableOn` and `unitClamp`, followed by exactly six theorems: `unitClamp_sequentiallyComputable`, `tolerantDifference_error`, `tolerantDifference_safe_updates`, `tolerantDifference_inconclusive`, `tolerantBisection_correct`, and `effectiveInversionOn_Icc`. The implementation runs exactly p dyadic steps for requested precision p. At depth s, a natural index j represents the bracket $[j2^{-s}, (j+1)2^{-s}]$, and the two names are queried at precision $q = p + d(p) + 3 + s$. Their signed numerators are compared with a four-unit margin. A certified sign selects the corresponding half; an inconclusive comparison records the midpoint, and every remaining fixed step doubles its numerator so that its real value is unchanged while its denominator advances to 2^p. Thus equality of represented reals is never tested. A run with no inconclusive branch ends in a bracket of width 2^{-p}, while an inconclusive branch is certified by the assumed inverse modulus.

The generic theorem assumes a computable dyadic forward evaluator, strict monotonicity, both maps of $[0,1]$, inverse identities there, a positive computable denominator d, and

$$|u - v| < d(p)^{-1} \Longrightarrow |g(u) - g(v)| < 2^{-p}.$$

It consumes this inverse modulus. The adjacent `EffectiveGapInverse.lean` layer derives one from a supplied computable positive rational dyadic-gap sequence. The hard-coded unit interval is also significant: no arbitrary-endpoint algorithm is asserted without computable endpoint data.

That gap layer has exactly four public definitions and four public theorems: `EffectivelyUniformContinuousOn`, the structure `ComputablePositiveRationalSequence`, `ComputablePositiveRationalSequence.value`, `ComputablePositiveRationalSequence.reciprocalDenominator`, `ComputablePositiveRationalSequence.reciprocalDenominator_spec`, `inverseModulus_of_positiveRationalGap`, `effectiveInversionOn_Icc_of_computablePositiveRationalGap`, and `clampedEffectiveInversion_of_computablePositiveRationalGap`. The sequence contains computable positive natural numerators and denominators; the reciprocal denominator is their natural quotient plus one, so it is computable, positive, and has reciprocal strictly below the encoded rational value. For a strict increasing inverse pair on $[0,1]$, the exact analytic hypothesis is

$$\alpha(p) \le f(x + 2^{-p}) - f(x) \quad \text{for every } x \in [0, 1 - 2^{-p}].$$

This yields the inverse modulus. A computable dyadic evaluator for f and interval maps for both functions then yield sequential computability and effective uniform continuity of g on $[0,1]$. The total theorem certifies exactly $x \mapsto g(\mathrm{unitClamp}(x))$; it agrees with g on the unit interval but makes no claim about the unclamped values of g outside it.

The specialization `FabiusInverseComputable.lean` has no public definition and exactly one public theorem, `fabiusInv_isComputableRealFunction`. The sequential clause uses the centered-spline evaluator, the Delta denominator, tolerant bisection on $[0,1]$, and the computable 1-Lipschitz clamp. Its effective-uniform- continuity clause is `fabiusInv_effectivelyUniformContinuous_logarithmicDelta`. Consequently the totalized Fabius inverse, not merely its

unit-interval restriction, satisfies `IsComputableRealFunction`. These two modules expose all nine fixed-depth/Fabius declarations just listed; the gap layer adds its eight declarations and no input-bit or running-time asymptotic.

The separate smooth bridge `FabiusInverseQuarterJet.lean` identifies the complete inverse jet at the quarter anchor. For $G(y) = \mathrm{fabiusInv}(F, h_F, y)$,

$$\frac{d^n}{dt^n}\left(G\left(\frac{5}{72}+t\right)-\frac{1}{4}\right)\Big|_{t=0} = n!\,[X^n]\big(\mathrm{QuadraticInverse.\,inverse}(4)\big),$$

and hence

$$G^{(m+1)}(5/72) = (m+1)!(-4)^m C_m.$$

The exact declarations are `iteratedDeriv_centeredFabiusInv_quarter_eq_quadraticInverse` and `iteratedDeriv_fabiusInv_five_seventy_two_succ`. They prove equality of every derivative jet, not equality with an analytic function on a neighborhood or a named nonzero flat-remainder decomposition.

Flatness of F becomes steepness of F^{-1}. Substituting $x = F^{-1}(y)$ into the effective estimate gives, on the stated shrinking window,

$$y \le 2^{\binom{n+1}{2}}\big(F^{-1}(y)\big)^n,$$

with a factorial-strengthened version as well. In particular $F^{-1}(y)/y \to +\infty$ as $y \downarrow 0$, and reflection supplies the right-endpoint counterpart. More sharply, the positive interior derivative itself tends to $+\infty$ at both ends, by `tendsto_deriv_fabiusInv_atTop_at_zero_right` and `tendsto_deriv_fabiusInv_atTop_at_one_left`. These are one-sided limits through $(0,1)$, not derivative values at the clamping points.

7.4 Exact inverse modulus and rigidity (post-snapshot)

The structural layer `InverseModulus.lean` now has an exhaustive public surface of one definition and thirty-one theorems. Relative to its earlier twenty-theorem surface, the eleven equality/rigidity declarations, in source order, are:

```
fabiusIntervalMass_eq_zero_of_add_nonpos
fabiusIntervalMass_eq_zero_of_one_le
strictMonoOn_fabiusIntervalMass_firstHalf
strictAntiOn_fabiusIntervalMass_secondHalf
fabiusReal_lt_fabiusIntervalMass_of_mem_Ioo
fabiusIntervalMass_eq_fabiusReal_iff
fabiusReal_sub_lt_sub
fabiusReal_sub_eq_sub_iff
fabiusReal_add_eq_iff
fabiusInv_sub_eq_sub_iff_of_mem_Icc
abs_fabiusInv_sub_eq_iff_of_mem_Icc
```

The first two record the zero-mass tails for nonnegative interval lengths. The global weak shape remains valid for every nonnegative length; at a nondegenerate length $0 < h \le 1$, the new strict branches are exactly $[-h, (1-h)/2]$ and $[(1-h)/2, 1]$, ending where the two constant tails begin. Within $[0,1]$, an interval of length h has the endpoint mass $F(h)$ exactly when $h = 0$, its left endpoint is zero, or its right endpoint is one. The strict-interior and subtraction wrappers give the same classification for the least-mass inequality, while constrained superadditivity is an equality exactly when one summand is zero or their sum is one.

Transport through the inverse classifies the unit-interval equality cases as well. For $0 \le u \le v \le 1$, the ordered inverse-gap bound is sharp exactly when $u = v$, $u = 0$, or $v = 1$. In the order-free

absolute bound, equality for $u, v \in [0,1]$ means that the inputs coincide or at least one is an endpoint. Both restrictions to the unit interval are essential: the totalized inverse has constant tails, which create additional equality configurations on all of $\mathbb{R}$. The preceding twenty declarations still supply the weak shape, sharp inequalities, attained exact moduli, and effective-injectivity implications; the next module turns the structural modulus into explicit recursive dyadic denominators.

7.5 Effective dyadic continuity of the noncomputable inverse

`FabiusInverseEffectiveContinuity.lean` supplies an explicit primitive-recursive modulus for the totalized inverse. Its two natural-valued denominators are

$$D_{\mathrm{fac}}(r) = 2^{\binom{r+1}{2}}(r+1)!, \qquad D_{\Delta}(r) = 2^{\binom{r+1}{2}}(r+1)^{r+1}.$$

The first is defined recursively and then identified with the displayed factorial form; the second is the elementary box-event denominator from the inverse-computability report. Both functions are primitive recursive and $D_{\mathrm{fac}}(r) \le D_{\Delta}(r)$ for every r.

Fix F of type `BoundedFabius` and let h_F witness `IsFabius` F. Only the sharp one-term recurrence estimate requires $r > 0$:

$$\frac{1}{2^{\binom{r}{2}}(r+1)!(2^r-1)} \le F(2^{-r}).$$

The consequences are total in r, including $r = 0$:

$$D_{\mathrm{fac}}(r)^{-1} \le F(2^{-r}), \qquad D_{\Delta}(r)^{-1} \le F(2^{-r}).$$

For every real u, v, either denominator D gives the strict and closed inverse moduli

$$|u-v| < D(r)^{-1} \Longrightarrow |F^{-1}(u) - F^{-1}(v)| < 2^{-r},$$
$$|u-v| \le D(r)^{-1} \Longrightarrow |F^{-1}(u) - F^{-1}(v)| \le 2^{-r}.$$

Here F^{-1} means the globally clamped `fabiusInv`, so no unit-interval hypothesis is placed on u or v. Taking the stronger factorial denominator at $r = n$, and using $n < 2^n$, proves `fabiusInv_effectivelyUniformContinuous`, namely `EffectivelyUniformContinuous` for that totalized inverse, in the repository's positive-precision reciprocal convention.

The module's exhaustive public inventory is two definitions and twelve theorems:

No.	Declaration	Formal content
1	`inverseFabiusFactorialDenominator`	Recursive natural-valued factorial denominator.
2	`inverseFabiusFactorialDenominator_eq`	Closed form $D_{\mathrm{fac}}(r)$.
3	`inverseFabiusFactorialDenominator_primrec`	Primitive recursiveness of D_{fac}.
4	`inverseFabiusDeltaDenominator`	Natural-valued report denominator D_{Δ}.
5	`inverseFabiusDeltaDenominator_primrec`	Primitive recursiveness of D_{Δ}.
6	`fabiusReal_inverse_two_pow_one_term_lower_bound`	One-term endpoint bound, under h_F and $r > 0$.
7	`inv_inverseFabiusFactorialDenominator_le_fabiusReal`	Factorial reciprocal bound for every r, under h_F.
8	`inverseFabiusFactorialDenominator_le_deltaDenominator`	$D_{\mathrm{fac}}(r) \le D_{\Delta}(r)$, with no Fabius hypothesis.

No.	Declaration	Formal content
9	`inv_inverseFabiusDeltaDenominator_le_fabiusReal`	Report reciprocal bound for every r, under h_F.
10	`abs_fabiusInv_sub_lt_inverse_two_pow_of_lt_factorialDenominator`	Strict factorial-threshold modulus for all real inputs.
11	`abs_fabiusInv_sub_le_inverse_two_pow_of_le_factorialDenominator`	Closed factorial-threshold modulus for all real inputs.
12	`abs_fabiusInv_sub_lt_inverse_two_pow_of_lt_deltaDenominator`	Strict report-threshold modulus for all real inputs.
13	`abs_fabiusInv_sub_le_inverse_two_pow_of_le_deltaDenominator`	Closed report-threshold modulus for all real inputs.
14	`fabiusInv_effectivelyUniformContinuous`	Effective uniform continuity with witness $D_{\mathrm{fac}}(n)$.

The boundary is important. The two denominator definitions and their primitive-recursion proofs are executable, while the definition `fabiusInv` remains noncomputable. An effective modulus alone is not a name-producing algorithm, so this module by itself proves neither `SequentiallyComputable` nor a combined `IsComputableRealFunction` theorem. The later logarithmic and tolerant-bisection modules provide the selector and name-producing algorithm; the exact-dyadic leaf supplies fixed-target ceiling leastness, while a named recursion certificate for that exact denominator and input-bit complexity remain open.

7.6 Generic formal implicit roots versus inverse-Fabius germs

`ImplicitPowerSeries.lean` isolates a purely formal root engine. `FormalImplicitRoot.existsUnique_isRoot_sub_mem` lifts a simple approximate root modulo an adic ideal in a complete commutative ring without a monicity assumption. Its power-series specialization `PowerSeries.Implicit.existsUnique_zeroConstant_root` says that a polynomial over $R[[X]]$ with vanishing parameter-constant constant coefficient and unit parameter-constant linear coefficient has a unique zero-constant formal root. The chosen root and its uniqueness interface are `PowerSeries.Implicit.root`, `PowerSeries.Implicit.constantCoeff_root`, `PowerSeries.Implicit.eval_root`, and `PowerSeries.Implicit.eq_root`.

The specialization `QuarterCatalanGerm.lean` has two public definitions and thirteen public theorems. Besides the explicit unique zero-constant solution of $D + 4D^2 = (4/9)X$, it proves that the distinguished `dyadicGermTwo` obeys the same scaled equation. The four bridge declarations are `dyadicGermTwo_functionalEquation`, `rescale_dyadicGermTwo_eq_quadraticInverse`, `dyadicGermTwo_eq_rescale_quadraticInverse`, and `coeff_dyadicGermTwo_succ`; rescaling by $9/4$ gives $\mathrm{QuadraticInverse}.\mathrm{inverse}(4)$, and the degree-$(m+1)$ coefficient before that rescaling is $(4/9)^{m+1}(-4)^m C_m$.

These declarations are formal algebra only. They neither define the inverse frontier's $\mathcal{A}_r$ or Q_{x_0}, nor prove convergence or analyticity of the resulting series, identification with a local inverse, entry into a finite-prefix plateau, a nonzero flat defect, or any Fabius quantile formula.

7.7 Exact analytic loci and non-elementarity

The smooth function is analytic precisely off its closed transition interval:

$$\mathrm{AnalyticAt}(F, x) \quad \Longleftrightarrow \quad x \notin [0, 1].$$

`NowhereAnalytic.lean` includes both endpoints; outside the interval the function is locally constant. `InverseNotElementary.lean` proves the same analytic locus for the totalized inverse.

In the interior, analyticity of the inverse would make F analytic by the inverse-function theorem; at the endpoints, the identity theorem conflicts with the constant tails and strict increase.

The generic modules `AlgebraicBranch.lean` and `ElementaryFunction.lean` show that the formal class `IsElementary` has dense analytic locus, as do continuous algebraic branches with elementary coefficients. `NotElementary.lean` therefore excludes agreement with F on any subset of $[0,1]$ having nonempty interior. `InverseBranch.lean` proves that a continuous right-inverse branch of a densely analytic function is analytically dense on its branch domain. The constructor `IsElementaryOrInverse.invBranch` closes the class under such branches on an open set U only when the chosen total extension is analytic on $\mathrm{interior}(U^c)$; branch-domain boundaries are deliberately excluded. Its Lambert-W theorem is a conditional criterion for any branch satisfying these hypotheses, not an instantiated mathlib declaration of W. `InverseNotElementary.lean` then excludes both F and its totalized inverse from this enlarged class on $(0,1)$.

Formalization-sensitive point. These are theorems about the explicitly defined Lean predicates `IsElementary` and `IsElementaryOrInverse`, not a claim about every possible computer-algebra language or every informal use of the word "closed form." The precise constructors and branch hypotheses in `ElementaryFunction.lean` and `InverseBranch.lean` are part of the statement.

7.8 Primitive-recursive splines and computable reals

The approximation branch also became effective. `FabiusComputability.lean` proves the uniform estimates

$$\sup_{x\in[0,1]} |S_p(x) - F(x)| \le 2^{-p}, \qquad \sup_{x\in\mathbb{R}} |S_p(x) - F_{\mathrm{gl}}(x)| \le 2^{-p},$$

and records the primitive-recursive modulus $d(n) = 2n$ coming from the sharp Lipschitz bound. `FabiusComputableSpline.lean` implements the centered Thue–Morse spline using signed pairs of natural numerators, proves its pieces primitive recursive, rounds to a dyadic name with an explicit error budget, and supplies two evaluators with the same error $5 \cdot 2^{-(p+3)}$. The first clamps its input to the bounded interval and packages `IsComputableRealFunction(fabiusRealF)`. The second retains the unrestricted signed spline code; `extendedFabiusSplineApproxPR_error` proves its global error, and `extendedFabius_isComputableRealFunction` packages the result for the signed extension of every F satisfying `IsFabiusF`. Its canonical specialization is `globalFabius_isComputableRealFunction`. Thus both bounded and signed-global computability theorems contain the two required clauses: preservation of computable real sequences and effective uniform continuity. The unrestricted evaluator is a primitive-recursive certificate rather than a complexity claim, since its finite fold length depends on the positive input numerator.

Uniform convergence also controls changing inputs. The theorem `abs_fabiusUniformSpline_sub_extendedFabius_le_add` states

$$|S_p(x) - F_{\mathrm{gl}}(y)| \le 2^{-p} + 2|x - y|,$$

and `fabiusUniformSpline_tendsto_extendedFabius_of_tendsto` concludes that $S_p(x_p) \to F_{\mathrm{gl}}(x)$ whenever $x_p \to x$.

8 The exact–analytic dyadic bridge

8.1 The purpose of `DyadicAnalytic.lean`

The exact evaluator is useful only after it is tied to the analytic function. `DyadicAnalytic.lean` is the central bridge. It proves, for every `IsFabius` candidate, the same finite recurrences that were defined independently in `Arithmetic.lean`; then it identifies the two by induction.

The proof has four main stages:

1. derive a finite Taylor identity on a dyadic cell;
2. specialize it to inverse powers of two and obtain an endpoint recurrence;
3. prove that the analytic inverse-power sequence equals the exact rational table; and
4. lift this equality to arbitrary dyadic numerators through the highest-bit recursion.

The surrounding modules `DyadicClosedForm.lean` and `DyadicCorrectness.lean` isolate the algebraic and algorithmic portions of this chain.

8.2 Finite Taylor expansion on a dyadic scale

Fix a sufficiently small dyadic base point. The iterated derivative formula expresses every derivative up to a chosen order through a scaled value of F. At the order where the scaled argument leaves the active interval and enters the zero tail, higher derivatives vanish. Taylor's theorem therefore terminates exactly rather than producing an infinite series with a remainder.

A representative identity, in the normalization used by the evaluator, is

$$F(2^{-m}+y)+F(y)=\sum_{k=0}^{m}\frac{2^{\binom{k+1}{2}}}{k!}F(2^{-(m-k)})\,y^k,$$

valid for $m\geq 1$ and $0\leq y\leq 2^{-m}$. Thus the highest-bit evaluator computes the displayed polynomial and subtracts the recursively evaluated $F(y)$. The domain and the subtraction are essential parts of the recurrence.

The proof does not invoke analyticity. It uses a finite-order Taylor theorem with a remainder, then proves the next derivative vanishes on the relevant segment. This is crucial: the Fabius function is smooth but nowhere analytic in the appropriate sense, so an argument based on convergence of an infinite Taylor series would be false.

Formalization-sensitive point. Finite Taylor identities are local consequences of compact support and the differential equation. They do not imply that the function equals its infinite Taylor series in a neighborhood. The formalization later turns the same exact truncation phenomenon into a proof of nonanalyticity.

8.3 Inverse-power values

For a candidate F, define informally

$$A_n(F)=F(2^{-n}).$$

The dyadic Taylor identity at a suitable endpoint gives a recurrence for $A_{n+1}(F)$ in terms of earlier $A_j(F)$. Boundary compatibility and symmetry supply the initial value $A_1(F)=1/2$.

Independently, `Arithmetic.lean` defines a rational sequence satisfying the same recurrence. Strong induction proves

$$A_n(F)=A_n^{\mathrm{exact}}$$

for every n. The induction step is predominantly algebraic, but the statement lives in $\mathbb{R}$; the proof must cast exact rational sums and products into reals. The source localizes these casts in lemmas asserting that the rational recurrence is preserved by the canonical embedding $\mathbb{Q}\hookrightarrow\mathbb{R}$.

8.4 From closed polynomial to Horner program

Once the analytic value is a finite polynomial in exact inverse-power values, `DyadicAnalytic.lean` invokes the algebraic theorem that the closed polynomial equals `fabiusTaylorHorner`. This is a clean interface point:

- analysis proves the value is the mathematical polynomial;
- arithmetic proves the polynomial has exact rational coefficients; and
- algorithmics proves Horner evaluation computes that polynomial.

No proof about derivatives needs to know how a coefficient list is stored, and no proof about lists needs to know the fundamental theorem of calculus.

8.5 Correctness of highest-bit reduction

Suppose the numerator decomposes as $a = 2^b + r$. The analytic Taylor theorem proves exactly the recurrence used by the recursive evaluator. The recursive call has numerator $r < a$, so strong induction on a applies. At the end, the theorem states that evaluating the exact program and embedding its rational output into $\mathbb{R}$ gives $F(a/2^n)$.

The proof handles endpoint and out-of-range cases before entering the recursion. This matches the program structure and avoids forcing the induction hypothesis to cover impossible branch conditions. Each branch ends with a theorem about the corresponding normalization operation: left-tail clamping, right-tail clamping, highest-bit Taylor reduction, or refinement. Reflection appears only in the separate signed global evaluator.

8.6 Representation independence

The same dyadic rational has infinitely many representations:

$$\frac{a}{2^n} = \frac{2a}{2^{n+1}} = \frac{4a}{2^{n+2}} = \cdots .$$

The evaluator's mathematical specification therefore includes refinement invariance. `DyadicCorrectness.lean` proves that one common factor of two can be introduced or removed without changing the result; induction yields invariance under any common power.

There are two possible proof routes. One can prove both representations equal the analytic function and conclude by transitivity, or prove refinement directly from the exact recursion. The repository contains strong exact correctness results so that the executable layer remains trustworthy even when used independently of analysis. Analytic equality then gives a second conceptual explanation.

8.7 Dyadic density and uniqueness

Let F and G satisfy `IsFabius`. Correctness gives

$$F(q) = G(q)$$

for every dyadic rational q. To extend this to an arbitrary real x, choose dyadic approximants such as

$$q_n = 2^{-n} \lfloor 2^n x \rfloor .$$

Then $q_n \to x$. Continuity gives

$$F(x) = \lim_n F(q_n) = \lim_n G(q_n) = G(x).$$

In Lean, density can be applied through a generic theorem saying continuous maps equal on a dense set are equal, or through an explicit approximating sequence. The repository's dyadic infrastructure supplies the density statement in the exact form required by the function-extensionality proof. Tail values allow the argument to focus on $[0, 1]$ when convenient.

8.8 The global dyadic theorem

`GlobalDyadic.lean` transfers exact evaluation from the bounded interval to `extendedFabius`. The result accepts unrestricted integer numerators and nonreduced dyadic representations. It combines:

1. arithmetic decomposition into an integer translate and a unit-interval remainder;
2. the translation law for the signed extension;
3. exact unit-interval correctness; and
4. representation invariance.

This stronger formulation is the one required by the q-binomial identities, where summation indices naturally produce numerators outside $[0, 2^n]$.

8.9 The dyadic derivative filtration

The zero-definition/six-theorem module `DyadicDerivativeFiltration.lean` gives the complete derivative trichotomy at reduced dyadics. Its public surface is `rvachevUp_eq_zero_of_one_le_abs`, `iteratedDeriv_rvachevUp_dyadic_eq_zero`, `iteratedDeriv_rvachevUp_dyadic_critical`, `dyadic_depth_eq_max_nonzero_iteratedDeriv`, `iteratedDeriv_rvachevUp_eq_extendedFabius`, and `iteratedDeriv_rvachevUp_dyadic_below`. The first four supply the support lemma, vanishing above a reduced dyadic's depth, its critical signed power of two, and exact depth detection. The final two are the new below-depth closure. In fact the first of them is stronger than a dyadic statement: for every natural m and every $x < 1$,

$$\mathrm{up}^{(m)}(x) = 2^{\binom{m+1}{2}} F_{\mathrm{gl}}(2^m(x+1)) \,.$$

Consequently, if $m < n$ and $a < 2^n$,

$$\mathrm{up}^{(m)}\left(\frac{a}{2^n}\right) = 2^{\binom{m+1}{2}} F_{\mathrm{gl}}\left(\frac{2^n + a}{2^{n-m}}\right).$$

Together with the critical and above-depth theorems, this exhausts the module's public declarations. The source statements concern the derivatives themselves; they do not add the frontier volume's normalized-by-π^m notation.

Architectural point. The dyadic bridge is the repository's main "commuting square." One path starts with an analytic function and derives a finite Taylor recurrence; the other starts with exact rational recurrences and an evaluator. The correctness theorem says both paths produce the same real number.

---

9 Moments, generating functions, and exact denominators

9.1 Analytic moments of the up-function

`AnalyticMoments.lean` develops integral interpretations of the exact moment sequences. Because up is smooth and compactly supported, every polynomial moment exists:

$$M_n = \int_{\mathbb{R}} x^n \,\mathrm{up}(x)\, \mathrm{d}x.$$

Evenness immediately gives $M_{2k+1} = 0$, while the exact rational sequence is indexed only by the even moments:

$$M_{2k} = (\mathrm{moment}(k) : \mathbb{R}).$$

The total mass $M_0 = 1$ is proved from the bounded differential equation and the fundamental theorem of calculus, rather than assumed as part of the definition.

The main recurrence is obtained by integration by parts and a dyadic change of variables. The precise coefficients depend on the chosen centering of up, but the structure is a finite convolution of lower moments with powers of two and binomial coefficients. The same recurrence was used to define `moment` in the exact layer.

9.1.1 Integration over compact support

Mathlib's integral on $\mathbb{R}$ is convenient for Fourier transforms, while interval integrals are convenient for integration by parts. The proof repeatedly converts between them using support lemmas. A standard sequence is:

1. prove the integrand vanishes outside a compact interval;
2. rewrite the whole-line integral as an integral over that interval;
3. apply an interval integration-by-parts theorem;
4. show endpoint terms vanish by support or flatness; and
5. transform the remaining integral by an affine substitution.

The support theorem is therefore not merely qualitative metadata; it is an active rewrite rule used throughout the analytic stack.

9.2 Identifying analytic and exact moments

After deriving the recurrence, the module proves by strong induction that

$$M_{2n} = (\mathrm{moment}(n) : \mathbb{R})$$

for every n. Raw odd moments are handled separately by evenness. The displayed formula intentionally suppresses namespace syntax; the Lean theorem states the rational cast explicitly.

The base case is total mass. In the induction step, the analytic recurrence is rewritten term by term using the induction hypotheses. A cast lemma turns the finite rational sum into the corresponding real sum. The exact recurrence then closes the goal.

The same method identifies the normalized integrals of up over $[0, 1]$ with `halfMoment`. The source-facing theorem starts at positive index, while `halfMoment_eq_integral_formula_all` packages the normalized zeroth value and the integral cases into one statement for every natural index. These moments are connected more directly to inverse-power Fabius values, so they become the analytic bridge between moment theory and the dyadic table.

9.3 Generating functions

The analytic moment generating function is represented both as an integral and as a convergent series. Compact support gives uniform bounds

$$|M_n| \le CR^n,$$

which are more than sufficient for convergence of

$$\sum_{n\ge 0} M_n \frac{z^n}{n!}$$

for every complex z. Dominated convergence justifies interchanging sum and integral, giving the usual identity with $\int e^{zx}\,\mathrm{up}(x)\,\mathrm{d}x$.

`MomentPowerSeries.lean` is a separate exact-algebra layer over `PowerSeries` over $\mathbb{Q}$. If

$$M(X)=\sum_{n\ge 0}\frac{\mathrm{moment}(n)}{(2n)!}X^n,\qquad S(X)=\sum_{n\ge 0}\frac{X^n}{(2n+1)!},$$

then its central identity is

$$M(4X)=M(X)S(X).$$

This is a coefficient theorem in exact rational power series, not an appeal to `FormalMultilinearSeries` or `HasFPowerSeriesAt`. Analytic modules later identify the evaluated series with transforms after proving convergence. The half-moment bridge also gives the exact inverse-power formula

$$F(2^{-n})=\frac{\mathrm{halfMoment}(n)}{n!\,2^{\binom{n}{2}}}.$$

9.4 Removable singularities

Expressions such as

$$\frac{e^z-1}{z}$$

appear in half-moment generating functions. The source defines a total function with value one at zero, then proves continuity and analyticity across the removable singularity. This avoids carrying $z\neq 0$ through every downstream identity.

The proof combines the exponential power series with division by z term by term. In Lean, defining the value at zero first and proving a global power-series expansion is often cleaner than starting with a piecewise quotient and repeatedly rewriting the nonzero branch.

9.5 Normalized half moments

`NormalizedMoments.lean` proves a closed denominator normalization of the form

$$\mathrm{halfMoment}(n)=\frac{H_n}{(n+1)!\prod_{j=1}^{n}(2^j-1)},$$

where H_n is the division-free natural numerator defined in `Arithmetic.lean`. The theorem is not obtained by asking Lean to normalize a large rational expression automatically. It is proved by strong induction with a deliberately chosen common denominator.

At the induction step, every earlier half moment has a denominator whose factorial and Mersenne products are prefixes of the target denominator. The missing suffix factors are inserted explicitly. Product-splitting lemmas show that multiplying an earlier term by its suffix gives the target common denominator. The natural numerator recurrence was designed to be exactly the resulting cleared equation.

9.6 Normalized even moments

`NormalizedEvenMoments.lean` proves the analogous expression for even centered moments. A representative denominator consists of an odd double factorial and an even-indexed Mersenne product:

$$M_{2n}=\frac{E_n}{(2n+1)!!\prod_{j=1}^{n}(2^{2j}-1)}.$$

The actual indexing in the source is chosen to align with its recursive sequence, but the structural point is the same. Odd moments vanish analytically, while even moments carry all arithmetic information.

The proof requires more combinatorial normalization than the half-moment case because binomial coefficients and parity restrictions appear in the recurrence. Factorial identities convert binomial coefficients into products compatible with the odd double factorial. The final division-free recurrence is again a natural-number equality.

9.7 Parity through Lucas' theorem

`Parity.lean` proves that the natural moment numerators are odd. The key combinatorial input is Lucas' theorem modulo two: a binomial coefficient $\binom{n}{k}$ is odd exactly when every one-bit of k occurs at a one-bit position of n. Consequently the number of odd entries in row n of Pascal's triangle is

$$2^{\mathrm{wt}_2(n)}.$$

The moment recurrence is reduced modulo two. Most terms vanish in pairs or carry explicit factors of two; the surviving binomial pattern is counted using Lucas' theorem. The source must bridge several representations of a finite range—natural inequalities, `Fin` types, and `Finset.range`—before applying mathlib's Lucas results. Dedicated cardinality lemmas hide this bridge from the final parity induction.

9.8 Reduced numerators, denominators, and two-adic valuation

`TwoAdic.lean` turns the normalization and parity theorems into exact assertions about reduced rational numerators and denominators. If a displayed quotient has odd numerator and a denominator whose power of two is known, then no cancellation of two-adic factors is possible. This determines the two-adic valuation of the reduced denominator.

The argument differs for odd and even indices. Odd-index statements reduce to the normalized even moments. Even-index statements use the half-moment recurrence together with the Lucas count of odd binomial coefficients. The formal proof includes lemmas relating:

- the numerator and denominator fields of a normalized rational;
- divisibility by two in $\mathbb{N}_0$ and $\mathbb{Z}$;
- coprimality in reduced form; and
- a valuation statement for the dyadic value $F(2^{-n})$.

The later leaf `PrimePowerBinomialValuation.lean` exposes the prime-power Pascal-row identity directly through `padicValNat`. Its exhaustive public API consists of exactly six theorems and no definitions: `primePowerChoose_padicValNat_add`, `primePowerChoose_padicValNat`, and `twoPowChoose_padicValNat`, together with `primePowerSubOneChoose_padicValNat`, `primePowerSubTwoChoose_padicValNat`, and `twoPowSubTwoChoose_padicValNat`. For every prime p, exponent m, and $0 < j \le p^m$, the first two state

$$\nu_p\binom{p^m}{j} + \nu_p(j) = m, \qquad \nu_p\binom{p^m}{j} = m - \nu_p(j).$$

The generic surface includes $j = p^m$ and $m = 0$, excluding only $j = 0$ because the valuation is totalized there. The third theorem is the strict dyadic-comb specialization $0 < j < 2^m$. The new generic companions prove that row $p^m - 1$ consists of p-adic units for $j < p^m$, and that $\nu_p\binom{p^m-2}{j-1} = \nu_p(j)$ for exactly $0 < j < p^m$; the last theorem is its dyadic wrapper. The upper endpoint is excluded because that binomial coefficient is zero there. No valuation-histogram count is proved.

9.9 Denominator bounds

`DenominatorBound.lean` and `HalfMomentDenominator.lean` assemble divisibility results into explicit bounds. Rather than attempting to compute the reduced denominator directly, the proof supplies a common denominator known to clear every term in a recurrence. The reduced denominator then divides that common denominator by the universal property of rational normalization.

This method is robust under changes of indexing and does not require unique factorization calculations for the full denominator. Exact two-adic theorems can be layered on top where a sharper result is needed.

9.10 Bernoulli recurrences and source-facing arithmetic

`BernoulliRecurrences.lean` translates some moment identities into Bernoulli-number language used by the arithmetic paper. The formalization keeps Bernoulli manipulations separate from the basic moment recurrence: the latter is elementary and executable, while the former uses a larger special-function and finite-sum API.

`PaperStatements.lean` then exposes propositions and theorems with numbering matching the paper. Its proofs are often short compositions of the stronger internal results described above. This is a good place to learn what each source theorem says, but not always the place to learn why it is true.

10 Fourier analysis and Rvachev's original characterization

10.1 Why the up-function is the Fourier object

The bounded Fabius function has nonzero constant tails and is not integrable on the line. The Rvachev up-function is compactly supported, smooth, and even, so it belongs to every L^p space and, after viewing it as a smooth function with compact support, defines a Schwartz function. This is the natural object for the Fourier-transform theorems in `FourierAnalytic.lean` and `OriginalUniqueness.lean`.

The original characterization asks for a smooth compactly supported function satisfying a refinement or derivative equation and a normalization. The repository formalizes both existence and uniqueness. The uniqueness proof is especially instructive because it turns a differential equation in physical space into a multiplicative recurrence in frequency space.

The source predicate retains the paper's explicit assumption that its dilation scale is positive. The smart constructor `IsOriginalFabius.mk_of_derivative_law` shows that this sign is actually forced by the remaining hypotheses: a mean-value point in $(-1, 0)$ reduces the derivative law to $k\varphi(2c + 1) = 1$ with a strictly positive second factor.

The two formal solution spaces are connected explicitly. The theorem `IsFabius.isOriginalFabius_rvachevUp` says that the Rvachev fold of every bounded Fabius solution satisfies the original compact-support characterization at scale two. Conversely, `isOriginalFabius_iff_existsUnique_isFabius` says that every original solution has scale two and is the fold of a unique bounded Fabius solution. This is the correct tail-free equivalence: a direct predicate iff for an arbitrary bounded function would be false because its fold never inspects values on $(1, \infty)$. The theorem `rvachevUp_eq_iff_eqOn_Iic_one` makes that loss of information exact: two folds agree if and only if their candidates agree on $(-\infty, 1]$. Accordingly, `isFabius_iff_isOriginalFabius_rvachevUp_and_rightTail` restores the sharp fixed-candidate iff by adjoining only the condition $F(x) = 1$ for $x > 1$.

10.2 Fourier conventions

Formal Fourier proofs are extremely sensitive to normalization. Mathlib's transform includes a fixed kernel convention; the source therefore writes explicit factors of π, 2π, and the imaginary unit rather than silently adopting a paper convention. The sinc factor is defined compatibly with that convention and extended continuously at zero.

A schematic transform identity is

$$\widehat{\mathrm{up}}_{2\pi}(z) = \mathrm{sinc}_{\mathrm{rad}}(\pi z)\,\widehat{\mathrm{up}}_{2\pi}(z/2).$$

Depending on whether the source theorem uses angular or ordinary frequency, the argument of sinc is adjusted. The Lean theorem's constants should be taken as authoritative; the important structural feature is multiplication by one sinc factor and halving of frequency.

10.3 Transforming the derivative equation

The source first proves that a candidate satisfying the original assumptions is compactly supported and smooth enough to be represented as a Schwartz map. It then applies the Fourier derivative theorem and the affine change-of-variable theorem. A derivative contributes a frequency factor; a dilation contributes both a reciprocal Jacobian and a rescaled frequency; translations contribute complex phases.

After simplification, the phases combine into a sine quotient, producing the sinc recurrence. The mass normalization gives

$$\widehat{\mathrm{up}}_{2\pi}(0) = \int_{\mathbb{R}} \mathrm{up}(x)\ \mathrm{d}x = 1.$$

This is the seed value that controls the limit after infinitely many refinements.

Lean pattern. The proof does not rewrite Fourier integrals by hand at every step. It first establishes membership in the function class required by mathlib, then invokes library theorems for derivative, translation, and dilation. Only the final algebra of complex exponentials is expanded. This concentrates the fragile measure-theoretic assumptions near the beginning of the proof.

10.4 Iteration to a finite product

Iterating the refinement identity n times gives

$$\widehat{\mathrm{up}}_{2\pi}(z) = \left(\prod_{j=0}^{n-1} \mathrm{sinc}_{\mathrm{rad}}\left(\frac{\pi z}{2^j}\right)\right) \widehat{\mathrm{up}}_{2\pi}\left(\frac{z}{2^n}\right).$$

This is proved by induction. The Lean induction must reconcile two equivalent ways of writing a finite product: appending the last factor versus splitting off the first. A small library of `Finset.range` product identities keeps the main theorem readable.

As $n \to \infty$, $z/2^n \to 0$, so continuity of the Fourier transform gives

$$\widehat{\mathrm{up}}_{2\pi}(z/2^n) \to 1.$$

The formal product proof uses the available bound $\mathrm{complexSinc}(w) - 1 = O(w)$ near zero, so the deviations are dominated by the geometric series $\sum_j |\pi z|/2^j$. The sharper classical quadratic expansion is not needed.

10.5 The infinite sinc product

`FourierProduct.lean` develops the technical product independently. It proves that complex sinc is entire, establishes pointwise multipliability and the finite-product recurrence, and identifies the product with the analytic Fourier transform. Continuity then follows through that identification rather than from a separate locally uniform product theorem in this module.

Infinite products in Lean can be represented through partial products and limits rather than through a single primitive operator. The source proves that the sequence is Cauchy using a summable bound on deviations from one. Special care is taken at zeros of a sinc factor: logarithmic product arguments are invalid there, whereas direct partial-product convergence remains sound.

The result is the celebrated formula

$$\widehat{\mathrm{up}}_{2\pi}(z) = \prod_{j\geq 0} \mathrm{sinc}_{\mathrm{rad}}\Big(\frac{\pi z}{2^j}\Big)$$

with the indexing adjusted to the normalization used by the code.

10.6 Generalized products and dyadic-order identifiability

For an exponent sequence $a : \mathbb{N}_0 \to \mathbb{N}_0$ with $\sum_{h\geq 0} a(h)/2^h < \infty$, the generalized product is

$$\Phi_a(z) = \prod_{h\geq 0} \mathrm{sinc}_{\mathrm{rad}}\Big(\frac{\pi z}{2^h}\Big)^{a(h)}.$$

The complete divisor API reduces its analytic order at a positive integer to the weighted dyadic multiplicity. At the power 2^n, this becomes

$$\mathrm{ord}_{z=2^n}\,\Phi_a = \sum_{h=0}^{n} a(h). \tag{1}$$

Thus the dyadic orders recover every exponent:

$$a(0) = \mathrm{ENat.toNat}(\mathrm{ord}_{z=1}\,\Phi_a)\,, \tag{2}$$

$$a(n+1) = \mathrm{ENat.toNat}(\mathrm{ord}_{z=2^{n+1}}\,\Phi_a) - \mathrm{ENat.toNat}(\mathrm{ord}_{z=2^n}\,\Phi_a)\,. \tag{3}$$

The use of $\mathrm{ENat.toNat}$ is safe here because (1) has already exhibited each order as a finite natural cast.

The proof is deliberately triangular. The valuation identity $\nu_2(2^n) = n$ turns each sampled order into the inclusive prefix $P_a(n)$. For $n = 0$, the sample is $z = 1$, not $z = 0$, and $P_a(0) = a(0)$; all these products have value one at the origin. The successor law $P_a(n+1) = P_a(n) + a(n+1)$ and natural cancellation give the consecutive-difference formula. If two full product functions are equal, applying `congrArg` to `analyticOrderAt` at each dyadic point makes all prefixes equal, so cancellation recovers the same sequence. Conversely, equal sequences give definitionally equal products. Hence, for admissible a, c,

$$\Phi_a = \Phi_c \quad\Longleftrightarrow\quad a = c.$$

Both summability hypotheses are required: the underlying `tprod` is totalized outside the admissible range.

This is identifiability from the divisor, meaning zeros with their analytic multiplicities. The unweighted zero set does not record the exponent sizes, and the product values at dyadic zeros are likewise insufficient because they merely record vanishing. The result does not identify a probability law and supplies no spectral-zeta or cumulant-sample recovery theorem; those interfaces remain open. The zero-definition/six-theorem module `GeneralizedRvachevIdentifiability.lean` contains the order formula, the head and successor decoders, the dyadic-order uniqueness theorem, its general-base additive-cancellation core, and `generalizedRvachevProduct_eq_iff`. This six-theorem source surface is included in the current walkthrough.

10.7 Fourier injectivity and uniqueness

If two original candidates satisfy the same hypotheses, the iteration argument gives the same Fourier transform for both. The source then uses injectivity of the Fourier transform on Schwartz functions to conclude equality. The final transition is:

1. package each smooth compactly supported function as a Schwartz map;
2. prove their transforms are equal as functions;
3. apply Schwartz-space Fourier injectivity; and
4. project the equality back to ordinary functions.

The theorem also shows that the scaling constant in the original functional equation is forced, rather than being an arbitrary parameter. Integrating the derivative law on $[-1, 0]$ and using the mass normalization leaves only the canonical value, which in the repository's normalization is two.

10.8 Moment series from the Fourier product

Expanding the transform at zero recovers moments. The logarithm of the sinc product yields cumulant-like sums, while direct multiplication gives a power series whose coefficients are rational combinations of powers of two. `FourierAnalytic.lean` supplies the entire transform and inversion interface; the moment-series identification itself is proved in `AnalyticMoments.lean`.

The proof again prefers an identity of entire functions followed by coefficient comparison. Establishing an entire power series permits differentiation at zero to extract moments without repeatedly justifying differentiation under the integral sign.

A current-tree extension names the individual even modes: write $A_n(z)$ for the value of `rvachevFourierMomentTerm` at z, n. The declarations `rvachevFourierMomentTerm_zero`, `rvachevFourierMomentTerm_scale`, and `hasSum_rvachevFourierMomentTerm` prove

$$A_0(z) = 1, \qquad A_n(cz) = (c^2)^n A_n(z), \qquad \mathrm{HasSum}(A_\bullet(z), \widehat{\mathrm{up}}_{2\pi}(z)).$$

The separate module `RvachevQBinomialFilter.lean` feeds this genuine series into the geometric Lagrange filter. For arbitrary $c, z \in \mathbb{C}$, natural order p, and Gaussian base $q = c^2$, it assumes only injectivity of $j \mapsto q^j$ on $\{0, \dots, p\}$ and proves the exact Gaussian `tsum` tail for the actual infinite Rvachev product. At $c = 1/2$, hence $q = 1/4$, the finite-node hypothesis is discharged internally. This is not an identification of a frontier report's finite prefix $P_{b,N}$, its quotient by the infinite product, or its Bell coefficients, and it supplies none of that report's sign, bound, uniform/derivative-convergence, or asymptotic conclusions.

10.9 Poisson summation

The original paper's Poisson-summation theorem is formalized in a dedicated module. Compact support and smoothness place the function in the Schwartz class, so mathlib's Poisson summation framework applies. The main work is normalization: the theorem from the library and the paper use different Fourier conventions and summation coordinates. The proof rewrites the transform through the sinc product and then simplifies the sampled factors.

For positive spacing u, the normalized identity has the form

$$\sum_{k\in\mathbb{Z}} \mathrm{up}(t + uk) = \frac{1}{u} \sum_{k\in\mathbb{Z}} \widehat{\mathrm{up}}_{2\pi}(k/u) e^{2\pi \mathrm{i} k t/u}.$$

The formal audit records two source corrections visible in this equation: the printed equation (25) omitted the translation variable t from the exponential, and a later displayed $1/a$ must disappear after both sides are multiplied by a.

The same Schwartz realization yields more than the summability needed here. The public theorem `rvachevFourier_real_iteratedDeriv_rapidDecay` bounds every real-axis derivative after multiplication by an arbitrary power of the frequency, while `rvachevFourier_real_rapidDecay` is the transform-only specialization. Thus the rapid decay invoked by Poisson summation is itself part of the reusable Lean API.

Compact support sharpens the $t = 0$ specialization on the full ray $a \geq \frac{1}{2}$. Only the lattice sites $-1, 0, 1$ remain, so

$$\sum_{m\in\mathbb{Z}} \mathrm{up}(am) = 1 + 2\,\mathrm{up}(a), \qquad a + 2a\,\mathrm{up}(a) = \sum_{m\in\mathbb{Z}} \widehat{\mathrm{up}}_{2\pi}(m/a).$$

The declarations `rvachev_lattice_sum_of_one_half_le` and `rvachev_poisson_support_specialization_of_one_half_le` require no upper bound $a \leq 1$; older two-bound declarations remain compatibility wrappers.

This is representative of source-facing modules: the analytic theorem already exists in a reusable library form, while the paper theorem is a carefully normalized corollary.

Proof architecture. The Fourier uniqueness proof is a rigidity argument. The differential/refinement equation repeatedly pushes all unknown information into the value of the transform at a frequency tending to zero. Normalization fixes that limiting value, so no freedom remains. □

11 Finite convolutions, probability, and approximation

11.1 The probabilistic representation

`ProbabilityRepresentation.lean` uses the product probability space $\Omega = [0,1]^{\mathbb{N}_0}$ and the geometrically weighted coordinate sum

$$X(\omega) = \sum_{n\geq 0} \frac{\omega_n}{2^{n+1}}.$$

Splitting off the first coordinate gives the exact self-similarity

$$X(\omega) = \frac{\omega_0 + X(\mathrm{tail}\,\omega)}{2}.$$

If $H(y) = \mathbb{P}[X \leq y]$, conditioning on ω_0 produces

$$H(y) = \int_0^1 H(2y - u)\,\mathrm{d}u.$$

The module proves globally that $H = F$, and identifies the folded function by

$$\mathrm{up}(x) = H(1 - |x|).$$

The existing `ProbabilityLaplaceMoments.lean` module now adds exactly two report-facing consequences. Atomlessness identifies the closed and open tails, so for every $t \geq 0$,

$$\mathbb{P}[X \geq t] = \mathbb{P}[X > t] = \mathrm{up}(t),$$

which is `weightedSumDistribution_real_Ici_eq_rvachevUp_of_nonneg`. For every natural $n \geq 1$,

$$\int_{\mathbb{R}} x^n\,\mathrm{d}\mu_X(x) = n\int_0^1 t^{n-1}\,\mathrm{up}(t)\,\mathrm{d}t,$$

which is `integral_pow_weightedSumDistribution_eq_mul_intervalIntegral_rvachevUp`. The expectation is over the full weighted-sum law; its compact support is consumed by the existing survival layer-cake theorem. Together with the global up/Fabius identities, these make `prop:up-tail` and `cor:up-moments` Exact. They add no density statement and the moment theorem deliberately assumes positive degree. This is the direct CDF model on $[0,1]$. It should not be conflated with the centered finite-convolution measures used by the original-paper approximants.

The generic weighted-law compatibility layer also supplies an exact Bochner barycenter. For norm-summable weights in a complete Borel real normed space,

$$\int_E x \, \mathrm{d}\mu_w(x) = \frac{1}{2} \cdot \sum_{n=0}^{\infty} w_n.$$

This is `integral_id_weightedUniformDistribution` in `WeightedUniformDistribution.lean`. It assumes no sign or order on the weights and uses bounded-series integrability plus central reflection, not a termwise integration or coordinate-independence expansion. That module now has exactly eleven public theorems and no public definitions.

The self-similarity of X is the probabilistic counterpart of the fixed-point integral equation. It supplies a second route to the same canonical bounded function once uniqueness of the `IsFabius` specification is available.

11.2 The reusable affine-law, geometric CDF, and density layer

The post-snapshot modules `ContinuousCDF.lean`, `AffineIndependentCopy.lean`, `GeometricUniformLaw.lean`, `GeometricUniformUniqueness.lean`, and `GeometricUniformCDF.lean` separate the measure-theoretic argument from its dyadic specialization. The generic CDF layer proves three facts for real probability laws. A law with null singletons has a continuous CDF; invariance under $y \mapsto a - y$ gives

$$H(a - x) = 1 - H(x);$$

and an everywhere pointwise derivative $H'(x) = h(x)$ identifies the law exactly as Lebesgue measure with density $\mathrm{ofReal} \circ h$. The last theorem assumes neither continuity of h, null singletons, nor prior absolute continuity. Monotonicity of a CDF supplies the derivative's nonnegativity and local integrability inside the proof. The declarations are `continuous_cdf_of_nullSingleton`, `cdf_reflection_sub`, and `measure_eq_withDensity_of_cdf_hasDerivAt`.

The generic independent-copy layer starts with an arbitrary measurable digit space D and a second-countable Borel real inner-product space E. For a digit law ρ, a map $d : D \to E$, a real scale q, and a candidate law μ on E, it packages

$$\mathcal{T}_{\rho,d,q}(\mu) = (d_*\rho) * ((q\,\cdot)_*\mu)$$

as `affineIndependentCopyLaw`. A measurable digit makes this a probability law when ρ and μ are probability laws, and for s-finite input measures the convolution is exactly the direct product pushforward

$$\mathcal{T}_{\rho,d,q}(\mu) = (\rho \otimes \mu)_*\big((u,x) \mapsto d(u) + qx\big).$$

These are `affineIndependentCopyLaw_isProbabilityMeasure` and `affineIndependentCopyLaw_eq_map_prod`. Characteristic functions then give the factorization, fixed-point recurrence, and its finite iterate:

$$\varphi_{\mathcal{T}_{\rho,d,q}(\mu)}(t) = \varphi_{d_*\rho}(t)\varphi_\mu(qt), \qquad \varphi_\mu(t) = \prod_{k=0}^{n-1} \varphi_{d_*\rho}(q^k t)\, \varphi_\mu(q^n t).$$

The corresponding declarations are `charFun_affineIndependentCopyLaw`, `charFun_eq_mul_charFun_of_affineIndependentCopy_fixedPoint`, and `charFun_iterate_of_affineIndependentCopy_fixedPoint`.

At compiled checkpoint `d312c0603`, the module's public crosswalk consists of the definition above and eight theorems. The final three declarations `eq_of_charFun_affine_recurrence`, `affineIndependentCopyLaw_fixedPoint_unique`, and `affineIndependentCopy_map_fixedPoint_unique` prove uniqueness when $|q| < 1$. Completeness of E is used only by these three uniqueness theorems; the earlier definition, probability, product-map, and characteristic-function formulas require no completeness. For the two fixed-point wrappers the digit is measurable and the digit and candidate laws are probability measures. The argument includes $q = 0$ and negative q, and assumes no support, density, or moment information.

For the geometric weights $w_n(q) = (1-q)q^n$, write μ_q for the law, F_q for `geometricUniformCDFq`, and, when $0 < q < 1$, set

$$f_q(x) = \frac{F_q(x/q) - F_q((x-(1-q))/q)}{1-q}.$$

The exhaustive `GeometricUniformLaw.lean` surface now has twenty-four public declarations. The new specialization `integral_id_geometricUniformDistribution_eq_one_half` proves

$$\int_{\mathbb{R}} x \, \mathrm{d}\mu_q(x) = \frac{1}{2} \qquad \text{whenever } |q| < 1.$$

It includes $q = 0$ and every negative contraction and uses no positivity hypothesis. The Lean definition `geometricUniformDensityq` is total in q, but only $0 < q < 1$ gives this classical density interpretation. The formal boundary is now exact: at $q = 0$ the selected quotient is identically zero, whereas for $q < 0$ it is nonpositive because the affine CDF interval reverses. Thus its `ENNReal.ofReal` clamp gives the zero `withDensity` measure for every $q \le 0$, and that measure is provably not μ_q when also $|q| < 1$. This does not contradict the absolute continuity of the law; an exact Lean theorem for the paper-level corrected signed-ratio density at negative q remains open. The single theorem in `GeometricUniformUniqueness.lean` specializes the generic product-map theorem to the already proved head–tail identity:

$$\mu = (\mathrm{Unif}[0,1] \otimes \mu)_*\big((u,x) \mapsto (1-q)u + qx\big) \quad \Longrightarrow \quad \mu = \mu_q.$$

This is `eq_geometricUniformDistribution_of_selfSimilar`; it holds for every $|q| < 1$, including $q = 0$ and negative q, among all probability measures, with no support, density, or moment hypothesis.

At the fixed dyadic ratios, `GeometricUniformMultisection.lean` splits the coordinates into `evenCoordinates` and `oddCoordinates` and proves the pointwise identity

$$Y_{1/2}(\omega) = \frac{2}{3} Y_{1/4}(\mathrm{evenCoordinates}\,\omega) + \frac{1}{3} Y_{1/4}(\mathrm{oddCoordinates}\,\omega).$$

The theorems `geometricUniformSeries_one_half_multisection`, `geometricUniformDistribution_one_half_multisection`, and `geometricUniformDistribution_one_half_conv_one_quarter` transport this through the independent product map to the corresponding affine image and convolution laws. This five-declaration leaf is the fixed normalized two-section result only; it does not assert a general-ratio multisection theorem.

The finite-tail dictionary is already stronger than a characteristic-function-only statement: `geometric_tail_dictionary_geometricUniform` packages finite measure, characteristic-function, MGF, and CGF factorizations. The later `GeometricSincFactorization.lean` evaluates the

scaled digit explicitly. If $\phi_q(t) = \mathrm{charFun}(\mu_q)(t)$, then for every real q with $|q| < 1$, every $m \in \mathbb{N}_0$, and every $t \in \mathbb{R}$,

$$\phi_q(t) = e^{\mathrm{i}(1-q^m)t/2} \left(\prod_{k=0}^{m-1} \mathrm{sinc}_{\mathrm{rad}}\left(\frac{(1-q)q^k t}{2} \right) \right) \phi_q(q^m t),$$

and the phase-bearing prefix obtained by deleting the residual factor $\phi_q(q^m t)$ converges pointwise to $\phi_q(t)$. The digit identity itself, $\varphi_{\eta_q}(t) = e^{\mathrm{i}(1-q)t/2}\,\mathrm{sinc}_{\mathrm{rad}}((1-q)t/2)$, is unconditional in real q. Thus the contraction-range result includes $q = 0$ and negative ratios; at $q = 1/2$ it has exact wrappers for the classical weighted-sum law.

The six-declaration inventory of `GeometricSincFactorization.lean` is `charFun_geometricUniformDigit`, `charFun_geometricUniformDistribution_prefix`, `charFun_geometricUniformDistribution_prefix_sinc`, `tendsto_prefix_sinc_charFun`, `charFun_weightedSumDistribution_prefix_sinc`, and `tendsto_prefix_sinc_charFun_weightedSumDistribution`. This is pointwise prefix convergence. The four new public theorems attached to `geometricSincProduct` in `GeometricReciprocalGamma.lean` are `hasProdLocallyUniformly_geometricSincProduct`, `geometricSincProductFactors_multipliable`, `hasProd_geometricSincProduct`, and `geometricSincProduct_differentiable`. For complex q with $\|q\| < 1$, they prove locally uniform convergence on $\mathbb{C}$ of the defining sinc products, pointwise multipliability and the exact `HasProd` identity, and entireness. Write $S_q(z)$ for `geometricSincProductqz`.

The new zero-definition, four-theorem module `GeometricSincCharacteristicFunction.lean` consists exactly of `charFun_geometricUniformDistribution_eq_phase_mul_geometricSincProduct` and `charFun_geometricUniformDistribution_eq_phase_mul_geometricReciprocalGamma`, followed by `tendstoLocallyUniformly_prefix_sinc_charFun` and `tendstoUniformlyOn_prefix_sinc_charFun`. Write $G_q(z)$ for `geometricReciprocalGamaqz`, and put

$$z_q(t) = \frac{(1-q)t}{2\pi}.$$

For every real $|q| < 1$ and $t \in \mathbb{R}$, including $q = 0$ and negative ratios, the first two theorems give the exact identities

$$\phi_q(t) = e^{\mathrm{i}t/2} S_q\big(z_q(t)\big) = e^{\mathrm{i}t/2} G_q\big(z_q(t)\big) G_q\big(-z_q(t)\big).$$

For

$$P_{q,m}(t) := e^{\mathrm{i}(1-q^m)t/2} \prod_{k=0}^{m-1} \mathrm{sinc}_{\mathrm{rad}}\left(\frac{(1-q)q^k t}{2} \right),$$

the last two prove that $P_{q,m}$ tends locally uniformly on the real frequency line to ϕ_q, and uniformly on every compact set $K \subseteq \mathbb{R}$. The Gamma-side locally uniform theorem concerns S_q as a complex-variable pure sinc product, whereas the full phase-bearing theorem is stated for real frequencies. No complex-frequency analogue of the latter or uniform-convergence claim on all of $\mathbb{R}$ is made. The one-definition, ten-theorem module `RvachevPochhammerFactorization.lean` gives a second, spectral description of this pure sinc product. It defines `complexQPochhammerInf` by

$$(a;q)_{\infty,\mathbb{C}} = \prod_{j=0}^{\infty} (1 - aq^j).$$

The five foundational declarations `complexQPochhammerInf_eq_tprod`, `complexQPochhammerInf_eq_qPochhammerInfIn`, `multipliable_one_sub_mul_pow_complex`, `hasProd_complexQPochhammerInf`, and `tendsto_finiteQPochhammerIn_complex` state its defining product, identify it with the generic public name, and prove, for exactly $\|q\| < 1$, multipliability, the named product, and convergence of the finite prefixes, with no restriction on a. The bridge theorem

is stronger in a different direction: for every complex a, q, with no hypothesis, it is a definitional equality between the two identical total `tprod`s. It therefore remains true in Mathlib's nonmultipliable junk-value regime, without asserting convergence there.

For every strict complex contraction q and every $z \in \mathbb{C}$, the remaining geometric layer starts from

$$\sum_{h,k\geq 0} \left\| \frac{q^{2h} z^2}{(k+1)^2} \right\| = \|z\|^2 \left(\sum_{h\geq 0} \|q\|^{2h} \right) \left(\sum_{k\geq 0} \frac{1}{(k+1)^2} \right) < \infty.$$

This is `summable_norm_sineTerm_qpow_pair`. Absolute multipliability lets `geometricSincProduct_eq_tprod_pair` flatten the scale-indexed Euler products and lets `geometricSincProduct_eq_tprod_complexQPochhammerInf` exchange the two indices, giving globally

$$S_q(z) = \prod_{k=0}^{\infty} \left(\frac{z^2}{(k+1)^2}; q^2 \right)_{\infty,\mathbb{C}}.$$

No division occurs, so the formula includes $q = 0$ and individual zero factors. The final declarations `rvachevFourierProduct_eq_tprod_complexQPochhammerInf` and `rvachevFourier_eq_tprod_complexQPochhammerInf` specialize to $q = 1/2$, nome $1/4$, first for the standalone Rvachev product and then for every bounded `IsFabius` witness. This factorization module itself does not classify the single-symbol zero lattice or multiplicities; the next leaf closes that fixed-nome problem. Neither module constructs a reciprocal zero–pole factorization or joint holomorphy in q, supplies a centered characteristic-function/MGF package, or treats $\|q\| > 1$. The separate normal-convergence leaf described below closes the former local-uniform outer-product boundary.

11.2.1 Entire fixed-nome q-Pochhammer symbol and simple zeros

The zero-definition, five-theorem module `QPochhammerEntire.lean` has the exhaustive public surface `hasProdLocallyUniformly_complexQPochhammerInf`, `complexQPochhammerInf_differentiable`, `complexQPochhammerInf_eq_zero_iff`, `complexQPochhammerInf_eq_zero_iff_eq_inv_pow`, and `analyticOrderAt_complexQPochhammerInf_of_eq_zero`. Fix a complex q under exactly $\|q\| < 1$, and put

$$P_q(a) = \texttt{complexQPochhammerInf a q} = (a;q)_{\infty,\mathbb{C}} = \prod_{j=0}^{\infty} (1 - aq^j).$$

The first theorem says that the finite prefixes converge locally uniformly as functions of a on all of $\mathbb{C}$; the second says that P_q is entire. The next two give the exact zero descriptions

$$P_q(a) = 0 \iff \exists j \in \mathbb{N}_0 :\ 1 - aq^j = 0, \tag{4}$$

$$\iff \exists j \in \mathbb{N}_0 :\ a = (q^j)^{-1} \qquad (q \neq 0), \tag{5}$$

and the final theorem proves analytic order one at every zero under the historical complex-product name. The generic-name order theorem is owned by `QPochhammerInfinite.lean`. The raw factor-zero line is primary because it also covers $q = 0$: in that case only the index-zero factor can vanish, at $a = 1$, and the zero is simple.

Here is the proof architecture. Take $f(w) = 1 - w$ and scales q^j in the generic summable-scale product engine. Since $\sum_j \|q^j\| < \infty$, $f(w) - 1 = -w = O(w)$, and f is entire, the engine gives the locally uniform product and holomorphy. Its no-hidden-zero theorem gives (4). Under $q \neq 0$, rewriting $1 - aq^j = 0$ as $aq^j = 1$ proves (5); conversely $q^j \neq 0$, so its inverse cancels.

For simplicity, first prove that a vanishing index is unique. At $q = 0$, a factor equation forces the index to be zero. At $q \neq 0$, two vanishing indices j, k give $a \neq 0$ and $q^j = q^k$; norms give

$\|q\|^j = \|q\|^k$, and the natural powers of a number in $(0,1)$ are injective. If j is the unique vanishing index, splitting the product after it yields

$$P_q(z) = (z-a)U(z), \qquad U(z) = -q^j(z;q)_j P_q(zq^{j+1}).$$

The cofactor U is entire. At a, its constant is nonzero (when $q=0$, uniqueness gives $j=0$); its finite prefix is nonzero by uniqueness; and its tail is nonzero because a tail zero would shift to a second vanishing factor with index $j+1+k$. Hence $U(a) \neq 0$, which is exactly analytic order one.

This is a fixed-q, single-symbol result. It proves no joint holomorphy in (a,q), outside-disk reciprocal formula, or centered characteristic-function/MGF wrapper. The separate three-theorem normal-convergence leaf proves the locally uniform outer spectral product $\prod_k (z^2/(k+1)^2; q^2)_\infty$ and its two dyadic specializations, but does not close those other compound spectral clauses and does not replace the public unconditional bridge `complexQPochhammerInf_eq_qPochhammerInfIn`.. This subsection records both current source surfaces and their exact boundaries.

Two further modules supply the finite and general-ring layers without replacing that `complexQPochhammerInf` interface. `QPochhammerDissection.lean` has zero definitions and exactly two theorems, `finiteQPochhammerIn_dissection` and `finiteQPochhammerIn_dissection_remainder`. Over an arbitrary commutative ring they prove the exact residue-class split at length rn and, under exactly $u \leq r$, its partial-period form with lengths $n+1$ for $s < u$ and n for $u \leq s < r$. No restriction on q, topology, division, cancellation, or nonvanishing is present.

The one-definition, twenty-nine-theorem module `QPochhammerInfinite.lean` has the exhaustive public surface `qPochhammerInfIn`; `qPochhammerInfIn_eq_tprod`, `summable_norm_mul_pow`, `one_sub_ne_zero_of_norm_lt_one`, `norm_mul_pow_self_lt_one`, `finiteQPochhammerIn_self_ne_zero`, `multipliable_one_sub_mul_pow_of_norm_lt_one`, `hasProd_qPochhammerInfIn`, `tendsto_finiteQPochhammerIn_qPochhammerInfIn`, `qPochhammerInfIn_eq_finite_mul_shift`, `qPochhammerInfIn_succ_shift`, `qPochhammerInfIn_eq_factor_mul`, `qPochhammerInfIn_dissection`, `qPochhammerInfIn_ne_zero`, `qPochhammerInfIn_eq_zero_iff`, and `qPochhammerInfIn_self_ne_zero`.

The unconditional normed-field support declarations are `qPochhammerInfIn_eq_tprod_smul` and `isBigO_one_sub_sub_one`; `summable_norm_pow_of_norm_lt_one` additionally assumes $\|q\| < 1$, while `differentiable_finiteQPochhammerIn` needs only a nontrivially normed field. A complete normed field, $\|q\| < 1$, and $q \neq 0$ give the reciprocal-power zero list in `qPochhammerInfIn_eq_zero_iff_exists_inv_pow`; the same strict contraction together with local compactness is required by `hasProdLocallyUniformly_qPochhammerInfIn` and `continuous_qPochhammerInfIn`. The purely algebraic identity `pow_sq_mul_finiteQPochhammerIn_inv_pow_self` holds in every field with $q \neq 0$.

11.2.2 Latest q-series limit, triple-product, and polynomial tranche

The downstream `GaussianBinomialPalindromic.lean` has no public definitions and exactly fourteen public theorems: `reflect_add_of_natDegree_le`, `reflect_one'`, `gaussianBinomial_natDegree_le`, `gaussianBinomial_zero_left`, `gaussianBinomial_diag'`, `reflect_gaussianBinomial`, `coeff_gaussianBinomial_reflect`, `coeff_gaussianBinomial_zero`, `coeff_gaussianBinomial_top`, `gaussianBinomial_natDegree`, `gaussianBinomial_monic`, `two_mul_derivative_gaussianBinomial_eval_one`, `coeff_gaussianBinomial_one_of_pos_of_lt`, and `coeff_gaussianBinomial_one`. Over every commutative semiring these give the general reflection helpers, the Gaussian degree bound, zero and diagonal values, reflection in degree $k(n-k)$, constant and top coefficients one, bounded-index coefficient reversal, and the division-free mean identity. The interior theorem gives linear coefficient one under exactly $0 < k < n$; the total theorem gives one in that case and zero otherwise, explicitly covering $k=0$, $k=n$, and $n<k$. Both hold over every commutative semiring. Exact degree and monicity alone require nontriviality.

The companion `GaussianBinomialPolynomialStructure.lean` has no public definitions and exactly five public theorems: `natDegree_gaussianBinomial_universal`, `gaussianBinomial_universal_monic`, `coeff_zero_gaussianBinomial_universal`, `gaussianBinomial_universal_reflect`, and `coeff_gaussianBinomial_universal_symm`. For $k \le n$, the universal polynomial `gaussianBinomial` $(X : \mathbb{N}_0[X])\, n\, k$ has exact natural degree $k(n-k)$, is monic, has constant coefficient one, and is fixed by reflection in that degree. If also $d \le k(n-k)$, the last theorem identifies the coefficients in degrees d and $k(n-k)-d$. With the existing Gaussian symmetry and reciprocity results this closes the canonical forward q-binomial structure theorem. The adjacent general-semiring theorem `coeff_gaussianBinomial_one` supplies the complete linear-coefficient classifier.

The `CentralQBinomialReduction.lean` surface has no definitions and six theorems: `finiteQPochhammerIn_mul_neg`, `finiteQPochhammerIn_two_mul`, `finiteQPochhammerIn_map_ringHom`, `central_gaussianBinomial_sq_mul_int`, `central_gaussianBinomial_sq_mul`, and `central_gaussianBinomial_sq_div`. The first five give sign pairing, even–odd dissection, naturality, and the integral and general division-free central squared-base reductions over commutative rings; the quotient wrapper is field-level and assumes its two denominators are nonzero. `CyclotomicFactorization.lean` has no definitions and seven theorems: `div_add_div_le_div`, `div_le_div_add_div_add_one`, `mem_range_and_mem_divisors_iff`, `finiteQPochhammerIn_X_eq_prod_cyclotomic`, `finiteQPochhammerIn_X_eq_gaussianBinomial_mul`, `prod_cyclotomic_pow_div_extend`, and `gaussianBinomial_X_eq_prod_cyclotomic`. They give exponent bounds, the divisor reindexing, commutative-ring finite-product identities, and the integral-domain Gaussian cyclotomic factorization.

Four further modules form the exhaustive root-of-unity and q-Catalan layer. `PrimitiveRootBlock.lean` is 0+3: `gaussianBinomial_isPrimitiveRoot_eq_zero`, `neg_one_pow_mul_pow_choose_two`, and `finiteQPochhammerIn_isPrimitiveRoot`. In a commutative integral domain, a primitive d-th root ζ kills the interior coefficients $\begin{bmatrix} d \\ k \end{bmatrix}_\zeta$; for $d > 0$, its top phase is -1 and the complete block is $(y;\zeta)_d = 1 - y^d$.

`QLucas.lean` is 0+7: `add_mul_add_sub_one`, `choose_two_add`, `coeff_finiteQPochhammerIn_neg_X`, `finiteQPochhammerIn_neg_X_block`, `coeff_block_pow_mul`, `pow_choose_two_add_mul_eq`, and `gaussianBinomial_q_lucas`. Its two natural quadratic, polynomial coefficient, block, and phase identities prove

$$\begin{bmatrix} ad+b \\ rd+s \end{bmatrix}_\zeta = \binom{a}{r} \begin{bmatrix} b \\ s \end{bmatrix}_\zeta$$

when $d > 0$, ζ is a primitive d-th root in a commutative integral domain, and $b, s < d$. Its local `two_mul_choose_two` helper is private; the unique public theorem of that name belongs to `QChuVandermonde.lean`.

`CyclotomicDivisibility.lean` is 0+3: `cyclotomic_exponent_eq_one_iff`, `cyclotomic_dvd_gaussianBinomial_iff`, and `gaussianBinomial_mul_isPrimitiveRoot`. For $k \le n$ and $d > 0$, the exponent is one exactly when $n \bmod d < k \bmod d$; over $\mathbb{Q}[X]$ this is the exact divisibility criterion for Φ_d. A primitive n-th root in a commutative integral domain also gives $\begin{bmatrix} an \\ bn \end{bmatrix}_\zeta = \binom{a}{b}$ when $n > 0$.

`QCatalan.lean` is 1+11. Its definition is `qCatalan`; its theorems are `map_qInt`, `qInt_X_monic`, `qInt_X_natDegree`, `X_sub_one_mul_qInt`, `qInt_X_eq_prod_cyclotomic`, `qInt_X_dvd_gaussianBinomial_rat`, `qInt_X_dvd_gaussianBinomial_int`, `qInt_X_mul_qCatalan`, `qCatalan_natDegree`, `qCatalan_eval_one_mul`, and `qCatalan_eval_one`. The q-integer identities give $[n+1]_X \mid \begin{bmatrix} 2n \\ n \end{bmatrix}_X$ over both $\mathbb{Q}[X]$ and $\mathbb{Z}[X]$; the integral quotient has degree $n(n-1)$ and evaluates at one to the ordinary Catalan number.

`NewtonInterpolation.lean` is 3+19. Its definitions are `newtonCoeff`, `nodeNewtonPoly`, and `newtonInterpolant`; its theorems are `newtonCoeff_eq`, `newtonCoeff_zero`, `newt`

onCoeff_mul_prod, nodeNewtonPoly_succ, eval_nodeNewtonPoly, degree_nodeNewtonPoly_lt, nodeNewtonPoly_eq_interpolate, eq_nodeNewtonPoly_of_eval_eq, coeff_nodeNewtonPoly_self, newtonCoeff_eq_sum, nodal_range_pow, prod_erase_pow_sub_pow, newtonCoeff_pow_eq_sum, newtonPoly_succ, eval_newtonPoly, degree_newtonPoly_lt, newtonPoly_eq_interpolate, eq_newtonPoly_of_eval_eq, and coeff_newtonPoly_self. Over a field this is the complete triangular-reconstruction, finite-node interpolation and uniqueness, divided-difference, and geometric-power-grid surface; its injectivity, nonzero-product, $q \neq 0$, and index hypotheses remain explicit. The node-qualified family avoids the existing scalar newtonPoly; the compatibility family is definitionally identical.

QBetaIntegral.lean is 1+8. Its definition is qBeta; its theorems are qNumber_pos, qBeta_term_eq, qBeta_eq_prod, qBeta_eq_qGamma, qBeta_comm, qBeta_pos, qBeta_add_one_left, and qBeta_add_one_right. For $0 < q < 1$, they evaluate the Jackson q-beta integral by infinite products and q-Gamma and give symmetry, positivity, and both recurrences under the displayed positive-argument hypotheses.

The zero-definition/fourteen-theorem GaussianBinomialPalindromic.lean surface is reflect_add_of_natDegree_le, reflect_one', gaussianBinomial_natDegree_le, gaussianBinomial_zero_left, gaussianBinomial_diag', reflect_gaussianBinomial, coeff_gaussianBinomial_reflect, coeff_gaussianBinomial_zero, coeff_gaussianBinomial_top, gaussianBinomial_natDegree, gaussianBinomial_monic, two_mul_derivative_gaussianBinomial_eval_one, coeff_gaussianBinomial_one_of_pos_of_lt, and coeff_gaussianBinomial_one. All are over an arbitrary commutative semiring. The Gaussian statements use $k \leq n$; coefficient reversal additionally bounds $j \leq k(n-k)$, while exact degree and monicity alone require a nontrivial coefficient semiring. The last theorem is the division-free identity $2G'_{n,k}(1) = k(n-k)\binom{n}{k}$. The final two declarations identify the linear coefficient as one exactly for $0 < k < n$, and as zero otherwise, including the zero, diagonal, and above-row boundaries. With row symmetry and reciprocity these results close the canonical forward structure theorem; no characteristic restriction, positivity, or unimodality claim is hidden in them.

The three definitions qDeriv, qExp, and qExpBig, together with the eight theorems qFactorial_mul_one_sub_pow, qFactorial_ne_zero, qDeriv_mul, hasSum_qExp, hasSum_qExpBig, qExp_mul_qExpBig_neg, qDeriv_qExp, and qDeriv_qExpBig, are the exhaustive public surface of QExponential.lean. The factorial identity is purely algebraic over a commutative ring, and the product rule is total over a normed field. The analytic statements use a complete normed field and $\|q\| < 1$: the small exponential series, reciprocal product, and its eigenfunction law require $\|(1-q)x\| < 1$ (with the displayed scaled argument in the eigenfunction theorem), whereas the big exponential series is global in x. Both q-derivative eigenfunction laws require $x \neq 0$; the big one needs no small-argument hypothesis.

The one-definition/seven-theorem JacksonIntegral.lean inventory is jacksonIntegral, qDeriv_jacksonIntegral, one_sub_mul_pow_mul_qDeriv, tendsto_jackson_sum_qDeriv, jacksonIntegral_qDeriv, tendsto_jackson_sum_parts, jackson_parts_of_tendsto, and jacksonIntegral_mul_qDeriv. It works over a complete normed field. The first fundamental-theorem direction assumes exactly $q \neq 1$, $x \neq 0$, and summability of the defining series. The telescoping identity itself is unconditional; the converse and integration-by-parts statements separate explicit convergence of partial sums from the stronger summable forms, and retain the boundary limit $G(aq^n) \to l$ or $f(aq^n)g(aq^n) \to l$ rather than silently replacing it by a zero limit.

The definition bilateralTheta and six theorems thetaExponent_add_one, pow_thetaExponent_add_one, hasSum_bilateralTheta, bilateralTheta_eq_prod, bilateralTheta_mul_left, and bilateralTheta_eq_zero_iff form all of ThetaQuasiPeriodicity.lean. The integer exponent recurrence is unconditional and its power form requires only $q \neq 0$. Over a complete normed field, the sum and product formulas assume $\|q\| < 1$ and $z \neq 0$; quasi-periodicity and the exact zero lattice $z = -q^m$, $m \in \mathbb{Z}$, additionally assume $q \neq 0$. The companion JacobiCubic.lean has no definitions and exactly the two theorems two_mul_add_one_le_three_pow

and hasSum_jacobi_cubic. The former is an unconditional natural-number bound; the latter is the complex HasSum cubic identity under only $\|q\| < 1$, including $q = 0$.

The zero-definition/ten-theorem QPochhammerLogDerivative.lean surface is one_sub_le_norm_one_sub_mul_pow, summable_pow_div_one_sub_mul_pow, summable_log_one_sub_mul_pow, one_sub_mul_pow_ne_zero, qPochhammerInfIn_eq_cexp_tsum_log, hasDerivAt_tsum_log_one_sub_mul_pow, hasDerivAt_qPochhammerInfIn, hasDerivAt_lambert_series, tsum_neg_pow_div_one_sub_mul_pow_eq, and hasDerivAt_qPochhammerInfIn_lambert. These are complex statements. The norm lower bound permits $\|q\| \le 1$ and $\|y\| < r$; logarithm summability holds for arbitrary a once $\|q\| < 1$. Nonvanishing, the exponential-product formula, the reciprocal and Lambert sums, and the final derivative formulas restrict to $\|a\| < 1$. The two termwise-differentiation theorems expose their working disc explicitly as $\|a\| < r < 1$.

Finally, QPochhammerOrderDerivative.lean has no definitions and exactly hasDerivAt_const_cpow', hasDerivAt_qPochhammerInfIn_mul_cpow, and hasDerivAt_qPochhammerC. The constant-base complex-power derivative requires $q \neq 0$. The composite and order derivatives add $\|q\| < 1$ and $\|aq^{\alpha}\| < 1$; they assert local complex derivatives in α, not a boundary continuation at $q = 0$ or $\|aq^{\alpha}\| = 1$.

Six further facade-reachable modules contribute exactly seventy public declarations. The small-module inventories are exhaustive: GaussianBinomialContinuity.lean has the three theorems continuous_gaussianBinomial, tendsto_gaussianBinomial_nhds_one, and gaussianBinomial_eq_finiteQPochhammerIn_div; QPascalSummation.lean has sum_gaussianBinomial_succ_mul, sum_gaussianBinomial_succ_mul', Commute.gaussianBinomial_left, and Commute.gaussianBinomial_right; and QuantumBinomial.lean has the two noncommutative theorems quantumPlane_mul_pow and quantum_binomial.

The one-definition/twenty-nine-theorem QBinomialTheoremInfinite.lean surface is gaussianMajorant, hasSum_of_tendsto_of_dominated, one_le_qPochhammerInfIn_neg, finiteQPochhammerIn_neg_le_qPochhammerInfIn, qPochhammerInfIn_nonneg, qPochhammerInfIn_le_finiteQPochhammerIn, qPochhammerInfIn_pos_of_lt_one, qPochhammerInfIn_zero_left, norm_finiteQPochhammerIn_le_neg_norm, finiteQPochhammerIn_norm_le_norm, norm_finiteQPochhammerIn_le, qPochhammerInfIn_norm_le_norm_finiteQPochhammerIn, finiteQPochhammerIn_ne_zero_of_norm_lt_one, gaussianBinomial_eq_div, gaussianMajorant_nonneg, norm_gaussianBinomial_le, tendsto_finiteQPochhammerIn_pow_atTop, norm_finiteQPochhammerIn_pow_sub_one_le_exp_of_norm_le_one, norm_finiteQPochhammerIn_pow_sub_one_le_exp, isBigO_finiteQPochhammerIn_pow_sub_one, tendsto_gaussianBinomial_atTop, tendsto_gaussianBinomial_add_const_atTop, tendsto_gaussianBinomial_add_atTop, isBigO_gaussianBinomial_sub_inv, isBigO_gaussianBinomial_add_sub_inv, summable_pow_choose_two_mul_pow, hasSum_euler_product, qPochhammerInfIn_ne_zero_of_norm_lt_one, hasSum_qBinomial_theorem, and hasSum_euler_reciprocal.

The five effective fixed-column declarations in QBinomialTheoremInfinite.lean are exact. They include the generalized product estimate

$$\|(q^m;q)_k - 1\| \le \exp(k\|q\|^m) - 1,$$

its geometric IsBigO consequence, the two additive Gaussian error rates, and the arbitrary-fixed-shift limit

$$\begin{bmatrix} n+r \\ k \end{bmatrix}_q \longrightarrow (q;q)_k^{-1}.$$

The downstream zero-definition/nine-theorem GaussianBinomialFixedColumnRate.lean surface is norm_finiteQPochhammerIn_pow_sub_one_le_exp', norm_finiteQPochhammerIn_pow_sub_one_le, norm_finiteQPochhammerIn_self_mul_gaussianBino

mial_sub_one_le, norm_gaussianBinomial_sub_inv_finiteQPochhammerIn_le, norm_gaussianBinomial_add_sub_inv_finiteQPochhammerIn_le, gaussianBinomial_fixedColumn_relativeError_isBigO, gaussianBinomial_shifted_fixedColumn_relativeError_isBigO, gaussianBinomial_fixedColumn_error_isBigO, and gaussianBinomial_shifted_fixedColumn_error_isBigO. In a normed commutative ring with normalized multiplicative norm, the upstream bound and the second downstream theorem bound $\|(q^m;q)_k - 1\|$ by $\exp(k\|q\|^m) - 1$ and by $ke^k\|q\|^m$; the third downstream theorem turns the latter into the denominator-free relative Gaussian bound for $k \le n$, so roots of unity are still covered. Over any normed field, $\|q\| < 1$ gives both nonasymptotic fixed/shifted additive errors, arbitrary-shift convergence, and the four relative/additive IsBigO bounds at q^{n-k+1} and q^{n+1}. No completeness or $q \neq 0$ assumption is present, and the constant is elementary rather than sharp. With the existing tendsto_gaussianBinomial_atTop, this proves all four clauses of thm:fixed-column-limit; the relative pair is exactly the two multiplicative $1 + O(\cdot)$ claims.

The zero-definition/two-theorem GaussianBinomialGreaterOneAsymptotics.lean leaf consists of gaussianBinomial_gt_one_fixedColumn_relativeError_isBigO and gaussianBinomial_gt_one_central_isEquivalent. For real $q > 1$ they prove the exact printed normalizations $(q^{-1};q^{-1})_k q^{-k(n-k)} \begin{bmatrix} n \\ k \end{bmatrix}_q - 1 = O((q^{-1})^{n-k+1})$ and $\begin{bmatrix} 2m \\ m \end{bmatrix}_q \sim q^{m^2}(q^{-1};q^{-1})_\infty^{-1}$. Natural subtraction is total, while the existing gaussianBinomial_inv is used only eventually for $k \le n$. Thus cor:qgreaterone is Exact; no shifted-central, non-real, or wider-nome result is claimed.

The two-definition/twenty-five-theorem JacobiTripleProduct.lean inventory is thetaExponent, pentagonalExponent, two_dvd_mul_sub_one, mul_sub_one_nonneg, two_mul_thetaExponent, thetaExponent_natCast, thetaExponent_neg_natCast, choose_two_add_choose_two_succ_eq, sum_mul_sum_eq_sum_antidiagonal, thetaCoeff_pair_eq, sum_antidiagonal_thetaCoeff, finite_triple_product_poly, pow_mul_finiteQPochhammerIn_div, neg_one_zpow_natCast_sub, finite_triple_product, norm_gaussianBinomialInt_le, tendsto_gaussianBinomialInt_central, summable_pow_thetaExponent_mul_zpow, hasSum_jacobi_triple_product, hasSum_jacobi_triple_product', mul_three_mul_sub_one_nonneg, two_dvd_mul_three_mul_sub_one, two_mul_pentagonalExponent, pentagonalExponent_eq, pentagonalExponent_pos, hasSum_pentagonal, and hasSum_pentagonal_pairs. Finally, RogersSzegoPolynomial.lean has the definition rogersSzego and exactly nine theorems: rogersSzego_zero, rogersSzego_one, rogersSzego_succ, one_sub_pow_succ_mul_gaussianBinomial_succ_succ, rogersSzego_dilation, rogersSzego_three_term, summable_norm_pow_div_finiteQPochhammerIn_self, sum_antidiagonal_euler_mul_euler, and hasSum_rogersSzego_generating. The six-module total is four definitions and seventy theorems, seventy-four declarations. This source inventory does not assert a fresh aggregate Lean build.

The definition of F_q, together with its monotonicity, range bounds, and measurability, is total in real q. If $|q| < 1$, then F_q is continuous and

$$F_q(1-x) = 1 - F_q(x), \qquad F_q(1/2) = 1/2.$$

For $0 \le q < 1$, it is exactly zero on $x \le 0$ and exactly one on $1 \le x$. Strict positivity of q is essential for solving the affine event inequality in the conditioning step. Under $0 < q < 1$, Lean proves on all of $\mathbb{R}$

$$F_q(x) = \int_0^1 F_q\left(\frac{x-(1-q)u}{q}\right) \mathrm{d}u, \tag{6}$$

$$= \frac{q}{1-q} \int_{(x-(1-q))/q}^{x/q} F_q(t)\, \mathrm{d}t, \tag{7}$$

$$F_q'(x) = f_q(x). \tag{8}$$

The final identity is pointwise `HasDerivAt`, not merely an almost-everywhere derivative statement. Consequently

$$\mu_q = \mathrm{Leb}\,\mathrm{withDensity}(\mathrm{ofReal} \circ f_q).$$

Both F_q and f_q are C^∞, proved by bootstrapping the affine derivative formula rather than by Fourier inversion. The density is continuous, nonnegative, and symmetric about $1/2$; it vanishes for $x \le 0$ and $1 \le x$. Its ordinary support is contained in $(0,1)$, its topological support is contained in $[0,1]$, and it has compact support. These are containment theorems for the density itself, not an assertion here that its nonzero set has closure exactly $[0,1]$.

At $q = 1/2$, `ProbabilityRepresentation.lean` supplies the exact all-real bridges

$$\mathrm{weightedSumCDF} = F_{1/2}, \qquad F_{1/2}(x) = F(x), \qquad f_{1/2}(x) = 2\,\mathrm{up}(2x-1).$$

Accordingly the dyadic smoothing, interval-integral, continuity, exterior-value, and reflection proofs now route through the reusable half-base API. There is still no Lean formalization of the paper-level corrected signed-ratio density for negative q, separately named centered random variable, or general-q centered density. The finite MGF/CGF dictionary and pointwise sinc-prefix limit above also stop short of explicit Bernoulli-cumulant and Bell-moment evaluations, their asymptotics, and log-concavity or shape-transition theorems.

11.3 Activation Taylor jets, square-summability, and effective dimension

The post-snapshot activation layer begins with four total real functions in `HyperbolicActivation.lean`. Of these, the local activation probability

$$p(x) := \mathrm{activationProbability}(x) = 1 - \frac{\tanh x}{x} \quad (x \neq 0), \qquad p(0) = 0,$$

is continuous, even, nonnegative, strictly positive exactly away from zero, and globally satisfies

$$0 \le p(x) < 1, \qquad p(x) \le \min\left(1, \frac{x^2}{3}\right).$$

The totalized quotient `tanhDiv` supplies the value at the origin; the proof never treats a removable singularity as an ordinary quotient. The identity `tanhDiv_eq_dslope` makes that totalization canonical: as functions,

$$\mathrm{tanhDiv} = \mathrm{dslope}(\tanh, 0).$$

Thus the value at zero is the ordinary divided-slope value selected by the derivative of $\tanh$, rather than an unrelated convention added merely to make the quotient total.

The short module `ActivationSeries.lean` extracts the rescaled one-coordinate estimate as `activationProbability_mul_le_quadratic`:

$$p(at) \le \frac{t^2}{3} a^2 \qquad (a, t \in \mathbb{R}).$$

For an arbitrary index type I and real family $w : I \to \mathbb{R}$ with $\sum_i w_i^2 < \infty$, the declarations `summable_activationProbability_mul_of_summable_sq` and `tsum_activationProbability_mul_le` then prove genuine summability for every t and the global budget

$$D_w(t) := \sum_{i \in I} p(w_i t) \le \frac{t^2}{3} \sum_{i \in I} w_i^2.$$

Here D_w is only convenient notation in this guide, not a named Lean definition. No ambient countability, positivity, or ℓ^1 hypothesis is imposed: the `Summable` hypothesis itself carries the required countable support, and weights may be signed or zero.

The asymptotic half of the argument deliberately does not manufacture a Taylor remainder. In `ActivationAsymptotics.lean`, one l'Hopital step applied to $(x - \tanh x)/x^3$ proves `tendsto_activationProbability_div_sq`; its derivative quotient is $(\mathrm{tanhDiv}\, x)^2/3$. Composition with a nonzero dilation, together with the identically-zero case, yields `tendsto_activationProbability_mul_div_sq`:

$$\lim_{\substack{t\to 0\\ t\neq 0}} \frac{p(at)}{t^2} = \frac{a^2}{3} \qquad (a \in \mathbb{R}).$$

Thus the statement explicitly includes $a = 0$, which is essential for sparse weight families.

There is now also a separate finite Taylor theorem. It is stronger in local information than the quadratic limit, but it is not used to prove that limit.

Theorem 11.1 (Exact finite activation Taylor jet)**.** *As $x \to 0$,*

$$\tanh x = x - \frac{x^3}{3} + \frac{2x^5}{15} - \frac{17x^7}{315} + \frac{62x^9}{2835} + O\big(\|x\|^{11}\big)\,,$$

and

$$p(x) = \frac{x^2}{3} - \frac{2x^4}{15} + \frac{17x^6}{315} - \frac{62x^8}{2835} + O\big(\|x\|^{10}\big)\,.$$

Because the exponent is even, the second remainder is equivalently $O(x^{10})$. These are respectively `tanh_sub_taylor_nine_isBigO`, `activationProbability_sub_taylor_eight_isBigO`, and `activationProbability_sub_taylor_eight_isBigO_pow` in `ActivationTaylor.lean`.

Proof architecture. The proof first obtains analyticity of real $\tanh$ from the analytic $\sinh$ and $\cosh$, using positivity of $\cosh$ to justify division. Rather than repeatedly differentiating a quotient, it differentiates the identity $\tanh x \cosh x = \sinh x$ with the iterated Leibniz rule. At the origin this gives the derivative table, in orders zero through ten,

$$(0, 1, 0, -2, 0, 16, 0, -272, 0, 7936, 0).$$

The analytic partial-sum remainder theorem then gives the degree-nine $O(\|x\|^{11})$ formula for $\tanh$. On a punctured neighborhood, the defining identity $p(x) = 1 - \tanh(x)/x$ divides that remainder by x; the already-totalized values patch the statement at $x = 0$. Finally, evenness of the tenth power converts the norm-power majorant to the ordinary x^{10} form. This finite derivative-table route is independent of the one-step l'Hopital proof above: either theorem can be used without importing the proof architecture of the other. □

For $t \neq 0$, the same global estimate gives

$$0 \le \frac{p(w_i t)}{t^2} \le \frac{w_i^2}{3}.$$

The right side is both the pointwise limit and a summable majorant. Tannery's theorem therefore gives, in `tendsto_tsum_activationProbability_mul_div_sq` from `ActivationSeriesAsymptotics.lean`,

$$\boxed{\lim_{\substack{t\to 0\\ t\neq 0}} \frac{D_w(t)}{t^2} = \frac{1}{3}\sum_{i\in I} w_i^2.}$$

This fork-and-join proof is useful Lean architecture: the non-asymptotic summability module stays independent of l'Hopital and Tannery, while the asymptotic module reuses its exact majorant.

The geometric specialization is intentionally thin. The structural module `GeometricActivationDimension.lean` defines

$$d_q(t) := \sum_{n\ge 0} p\big(q^n(1-q)t\big)$$

as `geometricActivationDimensionqt`, and its dyadic specialization as `dyadicEffectiveDimensiont`. The reusable theorem `hasSum_normalizedGeometricWeight_sq` records genuine convergence and the exact value

$$\sum_{n\geq 0}\bigl(q^n(1-q)\bigr)^2 = \frac{1-q}{1+q} \qquad (|q|<1).$$

Specializing the square-summable theorem gives `tendsto_geometricActivationDimension_div_sq` and `tendsto_dyadicEffectiveDimension_div_sq` in `GeometricActivationAsymptotics.lean`:

$$\boxed{\lim_{\substack{t\to 0\\ t\neq 0}} \frac{d_q(t)}{t^2} = \frac{1-q}{3(1+q)}, \qquad \lim_{\substack{t\to 0\\ t\neq 0}} \frac{d_{1/2}(t)}{t^2} = \frac{1}{9}.}$$

The analytic range $|q|<1$ includes negative ratios. These are deterministic activation-sum identities. The frontier report's positive-weight active-count interpretation uses $0<q<1$, and the formal declarations construct no Bernoulli family and assert no identity $d_{1/2}(t)=\mathbb{E}K_t$. The exact quadratic limits themselves do not supply Taylor coefficients. A sharp finite jet is now formalized independently as above, while an all-order Bernoulli coefficient formula and a theorem identifying its exact radius of convergence remain separate targets.

At source checkpoint `a345425d21d90e680bf15e34093af42c69c08a83`, the seven modules `HyperbolicActivation`, `ActivationTaylor`, `ActivationSeries`, `ActivationAsymptotics`, `ActivationSeriesAsymptotics`, `GeometricActivationDimension`, and `GeometricActivationAsymptotics` expose an exhaustive combined surface of six definitions and 99 theorems, 105 documented declarations in all.

11.4 Finite convolution measures

A formal convolution proof must verify that every measure involved is finite, usually a probability measure. The source defines the elementary measures, proves their support and total mass, and then uses induction to show the convolution remains a probability measure. Associativity is applied only after the necessary sigma-finiteness or finiteness instances are in scope.

The support of the n-fold convolution is the Minkowski sum of the elementary supports. Geometric decay keeps these supports in a fixed compact interval. This uniform support bound supplies tightness essentially for free and gives uniform moment bounds.

11.5 Weak convergence

`WeakConvergence.lean` proves convergence of the finite convolution measures by first establishing pointwise convergence of their characteristic functions and then applying mathlib's Lévy continuity theorem. It derives convergence against bounded continuous test functions as the reusable public form needed by later interval and distribution-function arguments.

The limiting density measure `rvachevMeasure` has exact topological support $[-1,1]$: every open set meeting $(-1,1)$ has positive mass, and the endpoints belong to the support even though the density vanishes there.

The test-function formulation avoids trying to prove pointwise convergence of densities too early. Weak convergence is stable under convolution and directly reflects the probabilistic construction. To recover the Fabius function, the code later specializes to approximations of interval indicators.

11.6 Step approximants

`StepApproximationLimit.lean` studies the explicit step functions arising from finite convolution coefficients. The coefficient arrays satisfy symmetry and unimodality properties. These imply

monotonicity of the approximants on each side of the center and provide pointwise upper and lower controls.

The desired limit is continuous, but indicator functions of intervals are not continuous test functions. The source uses an interval-average squeeze. Integrals over a small neighborhood converge by weak convergence; monotonicity bounds the point value of a step function between nearby interval averages. Shrinking the neighborhood and using continuity of the limit gives pointwise convergence.

11.7 The endpoint half-weight correction

Closed interval indicators double-count shared endpoints when adjacent steps are assembled. In ordinary integration this affects a measure-zero set, but a pointwise theorem about the approximating function sees the discrepancy. The formalization therefore uses a representative with half weight at step boundaries.

This correction is a good example of Lean revealing a genuine mathematical issue rather than a mere elaboration problem. The source paper's almost-everywhere intuition is adequate for an integral identity, but not for the stated pointwise approximation. Once the half-endpoint convention is adopted, neighboring intervals partition correctly and the convergence theorem has the intended endpoint values.

11.8 Coefficient unimodality

The finite convolution coefficients form rows with strong symmetry and unimodality. The proof proceeds through a recurrence: the next row is obtained by averaging shifted blocks of the previous row. Symmetry is immediate from the symmetric elementary measure; monotonicity requires comparing overlapping partial sums.

Lean handles the row as a finite function or list. The proof establishes index bounds before rewriting recurrence entries. Boundary entries, where one shifted block leaves the valid range, are split into separate cases. These technical cases explain why the coefficient lemmas occupy a substantial part of `StepApproximationLimit.lean`: the final analytic squeeze is comparatively short once monotonicity is available.

11.9 From distribution functions to the bounded Fabius function

The direct weighted sum $X \in [0,1]$ from `ProbabilityRepresentation.lean` has CDF F globally. The folded identity is stated directly as

$$\mathrm{up}(x) = \mathrm{weightedSumCDF}(1 - |x|) = \mathbb{P}[X \le 1 - |x|].$$

These are CDF and event-probability theorems; the module does not package a separate centered random variable. Its half-base bridges do now identify the explicit density from `GeometricUnifo rmCDF.lean` with $2\,\mathrm{up}(2x-1)$ on all of $\mathbb{R}$, as recorded above.

This gives an independent conceptual construction of the same object as the Banach fixed point. In the formal dependency graph it is normally developed after the canonical function is available, because identification is easier than rebuilding the entire `IsFabius` interface from measure convergence. Mathematically, however, it explains why the function is a distribution function.

11.10 Exact finite Taylor formulas and nonanalyticity

`OriginalPaperSupplement.lean` packages further theorems from the original paper. At centered dyadic points, sufficiently high derivatives vanish on one side because repeated scaling reaches the support boundary. Taylor's theorem becomes an exact finite polynomial identity on a small interval.

If the function were analytic at such a point, its Taylor series would terminate and the function would be a polynomial in a neighborhood. The differential equation and support properties rule

this out unless the function were trivial. Thus the exact local truncation contributes to a nowhere-analyticity theorem.

The formal proof distinguishes several notions:

- `ContDiff` smoothness;
- existence of derivatives of all orders at a point;
- equality to the Taylor series on a neighborhood; and
- mathlib's analytic-at predicate.

A proof that merely observes all derivatives vanish at a boundary point establishes flatness, not by itself nonanalyticity everywhere. The module supplies the additional translation and dyadic-density argument needed for the global conclusion.

11.11 Corrections recorded by the original-paper aggregator

`Paper05442.lean` and its supplement explicitly document several source-sensitive points. Among them are:

- finite convolution must be stated at the finite stage rather than with an already limiting object;
- step-function endpoints require half weights for pointwise identities;
- a factor of the transform variable is needed in one displayed Fourier equation;
- a formula involving a positive order requires the hypothesis $n > 0$; and
- a global dyadic identity needs the signed extension or an adjusted normalization.

These are not hidden as comments in low-level proofs. The aggregator treats them as part of the formal audit of the paper.

12 The two paper formalizations as navigable APIs

12.1 The first paper: `Paper05442.lean`

The module named after arXiv:1702.05442 imports the original uniqueness proof, probability representation, weak convergence, step approximation, Poisson summation, and supplementary source theorems. Its purpose is to certify coverage of the paper as a whole.

A useful reading order is:

1. `OriginalCharacterization.lean` and `OriginalUniqueness.lean` for the defining characterization and the equivalence of solution spaces;
2. `ProbabilityRepresentation.lean` for the random-series model;
3. `WeakConvergence.lean` for the measure limit;
4. `StepApproximationLimit.lean` for pointwise approximation;
5. `PoissonSummation.lean` for the sampled Fourier identity; and
6. `OriginalPaperSupplement.lean` for moments, Taylor formulas, rationality, and nonanalyticity.

The aggregator's theorem names are source-facing, but the mathematical objects remain those of the common core. This avoids a second, paper-specific definition of the function.

12.2 The arithmetic paper: `Paper06487.lean`

The second aggregator, named after arXiv:1702.06487, imports `Paper06487Supplement.lean`, which in turn draws on `PaperStatements.lean` and the arithmetic/analytic bridge modules. It covers the paper's propositions on moments, exact dyadic values, denominators, parity, and two-adic valuations.

The internal proof structure is more extensive than the paper numbering suggests:

exact recurrence $\longrightarrow$ normalized numerator $\longrightarrow$ parity/divisibility $\longrightarrow$ reduced rational theorem.

Each arrow is represented by one or more modules. The final paper theorem is often a cast or notation wrapper around an internal theorem with a stronger domain or a more explicit divisibility conclusion.

12.3 Statements that remain questions or conjectures

A formalization should not turn an open question into a theorem. `PaperStatements.lean` records the paper's Question 5, Definition 12, and Conjecture 16 as definitions or propositions expressing the claimed predicate, not as proofs of the unresolved assertion. This is part of the source-coverage discipline: every numbered item is represented, but its logical status is preserved.

The surrounding proved theorems may establish initial cases, necessary conditions, or denominator bounds relevant to the conjecture. Their names make clear when the result is a theorem about the conjectural predicate rather than a proof of the conjecture itself.

12.4 Why a separate coverage document matters

The repository's PAPER_COVERAGE.md maps source theorem numbers to Lean declarations. This is more than project management. It helps distinguish three questions:

1. Has the mathematical claim been formalized?
2. Where is the strongest internal theorem proved?
3. Which wrapper corresponds to the source numbering and notation?

Appendix B reproduces the navigational content of that map in compact form and adds the surrounding module families.

13 Thue–Morse and q-binomial identities

13.1 Why q-binomial algebra appears

The Thue–Morse sign converts binary digit structure into alternating sums. When a finite Taylor formula is applied repeatedly across dyadic cells, the resulting coefficients naturally involve Gaussian, or q-binomial, coefficients at $q = 1/2$. The repository develops this relationship in a series of modules ranging from exact finite identities to global analytic series.

The core finite identity has the flavor

$$\sum_{k=0}^{2^n-1} \varepsilon(k)(x+k)^m,$$

which vanishes for low degree and factors in a controlled way at the critical degree. q-binomial coefficients provide a compact description after binary decomposition. The Fabius values enter when the polynomial identity is inserted into a scaled finite Taylor expansion.

The same even/odd recursion gives the exact exponential factorization formalized in `ThueMorseExponential.lean`:

$$\sum_{r<2^n} (-1)^{\mathrm{wt}_2(r)} e^{rz} = \prod_{j<n} \bigl(1 - e^{2^j z}\bigr).$$

It is a finite identity, so no interchange of limits or infinite products is hidden in the proof.

13.2 Centered mixed differences

`ThueMorseSymmetricDifference.lean` turns the order-free operator of `ThueMorseMixedDifference.lean` into a centered Boolean-cube calculus. If a_i are half-steps indexed by a finite set s, then `symmetricMixedDifference` is the ordinary mixed difference with steps $-2a_i$, based at $x + \sum_{i\in s} a_i$. Its empty and singleton values are `symmetricMixedDifference_empty` and `symmetricMixedDifference_singleton`; peeling off a fresh index is `symmetricMixedDifference_insert`. The full expansion `symmetricMixedDifference_eq_sum_powerset_smul` is

$$\Delta^{\mathrm{sym}}_{a,s} f(x) = \sum_{t\subseteq s} (-1)^{|t|} f\left(x + \sum_{i\in s} a_i - 2\sum_{i\in t} a_i\right).$$

For a polynomial of degree at most $|s|$, `symmetricMixedDifference_polynomial_eq_coeff_card` extracts

$$[X^{|s|}]p \; |s|! \prod_{i\in s} (2a_i).$$

The strict-degree vanishing and first surviving monomial are `symmetricMixedDifference_polynomial_of_degree_lt` and `symmetricMixedDifference_pow_card`. The degree-valued hypothesis keeps the empty-set/zero-polynomial boundary honest.

The second public definition, `symmetricDyadicMixedDifference`, uses the half-steps $2^j h$, $j < m$. Its zero-order value is `symmetricDyadicMixedDifference_zero`; its centered Thue–Morse sum is `symmetricDyadicMixedDifference_eq_sum_thueMorseSign_smul`. With $h = 2^{-m}$, block reflection gives the increasing grid

$$\sum_{n<2^m} (-1)^m \varepsilon_n f\bigl(x - (1 - 2^{-m}) + 2n2^{-m}\bigr),$$

exactly `symmetricDyadicMixedDifference_inv_two_pow_eq_sum_thueMorseSign_smul`. The positive-order spelling with $m+1$ differences and mesh 2^{-m} is `symmetricDyadicMixedDifference_inv_two_pow_succ_eq_sum_thueMorseSign_smul`. These are all two definitions and eleven theorems in the module; it proves no analytic limit or differentiability statement.

The companion `ThueMorseCornerIntegral.lean` supplies precisely that local analytic step. Its one public definition, `centeredBoxIntegral`, recursively nests the intervals $[-a_i, a_i]$; its zero and successor equations are `centeredBoxIntegral_zero` and `centeredBoxIntegral_succ`. If $a_i \geq 0$ for $i < N$, I is open and order-connected, g is `ContDiffOn` of order N on I, and

$$[x - \sum_{i<N} a_i,\; x + \sum_{i<N} a_i] \subseteq I,$$

then

$$\Delta^{\mathrm{sym}}_{a,\{0,\dots,N-1\}} g(x) = \mathcal{B}_{a,N}(g^{(N)}; x).$$

This is `symmetricMixedDifference_range_eq_centeredBoxIntegral`; `symmetricMixedDifference_univ_eq_centeredBoxIntegral` is its `FinN`-indexed reindexing. Those four theorems plus the definition are the exhaustive 1+4 public surface.

The proof is an induction on N. Peeling the last symmetric difference reduces each Boolean-cube corner to an endpoint difference. The full-segment containment and order-connectedness put

every translated interval inside I, so the local interval fundamental theorem of calculus replaces that difference by an integral of g'. Finite linearity moves the corner sum through the outer integral, and the induction hypothesis applies to g'. Thus the theorem keeps the manuscript's local regularity rather than assuming global smoothness. It includes zero half-steps and $N = 0$, strengthening the printed positive-step case, but does not cover arbitrary signed half-steps. Together with the algebraic module it makes `thm:TM-corner` *Exact/Complete*; the following Walsh probability corollary remains separate.

13.3 The finite q-Pochhammer infrastructure

`HalfQBinomial.lean` develops exact q-binomial algebra at $q = 1/2$ over $\mathbb{Q}$. The finite q-Pochhammer product is represented without analytic convergence issues:

$$(a;q)_n = \prod_{j=0}^{n-1}(1 - aq^j).$$

At $q = 1/2$, clearing powers of two turns many factors into Mersenne numbers. This is why the Mersenne products introduced in `Arithmetic.lean` reappear.

The module defines a rational `halfQBinomial` and proves its q-Pascal recurrence. It then proves a finite q-binomial theorem by induction. The induction step is not a black-box appeal to an imported polynomial theorem; it matches the precise normalization required downstream. Summands are shifted and paired so the q-Pascal recurrence telescopes into the next product.

The exhaustive zero-definition/one-theorem companion `HalfQBinomialRootSimplicity.lean` supplies the missing multiplicity clause. For every $j < n$, `halfQBinomial_sum_rootMultiplicity_two_pow` proves over $\mathbb{Q}$ that the coefficientwise half-base polynomial has root multiplicity one at 2^j. Together with `halfQBinomial_sum_eq_zero_iff` and `gaussianBinomial_half_eq_halfQBinomial`, this makes the complete simple-root classification `cor:halfbase-root-locus` Exact. It makes no arbitrary-base, arbitrary-field, or general cyclotomic simplicity claim, and does not promote the probabilistic `cor:qbinom-inversion-law`, which remains Partial.

`QBinomialCauchy.lean` supplies the denominator-free finite q-Cauchy layer over every commutative ring. Its five public theorem names are `finite_qCauchy_identity`, the compatibility spelling `finiteQPochhammerIn_mul_eq_sum_gaussianBinomial`, `finite_qCauchy_identity_reflected`, `sum_qBernsteinBasis`, and `finite_qCauchy_second_identity`, with the single public definition `qBernsteinBasis`. All five theorems include degree zero, $q = 0$, roots of unity, positive characteristic, and zero divisors; none uses division, cancellation, or a nonvanishing hypothesis.

13.4 Finite moment functionals and affine Prouhet transport

`FinitePolynomialFunctional.lean` has no public definitions and exactly sixteen public theorems: `sum_weight_mul_eval₂_eq_sum_coeff_mul_moment`, `sum_weight_mul_eval₂_eq_eval₂_of_moments`, `sum_weight_mul_eval₂_eq_coeff_mul_moment`, `sum_weight_mul_eval₂_eq_topCoeff_mul_moment`, `sum_weight_mul_eval₂_eq_constantCoeff_mul_sum`, `sum_weight_mul_eval₂_eq_constantCoeff`, `sum_weight_mul_eval₂_eq_map_coeff_mul_of_moments`, `sum_weight_mul_eval₂_eq_map_coeff_mul_top_moment`, `sum_weight_mul_eval₂_eq_zero_of_degree_lt`, `sum_weight_mul_eval₂_congr_of_map_coeff_eq`, `sum_weight_mul_eval_eq_eval_of_moments`, `sum_weight_mul_eval_eq_coeff_mul_of_moments`, `sum_weight_mul_eval_eq_coeff_mul_top_moment`, `sum_weight_mul_eval_eq_zero_of_degree_lt`, `sum_weight_mul_eval_congr_of_coeff_eq`, and `sum_weight_mul_eval_affine_of_topCoeff_extractor`. The scalar-extension results work from an arbitrary semiring into a commutative semiring; the same-ring results need only a

commutative semiring. Expanding polynomial evaluation, interchanging the two finite sums, and substituting the prescribed monomial moments prove reproduction, selected coefficient extraction, strict-degree cancellation, and congruence. Repeated nodes and a zero selected moment are allowed.

The last theorem says that any same-ring functional extracting $[X^n]p\,c$ at nodes x_i extracts

$$b^n c[X^n]p$$

after the affine replacement $x_i \mapsto a + bx_i$. Its proof applies the original extractor to $p(a + bX)$ and computes the top coefficient; the zero-scale case is included, so neither $b \neq 0$ nor distinct transformed nodes is assumed. Composing this transport with `halfQBinomial_negativeDyadic_polynomial_sum_eq_mersenne` proves `cor:geometric-prouhet-affine` exactly under the established $\mathbb{Q}[X]$ convention, including $b = 0$ and $n = 0$.

13.5 Why prove a specialized q-binomial theorem?

Mathlib contains general polynomial and combinatorial infrastructure, but a specialized theorem at $q = 1/2$ gives the repository:

- exact rational coefficients with predictable denominators;
- direct compatibility with existing Mersenne products;
- a recurrence usable by kernel computation; and
- statements in the same finite-sum form as the Fabius formulas.

A general theorem would still require a substantial specialization layer. Proving the specialized identity directly keeps casts and nonvanishing side conditions under control.

13.6 Raw, scalar, and polished formulas

The q-binomial family is intentionally split into several modules:

- `FabiusRawQBinomialFormula.lean` and `FabiusRawQBinomialScalar.lean` establish algebraic sums before all Fabius-specific simplifications;
- `FabiusQBinomialFormula.lean` proves the exact finite identity in its principal form;
- `FabiusQBinomialScalarFormula.lean` specializes coefficient-valued statements to scalar values; and
- `FabiusDyadicQBinomialScalar.lean` inserts the exact dyadic evaluator.

This decomposition protects the core polynomial identity from unnecessary analytic dependencies. It also allows translation invariance to be proved at the polynomial level and reused after evaluation.

13.7 The unrestricted finite identity

A notable strength of the formal result is that it is not restricted to a reduced dyadic representative or to $0 \leq m \leq 2^n$. It is stated for arbitrary natural parameters in the range supported by the algebraic expression. The signed global extension absorbs the translation behavior that would otherwise force an artificial range hypothesis.

The proof proceeds by induction on the binary depth. Splitting a range into even and odd indices transforms the Thue–Morse sign into a difference of two scaled copies. The induction hypothesis applies to each copy, and the q-Pascal recurrence combines the coefficients. Polynomial degree bounds control vanishing terms.

Proof architecture. The even/odd split is the common engine behind Thue–Morse cancellation and q-binomial recursion. In Lean, the decisive step is a reindexing theorem that expresses a sum over

`Finset.range(2*N)` as sums over $2k$ and $2k+1$. Once that lemma has the right shape, the sign and power transformations simplify automatically. □

13.8 Translation invariance as a polynomial theorem

One of the strongest internal statements says that the relevant finite q-expression is independent of a translation parameter because it is a polynomial of degree below the cancellation threshold. This is proved before specializing the parameter to real or complex values. As a result, common translation invariance holds over a broad coefficient ring and later applies uniformly to $\mathbb{R}$ and $\mathbb{C}$.

This is a recurring proof-design strategy: prove a fact at the most algebraic level where it is true, then transport it through evaluation homomorphisms. Analytic specializations become one-line consequences of polynomial extensionality.

13.9 Polynomial constancy and Taylor reduction

In `FabiusQBinomialTaylor.lean`, q is a common translation indeterminate, not an asymptotic expansion parameter. Complete-block Thue–Morse cancellation proves that the finite numerator is a constant polynomial in q. Evaluation is therefore independent of the translation over every characteristic-zero field. After normalization and reindexing, the constant is identified exactly with `fabiusReductionSum`.

This algebraic constancy is distinct from the discrete-limit approximation `fabiusDiscreteLimitApproximationKqxn`. A finite approximation there can genuinely depend on a complex q; later complex-shift estimates prove only that this dependence vanishes in the prescribed limit.

13.10 The global binary-reduction series

`FabiusBinaryReductionSeries.lean` and `FabiusGlobalQBinomialSeries.lean` pass from finite identities to an infinite series representing the signed extension. The binary reduction follows the same highest-bit philosophy as the exact evaluator, but now the steps are organized as a global q-series.

A genuine scale-zero term forces the outer index to start at $m = 0$. Its previous prefix is the half-scale floor $\lfloor x/2 \rfloor$, so the zeroth coefficient distinguishes even and odd unit blocks of the signed extension. It vanishes for $x < 1$ and equals one at $x = 1$; hence the older $m = 1$ presentation survives only on $0 \le x < 1$.

The exact finite telescope leaves a signed residual whose uniform geometric bound is at most 2^{1-N} through inclusive scale $N \ge 1$, for every real input. It therefore tends uniformly to zero on the whole real line. The summand norms are pointwise summable, giving all-real `HasSum` and `tsum` identities. `FabiusGlobalQBinomialSeries.lean` proves that its literal real or complex q-summand is pointwise equal to the q-independent binary-reduction summand before either side is summed. The resulting real and complex series represent the signed extension for every $x \in \mathbb{R}$; the earlier nonnegative-input statements remain compatibility wrappers.

13.11 Parity-power series

`FabiusParityPowerSeries.lean` rewrites the corrected scale-zero binary-reduction series in the draft's parity-power notation. The outer sum starts at $m = 0$ and equals the signed global Fabius extension for every $x \in \mathbb{R}$, with the nonpositive side termwise zero; the original series starting at $m = 1$ is recovered only on $0 \le x < 1$. Its integer exponent is $\kappa_n = \binom{n+1}{2} - 1$, so $\kappa_0 = -1$, not a truncated natural subtraction; the scale-zero identity also uses Lean's convention $0^0 = 1$. The proof identifies the inner finite sum with one half of `fabiusReductionSum` and then reuses the binary telescope.

14 Discrete limits and complex shifts

14.1 The shape of the limit theorem

The discrete-limit project starts from finite q-binomial/Thue–Morse rows depending on a real location x and a complex parameter q. For nondyadic x, an individual finite row can depend genuinely on q. The theorem is that, after the prescribed normalization and as the row depth tends to infinity, the limit exists and is independent of q, equaling the relevant Fabius expression.

This distinction between finite-stage dependence and limiting independence is explicitly formalized. It prevents an overstrong claim that each approximant is constant in the parameter.

14.2 Uniform spline approximants

`FabiusUniformSpline.lean` introduces centered finite Thue–Morse splines. These are piecewise-polynomial functions whose coefficients encode a finite convolution row. They are uniformly bounded, vanish on the nonpositive half-line, and repeat on the nonnegative axis as signed two-unit blocks. `FabiusComputability.lean` proves the global error estimate

$$\sup_{x\in\mathbb{R}} |S_p(x) - F_{\mathrm{gl}}(x)| \le 2^{-p} \qquad (p \ge 1).$$

Thus they converge uniformly on all of $\mathbb{R}$, while their bounded-CDF specialization converges uniformly to F on $[0,1]$.

On the fundamental interval the source identifies the spline with the CDF of a centered finite weighted sum. This yields monotonicity, range and support bounds, and pointwise convergence; a block-translation identity extends those facts across the nonnegative axis.

14.3 Complex-shift control

`FabiusComplexShiftSpline.lean` and `FabiusDiscreteLimitComplexShift.lean` control evaluating a finite spline at a complex displacement. A Taylor expansion separates the real centered value from correction terms. Uniform derivative bounds, together with the scale of the shift, show the corrections vanish in the limit.

The proof is careful not to use order arguments on $\mathbb{C}$. Norm bounds replace real inequalities, and factorial denominators are estimated in $\mathbb{R}$ before being cast into complex norms. The finite nature of the Taylor expansion is again important: each finite spline has bounded degree, so no complex analytic remainder theorem is required.

14.4 The Toeplitz averaging mechanism

`FabiusDiscreteLimitToeplitz.lean` rewrites the normalized row as a Toeplitz-type weighted average of centered spline values. The weights need not be nonnegative: their row sum is one and their total variation is uniformly bounded, while the sampled spline degrees all tend to infinity. These facts transfer convergence of the splines to convergence of the averages.

The finite prefix is encoded safely as the natural floor

$$\texttt{fabiusDiscreteLimitRangeLength}(x,p) = \left(\left\lfloor 2^p x + \tfrac{1}{2}\right\rfloor\right)_+,$$

so no negative length is coerced into a natural number. For $x \ge 0$, its integer cast is $\left\lfloor 2^p x - \frac{1}{2}\right\rfloor + 1$, and the length is zero exactly when $2^p x < \frac{1}{2}$. The exact row-mass theorem gives sum 1. A lower bound $\mathrm{halfQPochhammer}(n) \ge 1/4$ yields the explicit total-variation bound 16, independent of the row. Finally, the sampled indices are shown to escape every fixed finite set uniformly. These are the concrete inputs to the abstract Toeplitz passage; positivity of the weights is neither assumed nor needed.

The generic theorem `tendsto_weighted_rows_of_tendsto` isolates four hypotheses:

1. every finite row has sum 1;
2. the row total variations have one uniform bound;
3. every sampled index in a row eventually lies beyond each prescribed cutoff; and
4. the sampled sequence $H(n)$ tends to its proposed limit.

No positivity or order structure on the coefficient field is required. This abstraction is useful beyond the specific Fabius application and keeps the final limit proof from becoming a nested epsilon argument. Searchable entry points are `sum_range_discreteLimitWeight`, `one_fourth_le_halfQPochhammer`, `sum_abs_discreteLimitWeight_le`, and `tendsto_weighted_rows_of_tendsto`.

14.5 Final reindexing and identification

`FabiusDiscreteLimitIntegration.lean` performs the exact outer q-binomial reindexing and applies the Toeplitz theorem to the complex-shift spline limit. It identifies the result with the signed global Fabius function and with the previously proved binary and literal q-binomial series. The main nonnegative-input theorem is strengthened by exact empty-prefix vanishing to every real input.

14.6 Proof flow of the discrete-limit family

A recommended source order is:

1. `FabiusRawQBinomialFormula.lean`;
2. `FabiusQBinomialFormula.lean`;
3. `FabiusBinaryReductionSeries.lean`;
4. `FabiusUniformSpline.lean`;
5. `FabiusComplexShiftSpline.lean`;
6. `FabiusDiscreteLimitToeplitz.lean`;
7. `FabiusDiscreteLimitComplexShift.lean`; and
8. `FabiusDiscreteLimitIntegration.lean`.

The sequence moves from exact finite algebra, through real approximation, to complex perturbation, averaging, and final analytic identification.

Formalization-sensitive point. When reading these files, distinguish the row index from the polynomial degree, the dyadic scale, and the q-parameter. Informal drafts often use a single letter for more than one role. The Lean development uses separate variables and explicit type annotations, which can make theorem statements longer but prevents illegal substitutions.

---

15 The logarithmic scale near the flat endpoint

15.1 Why ordinary Taylor asymptotics fail

Every derivative of the bounded Fabius function vanishes at zero, yet $F(x) > 0$ for every $x > 0$. Thus no nonzero power of x is the leading term. The correct scale is logarithmic and, at first order, quadratic in $-\log x$.

The asymptotic development introduces

$$\phi(t) = F(2^{-t}), \qquad g(t) = -\log \phi(t).$$

As $x \downarrow 0$, the variable $t = -\log_2 x$ tends to infinity. This turns dyadic scaling into unit translation and is the natural coordinate for the delay equation.

15.2 The exact logarithmic delay identity

`FabiusLogScale.lean` differentiates $\phi(t) = F(2^{-t})$. For $t \geq 1$, the argument 2^{-t} lies in the left branch where the differential equation applies. The chain rule gives

$$\phi'(t) = -(\log 2)2^{-t}F'(2^{-t}) = -2(\log 2)2^{-t}F(2^{1-t}).$$

Since $\phi(t-1) = F(2^{1-t})$, this becomes a delay relation. Rewriting in terms of $g = -\log\phi$ and its derivative produces the exact identity formalized in the module:

$$g(t) - g(t-1) = \log g'(t) - \log(\log 2) - (1-t)\log 2, \qquad t \geq 1.$$

The source first proves positivity of $F(2^{-t})$ and the relevant logarithmic slope, so every logarithm is legal.

Formalization-sensitive point. The threshold $t \geq 1$ is essential. The bounded function's derivative law does not hold on the same branch for all real t. An informal draft treated the delay identity as global; the Lean theorem records the correct domain and downstream asymptotic arguments are formulated with eventual filters.

15.3 Positivity near zero

Taking a logarithm requires strict positivity, not merely nonnegativity. The repository obtains positivity from monotonicity and exact dyadic values. Given $x > 0$, choose a sufficiently large inverse power of two below x. Exact arithmetic shows $F(2^{-n}) > 0$, and monotonicity gives $F(x) \geq F(2^{-n}) > 0$.

This proof is representative of the interaction between exact and analytic layers: a qualitative analytic property at every positive real point is obtained from exact values on a dense cofinal sequence.

15.4 The quadratic logarithmic law

`FabiusLogSquaredAsymptotic.lean` proves

$$\frac{g(t)}{t^2} \longrightarrow \frac{\log 2}{2}.$$

Equivalently,

$$\log F(x) \sim -\frac{(\log(1/x))^2}{2\log 2} \qquad (x \downarrow 0).$$

The proof first establishes sufficiently sharp estimates at natural values of t, where the exact inverse-dyadic recurrence is available. It then extends them to real t by monotonicity and floor/ceiling comparison.

If $n \leq t < n+1$, monotonicity gives

$$g(n) \leq g(t) \leq g(n+1)$$

with the inequality direction determined by the decreasing behavior of $F(2^{-t})$. Dividing by t^2 and using $n/t \to 1$ transfers the natural asymptotic to real arguments.

15.5 A quantitative coarse bound

The same module proves a bound of the form

$$\left| g(t) - \frac{\log 2}{2} t^2 \right| \le Ct \log t$$

for all sufficiently large t. This is weaker than the eventual sharp formula but strong enough to establish flatness and compare F with elementary scales.

Such a concrete inequality is more useful in Lean than a bare $o(t^2)$ statement. It can be inserted into exponential comparisons with explicit thresholds, and it supports later proofs that every power of x dominates $F(x)$ while every $e^{-c/x}$ is dominated by $F(x)$.

15.6 Filter formulations

The repository states asymptotics both in the $t \to \infty$ coordinate and as $x \to 0^+$. Conversion lemmas relate the filters under $x = 2^{-t}$. A typical proof establishes a limit along `atTop`, composes with the continuous map $t \mapsto 2^{-t}$, and then identifies the image filter with `nhdsWithin0(Set.Ioi0)`.

Explicit conversion lemmas are preferable to ad hoc substitutions in each theorem. They ensure that positivity domains and logarithm definitions are carried through uniformly.

16 Negative Laplace analysis and the periodic correction

16.1 The negative moment generating function

The repository's entire generating function is totalized as

$$G(z) = 1 + z \int_0^1 \mathrm{up}(t) e^{zt} \, \mathrm{d}t.$$

For $s > 0$, `NegativeLaplace.lean` identifies its negative-axis value with the exact product

$$e^{\mathrm{negativeLaplaceLog}(s)} = G(-s) = \prod_{j \ge 1} \frac{1 - e^{-s/2^j}}{s/2^j}.$$

The companion `LaplaceTransform.lean` proves the equivalent bounded-function formula

$$\frac{G(-s)}{s} = \int_{(0,\infty)} F(t) e^{-st} \, \mathrm{d}t.$$

The product's logarithm is

$$L(s) = \sum_{j \ge 1} \left[\log(1 - e^{-s/2^j}) - \log(s/2^j) \right].$$

`NegativeLaplace.lean` constructs this series carefully and proves absolute convergence after grouping terms in a form adapted to small and large arguments.

16.2 The kernel and its bounds

The central scalar kernel is the logarithm of a normalized Bose factor. For small u, the quotient $(1 - e^{-u})/u$ tends to one, so its logarithm is $O(u)$. For large u, the exponentially small correction $\log(1 - e^{-u})$ is controlled by a geometric series.

The source proves elementary inequalities before summing over dyadic scales. Typical estimates use

$$0 \le -\log(1-v) \le \frac{v}{1-v} \qquad (0 \le v < 1),$$

with $v = e^{-u}$. The resulting majorants are summable after the dyadic split at the scale where $s/2^j$ crosses one.

16.3 The exact dilation equation

The product shifts cleanly under $s \mapsto 2s$. Removing the first factor and reindexing the rest gives an exact functional equation for $L(s) = \log G(-s)$. Solving the associated difference equation in the logarithmic variable $u = \log_2 s$ produces a quadratic polynomial plus a periodic remainder.

The formal decomposition is

$$L(s) = -\frac{(\log s)^2}{2\log 2} + \frac{1}{2}\log s + R(\log_2 s) + E(s),$$

where $R(u+1) = R(u)$ and the forward-tail error satisfies an explicit exponential bound. A representative bound proved in the module is

$$|E(s)| \le \frac{e^{-s}}{(1-e^{-s})^2},$$

and for s beyond a fixed threshold this is simplified to a constant multiple of e^{-s}.

The sign information is equally exact on the positive half-line. Every logarithmic factor and the full logarithmic product are strictly negative, as is the forward tail. Consequently `negativeLaplaceTailError`, defined as the negative of that tail, is strictly positive and its absolute value can be removed.

The reusable differentiation step is isolated in `ScalingRecurrence.lean`, a Mathlib-only module independent of the Fabius definitions. It differentiates multiplicative scaling recurrences for an arbitrary dilation $q > 0$, in both vector-valued and real-scalar forms, with the outgoing weight normalized by the caller. Its inert-parameter version acquires no factor of q, because differentiation is in the other variable; dyadic wrappers feed these results into the ordinary and vertical negative-Laplace derivative layers.

16.4 How the periodic function is constructed

The periodic term is not guessed from numerics. The code builds it by comparing the exact logarithm with a particular quadratic solution of the dilation equation. The difference is invariant under doubling up to a forward tail. Iterating the dilation toward infinity makes that tail summable; adding its convergent correction yields an exactly invariant function of $\log_2 s$.

Concretely, one introduces a defect

$$D(s) = L(s) + \frac{(\log s)^2}{2\log 2} - \frac{1}{2}\log s.$$

The dilation identity shows that $D(2s) - D(s)$ is an exponentially small kernel. A forward sum of these defects corrects D into a function satisfying $P(2s) = P(s)$. Writing $R(u) = P(2^u)$ gives period one.

Proof architecture. The periodic correction is the homogeneous solution of a dyadic difference equation. The quadratic logarithmic terms form a particular solution. Exponentially small forward tails determine which periodic solution corresponds to the actual infinite product. □

16.5 Continuity and regularity

`PeriodicCorrection.lean` proves continuity of R. The defining correction is a locally uniformly convergent series on positive compact intervals. The proof covers a compact interval $[a, b] \subset (0, \infty)$, bounds every tail term uniformly by a summable geometric sequence depending on a, and applies a Weierstrass-type theorem.

Later modules strengthen regularity. `PeriodicRegularity.lean` differentiates the correction series term by term after proving uniform convergence of derivative tails. `PeriodicMean.lean` computes or normalizes its mean. `PeriodicFourier.lean` identifies Fourier coefficients through Mellin-transform data.

16.6 Mellin–Bose analysis and finite parts

`MellinBose.lean` proves, first for real $a > 0$,

$$\int_0^\infty x^{a-1} \log(1 - e^{-x}) \, \mathrm{d}x = -\Gamma(a)\zeta(a+1),$$

by expanding the logarithm into an absolutely integrable exponential series and integrating term by term. `MellinFinitePart.lean` regularizes the double pole of $\Gamma(s)\zeta(1+s)$ at $s = 0$, while `BoseFinitePartIntegral.lean` realizes the finite part as convergent split integrals after subtracting the small-x singular term.

The local constants are normalized explicitly. `GammaSecondOrder.lean` proves the sharp complex remainder

$$\Gamma(1+z) = 1 - \gamma z + \left(\frac{\gamma^2}{2} + \frac{\pi^2}{12}\right) z^2 + O(z^3) \qquad (z \to 0),$$

while `StieltjesConstant.lean` fixes γ_1 by declaring the derivative of `zetaRegularReal` at one to be $-\gamma_1$, recorded in `HasDerivAt` form. Thus the sign of the linear coefficient in $\zeta(1+s) = s^{-1} + \gamma - \gamma_1 s + O(s^2)$ is part of the checked interface.

`PeriodicFourier.lean` treats the zero mode through the mean calculation and the nonzero modes by the substitution $x = 2^t$, dyadic interval decompositions, an identity-theorem extension to the open right half-plane, and continuity to the punctured imaginary axis. It obtains the Gamma–zeta Fourier coefficients and absolute Fourier reconstruction without a residue calculation, contour truncation, or Mellin inversion.

16.7 Why the periodic term cannot be dropped

The oscillatory correction $R(\log_2 s)$ is bounded but nonconstant. Therefore it is lower order than the quadratic logarithmic term but not an error tending to zero. Any asymptotic formula claiming an $o(1)$ or $O(1/\log s)$ residual after omitting R is false.

`FabiusWikipediaObstruction.lean` and related audit modules formalize this obstruction. They prove nonconstancy through a nonzero Fourier coefficient or a pair of points with unequal values. Sampling along two dyadic phase subsequences then produces distinct limit points for the alleged residual.

16.8 Identification with the generating function

After constructing the product and its logarithm, `NegativeLaplace.lean` proves equality with the analytic generating function defined earlier. The route is:

1. establish the product formula for finite truncations;
2. prove convergence to the infinite product;

3. establish the same functional equation and normalization for the generating function; and
4. use uniqueness or direct limit passage to identify them.

This closes the loop between exact moment series, Fourier/probability products, and the asymptotic Laplace object.

17 Bromwich inversion and the sharp saddle point

17.1 From a transform to the endpoint density

For every $r > 0$, `FabiusBromwichInput.lean` proves the exact Fourier–Bromwich identity

$$F(x) = e^{rx} \int_{\mathbb{R}} e^{2\pi i \xi x} \frac{G(-(r + 2\pi i\xi))}{r + 2\pi i\xi} \, d\xi.$$

Thus the vertical coordinate $z = r + 2\pi i\xi$ has positive real part, while the generating function is sampled at the negative argument $-z$. The theorem comes directly from Fourier inversion of the exponentially tilted CDF and requires no contour-shifting assumption.

`BromwichSaddle.lean` supplies the generic quantitative framework and `FabiusBromwichInput.lean` discharges its Fabius-specific integrability and decay hypotheses. For asymptotic extraction one then chooses the explicit lower-Lambert reference radius and splits the resulting real integral into central and tail regions.

17.2 The Lambert phase

The dominant balance introduces a lower-Lambert reference coordinate. On its natural small-positive-x domain the repository defines $\lambda(x)$ so that

$$\lambda(x)\, 2^{-\lambda(x)} = x.$$

Equivalently, if $x = 2^{-t}$, then

$$\lambda(2^{-t}) - \log_2 \lambda(2^{-t}) = t.$$

These are exact algebraic identities for the chosen coordinate. The formal module deliberately does not claim that this reference coordinate is the exact critical point of every residual term in the full contour phase.

The advantage of retaining the exact Lambert phase is that all-order expansions can be stated without repeatedly composing truncated asymptotic series. The leading behavior is

$$\lambda(2^{-t}) = t + \log_2 t + o(1),$$

or the corresponding expression after substituting $t = -\log_2 x$. The exact relation captures the corrections needed in both the exponential and Gaussian normalization.

`FabiusLambertPhase.lean`, `FabiusLambertRates.lean`, and `FabiusLambertSaddle.lean` establish the domain, monotonicity, growth, derivative bounds, and exact lower-Lambert identities. Separate modules for small-argument expansions later translate λ into elementary logarithms to any fixed order.

The reciprocal scale conversion is algebraically total: `smallArgumentLog_inv_eq_all` holds for every real input, and `smallArgumentLog_inv_isBigO` holds at an arbitrary filter. The small-positive filter reappears only when these identities are transferred to an endpoint asymptotic theorem.

17.3 Exact reduction at the saddle

`FabiusSharpExactReduction.lean` packages the inversion calculation into an exact error decomposition. In logarithmic form, it has the conceptual shape

$$\log F(x) - M(x) = \log\bigl(\text{normalized saddle mass}\bigr) + \text{forward-tail correction},$$

where $M(x)$ contains the explicit quadratic-logarithmic, Lambert, Gaussian, and periodic pieces.

This theorem is the critical interface between contour analysis and asymptotic algebra. All later approximations can focus on two controlled terms: a normalized central integral tending to one, and an exponentially flat tail inherited from the negative-Laplace product.

17.4 Central and tail decomposition

Write the vertical contour variable as $s = \sigma + \mathrm{i}y$. The integral is split at a central radius $|y| \le Y(\lambda)$:

$$I = I_{\mathrm{central}} + I_{\mathrm{tail}}.$$

Near the saddle, a Taylor expansion of the exponent begins with a negative quadratic term. Away from the saddle, a minor-arc estimate shows strict decay of the real part.

The module family includes `FabiusSaddleCentral.lean`, `FabiusSaddleTail.lean`, `FabiusLambertMinorArc.lean`, and reference-weight modules. The split allows different techniques:

- local cumulant expansions and Gaussian comparison in the center;
- coarse but uniform exponential inequalities in the tail.

Trying to use the local Taylor series on the entire contour would give an unjustified remainder.

17.5 The Gaussian reference weight

After rescaling y by the square root of the second derivative at the saddle, the leading central factor is $e^{-u^2/2}$. `FabiusSaddleReferenceWeight.lean` and `FabiusSaddleReferenceTail.lean` collect exact Gaussian integrals and tail bounds in the normalization used by the contour.

The normalized saddle mass is divided by the Gaussian main integral so its limit should be one. This normalization simplifies logarithms: once the ratio is $1 + O(1/\lambda)$, its logarithm is controlled by a standard $\log(1+u)$ estimate.

17.6 Cumulants and derivative bounds

The logarithm of the negative-Laplace product has derivatives expressible as dyadic sums of kernel derivatives. `LaplaceCumulantAsymptotics.lean`, `FabiusLambertDerivativeBounds.lean`, and related modules prove uniform estimates for these cumulants at the saddle.

The same cumulant module packages Bell-polynomial inversion of normalized Laplace moments through order four. The first identity is already exact without side conditions:

$$\mathrm{normalizedLaplaceMoment}_1(s) = -L'(s).$$

The second derivative determines the Gaussian width. Higher derivatives determine correction polynomials. The source proves bounds of the form

$$\kappa_r(\lambda) = O(\lambda^{1-r/2})$$

after the chosen normalization. Exact exponents vary with convention, but the hierarchy guarantees that each additional Taylor order contributes another inverse power of the large phase.

Two vertical-log modules make the derivative interface explicit. For every m, `exists_norm_iteratedDeriv_negativeLaplaceVerticalLog_rpow_le_add_one` supplies $C \geq 0$ such that the $(m+1)$-st vertical derivative at radius 2^b is bounded by $C(b+1)$ for $b \geq 0$ and $|\theta| \leq 1$, including the base radius $2^0 = 1$. The legacy Cb estimate is retained for $b \geq 1$. At any radius $r > 0$, `iteratedDeriv_negativeLaplaceVerticalLog_at_zero_eq_ordinary_all` identifies every zero-axis jet: order zero is normalized to zero, while order $n > 0$ is $(r\mathrm{i})^n$ times the n-th ordinary derivative of the negative-Laplace logarithm at r.

17.7 Minor arcs and tail decay

A complex product may have oscillatory peaks away from zero. `FabiusLambertMinorArc.lean` proves that, outside the central window, enough sinc/Bose factors lose modulus to force decay. The proof divides the frequency axis into ranges:

1. a near-central range where a quadratic cosine or exponential inequality applies;
2. an intermediate range where a fixed block of dyadic factors supplies uniform loss; and
3. a far range controlled by elementary transform decay and integration by parts.

Each range has a separate numerical inequality, making the final integral estimate a finite case split rather than an opaque global bound.

17.8 The first correction

`FabiusFirstSaddleCorrection.lean` computes the first non-Gaussian term. Cubic and quartic derivatives of the phase contribute through Gaussian moments. Odd terms integrate to zero against the even Gaussian; the first surviving coefficient combines the quartic term and the square of the cubic term.

The code represents the correction as a polynomial in the rescaled central variable. Exact Gaussian moments are evaluated recursively or through double factorials. The resulting rational coefficient is then inserted into the normalized saddle mass expansion.

17.9 The corrected sharp formula

`FabiusSharpAsymptotic.lean` states the sharp logarithmic asymptotic in the repository's exact normalization. Writing Ψ for the bounded nonconstant period-one correction and C_* for the proved normalization constant, its main term is

$$\log F(x) = -\frac{\log 2}{2}\lambda(x)^2 + \left(1 + \frac{\log 2}{2}\right)\lambda(x) - \frac{1}{2}\log \lambda(x) + C_* + \Psi\big(\lambda(x)\big) + O\left(\frac{1}{\lambda(x)}\right),$$

where $\lambda(x)$ is the exact lower-Lambert coordinate. In particular, the quadratic coefficient is $-\log 2/2$; changing logarithm bases without changing the definition of λ would give a different and incorrect coefficient.

Exponentiating gives an asymptotic equivalence for $F(x)$ itself. The proof first bounds the logarithmic error, then uses continuity of the exponential and the fact that $e^{O(1/\lambda)} = 1 + O(1/\lambda)$.

17.10 Failure of an uncorrected elementary formula

A simpler formula obtained by dropping the periodic term may still reproduce the leading $(\log x)^2$ scale. It does not have a residual of order $1/(-\log x)$. The repository proves this by choosing two phase subsequences along which the periodic correction approaches different values. The residual therefore has distinct limit points and cannot tend to zero.

Similarly, a proposed quotient of elementary exponentials is shown to decay on the wrong scale. `FabiusQuotientExponentialMismatch.lean` compares logarithms and proves the quotient is not asymptotically equivalent to F, even if a finite numerical range looks persuasive.

Architectural point. The sharp theorem does not emerge from manipulating the delay equation alone. The delay equation explains the coarse quadratic scale. The periodic term and correct constant require the exact transform product; the Gaussian factor and inverse-phase corrections require Bromwich inversion and saddle analysis.

---

18 All-order asymptotic expansions

18.1 Why the exact phase is retained

An all-order expansion can be expressed either in the exact saddle variable $\lambda(x)$ or directly in powers of $-\log x$ and iterated logarithms. The repository takes the exact-phase expansion as primary. This avoids a common error: composing two finite asymptotic series and claiming an arbitrary-order remainder without proving uniform control of the composition.

The exact-phase theorem has the form

$$\mathcal{M}(x) \sim \sum_{j\geq 0} \frac{c_j(\lambda(x))}{\lambda(x)^j},$$

where $\mathcal{M}(x)$ is the normalized saddle mass and each c_j is a bounded period-one coefficient function. For every N, the difference from the first N terms is $O(\lambda^{-N})$. Replacing λ by a finite elementary-log approximation inside these oscillatory coefficients requires a separate composition theorem and is not part of this statement.

18.2 Formal logarithm of the Laplace product

`FabiusLambertFormalLog.lean` extracts a formal series for the local phase. It treats logarithms, exponentials, and reciprocal series in a truncated polynomial ring. The goal is algebraic: compute coefficients that must occur if a sufficiently smooth analytic remainder is controlled.

Separating formal algebra from analytic remainder estimates is essential. Formal power-series identities can be checked by ring normalization and coefficient extensionality. Their analytic validity is supplied later by bounds on the omitted terms.

18.3 Higher small-argument expansion

`FabiusLambertHigherExpansion.lean` and `FabiusLambertAllOrderSmallArgument.lean` prove the expansion of the exact Lambert phase in terms of

$$L = \log(1/x), \qquad \ell = \log L.$$

For each fixed order, λ is represented by a polynomial in $1/L$ whose coefficients are polynomials in ℓ, plus a rigorously bounded remainder.

The proof uses the implicit saddle equation. An approximate phase is substituted into the equation; formal algebra shows cancellation through the desired order. Monotonicity and a derivative lower bound for the implicit function turn the equation residual into a bound on the phase error.

18.4 Gaussian polynomial contractions

The central exponential has an expansion

$$e^{-u^2/2}\exp\left(\sum_{r\geq 3}\lambda^{1-r/2}a_r(iu)^r\right).$$

Truncating the second exponential produces polynomials in u and $1/\sqrt{\lambda}$. Odd monomials vanish after integration; even monomials contract to Gaussian moments. The modules `GaussianPolynomialContraction.lean`, `GaussianPolynomialWholeIntegral.lean`, and the tail modules formalize this mechanism.

A polynomial is represented by finitely supported coefficients. The contraction functional maps u^{2k} to $(2k-1)!!$ and odd powers to zero. Proving that contraction commutes with finite sums and scalar multiplication turns the central coefficient calculation into exact algebra.

18.5 Coefficient recurrences

`FabiusSaddleCoefficientRecurrence.lean`, `FabiusSaddlePolynomialCoefficients.lean`, and `FabiusSaddleExpansionCoefficients.lean` organize the coefficients recursively. Each order depends only on finitely many cumulants and lower coefficients. This recurrence remains the executable foundation even though newer modules also solve its Fabius-specific pieces in closed form.

The recurrence is justified by differentiating or multiplying truncated formal series and comparing coefficients. Lean's finite-support polynomial representation ensures every coefficient sum is finite. Index constraints are handled with `Finset.antidiagonal` or explicit bounded ranges.

`FabiusSaddleJetClosedForm.lean` solves the differential recurrence for every periodic saddle jet, while `FabiusSaddleExponentClosedForm.lean` reduces each exponent coefficient to one jet term plus a universal jet-free tail. The jet weights are the coefficients of

$$\prod_{k=1}^{n}(X-k).$$

With signed first-kind Stirling numbers defined by $X(X-1)\cdots(X-n+1)=\sum_m s(n,m)X^m$, `FabiusSaddleJetStirling.lean` identifies the weight at X^m as $s(n+1,m+1)$, not $s(n,m)$. Thus the closed form is finite and machine-checkable, while the recurrence remains the convenient generator. `FabiusSaddleLeadingCoefficient.lean` additionally isolates the top-jet coefficient of every saddle *mass* coefficient. Its header notes the analogous logarithmic transfer, but that transfer is not proved in this module.

18.6 Universal endpoint-transfer polynomials and finite expectations

Two explicitly post-snapshot modules isolate the finite algebra behind an endpoint-transfer step. `EndpointTransferPolynomials.lean` defines universal polynomials $P_r\in\mathbb{Q}[X]$ by the formal identity

$$\exp\left(-\sum_{j\geq 2}\frac{X^j t^{j-1}}{j}\right)=\sum_{r\geq 0}P_r(X)t^r.$$

Theorems `endpointTransferPolynomial_succ`, `endpointTransferPolynomial_eq_partitionExpSum`, and `endpointTransferSeries_eq_exp_subst` connect the recursive, weighted-partition, and formal-series descriptions. The evaluation theorems `aeval_endpointTransferPolynomial` and `map_endpointTransferSeries` show that the construction is natural in

every commutative rational algebra. These are formal-algebra statements: they assert no analytic convergence, branch choice, or remainder estimate.

`PolynomialExpectationCumulant.lean` supplies the complementary finite integration bridge. The theorem `integral_eval₂_eq_sum_moment` expands the integral of a polynomial evaluation into coefficient-weighted raw moments, assuming integrability only for powers in the polynomial's support. The theorem `integral_eval₂_eq_sum_completeBell_momentCumulant_with_mass_correction` rewrites those moments through the complete Bell transform of their formal cumulants and adds the exact constant-term correction $\varphi(p_0)(m_0-1)$. The normalized theorem `integral_eval₂_eq_sum_completeBell_momentCumulant_of_moment_zero_eq_one` and the probability-measure specialization `integral_eval₂_eq_sum_completeBell_momentCumulant` remove that correction. Taking $p = P_r$ composes the two finite interfaces, but neither module proves the analytic endpoint-transfer expansion, Taylor remainders, or tail estimates.

18.7 Geometric principal specialization and all residual moments

At post-snapshot checkpoint `1e761ed77583e9870ceeaeb2c74ac824094f0f3f`, the finite interpolation algebra acquires a denominator-free geometric endpoint. Five reusable bridges in `CompleteHomogeneous.lean` are `completeHomogeneousEval_smul`, `completeHomogeneousEval_option_zero`, `completeHomogeneousEval_fin_succ`, `completeHomogeneousEvalOn_range`, and `completeHomogeneousEvalAt_zero`. They record homogeneity, removal of an adjoined zero variable, the head–tail recurrence on a nonempty `Fin` family, reindexing of a natural-number range by `Fin`, and specialization of the distinguished target to zero.

The later `CompleteHomogeneousGenerating.lean` packages these evaluations as the formal power series `completeHomogeneousGeneratingSeriesOn`. Its coefficient and empty-family interfaces are `coeff_completeHomogeneousGeneratingSeriesOn` and `completeHomogeneousGeneratingSeriesOn_empty`, while `completeHomogeneousGeneratingSeriesOn_insert` is the adjoining-variable geometric recurrence over every commutative semiring. Over a commutative ring, `one_sub_mul_completeHomogeneousGeneratingSeriesOn_insert` clears one linear denominator, `prod_one_sub_mul_completeHomogeneousGeneratingSeriesOn` clears the whole finite denominator product, and `completeHomogeneousGeneratingSeriesOn_eq_invOfUnit_prod` identifies the coefficient series with its canonical formal reciprocal. These are finite formal-series identities, valid without distinct values, division, cancellation, or domain hypotheses. They assert no analytic evaluation or convergence and do not prove the Bell/generalized-harmonic power-sum presentation.

The separate `CompleteHomogeneousBell.lean` closes exactly this finite conversion. For a finite family a_i, it defines `completeHomogeneousPowerSum` and `completeHomogeneousBellInput` by

$$p_k = \sum_i a_i^k, \qquad \kappa_0 = 0, \qquad \kappa_{k+1} = k!\, p_{k+1}.$$

Its strongest theorem is the division-free commutative-semiring identity

$$\mathrm{Bell.\,complete}(\kappa)_n = n!\, h_n(a),$$

named `bellComplete_completeHomogeneousBellInput`. Only after passing to a commutative $\mathbb{Q}$-algebra does `completeHomogeneousEvalOn_eq_factorialNormalize_completeBellPolynomial` state the report form

$$h_n(a) = \mathrm{factorialNormalize}\big(\mathrm{completeBellPolynomial}(\kappa)\big)_n.$$

Thus the generic theorem retains positive characteristic, zero divisors, nilpotents, repeated values, and the zero ring; the normalized report theorem uses the rational-algebra layer explicitly. Neither declaration is an analytic generating-series or convergence theorem.

Over every commutative semiring, without division or a hypothesis on q, `GeometricCompletHomogeneous.lean` proves

$$h_n(1, q, \dots, q^p) = \begin{bmatrix} n+p \\ p \end{bmatrix}_q, \qquad h_n(c, cq, \dots, cq^p) = c^n \begin{bmatrix} n+p \\ p \end{bmatrix}_q.$$

The exact declarations are `completeHomogeneousEval_geometric`, `completeHomogeneousEval_scaled_geometric`, and the range-indexed bridge `completeHomogeneousEvalOn_range_pow_eq_gaussianBinomial`.

The commutative-ring theorems in `GeometricResidualMoments.lean` assume polynomial exactness directly on the displayed scaled nodes:

$$\sum_{k=0}^{p} w_k (cq^k)^d = 0^d \quad (0 \le d \le p).$$

They then give, for $r \ge 0$ and $m > 0$,

$$\sum_{k=0}^{p} w_k (cq^k)^{p+1+r} = c^{p+1+r}(-1)^p q^{\binom{p+1}{2}} \begin{bmatrix} p+r \\ p \end{bmatrix}_q, \qquad \sum_{k=0}^{p} w_k (cq^k)^m = c^m (-1)^p q^{\binom{p+1}{2}} \begin{bmatrix} m-1 \\ p \end{bmatrix}_q.$$

The theorem with this direct scaled-node hypothesis is strictly more general than a result that first assumes exactness before dilation: it remains valid when c is zero or a zero divisor. The unscaled declarations are `sum_weight_mul_geometric_pow_succ_add` and `sum_weight_mul_geometric_pow_of_pos`; the maximally general scaled forms are `sum_weight_mul_scaled_geometric_pow_succ_add` and `sum_weight_mul_scaled_geometric_pow_of_pos`.

For a field, finite-node injectivity is used only to obtain those low moments from the geometric Lagrange row. The five resulting declarations are `sum_geometricLagrangeWeight_mul_pow_succ_add`, `sum_geometricLagrangeWeight_mul_pow_of_pos`, `sum_geometricLagrangeWeight_mul_scaled_geometric_pow_of_pos`, and `sum_geometricLagrangeWeight_mul_shifted_pow_of_pos`, together with `sum_geometricLagrangeWeight_mul_eval_scaled_geometric`. The last theorem says, for every polynomial P of natural degree at most p,

$$\sum_{k=0}^{p} \lambda_{p,k}^{(q)} P(cq^k) = P(0),$$

under injectivity of $k \mapsto q^k$ on the finite node range. Its scale c is arbitrary, including zero, so it subsumes the manuscript's $c \neq 0$ case. Along with the scaled positive-moment theorem it makes `cor:scaled-geometric-moments` Exact by composition. In the shifted form the nodes are $q^{start+k}$, and the exponent of q is exactly $start\, m + \binom{p+1}{2}$. These seventeen declarations are finite algebra: by themselves they assert no analytic Richardson remainder, convergence rate, or sign bound for a sampled function.

18.8 The recursive geometric-uniform moment polynomial

The source-only algebraic leaf `GeometricUniformMomentPolynomial.lean` has one public definition, `geometricUniformMomentPolynomial`, and exactly eight public theorems. The base and residual-product recurrence are `geometricUniformMomentPolynomial_zero` and `geometricUniformMomentPolynomial_succ`; `geometricUniformMomentPolynomial_natDegree_le` proves the triangular bound, and `geometricUniformMomentPolynomial_eval_zero` gives the reciprocal factorial value at zero. The remaining four theorems, `geometricUniformMomentPolynomial_one` through `geometricUniformMomentPolynomial_four`, are the displayed small

polynomials. The finite residual product makes this recursively defined family meaningful at $q = 0$, $q = 1$, and roots of unity without a quotient.

The companion `GeometricUniformMomentPolynomialBridge.lean` has no public definitions and exactly one public theorem, `geometricUniformMomentPolynomial_eval₂_eq_mgf_taylorCoefficient`. For every real q with $|q| < 1$ and every n, it identifies the recursive polynomial with the finite-q-Pochhammer normalization of the n-th Taylor coefficient of the genuine geometric-uniform moment generating function. This exhaustive $0+1$ bridge makes `p7:eq:Pn-def` Exact in the real contraction regime, including signed q and $q = 0$.

The subsequent source-only leaf `GeometricUniformComplexMomentProduct.lean` has the exhaustive public surface one definition, `geometricUniformComplexMomentProduct`, and three theorems, `hasProdLocallyUniformly_geometricUniformComplexMomentProduct`, `differentiable_geometricUniformComplexMomentProduct`, and `geometricUniformMomentPolynomial_eval₂_eq_complexMomentProduct_taylorCoefficient`. For every complex q with $\|q\| < 1$, it constructs the actual infinite product, proves locally uniform convergence on the complex plane, and proves complex differentiability on the whole plane, then identifies the recursive polynomial with the finite-q-Pochhammer normalization of its n-th Taylor coefficient. This is a complex analytic analogue, not a complex probability-moment extension of `p7:eq:Pn-def`; that label remains Exact only in the stated real contraction regime.

The next source-only leaf `GeometricUniformExteriorComplexMomentGerm.lean` also has the exhaustive surface one definition, `geometricUniformExteriorComplexMomentGerm`, and two theorems, `analyticAt_geometricUniformExteriorComplexMomentGerm` and `geometricUniformMomentPolynomial_eval₂_eq_exteriorComplexMomentGerm_taylorCoefficient`. For every complex q with $1 < \|q\|$, it defines the actual reciprocal germ $(\mathcal{A}_{q^{-1}}(-z))^{-1}$, proves analyticity at the origin, and identifies its normalized Taylor coefficient with the same recursive polynomial. No boundary claim at $\|q\| = 1$ is made by this analytic leaf.

The sharp-degree source-only leaf `GeometricUniformMomentPolynomialDegree.lean` has no public definitions and exactly three public theorems: `coeff_geometricUniformMomentPolynomial_choose_two`, `coeff_geometricUniformMomentPolynomial_choose_two_sub_one`, and `geometricUniformMomentPolynomial_natDegree_eq`. If $T_n = \binom{n}{2}$, they give $[q^{T_n}]\mathcal{P}_n = B_n(1)/n!$, $[q^{T_n-1}]\mathcal{P}_n = -B_n(1)/n! + B_{n-1}(1)/(2(n-1)!)$ for $n \geq 2$, and exact degree T_n for even n or $n = 1$, versus $T_n - 1$ otherwise. Lean writes $B_n(1)$ as `bernoulli'n`; equivalently it is $(-1)^n B_n$ for Mathlib's negative convention $B_1 = -1/2$. Together with the algebraic and analytic leaves above, this makes `p7:thm:Pn` Exact, and it makes the q-monograph proposition `prop:qF-P-degree-sharp` Exact.

The final rational-coefficient leaf `GeometricUniformMomentRatFunc.lean` has one public definition, `geometricUniformMomentRatFunc`, and exactly four public theorems: `qFactorial_mul_geometricUniformMomentRatFunc`, `eval_geometricUniformMomentRatFunc_eq_complexMomentProduct_taylorCoefficient`, `eval_geometricUniformMomentRatFunc_eq_exteriorComplexMomentGerm_taylorCoefficient`, and `eval_geometricUniformMomentRatFunc_one`. It defines the single rational coefficient $a_n = \mathcal{P}_n/[n]_q!$, proves globally in `RatFunc` that $[n]_X! a_n = \mathcal{P}_n$, and identifies its safe evaluations with the actual inner and exterior Taylor coefficients under $\|q\| < 1$ and $1 < \|q\|$. At $q = 1$ it uses the continued polynomial identity $[n]_1! = n!$, not the literal totalized $(q;q)_n/(1-q)^n$, whose positive-degree scalar presentation is $0/0$. Together with the preceding algebra, this exhaustive $1+4$ surface makes the canonical q-monograph `thm:qF-moment-polynomial` Exact. It does not evaluate through genuine poles at nontrivial roots of unity or assert a unit-circle analytic continuation; the broader MGF, germ-uniqueness, and pole-divisor claims remain Partial at those boundaries.

18.9 Reciprocity of the inner and exterior moment germs

The source-only `GeometricUniformMomentReciprocity.lean` leaf has exactly one public definition, `geometricUniformComplexMomentGerm`, and five public theorems: `geometricUniformComplexMomentGerm_of_norm_lt_one`, `geometricUniformComplexMomentGerm_of_one_lt_norm`, `analyticAt_geometricUniformComplexMomentGerm`, `geometricUniformComplexMomentGerm_reciprocity`, and `geometricUniformComplexMomentGerm_moment_convolution`. The combined germ is the entire inner product when $\|q\| < 1$ and the exterior reciprocal germ when $1 < \|q\|$. For $q \neq 0$ and $\|q\| \neq 1$, the reciprocity theorem states as an `EventuallyEq` at zero that

$$M_q(z)M_{q^{-1}}(-z) = 1.$$

This filter-level conclusion is the precise Lean form of equality as analytic germs. It is intentionally not a global pointwise identity: the relevant inner product is one at zero, so continuity makes it nonzero on a neighbourhood, but it can vanish farther away and Lean's inverse is total at those zeros.

For the proof, if $\|q\| < 1$, then $1 < \|q^{-1}\|$ and the definition of the exterior branch rewrites $M_{q^{-1}}(-z)$ as $M_q(z)^{-1}$; continuity supplies the neighbourhood on which cancellation is legitimate. If $1 < \|q\|$, apply the same argument to q^{-1} with the factors reversed. Differentiating this local identity, applying Leibniz, and differentiating the negation map gives

$$\sum_{k=0}^{n} \binom{n}{k} (-1)^{n-k} M_q^{(k)}(0) M_{q^{-1}}^{(n-k)}(0) = \delta_{n0}.$$

These derivatives are the manuscript's d_n, so the theorem and its coefficient form make `thm:qF-reciprocity` Exact. No claim is made at $q = 0$ or $\|q\| = 1$, on an explicit maximal disc, or about a global reciprocal identity, Mahler-germ uniqueness, or the exterior pole divisor.

18.10 Closed Rvachev survival and positive raw moments

The existing module `ProbabilityLaplaceMoments.lean` now adds exactly two public theorems. The first, `weightedSumDistribution_real_Ici_eq_rvachevUp_of_nonneg`, states for every bounded Fabius solution and every $t \geq 0$ that

$$\mathrm{weightedSumDistribution}([t, \infty)) = \mathrm{up}(t).$$

It obtains the closed ray from the existing strict-survival identity using the proved nullity of every singleton. Together with the global reflection identity and the nonnegative-ray formula $\mathrm{up}(t) = 1 - F(t)$, this makes the q-monograph label `prop:up-tail` Exact with its literal $\mathbb{P}(X \geq t)$ convention. The second theorem, `integral_pow_weightedSumDistribution_eq_mul_intervalIntegral_rvachevUp`, states for every natural $n \geq 1$ that

$$\int_{\mathbb{R}} x^n \, \mathrm{d}\mu_X(x) = n \int_0^1 t^{n-1} \, \mathrm{up}(t) \, \mathrm{d}t.$$

It is the exact full-law expectation in `cor:up-moments`. The restriction $n \geq 1$ is essential: at $n = 0$ the left side is one and the displayed right side is zero. Neither theorem adds a fractional-, complex-, or general-q moment statement.

18.11 Fixed-order phase-locked Lambert extraction

The later current-tree phase-extraction tranche supplies the analytic layer for the actual logarithmic Fabius samples. At the exact lower-Lambert nodes

$$x_{\lambda,j} = (\lambda + j)2^{-(\lambda+j)},$$

`FabiusLambertPhaseLockedPullback.lean` pulls back the full small-argument expansion one fixed node at a time. Periodicity freezes `negativeLaplacePsi` and every saddle-log coefficient at the common base phase λ, while the powers retain the shifted scale $(\lambda + j)^{-1}$.

Before any estimate is used, `LambertPhaseLockedBell.lean` specializes the finite Bell algebra to the shifted reciprocal alphabet. Its definitions `shiftedReciprocalPowerSum` and `shiftedReciprocalBellInput` encode

$$p_k^{(r)}(\lambda) = \sum_{j=0}^{r} (\lambda + j)^{-k}, \qquad \kappa_{k+1}^{(r,\lambda)} = k!\, p_{k+1}^{(r)}(\lambda).$$

The exact declaration `completeHomogeneousEvalOn_shiftedReciprocal_eq_bell` converts the complete homogeneous evaluation to the factorially normalized complete Bell polynomial. The all-offset and $s \geq 1$ Lagrange-moment forms are `sum_shiftedReciprocalLagrangeWeight_mul_invPow_card_add_eq_bell` and `sum_shiftedReciprocalLagrangeWeight_mul_invPow_eq_bell`. These normalized identities are stated over characteristic-zero fields (the direct complete-homogeneous rewrite uses the rational-algebra structure). Their shifted-reciprocal definitions use total field inversion, so they impose no positivity or nonvanishing assumption on λ. They remain finite algebra and provide no asymptotic control.

Two independent asymptotic tools make the finite extrapolation legitimate. `completeHomogeneousEvalOn_isBigO_pow` in `CompleteHomogeneousAsymptotics.lean` carries coordinatewise $\mathcal{O}(g)$ estimates through a fixed degree-n complete homogeneous evaluation, including $n = 0$. In `LambertReciprocalAsymptotics.lean`, the reciprocal nodes are Θ-equivalent to λ^{-1}, each fixed row coefficient and the row's total variation are $\mathcal{O}(\lambda^r)$, and `shiftedReciprocalLagrangeWeight_mul_invPow_isBigO_atTop` absorbs that growth into a sufficiently high inverse power.

Let $Y_j(\lambda)$ be the logarithmic sample with the nonperiodic Wikipedia/Lambert main term removed, and let $E_r(\lambda)$ be its order-r reciprocal-Lagrange combination. These are `fabiusPhaseLockedReducedSample` and `fabiusPhaseLockedPeriodicEstimator`; the exact higher moments and their finite sum are `fabiusPhaseLockedResidualTerm` and `fabiusPhaseLockedResidualPartialSum`. Write Ψ for `negativeLaplacePsi`. The leaf module `FabiusLambertPhaseExtractionBell.lean` leaves both definitions unchanged and proves the exact declaration

$$\begin{aligned} R_{r,n}(\lambda) = (-1)^r &\left(\prod_{j=0}^{r} (\lambda + j)^{-1} \right) \\ &\times \mathrm{factorialNormalize}\big(\mathrm{completeBellPolynomial}(\kappa^{(r,\lambda)})\big)_n \\ &\times c_{r+1+n}(\lambda). \end{aligned}$$

as `fabiusPhaseLockedResidualTerm_eq_bell`; `fabiusPhaseLockedResidualPartialSum_eq_sum_bell` rewrites each term in the exact finite partial sum. These are algebraic wrappers, not new analytic claims.

For every fixed $r, S \in \mathbb{N}_0$,

$$E_r(\lambda) - \Psi(\lambda) - \sum_{n=0}^{S-1} R_{r,n}(\lambda) = \mathcal{O}\big(\lambda^{-(r+1+S)}\big) \qquad (\lambda \to +\infty).$$

This is `fabiusPhaseLockedPeriodicEstimator_sub_residual_isBigO` in `FabiusLambertPhaseExtraction.lean`; its proof requests the single-node expansion at order $2r + S + 1$, exactly compensating for the $\mathcal{O}(\lambda^r)$ row. The same module gives the $\mathcal{O}(\lambda^{-(r+1)})$ estimator bound without residual subtraction, the one-order improvement after subtracting the first omitted moment, and convergence to `negativeLaplacePsi` at a fixed phase along $\lambda = a + n$. The result is a fixed-r, S Poincaré statement: it does not assert growing-order uniformity, convergence of the formal complete-homogeneous generating series or of an infinite residual series, a multiplicative or exponentiated Bell

relative-error hierarchy, a higher extractor, or a Lambert sign/bracketing theorem. The three Bell modules close only the finite complete-homogeneous/Bell/generalized-harmonic conversion and the displayed exact residual-term and partial-sum rewrites.

18.12 Unconditionally summable analytic filters

At compiled full-facade checkpoint `22f801337`, `AnalyticSeriesFilter.lean` supplies the pointwise convergent-series counterpart of that finite algebra. For a finite set of nodes ξ_i, weights w_i, coefficients a_m, and a scale z, it defines

$$\mathcal{L}_z(a) = \sum_i \sum_{m\geq 0} w_i\big((\xi_i z)^m a_m\big).$$

Here the displayed products are scalar actions in Lean. The generic declarations `summable_finiteAnalyticSeriesFilter_diagonal` and `finiteAnalyticSeriesFilter_eq_tsum` work over a commutative semiring acting on a normed additive group. If every *weighted* sampled series is unconditionally summable, they prove the exact diagonalization

$$\mathcal{L}_z(a) = \sum_{m\geq 0} \left(\sum_i w_i \xi_i^m\right) z^m a_m.$$

A zero weight therefore creates no convergence obligation at its node.

When the row reproduces a target through degree p, `finiteAnalyticSeriesFilter_eq_head_add_tail_of_exact` splits off the exact finite head and leaves an explicit infinite tail. Its target-zero specialization preserves a_0. Over a commutative ring with continuous constant scalar multiplication (each fixed scalar acts continuously), the geometric theorem `geometricAnalyticSeriesFilter_eq_constant_add_gaussian_tsum` identifies the finite row-moment factor in every residual tail term as

$$(-1)^p q^{\binom{p+1}{2}} \begin{bmatrix} p+r \\ p \end{bmatrix}_q.$$

Assuming $j \mapsto q^j$ is injective on $\{0,\dots,p\}$, the geometric-Lagrange wrapper over a scalar field and its shifted-scale form are `geometricLagrangeAnalyticSeriesFilter_eq_constant_add_gaussian_tsum` and `geometricLagrangeAnalyticSeriesFilter_shifted`. Their summability hypotheses are imposed only at nodes whose Lagrange weight is nonzero.

This is an exact pointwise `tsum` theorem, using Mathlib's unconditional summation semantics. The generic module itself does not cover conditionally convergent boundary series and proves no uniform convergence, error bound, residual sign, or asymptotic acceleration rate. Separately, `RvachevQBinomialFilter.lean` uses the named modes from `AnalyticMoments.lean` to instantiate the filter for the actual infinite Rvachev sinc product, with only finite-power injectivity in the generic complex theorem and no hypothesis at $c = 1/2$. That theorem does not identify the finite prefixes $P_{b,N}$ or their quotient and Bell coefficients; conditional-boundary, sign/bound, uniform/derivative, and asymptotic claims remain separate. A general bridge from an arbitrary formal `PowerSeries` evaluation to these coefficient hypotheses is also still separate work.

18.13 Uniform central remainders

Formal coefficients alone do not prove an asymptotic expansion. `FabiusSaddleCentralAllOrders.lean` bounds the difference between the exact central integrand and its truncated polynomial approximation uniformly on the growing central window.

The window grows slowly enough that the normalized Taylor remainder tends to zero at the requested inverse-phase rate, but fast enough that the omitted Gaussian tail is negligible. Choosing this window is a balancing act. The proof records explicit inequalities among its powers of λ, so later estimates reduce to comparisons of monomials.

18.14 All-order tails

`FabiusSaddleTailAllOrders.lean`, `GaussianPolynomialTailAllOrders.lean`, and `FabiusLambertAllOrderRemainder.lean` show that tail errors are smaller than every prescribed inverse power. Exponential decay is converted into polynomial decay using standard eventual inequalities:

$$e^{-c\lambda^{\alpha}} = O(\lambda^{-N}) \quad \text{for every fixed } N.$$

The source proves such comparisons with explicit thresholds and reuses them through Big-O transitivity.

18.15 Assembling the normalized mass expansion

`FabiusSaddleMassAllOrders.lean` combines central coefficient extraction and tail flatness. For each N, it proves

$$\mathcal{M}(x) - \sum_{j=0}^{N-1} \frac{c_j(\lambda(x))}{\lambda(x)^j} = O(\lambda(x)^{-N}).$$

The theorem is formulated through a structure or predicate such as `HasAsymptoticExpansion`, allowing generic lemmas for truncation, addition, multiplication, and logarithms.

Two generic interfaces make the all-order statement reusable. `SaddleExpansionFlat.lean` proves that, once one full expansion is known, another function has the same coefficient family exactly when their difference is $O(\mathrm{scale}^N)$ for every N; this is the natural uniqueness statement modulo flat remainders without assuming a nondegenerate scale. `SaddleExpansionSmallArgument.lean` transfers full expansions in both directions between the exact logarithmic coordinate and $x \to 0^+$, for scales in arbitrary normed fields and functions valued in arbitrary normed vector spaces.

18.16 From mass to logarithm

Since $c_0 = 1$, the normalized mass is eventually close to one. A formal logarithm of the coefficient series gives

$$\log \mathcal{M}(x) \sim \sum_{j \geq 1} \frac{d_j(\lambda(x))}{\lambda(x)^j},$$

where the d_j are again bounded period-one functions. The analytic proof uses a Taylor theorem for $\log(1+u)$ with a uniform remainder on a small neighborhood. Positivity of the exact mass, inherited from the original integral representation, ensures the logarithm is well-defined.

At the formal level, `SaddleLogExpansionAlgebra.lean` proves that the recursive logarithm and exponential coefficient transforms are exact inverses in every commutative $\mathbb{Q}$-algebra. Unit and zero constant-term hypotheses give identities at all indices; at every positive index the inverse identities need no normalization because the recurrences do not read the zeroth input coefficient. `SaddleLogExpansionPowerSeries.lean` then identifies the recursive logarithm with Mathlib's `PowerSeries.logOf`.

`FabiusSecondSaddleCorrection.lean` computes d_2 explicitly as a finite polynomial in the first four bounded exponent jets, and `FabiusSecondSaddleExpansion.lean` substitutes it into the public three-term statement:

$$\log F(x) = \mathrm{Main}(x) + \frac{A_1(\lambda(x))}{\lambda(x)} + \frac{A_2(\lambda(x))}{\lambda(x)^2} + O\big(\lambda(x)^{-3}\big)\,.$$

Both corrections are explicit continuous period-one functions; later regularity results may provide stronger smoothness where stated. The theorem does not assert that either correction is nowhere zero.

18.17 Transfer to $F(x)$

`FabiusFullAsymptoticExpansion.lean` inserts the logarithmic mass expansion into the exact saddle reduction, adds the periodic and explicit main terms, and absorbs the exponentially small forward tail. The result is an all-order logarithmic expansion for $F(x)$ at zero.

The primary remainder is $O(\lambda^{-N})$. A weaker but simpler corollary uses $\lambda(x) \asymp -\log x$ to obtain

$$O\big((-\log x)^{-N}\big).$$

18.18 First-order and Wikipedia-facing corollaries

Modules with names such as `FabiusWikipediaMain.lean` and `FabiusWikipediaExpansion.lean` package concise formulas suitable for comparison with public summaries. They separate what is correct at leading order from what is false at the claimed next order. The obstruction module demonstrates that a nonconstant dyadic-periodic correction cannot be absorbed into a vanishing error.

18.19 The proof-audit role of the asymptotic aggregator

`PaperFabiusAsymptotic.lean` is not just a theorem list. Its header records which informal claims are supported, which need corrected domains or periodic terms, and which are disproved. The formal development therefore functions as an audit trail:

- coarse log-squared behavior is validated;
- the exact delay relation is restricted to its true domain;
- a claimed overly small residual is refuted;
- the periodic Lambert-phase formula is proved independently through transforms and saddles; and
- arbitrary-order corrections are established at the exact phase.

19 Legendre expansions and polynomial approximation

19.1 Why Legendre polynomials are a natural basis

The up-function is supported on $[-1, 1]$, is even, and is infinitely differentiable. Legendre polynomials are orthogonal on exactly this interval with respect to Lebesgue measure, so they provide a canonical spectral expansion:

$$\mathrm{up}(x) = \sum_{n \geq 0} a_n P_n(x).$$

Evenness forces every odd coefficient to vanish. The repository does not stop at a formal L^2 expansion: it evaluates the even coefficients exactly through moments, proves absolute and uniform convergence, translates the series back to the global Fabius function, and proves least-squares optimality of finite truncations.

19.2 Building the polynomial basis

`LegendrePolynomial.lean` develops the required Legendre facts in the normalization used by the project. The polynomials are defined through Rodrigues' formula or an equivalent derivative representation:

$$P_n(x) = \frac{1}{2^n n!} \frac{\mathrm{d}^n}{\mathrm{d}x^n} (x^2 - 1)^n.$$

The module proves degree, parity, endpoint values, differential equation, and derivative bounds.

A bespoke development is useful because later coefficient formulas require explicit monomial coefficients. A black-box orthogonal-polynomial theorem would provide orthogonality but not necessarily the exact finite sum compatible with the moment sequence.

19.3 Sturm–Liouville orthogonality

The Legendre differential equation is

$$\big((1-x^2)P_n'(x)\big)' + n(n+1)P_n(x) = 0.$$

Multiplying the equation for P_n by P_m, the equation for P_m by P_n, subtracting, and integrating gives

$$\big(n(n+1) - m(m+1)\big)\int_{-1}^{1} P_n(x)P_m(x)\,\mathrm{d}x = 0.$$

Boundary terms vanish because of the factor $1-x^2$. For $m \neq n$, the eigenvalues differ, proving orthogonality.

`FabiusLegendreSeries.lean` carries out this argument using interval integration by parts. The proof explicitly verifies differentiability of the polynomial expressions and endpoint vanishing. The eigenvalue inequality is discharged in natural arithmetic and cast to reals only at the final multiplication step.

19.4 Norms and pointwise bounds

The exact norm is

$$\int_{-1}^{1} P_n(x)^2\,\mathrm{d}x = \frac{2}{2n+1}.$$

The source derives it from Rodrigues' formula and repeated integration by parts, or from a recurrence plus normalization depending on the helper theorem used. This norm fixes the Fourier–Legendre coefficient:

$$a_n = \frac{2n+1}{2}\int_{-1}^{1} \mathrm{up}(x)P_n(x)\,\mathrm{d}x.$$

A bound $|P_n(x)| \le 1$ on $[-1,1]$ is proved from the differential equation and an extremum argument. This simple uniform bound is the last ingredient needed to convert absolute convergence of coefficients into uniform convergence of the function series.

19.5 Vanishing of odd coefficients

The parity theorem states

$$P_n(-x) = (-1)^n P_n(x).$$

Since up is even, the integrand $\mathrm{up}(x)P_n(x)$ is odd when n is odd. A symmetric-interval integral of an odd integrable function is zero. In Lean, the proof either invokes a generic odd-integral lemma or performs the substitution $x \mapsto -x$ and adds the two integral identities.

The resulting series is indexed naturally by P_{2n}. This is not only cosmetic: the exact coefficient formula then uses the even moment M_{2k}, for which the arithmetic normalization and denominator theory are available.

19.6 Exact coefficient evaluation

`FabiusLegendreCoefficients.lean` expands P_{2n} in monomials:

$$P_{2n}(x) = \sum_{k=0}^{n} c_{n,k} x^{2k}.$$

Finite linearity of the integral gives

$$a_{2n} = \frac{4n+1}{2} \sum_{k=0}^{n} c_{n,k} M_{2k}.$$

Every M_{2k} is the exact rational `moment` value, so a_{2n} is rational. The source also substitutes formulas expressing moments through inverse-power Fabius values, producing a coefficient formula entirely in terms of the exact dyadic table.

The proof is layered:

1. establish the explicit polynomial coefficient theorem;
2. turn polynomial evaluation into a finite sum;
3. commute the finite sum with the integral;
4. replace each monomial integral by the analytic moment;
5. identify the moment with the exact rational sequence; and
6. simplify casts and normalizations.

At no point is an infinite series exchanged with the integral during coefficient evaluation.

19.7 Generic convergence infrastructure

`LegendreSeriesConvergence.lean` develops a theorem for smooth functions whose repeated derivatives satisfy suitable endpoint or norm bounds. Integration by parts transfers derivatives from the function to the Legendre eigenstructure, yielding decay of coefficients faster than any fixed power of n.

For the Fabius up-function, compact support and smoothness supply these hypotheses to every order. Consequently the coefficient sequence is absolutely summable. Since $|P_n(x)| \leq 1$, the Weierstrass M-test gives uniform convergence on $[-1, 1]$.

The generic theorem is separated from Fabius-specific coefficient arithmetic. This permits future reuse for other compactly supported smooth functions and keeps the main convergence proof independent of exact moment formulas.

19.8 Identifying the uniform limit

Orthogonality first shows that the partial sums are the L^2 projections of up. Absolute/uniform convergence produces a continuous limit S. To prove $S = \mathrm{up}$, the source may use one of two equivalent routes:

- completeness of Legendre polynomials in $L^2[-1, 1]$, followed by equality of continuous representatives; or
- a generic convergence theorem already identifying the pointwise limit for sufficiently smooth functions.

The final theorem states a `HasSum` at each point, then upgrades to uniform convergence of the function series.

Endpoints are included. This matters because pointwise convergence theorems for orthogonal series often state only interior convergence. The absolute coefficient bound and $|P_n(\pm 1)| = 1$ handle endpoints directly.

19.9 Translated series for the global Fabius function

`FabiusTranslatedLegendreSeries.lean` transports the up-function expansion to an interval such as $[0, 2]$ where a translate agrees with the signed global Fabius function. The transformation replaces $P_{2n}(x)$ by $P_{2n}(x - 1)$ or the corresponding affine normalization.

The module also expands translated Legendre polynomials into ordinary monomials. This produces an explicit power-polynomial series in the translated coordinate, with exact rational coefficients. Uniform convergence follows from the original series under composition with the affine map.

19.10 Least-squares optimality

`FabiusLegendreLeastSquares.lean` proves that the partial Legendre sum is the unique least-squares approximant among polynomials of the corresponding degree. Evenness gives a useful strengthening: the visible even sum through P_{2N} is the full orthogonal projection through degree $2N + 1$, because the next odd coefficient vanishes. It is therefore uniquely least-squares optimal among *all* real polynomials of degree at most $2N + 1$, one degree beyond its own visible degree. Let

$$S_N^{\mathrm{even}} = \sum_{n=0}^{N} a_{2n} P_{2n}.$$

For any polynomial p with $\deg p \le 2N + 1$,

$$\|\mathrm{up} - p\|_2^2 = \|\mathrm{up} - S_N^{\mathrm{even}}\|_2^2 + \|S_N^{\mathrm{even}} - p\|_2^2.$$

This Pythagorean identity follows because $\mathrm{up} - S_N^{\mathrm{even}}$ is orthogonal to every polynomial in that degree class.

The formal proof represents the finite span either as a submodule of L^2-type functions or directly through polynomial coefficients. It proves orthogonality term by term and then expands the square of the norm. Uniqueness follows because equality of errors forces $\|S_N^{\mathrm{even}} - p\|_2 = 0$, hence equality almost everywhere; polynomial continuity upgrades this to pointwise polynomial equality.

19.11 Central cancellation in the finite even-mode synthesis

The existing Legendre translate synthesis has the exact mesh $M = 4^n$ for the even mode. Evaluating it at zero gives the representation-frontier corollary

$$Q_{2n}(0) + 2 \sum_{0<k<M} Q_{2n}(k/M)\,\mathrm{up}(k/M) = (-1)^n \binom{2n}{n}.$$

`RvachevLegendreCentralSum.lean` has no public definitions and exactly three public theorems: `eval_legendrePolynomial_even_zero`, `eval_rvachevLegendreDeconvolutionPolynomial_even`, and `rvachevLegendreCentralSum`. The last quantifies over every `F : BoundedFabius` satisfying `IsFabiusF` and includes $n = 0$.

The proof starts from the finite synthesis block $-2M < k < 2M$. Compact support removes the terms with $M \le |k| < 2M$. The second theorem makes the deconvolved even mode even; evenness of up then pairs k with $-k$, and the central term uses $\mathrm{up}(0) = 1$. Clearing $M = 4^n$ and applying the first theorem, $P_{2n}(0) = (-1)^n 4^{-n} \binom{2n}{n}$, proves the formula. Thus `cor:leg-central-sum` is *Exact/Complete*. This result does not add a Jacobi closed form, all-degree decoder parity or rationality, reverse spectral closure, mesh minimality, or a larger Lagrange right inverse.

19.12 Finite Legendre–Rvachev biorthogonality

The subsequent exhaustive one-definition/one-theorem leaf `RvachevLegendreBiorthogonality.lean` defines `rvachevLegendreAnalysisKernel` by

$$\Lambda_m(c) = \frac{2m+1}{2}\int_{-1}^{1} \mathrm{up}(x-c)P_m(x)\,\mathrm{d}x$$

and proves `rvachevLegendreBiorthogonality`. Thus `def:leg-Lambda` is Exact. For every `F:BoundedFabius` satisfying `IsFabiusF`, natural $M \neq 0$, arbitrary ℓ, m, and $\ell \le \nu_2(M)$, its literal open-block identity is

$$\frac{1}{M}\sum_{-2M<k<2M} Q_\ell^-(k/M)\Lambda_m(k/M) = \delta_{m\ell}.$$

There is no restriction on the analysis degree m. The proof inserts the degree-ℓ finite synthesis on $[-1,1]$, commutes its finite sum with the normalized Legendre integral, and applies orthogonality. Hence `thm:leg-biorthogonality` is *Exact/Complete*, including degree zero. The larger `thm:leg-Lambda` support, smoothness, parity, origin-value, and Fourier–Bessel claims remain Partial, and `cor:leg-biorthogonal-matrices` remains Partial: the reverse projector, its rank and trace, and Cauchy–Binet are not formalized, and the leaf proves no dyadic rationality theorem for Λ_m.

The later module `FabiusLegendreEnergy.lean` closes the coefficient energy side of the same expansion. It defines the polynomial-form blocks $B_n = u_n P_{2n}$ and proves their complete orthogonality, including the diagonal value $2u_n^2/(4n+1)$. It then proves Parseval for up, the exact shifted coefficient tail for every partial-sum squared error in both `HasSum` and `tsum` form, and

$$\int_0^1 F(t)^2\,dt = \sum_{n\ge 0}\frac{u_n^2}{4n+1}.$$

The block is defined directly as a polynomial. These declarations do not formalize its finite up-translate realization, the separate Fourier-product or infinite-sinc integral for this energy, or rationality of its partial sums.

19.13 Executable Gaunt coefficients and finite up-law Gram sums

The post-snapshot module `LegendreGaunt.lean` closes the finite triple-product calculation over the rationals. Its exhaustive public API consists of four definitions, `legendreLebesgueMomentRat`, `legendreGauntRat`, `legendreGaunt`, and `legendreProductLinearizationCoeffRat`, and twelve theorems, `legendreLebesgueMomentRat_even`, `legendreLebesgueMomentRat_odd`, `legendreLebesgueMomentRat_cast`, `legendreGauntRat_eq_momentFunctional`, `legendreGauntRat_cast`, `legendreGauntRat_swap_left`, `legendreGauntRat_swap_right`, `legendrePolynomial_mul_eq_sum_gaunt`, `legendrePolynomialRat_mul_eq_sum_gaunt`, `legendreGauntRat_eq_zero_of_odd_sum`, `legendreGauntRat_eq_zero_of_add_lt`, and `legendreGauntRat_eq_zero_of_triangle_violation`. Writing μ_n^{L} for `legendreLebesgueMomentRat`(n), the definition and its first three theorems say exactly

$$\mu_n^{\mathrm{L}} = \begin{cases} 2/(n+1), & n \text{ even},\\ 0, & n \text{ odd}, \end{cases} \qquad (\mu_n^{\mathrm{L}} : \mathbb{R}) = \int_{-1}^{1} x^n\,\mathrm{d}x.$$

Thus $\mu_{2n}^{\mathrm{L}} = 2/(2n+1)$ and $\mu_{2n+1}^{\mathrm{L}} = 0$. If $p_{n,a} = [X^a]P_n$ denotes the executable rational Legendre coefficient, the rational Gaunt definition is the bounded sum

$$G_{ijk}^{\mathbb{Q}} = \sum_{a=0}^{i}\sum_{b=0}^{j}\sum_{c=0}^{k} p_{i,a}p_{j,b}p_{k,c}\,\mu_{a+b+c}^{\mathrm{L}},$$

while `legendreGaunt`(i,j,k) is

$$G_{ijk} = \int_{-1}^{1} P_i(x)P_j(x)P_k(x)\,\mathrm{d}x, \qquad L_{ijk} = \frac{2k+1}{2}G^{\mathbb{Q}}_{ijk}$$

is `legendreProductLinearizationCoeffRat`(i,j,k). The moment-functional theorem identifies the bounded sum algebraically, and the cast theorem gives $(G^{\mathbb{Q}}_{ijk} : \mathbb{R}) = G_{ijk}$. The two swap theorems exchange the first pair or the last pair of indices. The real and rational product theorems then give, without extra hypotheses,

$$P_iP_j = \sum_{k=0}^{i+j} L_{ijk}P_k$$

in $\mathbb{R}[X]$ and $\mathbb{Q}[X]$, respectively.

All four definitions are total at every $i,j,k \in \mathbb{N}_0$. The only hypotheses in the public theorem surface occur in the three support results: $\mathrm{Odd}(i+j+k)$ implies $G^{\mathbb{Q}}_{ijk} = 0$; $i+j<k$ implies the same; and any strict triangle violation

$$i+j<k \ \vee\ i+k<j \ \vee\ j+k<i$$

implies the same. These are necessary support zeros only. There is no converse support theorem, zero-row Wigner-square identification, factorial closed form, positivity theorem, or asymptotic assertion in `LegendreGaunt.lean` itself. The next focused leaf closes the first four gaps for the total squared zero-row integer-index datum and sharpens support to an equivalence.

The exhaustive public API of `LegendreGauntClosedForm.lean` consists of exactly two definitions, `legendreGauntAdmissible` and `legendreWignerThreeJZeroSqRat`, and exactly twenty-five theorems: `legendreGauntAdmissible_iff_exists_pairwise_add`, `legendreGauntAdmissible_pairwise_add`, `legendreWignerThreeJZeroSqRat_pairwise_add`, `legendreWignerThreeJZeroSqRat_pairwise_add_factorial`, `legendreWignerThreeJZeroSqRat_eq_factorial_of_halfSum`, `legendreWignerThreeJZeroSqRat_eq_zero_of_not_admissible`, `legendreGauntRat_add_boundary`, `legendreGauntRat_add_boundary_eq_two_mul_wignerThreeJZeroSqRat`, `legendreGauntRat_zero_left`, `legendreGauntRat_zero_left_eq_two_mul_wignerThreeJZeroSqRat`, `legendreGauntRat_pairwise_add_eq_two_mul_wignerThreeJZeroSqRat`, `legendreGauntRat_eq_zero_of_not_admissible`, `legendreGauntRat_eq_two_mul_wignerThreeJZeroSqRat`, `legendreGaunt_eq_two_mul_wignerThreeJZeroSqRat`, `legendreWignerThreeJZeroSqRat_pos_iff_admissible`, `legendreWignerThreeJZeroSqRat_nonneg`, `legendreWignerThreeJZeroSqRat_eq_zero_iff_not_admissible`, `legendreGauntRat_pos_iff_admissible`, `legendreGauntRat_eq_zero_iff_not_admissible`, `legendreGaunt_pos_iff_admissible`, `legendreGaunt_eq_zero_iff_not_admissible`, `legendreGauntRat_nonneg`, `legendreGaunt_nonneg`, `legendreProductLinearizationCoeffRat_eq_mul_wignerThreeJZeroSqRat`, and `legendreProductLinearizationCoeffRat_pos_iff_admissible`.

Admissibility means even total degree and the three weak triangle inequalities; it is equivalent to

$$(i,j,k) = (b+c, a+c, a+b) \qquad (a,b,c \in \mathbb{N}_0).$$

The total rational datum $W^{2,\mathbb{Q}}_{ijk}$ is zero outside this support. In pairwise-sum coordinates it is

$$W^{2,\mathbb{Q}}_{b+c,a+c,a+b} = \frac{\binom{2a}{a}\binom{2b}{b}\binom{2c}{c}}{(2(a+b+c)+1)\binom{2(a+b+c)}{a+b+c}},$$

with both pairwise-coordinate and half-sum factorial forms. The half-sum theorem assumes exactly $i+j+k = 2s$ and $i,j,k \le s$. The global identities are

$$G^{\mathbb{Q}}_{ijk} = 2W^{2,\mathbb{Q}}_{ijk}, \qquad G_{ijk} = 2\big(W^{2,\mathbb{Q}}_{ijk} : \mathbb{R}\big), \qquad L_{ijk} = (2k+1)W^{2,\mathbb{Q}}_{ijk}.$$

The square datum and both Gaunt coefficients are positive exactly on the admissible support, vanish exactly off it, and are nonnegative everywhere; L_{ijk} has the same sharp positivity support. Separate central-binomial boundary theorems cover $k = i + j$ and $i = 0$. This total object is only the *squared zero-row integer-index* datum: there is no signed symbol or phase convention, half-integer or nonzero-magnetic-index theory, general $3j/6j/9j$ symbol, Wigner orthogonality, or recoupling identity.

The companion `FabiusLegendreGaunt.lean` converts those product linearizations into finite formulas for the Rvachev Legendre Gram entries. Its exhaustive public API has one definition, `canonicalRvachevFullLegendreCoefficientRat`, and eight theorems, `canonicalRvachevFullLegendreCoefficientRat_even`, `canonicalRvachevFullLegendreCoefficientRat_odd`, `canonicalRvachevFullLegendreCoefficientRat_cast`, `canonicalRvachevFullLegendreCoefficientRat_eq_normalized_moment`, `rvachevLegendreGramEntryRat_eq_sum_full_gaunt`, `rvachevLegendreGramEntryRat_eq_sum_gaunt`, `rvachevLegendreGramMatrixRat_apply_eq_sum_gaunt`, and `upLegendreGramMatrix_apply_eq_sum_gaunt`. If $c_r^{\mathbb{Q}}$ is `canonicalRvachevLegendreCoefficientRat`(r), its full extension is

$$\widehat{c}_k^{\mathbb{Q}} = \begin{cases} c_{k/2}^{\mathbb{Q}}, & 2 \mid k, \\ 0, & 2 \nmid k. \end{cases}$$

The even and odd theorems give $\widehat{c}_{2n}^{\mathbb{Q}} = c_n^{\mathbb{Q}}$ and $\widehat{c}_{2n+1}^{\mathbb{Q}} = 0$. Unconditionally in k, the normalized-moment theorem says that $\widehat{c}_k^{\mathbb{Q}}$ is $(2k+1)/2$ times `momentFunctional` with moment sequence `rvachevRawMomentRat`, evaluated at `legendrePolynomialRat`(k). In abbreviated notation,

$$\widehat{c}_k^{\mathbb{Q}} = \frac{2k+1}{2}\, \mathcal{M}_{\mathrm{Rv}}(P_k^{\mathbb{Q}}).$$

For arbitrary natural i, j, write $R_{ij}^{\mathbb{Q}}$ for `rvachevLegendreGramEntryRat`(i, j). The two rational entry theorems state exactly

$$R_{ij}^{\mathbb{Q}} = \sum_{k=0}^{i+j} \widehat{c}_k^{\mathbb{Q}} G_{ijk}^{\mathbb{Q}} = \sum_{r=0}^{\lfloor (i+j)/2 \rfloor} c_r^{\mathbb{Q}} G_{ij,2r}^{\mathbb{Q}}.$$

The rational matrix theorem supplies the second equality entrywise for every $i, j \in \mathrm{Fin}\, n$.

Exactly two public theorems in this companion require analytic hypotheses. Given F of type `BoundedFabius` and a proof h_F of `IsFabiusF`, `canonicalRvachevFullLegendreCoefficientRat_cast` identifies the real cast of $\widehat{c}_k^{\mathbb{Q}}$ with `rvachevFullLegendreCoefficient`(F, k), and `upLegendreGramMatrix_apply_eq_sum_gaunt` states, writing $D_{n,ij}^F$ for `upLegendreGramMatrix`(F, n, i, j),

$$D_{n,ij}^F = \sum_{r=0}^{\lfloor (i+j)/2 \rfloor} c_r(F)\, G_{ij,2r},$$

where $c_r(F)$ is `rvachevLegendreCoefficient`(F, r), for $i, j \in \mathrm{Fin}\, n$. The other six theorems are unconditional finite identities. No infinite Legendre series is interchanged with integration, and this module itself proves no Wigner-square or factorial formula, Christoffel reconstruction, or new positivity or asymptotic result. The preceding `LegendreGauntClosedForm.lean` supplies the squared integer zero-row datum, factorial forms, sharp support, and positivity independently. The final finite composition leaf makes that substitution; neither companion chooses a signed phase or treats half-integer or nonzero magnetic indices, orthogonality, or recoupling.

`FabiusLegendreGauntClosedForm.lean` has no public definitions and exactly three public theorems: `rvachevLegendreGramEntryRat_eq_two_mul_sum_wignerThreeJZeroSqRat`, `rvachevLegendreGramMatrixRat_apply_eq_two_mul_sum_wignerThreeJZeroSqRat`, and

upLegendreGramMatrix_apply_eq_two_mul_sum_wignerThreeJZeroSqRat. For arbitrary natural i, j, the first is the unconditional rational identity

$$R^{\mathbb{Q}}_{ij} = 2 \sum_{r=0}^{\lfloor (i+j)/2 \rfloor} c^{\mathbb{Q}}_r W^{2,\mathbb{Q}}_{ij,2r}.$$

For $i, j \in \operatorname{Fin} n$, the executable rational matrix theorem is

$$R^{\mathbb{Q}}_{n,ij} = 2 \sum_{r=0}^{\lfloor (i+j)/2 \rfloor} c^{\mathbb{Q}}_r W^{2,\mathbb{Q}}_{ij,2r}.$$

The third theorem assumes exactly F of type BoundedFabius and h_F of type IsFabiusF; for $i, j \in \operatorname{Fin} n$ it states

$$D^F_{n,ij} = 2 \sum_{r=0}^{\lfloor (i+j)/2 \rfloor} c_r(F)\big(W^{2,\mathbb{Q}}_{ij,2r} : \mathbb{R}\big).$$

Thus the exact boundary is finite rational entry, rational matrix, and real up-law matrix sums against the total squared zero-row integer datum. The construction is a finite substitution only: it chooses no signed $3j$ phase and adds no half-integer, nonzero-magnetic-index, general Wigner-symbol, orthogonality, recoupling, or infinite-series result.

19.13.1 Expanded zero-row Wigner-square formula and proof ledger

The limitations just stated remain exact descriptions of the two predecessor modules. The downstream core LegendreGauntClosedForm.lean closes precisely the integer-index, zero-row *squared* problem. It has exactly two public definitions: legendreGauntAdmissible and legendreWign erThreeJZeroSqRat. Write

$$\mathsf{A}(i,j,k) \Longleftrightarrow (i+j+k) \bmod 2 = 0 \ \wedge\ i \le j+k \ \wedge\ j \le i+k \ \wedge\ k \le i+j,$$

put $C_n = \binom{2n}{n}$, and let $s = \lfloor (i+j+k)/2 \rfloor$. The second definition is the total rational datum

$$W^2_{ijk} = \begin{cases} \dfrac{C_{s-i} C_{s-j} C_{s-k}}{(2s+1) C_s}, & \mathsf{A}(i,j,k), \\ 0, & \neg \mathsf{A}(i,j,k). \end{cases}$$

The notation W^2 is expository: Lean defines this square datum directly, not as the square of a separately defined signed symbol.

Its exhaustive public API then has exactly twenty-five theorems, in source order: legendre GauntAdmissible_iff_exists_pairwise_add, legendreGauntAdmissible_pairwi se_add, legendreWignerThreeJZeroSqRat_pairwise_add, legendreWignerThreeJ ZeroSqRat_pairwise_add_factorial, legendreWignerThreeJZeroSqRat_eq_facto rial_of_halfSum, legendreWignerThreeJZeroSqRat_eq_zero_of_not_admissible, legendreGauntRat_add_boundary, legendreGauntRat_add_boundary_eq_two_mul_w ignerThreeJZeroSqRat, legendreGauntRat_zero_left, legendreGauntRat_zero_le ft_eq_two_mul_wignerThreeJZeroSqRat, legendreGauntRat_pairwise_add_eq_t wo_mul_wignerThreeJZeroSqRat, legendreGauntRat_eq_zero_of_not_admissible, legendreGauntRat_eq_two_mul_wignerThreeJZeroSqRat, legendreGaunt_eq_two_m ul_wignerThreeJZeroSqRat, legendreWignerThreeJZeroSqRat_pos_iff_admissible, legendreWignerThreeJZeroSqRat_nonneg, legendreWignerThreeJZeroSqRat_eq_zer o_iff_not_admissible, legendreGauntRat_pos_iff_admissible, legendreGauntRat _eq_zero_iff_not_admissible, legendreGaunt_pos_iff_admissible, legendreGaun

`t_eq_zero_iff_not_admissible`, `legendreGauntRat_nonneg`, `legendreGaunt_nonneg`, `legendreProductLinearizationCoeffRat_eq_mul_wignerThreeJZeroSqRat`, and `legendreProductLinearizationCoeffRat_pos_iff_admissible`.

The first structural theorem gives the exact triangle-coordinate parametrization

$$\mathsf{A}(i,j,k) \iff \exists a,b,c \in \mathbb{N}_0, \qquad i = b + c, \quad j = a + c, \quad k = a + b.$$

If $S = a + b + c$, the two closed forms on this support are

$$W^2_{b+c,a+c,a+b} = \frac{C_a C_b C_c}{(2S+1)C_S} = \frac{(S!)^2(2a)!(2b)!(2c)!}{(2S+1)!(a!)^2(b!)^2(c!)^2}.$$

The separately named half-sum theorem says that, under the explicit hypotheses $i + j + k = 2s$ and $i, j, k \le s$,

$$W^2_{ijk} = \frac{(s!)^2(2(s-i))!(2(s-j))!(2(s-k))!}{(2s+1)!((s-i)!)^2((s-j)!)^2((s-k)!)^2}.$$

Those dominance hypotheses matter because all indices and subtractions in the Lean statement are natural-number operations.

For $G^{\mathbb{Q}}_{ijk} = \texttt{legendreGauntRat}(i,j,k)$, the public boundary formulas are

$$G^{\mathbb{Q}}_{i,j,i+j} = \frac{2C_iC_j}{(2(i+j)+1)C_{i+j}} = 2W^2_{i,j,i+j}, \qquad G^{\mathbb{Q}}_{0,j,k} = \begin{cases} 2/(2j+1), & j = k, \\ 0, & j \neq k, \end{cases} = 2W^2_{0,j,k}.$$

The identification is global, not merely a necessary support statement:

$$G^{\mathbb{Q}}_{ijk} = 2W^2_{ijk}, \qquad \texttt{legendreGaunt}(i,j,k) = 2(W^2_{ijk} : \mathbb{R}).$$

For each of W^2_{ijk}, $G^{\mathbb{Q}}_{ijk}$, and the real Gaunt integral, the API separately proves positivity exactly on A, zero exactly off A, and global nonnegativity. Finally,

$$\texttt{legendreProductLinearizationCoeffRat}(i,j,k) = (2k+1)W^2_{ijk},$$

and that coefficient is positive exactly on $\mathsf{A}(i,j,k)$.

Proof architecture. The proof stays inside the executable rational Legendre layer. Let $\lambda_n = [X^n]P_n = 2^{-n}C_n$. In the finite product linearization of P_iP_j, only its P_{i+j} term reaches degree $i + j$, so

$$\lambda_i\lambda_j = \frac{2(i+j)+1}{2} G^{\mathbb{Q}}_{i,j,i+j}\lambda_{i+j}.$$

Solving this identity gives the degenerate-triangle boundary above. The zero-left law uses the diagonal boundary together with the already proved triangle zeros.

The private engine next derives the rational Bonnet recurrence

$$XP_n = \frac{n+1}{2n+1}P_{n+1} + \frac{n}{2n+1}P_{n-1}.$$

Applying the rational moment functional to $(XP_i)P_jP_k = P_i(XP_j)P_k$ gives

$$\frac{i+1}{2i+1}G^{\mathbb{Q}}_{i+1,j,k} + \frac{i}{2i+1}G^{\mathbb{Q}}_{i-1,j,k} = \frac{j+1}{2j+1}G^{\mathbb{Q}}_{i,j+1,k} + \frac{j}{2j+1}G^{\mathbb{Q}}_{i,j-1,k}.$$

The proposed central-binomial quotient obeys the corresponding recurrence by $(n+1)C_{n+1} = 2(2n+1)C_n$. A nested induction in the pairwise coordinates uses the degenerate-triangle values on its two boundary faces, the recurrence in the interior, and cancellation of a strictly positive rational leading coefficient. Thus $G^{\mathbb{Q}} = 2W^2$ for all pairwise-sum triples. The coordinate equivalence

covers the admissible support; the predecessor parity and strict-triangle zeros settle its complement. Replacing C_n by $(2n)!/(n!)^2$, and using positivity of central binomials, yields the factorial, exact-support, and sign packages. The real result is then the existing cast theorem. In particular, the proof imports no external Wigner integral evaluation or representation-theory theorem. □

The wrapper `FabiusLegendreGauntClosedForm.lean` has no public definition and exactly three public theorems: `rvachevLegendreGramEntryRat_eq_two_mul_sum_wignerThreeJZeroSqRat`, `rvachevLegendreGramMatrixRat_apply_eq_two_mul_sum_wignerThreeJZeroSqRat`, and `upLegendreGramMatrix_apply_eq_two_mul_sum_wignerThreeJZeroSqRat`. The first two are unconditional rational identities:

$$R_{ij}^{\mathbb{Q}} = 2 \sum_{r=0}^{\lfloor (i+j)/2 \rfloor} c_r^{\mathbb{Q}} W_{i,j,2r}^2,$$

first for `rvachevLegendreGramEntryRat` and then entrywise for `rvachevLegendreGramMatrixRat`. The third requires F : `BoundedFabius` and h_F : `IsFabius` F, and for $i, j : \mathrm{Fin}\, n$ states

$$\begin{aligned} &\texttt{upLegendreGramMatrix}(F, n, i, j) \\ &\quad = 2 \sum_{r=0}^{\lfloor ((i:\mathbb{N}_0)+(j:\mathbb{N}_0))/2 \rfloor} \texttt{rvachevLegendreCoefficient}(F, r) \\ &\qquad \times \left(W^2_{(i:\mathbb{N}_0),(j:\mathbb{N}_0),2r} : \mathbb{R} \right). \end{aligned}$$

All three sums are finite, with Lean's actual index set $\texttt{range}(\lfloor (i+j)/2 \rfloor + 1)$. Their proofs simply substitute the global Gaunt identity into the three predecessor sums and move the scalar 2 outside.

The boundary of this API is deliberate and exact. It defines only a total rational square datum for natural angular indices and a zero magnetic row. There is no signed symbol and no Condon–Shortley or other phase convention; in particular, there is no Lean theorem whose left side is the square of a formal signed Wigner term. There are no half-integer or nonzero-magnetic-index symbols, no general $3j$, $6j$, or $9j$ theory, and no Wigner orthogonality or recoupling result. The wrapper is a finite-sum transport only and asserts no infinite Legendre-series interchange.

19.14 Computational significance

The exact coefficient theorem gives a certified way to produce rational polynomial approximants. The least-squares theorem certifies optimality in L^2, while uniform convergence certifies pointwise approximation. This is a rare combination:

- exact arithmetic coefficients;
- kernel-checked analytic convergence; and
- an optimality theorem for every finite truncation.

The evaluator and moment recurrences can be used to compute coefficients without numerical integration.

20 The k-fold Thue–Morse draft audit

20.1 Purpose of the audit

`PaperKFoldThueMorse.lean` collects a separate line of work concerning k-fold Thue–Morse constructions and a related informal draft. Its role is both constructive and diagnostic: it proves exact identities and corrected approximation statements, but it also formalizes counterexamples to several literal claims in the draft.

This part of the repository illustrates a mature use of formalization. A failed proof is not treated as an obstacle to be papered over; the surrounding hypotheses are tested, false versions are refuted, and repaired versions are isolated.

20.2 Exact finite identities

The Thue–Morse modules prove prefix-sum cancellation, long zero runs of finite differences, convolution formulas, and generating-series identities. Repeated finite difference annihilates polynomials below a threshold degree, which explains the high-order cancellation of the sign sequence.

The full row-level symmetry is sharper than the endpoint cancellation. If $k, r, d \in \mathbb{N}_0$, $k \le r$, and $k + d < 2^r$, then

$$\sigma_k(2^r - k - 1 - d) = (-1)^{r-k}\sigma_k(d), \qquad \sigma_k(2^r - k - 1 - d) = 0 \iff \sigma_k(d) = 0.$$

These are exactly `iteratedPrefix_dyadic_reverse_window` and `iteratedPrefix_dyadic_reverse_window_eq_zero_iff` in `ThueMorsePrefix.lean`. At $k = 0$, the identity is the binary complement theorem `thueMorseSign_dyadic_complement` from `DyadicClosedForm.lean`; at $d = 0$, the statement specializes, using `iteratedPrefix_at_zero`, namely $\sigma_k(0) = 1$, to the sharp left boundary `iteratedPrefix_before_dyadic_run` of the final dyadic zero run. The terminal zero run and the value -1 at its right boundary are separate theorems, exposed by `iteratedPrefix_dyadic_endpoint`, `iteratedPrefix_dyadic_zero_reverse`, `iteratedPrefix_dyadic_zero_run`, and `iteratedPrefix_at_dyadic`. In particular, the reflected-zero equivalence pairs zeros in the pre-run window but does not assert that the window is zero-free.

The proof mirrors the finite-difference geometry. Dyadic bit complementation settles order zero, the left boundary seeds offset zero, and a nested induction uses $\sigma_{k+1}(n+1) - \sigma_{k+1}(n) = \sigma_k(n+1)$ to propagate the same signed reflection in both prefix order and offset. Since the sign is always ± 1, the zero-locus equivalence follows without any cancellation hypothesis.

The formal proofs use induction on binary depth and repeated even/odd splitting. Convolution associativity is kept finite, so all sums are over `Finset.range` and can be reindexed explicitly. Generating polynomials turn convolution into multiplication, allowing coefficient comparison to replace nested sum manipulations.

`ThueMorseBinomialLog.lean` also makes the zero–one convention exact. For the integer sum $X(n) = \sum_{k=0}^{n} (-1)^{\binom{n}{k}}$, it proves $n + 1 - X(n) = 2^{\mathrm{wt}(n)+1}$, so the ensuing base-two logarithm is exact. If $t_n = \mathrm{wt}(n) \bmod 2$, the resulting rational identity is

$$t_n = \frac{1 + (-1)^{\log_2(n+1-X(n))}}{2}.$$

The module also identifies, in every ring, $(-1)^{t_n}$ with the cast of the signed Thue–Morse convention used by the real and complex q-series.

20.3 The failed polygon and the corrected grid samples

The literal grid of the draft uses $S_j^k / 2^{\binom{k}{2}}$ at $j/2^k$. At $x = 1$ its value is $-2^{-\binom{k}{2}}$, so neither the grid nor either intended polygonal interpolation can converge there to a function with value one. This is formalized by `paperPrefixGridValue_endpoint_not_tendsto_one`, `paperPrefixPolygon_endpoint_not_tendsto_one`, and `paperPrefixPolygonReal_endpoint_not_tendsto_one` in `ThueMorseGenerating.lean`.

The repaired theorem uses the shifted, dyadic-floor samples `correctedPrefixGridSample`, not a continuous polygon. Away from the right endpoint each sample is exactly `stepApproximantn` at `correctedPrefixCellCenternx`; those centers tend to $2x - 1$. The endpoint is handled separately, giving pointwise convergence on $[0, 1]$ to $\mathrm{up}(2x - 1)$, and sampling at $x/2$ recovers $F(x)$. No quantitative uniform rate or continuity of the sample function is asserted.

20.4 Formal counterexamples

`DraftCounterexamples.lean` gives exact witnesses for three distinct failures. At $k = 2, j = 1$, the proposed slope is -2, whereas $F'(1/4) = 1$; the error is therefore $3 > 4h = 1$, contradicting the claimed local bound. For every $x > 0$, the proxy ratio from equation (7) is

$$\frac{x\,2^{n+1}}{n+1} \longrightarrow +\infty,$$

so the terms are unbounded and the displayed maximum does not exist. Finally, under the draft's equation (8), the Stirling proxy retains an additive $+n$ term; equation (10) cannot absorb it into $O(\log n)$.

These are arithmetic or elementary asymptotic counterexamples, not subsequence arguments imported from the coarse Fabius endpoint theorem. Each negative result is stated independently of the prose whose claim it refutes.

20.5 Repairing the lower Lambert branch

The draft uses a lower branch of a Lambert-type inverse. The repository develops both real branches with explicit domains, order, and inversion theorems; no appeal is made to an ambiguous multivalued inverse. The raw endpoint continuity layer is now closed: in `LowerLambertW.lean`, `lowerLambertW_continuousWithinAt_branchPoint` and `lowerLambertW_continuousOn_Ico` give continuity of W_{-1} on $[-\exp(-1), 0)$, while `PrincipalLambertW.lean` supplies `principalLambertW_continuousWithinAt_branchPoint` and `principalLambertW_continuousOn_Ici` on $[-\exp(-1), \infty)$. At the ordinary point zero, `deriv_principalLambertW_zero` states $\operatorname{deriv} W_0(0) = 1$, and `principalLambertW_isEquivalent_zero` packages $W_0(z) \sim z$. The later module `LambertWCurvature.lean` closes the regular-domain second-derivative and shape layer. On either smooth branch it uses the nonsingular exponential form

$$W''(z) = -\frac{e^{-2W(z)}(W(z)+2)}{(W(z)+1)^3}.$$

For the principal branch, `deriv_principalLambertW`, `deriv_principalLambertW_hasDerivAt`, and `deriv_deriv_principalLambertW` give the first derivative and the derivative-of-derivative formula throughout $(-e^{-1}, \infty)$, including $z = 0$. The simplification `deriv_deriv_principalLambertW_zero` gives $W_0''(0) = -2$, `deriv_deriv_principalLambertW_neg` proves strict negativity on the open domain, and `strictConcaveOn_principalLambertW` extends strict concavity to the closed natural domain $[-e^{-1}, \infty)$.

For W_{-1}, `deriv_lowerLambertW_hasDerivAt` and `deriv_deriv_lowerLambertW` give the same formula on $(-e^{-1}, 0)$. The three exact sign declarations `deriv_deriv_lowerLambertW_pos_iff`, `deriv_deriv_lowerLambertW_neg_iff`, and `deriv_deriv_lowerLambertW_eq_zero_iff` locate the unique change at $z = -2e^{-2}$: the second derivative is positive to its left, negative to its right, and zero exactly there. Consequently `strictConvexOn_lowerLambertW_left` proves strict convexity on $[-e^{-1}, -2e^{-2}]$, while `strictConcaveOn_lowerLambertW_right` proves strict concavity on $[-2e^{-2}, 0)$. These are regular-interior derivative results plus shape theorems that include the finite branch point and shared inflection endpoint; the lower domain remains open at zero.

20.5.1 A global coordinate for the branch pair

The two branches on their common open domain admit a more economical description than two unrelated inverse functions. For $-\exp(-1) < x < 0$, define

$$a = W_0(x), \qquad b = W_{-1}(x), \qquad \delta = a - b > 0.$$

The exact forward identities are

$$a = -\frac{\delta}{\exp(\delta)-1}, \qquad b = -\frac{\delta\exp(\delta)}{\exp(\delta)-1} = -\frac{\delta}{1-\exp(-\delta)},$$
$$x = -\frac{\delta}{\exp(\delta)-1}\exp\left(-\frac{\delta}{\exp(\delta)-1}\right).$$

They come from one cancellation. Since $a\exp(a) = b\exp(b) = x$ and $a, b \neq 0$, division gives $b/a = \exp(a-b) = \exp(\delta)$. Thus $b = a\exp(\delta)$ and $\delta = a - b = a(1-\exp(\delta))$, which yields the first two formulas; the defining equation yields the third.

The converse is just as explicit. For $\delta > 0$, define

$$A(\delta) = -\frac{\delta}{\exp(\delta)-1}, \quad B(\delta) = -\frac{\delta\exp(\delta)}{\exp(\delta)-1}, \quad X(\delta) = A(\delta)\exp(A(\delta)).$$

The strict exponential tangent inequalities give $-1 < A(\delta) < 0$ and $B(\delta) < -1$. Direct algebra gives $B = A - \delta$ and $B\exp(B) = A\exp(A)$, so branch uniqueness proves $W_0(X(\delta)) = A(\delta)$, $W_{-1}(X(\delta)) = B(\delta)$, and $W_0(X(\delta)) - W_{-1}(X(\delta)) = \delta$. Conversely, substituting the gap of x into X returns x. Hence the branch-gap map is a bijection

$$(-\exp(-1), 0) \longleftrightarrow (0, \infty).$$

With $t = \exp(\delta) > 1$, the inverse coordinate becomes

$$W_0 = -\frac{\log t}{t-1}, \qquad W_{-1} = -\frac{t\log t}{t-1}, \qquad x = -\frac{\log t}{t-1}\,t^{-1/(t-1)}.$$

Adding, multiplying, and dividing the paired formulas gives

$$\frac{W_{-1}(x)}{W_0(x)} = \exp(\delta), \qquad W_0(x) + W_{-1}(x) = -\delta\coth(\delta/2), \qquad W_0(x)W_{-1}(x) = \frac{\delta^2}{4\sinh^2(\delta/2)}.$$

For $y > 0$, the inequalities $\sinh y > y$ and $y\coth y > 1$ prove the strict consequences

$$W_0(x) + W_{-1}(x) < -2, \qquad 0 < W_0(x)W_{-1}(x) < 1.$$

The Lean layering follows the proof rather than merging all of this into one opaque theorem. `LambertWBranchPairing.lean` has no definitions and exactly seven theorems: `principalLambertW_sub_lowerLambertW_pos`, `lowerLambertW_eq_principalLambertW_mul_exp_gap`, `principalLambertW_eq_neg_gap_div`, `lowerLambertW_eq_neg_gap_mul_exp_div`, `lowerLambertW_eq_neg_gap_div_one_sub_exp_neg`, `eq_neg_gap_div_mul_exp`, and `principalLambertW_lowerLambertW_eq_of_exp_gap`. `LambertWGapBijection.lean` has the four definitions `gapPrincipal`, `gapLower`, `gapArg`, and `branchGap`, followed by exactly sixteen theorems: `gap_denominator_pos`, `gapPrincipal_mem_Ioo`, `gapLower_eq_mul_exp`, `gapLower_eq_sub`, `gapLower_lt_neg_one`, `gapLower_mul_exp`, `gapArg_mem_Ioo`, `principalLambertW_gapArg`, `lowerLambertW_gapArg`, `branchGap_gapArg`, `gapArg_branchGap`, `branchGap_invOn`, `branchGap_bijOn`, `principalLambertW_gapArg_log`, `lowerLambertW_gapArg_log`, and `gapArg_log`. The zero-definition/nine-theorem `LambertWBranchSymmetry.lean` surface is `lowerLambertW_div_principalLambertW_eq_exp_branchGap`, `principalLambertW_add_lowerLambertW_eq_exp_branchGap`, `principalLambertW_add_lowerLambertW_eq_cosh_div_sinh_branchGap`, `principalLambertW_mul_lowerLambertW_eq_exp_branchGap`, `principalLambertW_mul_lowerLambertW_eq_sinh_sq_branchGap`, `principalLambertW_add_lowerLambertW_lt_neg_two`, `principalLambertW_mul_lowerLambertW_pos`, `principalLambertW_mul_lowerLambertW_lt_one`, and `principalLambertW_mul_lowerLambertW_mem_Ioo`.

The openness of both intervals is substantive. At the finite branch point the gap is zero, the rational formulas have a zero denominator, and their limiting sum and product are -2 and 1, so the strict inequalities do not extend there. At zero the classical lower branch is singular; the theorems make no assertion about its all-real totalization.

The separate leaf `LambertWBranchGapBernoulli.lean` has no definitions and exactly five theorems: `summable_norm_bernoulli_mul_pow_div_factorial`, `summable_bernoulli_mul_pow_div_factorial_iff`, `hasSum_bernoulli_mul_pow_div_factorial`, `hasSum_bernoulli_mul_pow_div_factorial_complex_iff`, and `principalLambertW_lowerLambertW_eq_bernoulliSeries`. They prove absolute summability of $\sum_{n\ge 0} B_n z^n/n!$ for real z with $|z| < 2\pi$, its exact complex convergence locus

$$\sum_{n\ge 0} B_n w^n/n! \text{ is summable} \quad \Longleftrightarrow \quad |w| < 2\pi,$$

its actual real `HasSum` value $z/(\exp z - 1)$ when $z \neq 0$, its actual complex `HasSum` value—the inverse of `complexExpm1Div` at w—throughout that disk, and, on $-e^{-1} < x < 0$ with $0 < \delta < 2\pi$,

$$W_0(x) = -\sum_{n\ge 0} B_n \frac{\delta^n}{n!}, \qquad W_{-1}(x) = -\delta - \sum_{n\ge 0} B_n \frac{\delta^n}{n!}.$$

Together with the preceding three finite branch-coordinate modules, this is an exhaustive four-module surface of four definitions and thirty-seven theorems, forty-one public declarations in all.

The analytic proof is explicit. Put

$$q = \frac{|z|}{2\pi} < 1, \qquad c_r = \frac{B_{2(r+1)}}{2(r+1)(2(r+1))!}.$$

The even-zeta majorant from `BasicBernoulliLog.lean` gives

$$|c_r z^{2(r+1)}| \le \frac{\pi^2}{6} q^{2(r+1)}.$$

The odd Bernoulli numbers beyond B_1 vanish, and an even EGF term is $2(r+1)c_r z^{2(r+1)}$. Hence, for every $n \ge 2$,

$$\left|\frac{B_n z^n}{n!}\right| \le n\frac{\pi^2}{6} q^n.$$

Comparison with the convergent series $\sum nq^n$, plus the two initial terms, proves absolute summability.

For complex w, sufficiency follows from the same comparison because each term norm depends only on $|w|$. For necessity, put

$$Z_{r+1} = \sum_{m=1}^{\infty} m^{-2(r+1)} \ge 1.$$

The exact even-Bernoulli identity used by the Lean proof is

$$\left|\frac{B_{2(r+1)} w^{2(r+1)}}{(2(r+1))!}\right| = 2Z_{r+1}\left(\frac{|w|}{2\pi}\right)^{2(r+1)}.$$

When $|w| \ge 2\pi$, every displayed even-index term norm is at least 2, so the terms fail to tend to zero. Hence the complex series is summable exactly inside the disk and diverges at every boundary or exterior point.

For evaluation set

$$f_n = \frac{B_n z^n}{n!}, \qquad g_n = \frac{z^n}{n!} - \mathbf{1}_{\{n=0\}}.$$

Both sequences are absolutely summable: this was just proved for f, while for g it follows from absolute summability of the exponential series and the finite-support correction. Its sum is $\exp z - 1$. The formal identity

$$\left(\sum_{n\geq 0} \frac{B_n}{n!} X^n\right)(\exp X - 1) = X$$

says, after multiplying the coefficient of degree n by z^n, that

$$\sum_{i+j=n} f_i g_j = \begin{cases} z, & n = 1, \\ 0, & n \neq 1. \end{cases}$$

The absolutely convergent Cauchy-product theorem therefore yields, over $\mathbb{C}$,

$$\left(\sum_{n\geq 0} f_n\right)(\exp z - 1) = z.$$

If $z \neq 0$, this identity itself forces $\exp z - 1 \neq 0$, and division proves the complex quotient evaluation. At $z = 0$, the series reduces to its constant term $B_0 = 1$. The definition `complexExpm1Div` records exactly these two cases: it is $(\exp z - 1)/z$ away from zero and 1 at zero, so its inverse is the canonical removable value of the Bernoulli EGF. The theorem `hasSum_bernoulli_mul_pow_div_factorial_complex_iff` packages this actual value together with the exact open-disk criterion. For the branch wrapper, positivity of δ turns $\delta < 2\pi$ into $|\delta| < 2\pi$; substituting the EGF in $W_0 = -\delta/(\exp\delta - 1)$ and then using $W_{-1} = W_0 - \delta$ proves the paired series.

The last theorem is the exact Lean crosswalk for the compact `eq:pair-Bernoulli-general` in the retained Lambert guide, and the complex `HasSum` equivalence makes the radius-2π, boundary divergence, and canonical-removable reading of `eq:bernoulli-gen` Exact. This does not assert the false literal totalized quotient at the origin—where Lean evaluates $0/(\exp 0 - 1)$ as zero—or holomorphy of a named sum function. No other label in the surrounding corollary is promoted, and higher or convergent Puiseux/logarithmic expansions remain open.

The next two raw-branch leaves resolve the formerly open branch-point geometry. Write

$$b = -\exp(-1), \qquad \Delta(z) = z - b = z + \exp(-1), \qquad \mathcal{S}(z) = \sqrt{2\exp(1)\big(z + \exp(-1)\big)}.$$

All limits here use the right-hand filter $z \downarrow b$. The compiled statements are

$$\operatorname{deriv} W_0(z) \longrightarrow +\infty, \qquad \operatorname{deriv} W_{-1}(z) \longrightarrow -\infty,$$

and, for the secants drawn from the common endpoint $(b, -1)$,

$$\frac{W_0(z) + 1}{z + \exp(-1)} \longrightarrow +\infty, \qquad \frac{W_{-1}(z) + 1}{z + \exp(-1)} \longrightarrow -\infty.$$

In particular neither branch has a finite right derivative: both fail

$$\mathrm{DifferentiableWithinAt}_{\mathbb{R}}(W_j, \mathrm{Ioi}(b), b).$$

The two totalized all-real branch functions also fail

$$\mathrm{DifferentiableAt}_{\mathbb{R}}(W_j, b).$$

The human proof follows the source layering. Endpoint continuity gives $W_j(z) \to -1$, and the inverse derivative is

$$\frac{1}{\exp(W_j(z))(W_j(z)+1)}.$$

Its denominator tends to zero from above on the principal branch and from below on the lower branch, giving the two signed infinite derivative limits. Principal concavity bounds its endpoint secant from below by the moving derivative; lower convexity on the left shape interval bounds its endpoint secant from above by the moving derivative. Those inequalities force the same signed limits for the secants. A standard one-sided derivative-divergence lemma rules out a finite within-set derivative, and ordinary differentiability would imply that prohibited right-hand differentiability.

The complete public surface of `LambertWBranchPointGeometry.lean` is exactly these eight theorems:

Declaration	Formal content
`tendsto_deriv_principalLambertW_branchPoint_atTop`	Principal derivative tends to $+\infty$ from the right.
`tendsto_deriv_lowerLambertW_branchPoint_atBot`	Lower derivative tends to $-\infty$ from the right.
`tendsto_principalLambertW_secantSlope_branchPoint_atTop`	Principal endpoint secant slope tends to $+\infty$.
`tendsto_lowerLambertW_secantSlope_branchPoint_atBot`	Lower endpoint secant slope tends to $-\infty$.
`principalLambertW_not_differentiableWithinAt_branchPoint`	No finite principal right derivative on $\mathrm{Ioi}(b)$.
`lowerLambertW_not_differentiableWithinAt_branchPoint`	No finite lower right derivative on $\mathrm{Ioi}(b)$.
`principalLambertW_not_differentiableAt_branchPoint`	The totalized principal branch is not differentiable at b.
`lowerLambertW_not_differentiableAt_branchPoint`	The totalized lower branch is not differentiable at b.

The asymptotic leaf proves the first square-root term. For the forward map $\phi(w) = w \exp w$, one application of L'Hôpital's rule gives

$$\frac{\phi(w) + \exp(-1)}{(w+1)^2} \longrightarrow \frac{\exp(-1)}{2} \qquad (w \to -1,\ w \neq -1),$$

because the differentiated quotient cancels to $\exp(w)/2$ away from -1. Composing with each branch, rewriting $W_j(z)\exp(W_j(z)) = z$, and inverting the nonzero limit yields

$$\frac{(W_j(z)+1)^2}{z + \exp(-1)} \longrightarrow 2\exp(1), \qquad j \in \{0, -1\}.$$

Thus both squared displacements are asymptotic to $2\exp(1)(z + \exp(-1))$. The branch signs select the square root, with no sign ambiguity:

$$W_0(z) + 1 \sim +\mathcal{S}(z), \qquad W_{-1}(z) + 1 \sim -\mathcal{S}(z) \qquad (z \downarrow b).$$

The complete public inventory of `LambertWBranchPointAsymptotics.lean` is one noncomputable definition and eight theorems:

Declaration	Formal content
`lambertWBranchPointScale`	Definition of $\sqrt{2\exp(1)(z+\exp(-1))}$.
`lambertWBranchPointScale_pos`	Positivity for $b < z$.
`lambertWBranchPointScale_sq`	Exact scale-square identity for $b < z$.
`tendsto_principalLambertW_add_one_sq_div_branchPoint`	Principal squared-ratio limit $2\exp(1)$.
`tendsto_lowerLambertW_add_one_sq_div_branchPoint`	Lower squared-ratio limit $2\exp(1)$.
`principalLambertW_add_one_sq_isEquivalent_branchPoint`	Principal squared-displacement equivalence.
`lowerLambertW_add_one_sq_isEquivalent_branchPoint`	Lower squared-displacement equivalence.
`principalLambertW_add_one_isEquivalent_branchPoint`	Signed principal equivalence to $+\mathcal{S}$.
`lowerLambertW_add_one_isEquivalent_branchPoint`	Signed lower equivalence to $-\mathcal{S}$.

This closes only the leading square-root (first Puiseux) equivalence. No $O(\Delta)$ remainder, higher coefficient, or convergent/full Puiseux expansion has been proved. Named endpoint derivative, secant, and square-root wrappers for the generic power–exponential phases or their Fabius specializations also remain open, as does the generic $m \pm \sqrt{m}$ threshold/sign/shape layer. These two leaves are a current-tree tranche compiled on 30 August 2026: the inventory is exactly eight public geometry theorems and one public asymptotic definition plus eight public asymptotic theorems, excluding private proof helpers. They are recorded without inventing an aggregate commit hash and are not attributed to the older title-page snapshot.

The reusable layer treats

$$S_{m,A,\beta}(\lambda) = A\lambda^m e^{-\beta\lambda}, \qquad P_{m,A,\beta} = A\left(\frac{m}{\beta}e^{-1}\right)^m,$$

under $m \in \mathbb{N}_{>0}$, $A > 0$, and $\beta > 0$. `PowerExponentialLambertCalculus.lean` proves `principalPowerExponentialPhase_continuousOn_Icc` on $[0, P_{m,A,\beta}]$ and `lowerPowerExponentialPhase_continuousOn_Ioc` on $(0, P_{m,A,\beta}]$. Both proofs compose the raw endpoint theorems with the globally continuous normalized argument `powerExponentialLambertArgument_continuous`. For x in the latter interval and $\lambda \geq 0$, `powerExponentialSaddle_eq_iff_eq_principal_or_eq_lower` is the exact equivalence

$$S_{m,A,\beta}(\lambda) = x \quad \Longleftrightarrow \quad \lambda = \lambda_0(x) \text{ or } \lambda = \lambda_{-1}(x).$$

The two values are unequal when $0 < x < P_{m,A,\beta}$, by `principalPowerExponentialPhase_ne_lowerPowerExponentialPhase`. Both classification results live in `PowerExponentialLambertInverse.lean`. The hypothesis $\lambda \geq 0$ matters when m is even, because negative inputs can produce further positive profile values.

The next import, `PowerExponentialLambertAsymptotics.lean`, defines the intrinsic scale

$$\epsilon(x) = \frac{\beta}{m}\left(\frac{x}{A}\right)^{1/m}$$

and proves exactly

$$\lambda_0(x) \sim \left(\frac{x}{A}\right)^{1/m}, \qquad \lambda_{-1}(x) \longrightarrow +\infty, \qquad \lambda_{-1}(x) - M(x) \longrightarrow 0 \quad (x \downarrow 0),$$

where

$$M(x) = -\frac{m}{\beta}\big(\log \epsilon(x) - \log|\log \epsilon(x)|\big).$$

The declarations are `powerExponentialLambertEpsilon`, `principalPowerExponentialPhase_isEquivalent_rpow`, `tendsto_lowerPowerExponentialPhase_nhdsGT_zero_atTop`, `lowerPowerExponentialPhaseIntrinsicMain`, and `lowerPowerExponentialPhase_sub_intrinsicMain_tendsto_zero`. This generic result is intentionally only the intrinsic two-term ϵ-expansion with $o(1)$ remainder. A cleaned $L = \log(A/x)$ formula, additional terms or a complete series, a named generic-phase branch-point vertical-tangent/secant wrapper, and a transported leading square-root law remain outside this tranche. The raw branch-point statements above do not by themselves create those scaled phase wrappers.

The separate `PowerExponentialLambertCurvature.lean` does supply the generic regular-interior second derivative. If λ denotes either scaled branch and $0 < x < P_{m,A,\beta}$, then, under the same hypotheses $m \in \mathbb{N}_{>0}$, $A > 0$, and $\beta > 0$,

$$\lambda''(x) = \frac{\lambda(x)\big(m - (m - \beta\lambda(x))^2\big)}{x^2\big(m - \beta\lambda(x)\big)^3}.$$

The complete public surface of that module consists of the two derivative-of-derivative witnesses `deriv_principalPowerExponentialPhase_hasDerivAt` and `deriv_lowerPowerExponentialPhase_hasDerivAt`, together with their value corollaries `deriv_deriv_principalPowerExponentialPhase` and `deriv_deriv_lowerPowerExponentialPhase`. The factorization suggests candidate thresholds $\beta\lambda = m \pm \sqrt{m}$, but generic sign, zero-set, threshold, inflection, and strict-convexity/concavity packages have not been formalized and are not claimed here.

Finally `PowerExponentialLambertFabius.lean` specializes $(m, A, \beta) = (1, 1, \log 2)$. It defines `fabiusPrincipalLambertPhase`, identifies the other branch with `fabiusLambertPhase`, and exports `fabiusSaddle_eq_iff_eq_principal_or_eq_lower`, `fabiusPrincipalLambertPhase_ne_fabiusLambertPhase`, `fabiusPrincipalLambertPhase_continuousOn_Icc`, and `fabiusLambertPhase_continuousOn_Ioc`. `fabiusPrincipalLambertPhase_isEquivalent_id` gives the specialized principal asymptotic $\lambda_0(x) \sim x$; the generic lower divergence and intrinsic expansion specialize to the existing lower Fabius phase.

The curvature specialization is isolated in `PowerExponentialLambertFabiusCurvature.lean`. Write

$$x_* := \frac{2e^{-2}}{\log 2}, \qquad P := \frac{e^{-1}}{\log 2}.$$

For the lower phase it proves $x_* \in (0, P)$, $\lambda_{-1}(x_*) = 2/\log 2$, and, for $0 < x < P$,

$$\lambda''_{-1}(x) = \frac{(\log 2)\lambda_{-1}(x)^2\big(2 - (\log 2)\lambda_{-1}(x)\big)}{x^2\big(1 - (\log 2)\lambda_{-1}(x)\big)^3}.$$

Its second derivative is positive exactly when $x < x_*$, negative exactly when $x_* < x$, and zero exactly when $x = x_*$. Thus x_* is the exact formalized inflection input: the phase is strictly convex on $(0, x_*]$ and strictly concave on $[x_*, P]$.

The principal specialization deliberately uses the raw-W_0 curvature API rather than restricting the generic positive-input formula. Globally,

$$\lambda_0(x) = -\frac{W_0(-x\log 2)}{\log 2};$$

for every $x < P$,

$$\lambda_0'(x) = \frac{1}{e^{W_0(-x\log 2)}(W_0(-x\log 2) + 1)},$$
$$\lambda_0''(x) = \frac{(\log 2)e^{-2W_0(-x\log 2)}(W_0(-x\log 2) + 2)}{(W_0(-x\log 2) + 1)^3} > 0.$$

In particular $\lambda_0''(0) = 2\log 2$, and the phase is strictly convex on the full endpoint-inclusive half-line $(-\infty, P]$, including all negative inputs and zero. The leaf's complete public surface is one noncomputable definition and eighteen theorems:

No.	Declaration	Formal content
1	`fabiusLambertInflectionInput`	Definition of $x_* = 2e^{-2}/\log 2$.
2	`fabiusLambertInflectionInput_mem_Ioo`	$x_* \in (0, P)$.
3	`fabiusLambertPhase_inflectionInput`	$\lambda_{-1}(x_*) = 2/\log 2$.
4	`deriv_deriv_fabiusLambertPhase`	Exact lower-phase second derivative on $(0, P)$.
5	`deriv_deriv_fabiusLambertPhase_pos_iff`	For $0 < x < P$, $0 < \lambda_{-1}''(x) \iff x < x_*$.
6	`deriv_deriv_fabiusLambertPhase_neg_iff`	For $0 < x < P$, $\lambda_{-1}''(x) < 0 \iff x_* < x$.
7	`deriv_deriv_fabiusLambertPhase_eq_zero_iff`	For $0 < x < P$, $\lambda_{-1}''(x) = 0 \iff x = x_*$.
8	`deriv_deriv_fabiusLambertPhase_inflectionInput`	The simplified equality $\lambda_{-1}''(x_*) = 0$.
9	`strictConvexOn_fabiusLambertPhase_left`	Strict convexity on $\mathrm{Ioc}(0, x_*)$.
10	`strictConcaveOn_fabiusLambertPhase_right`	Strict concavity on $\mathrm{Icc}(x_*, P)$.
11	`fabiusPrincipalLambertPhase_eq_principalLambertW`	Global affine-W_0 coordinate formula.
12	`fabiusPrincipalLambertPhase_continuousOn_Iic`	Continuity on $\mathrm{Iic}(P)$.
13	`fabiusPrincipalLambertPhase_hasDerivAt`	First-derivative witness for every $x < P$.
14	`deriv_fabiusPrincipalLambertPhase`	Exact first-derivative value for every $x < P$.
15	`deriv_fabiusPrincipalLambertPhase_hasDerivAt`	Derivative-of-derivative witness for every $x < P$.
16	`deriv_deriv_fabiusPrincipalLambertPhase`	Exact principal second-derivative value for every $x < P$.
17	`deriv_deriv_fabiusPrincipalLambertPhase_pos`	Strict positivity for every $x < P$.
18	`deriv_deriv_fabiusPrincipalLambertPhase_zero`	The simplified equality $\lambda_0''(0) = 2\log 2$.
19	`strictConvexOn_fabiusPrincipalLambertPhase`	Strict convexity on $\mathrm{Iic}(P)$.

The import ladder reflects the proof structure: `PowerExponentialLambert.lean` imports the raw lower and principal branches, then `PowerExponentialLambertCalculus.lean`, `PowerExponentialLambertInverse.lean`, and `PowerExponentialLambertAsymptotics.lean` add continuity, classification, and asymptotics in that order. `PrincipalLambertW.lean` itself imports `LowerLambertW.lean`; `LambertBranchDichotomy.lean` imports the principal leaf; and `LambertWCurvature.lean` imports that pair layer. `LambertWBranchPointGeometry.lean` then imports exactly the raw curvature leaf, and `LambertWBranchPointAsymptotics.lean` imports the geometry leaf together with Mathlib's specific-asymptotics and L'Hôpital modules. Thus

the raw dependency order is

$$\text{lower} \longrightarrow \text{principal} \longrightarrow \text{branch dichotomy} \longrightarrow \text{curvature}$$
$$\longrightarrow \text{branch-point geometry} \longrightarrow \text{branch-point asymptotics}.$$

`PowerExponentialLambertCurvature.lean` imports the generic calculus layer and contains only the four branch-generic second-derivative declarations above. The non-curvature Fabius bridge imports the asymptotic layer together with the older generalized-coordinate and Fabius saddle modules, while `PowerExponentialLambertFabiusCurvature.lean` imports that bridge and both curvature leaves. The umbrella `FabiusFunction.lean` lists `LambertWCurvature.lean`, `LambertWBranchPointGeometry.lean`, and `LambertWBranchPointAsymptotics.lean` consecutively in dependency order before the generic power–exponential leaves; it also exposes the two scaled curvature modules. These lemmas support the corrected comparisons in both the k-fold project and the principal Fabius saddle analysis.

20.6 Decay comparisons

`FabiusFlatness.lean` and `FabiusDecayComparison.lean` convert the log-squared law into two useful endpoint comparisons. For every $m > 0$,

$$F(x) = o(x^m) \qquad (x \downarrow 0),$$

so the function is smaller than every power. On the other hand, for every $c > 0$,

$$e^{-c/x} = o(F(x)),$$

so it is much larger than an $e^{-c/x}$ flat function.

Taking logarithms reduces these statements to comparing $(\log(1/x))^2$ with $\log(1/x)$ and with $1/x$. The code packages positivity and eventual monotonicity so the exponential comparison can use standard filter lemmas.

21 Recurring proof-engineering patterns

21.1 Strong induction as the default arithmetic engine

Many sequences in the project depend on all smaller indices, not only on the predecessor. The idiomatic proof begins with strong induction:

```
refine Nat.strong_induction_on n ?_
intro n ih
-- ih : forall m < n, P m
```

A finite sum over `Finset.rangen` then supplies the inequality needed to invoke `ih`. It is often worth proving a helper lemma that converts membership in the range directly into the strict inequality expected by the hypothesis.

Strong induction also matches the highest-bit evaluator, whose recursive remainder is smaller but not necessarily one less. Using the same well-founded order for definition and correctness proof keeps the program and theorem structurally aligned.

21.2 Finite sums and index transport

The repository moves frequently among:

- `Finset.rangen`;

- subtype indices `Finn`;
- integer intervals;
- list positions; and
- antidiagonals describing coefficient convolution.

Rather than rewriting all indices inside a large expression, the code usually proves an equivalence between finite index types and invokes a reindexing theorem. A clean bijection proof has four parts: the forward map, inverse map, membership preservation, and left/right inverse equalities.

For even/odd splits, a specialized theorem is especially valuable:

$$\sum_{j<2N} f(j) = \sum_{k<N} f(2k) + \sum_{k<N} f(2k+1).$$

Once established, it drives Thue–Morse identities, convolution recurrences, and parity counts.

21.3 Casts as explicit interfaces

Exact theorems live in $\mathbb{N}_0$, $\mathbb{Z}$, or $\mathbb{Q}$; analytic theorems live in $\mathbb{R}$ or $\mathbb{C}$. Lean will insert canonical casts, but it will not silently prove that a finite sum commutes with the cast or that a natural exponent has the intended target type.

The code therefore proves and reuses lemmas of the forms

$$\left(\sum_{k\in s} q_k : \mathbb{Q}\right)\uparrow^{\mathbb{R}} = \sum_{k\in s}(q_k : \mathbb{R}),$$
$$(2^n : \mathbb{Q})\uparrow^{\mathbb{R}} = (2 : \mathbb{R})^n,$$
$$\left(\binom{n}{k} : \mathbb{N}_0\right)\uparrow^{\mathbb{Q}} = \binom{n}{k} \text{ interpreted in } \mathbb{Q}.$$

In actual Lean syntax the upward arrows are ordinary coercions. Tactics such as `norm_cast`, `exact_mod_cast`, and `push_cast` are used after the expression is put in a form they recognize.

Lean pattern. Do not ask `norm_cast` to solve a goal containing unreduced divisions, nested sums, and conditionals. First rewrite the exact recurrence, move casts through finite sums, simplify powers, and only then invoke cast normalization. The repository's helper lemmas reflect this staged approach.

21.4 Pointwise derivatives instead of raw `deriv`

A theorem of type `HasDerivAtff'x` carries both differentiability and the derivative value. It composes under addition, multiplication, affine maps, logarithms, and exponentials. By contrast, an equality involving `derivfx` often creates a side goal proving `DifferentiableAtfx` before it can be rewritten.

Accordingly, the calculus modules first prove `HasDerivAt` identities and derive `deriv` corollaries only when a source theorem is stated in that notation. Iterated derivatives are handled through `iteratedDeriv` or `ContDiff` lemmas with explicit order parameters.

21.5 Piecewise functions and boundary glue

The bounded function, fixed-point transform, splines, and endpoint representatives are all piecewise. The source follows a stable pattern:

1. define each branch as a total function;

2. prove branch continuity or differentiability;
3. prove equality of branch values at the boundary;
4. apply a piecewise glue theorem; and
5. derive simplified branch lemmas for later rewriting.

The simplified lemmas are often tagged for the simplifier only when their conditions are syntactically obvious. Overly aggressive `simp` tags on overlapping branches can create loops or choose an unintended normal form.

21.6 Local finiteness before infinite-sum algebra

For translate sums, the code first proves finite support at a point or on a neighborhood. Only then does it reindex or differentiate a `tsum`. This is safer than invoking general unconditional summability theorems on a series whose convergence is really trivial by support.

A typical theorem states that the summand is zero outside a finite integer interval. From this it derives summability, rewrites the `tsum` as a `Finset.sum`, performs the algebra, and converts back only if the public theorem uses `tsum` notation.

21.7 Integrability from support and bounds

The most common integrability proof has three ingredients:

- continuity or measurability of the integrand;
- compact support; and
- a uniform bound on that support.

The repository builds reusable support lemmas for F, up, their derivatives, and affine transforms. Polynomial or exponential multipliers remain bounded on a compact support, so moment and transform integrability follows from a generic compact-support theorem.

This style avoids global estimates that are stronger than needed. For instance, $x^n \,\mathrm{up}(x)$ is integrable not because x^n is globally bounded, but because the product vanishes outside $[-1, 1]$.

21.8 Interchanging sums, limits, derivatives, and integrals

Every interchange in the project is supported by a named convergence theorem:

- finite sums commute with everything by algebra;
- infinite sums commute with integrals under a summable integrable majorant;
- limits pass under integrals by dominated or bounded convergence;
- derivatives pass through series under local uniform convergence of derivative series; and
- contour limits use explicit uniform tail bounds.

The proof normally constructs the majorant first, proves its summability or integrability, and only then invokes the interchange theorem. This order makes failures local: if an informal series is not uniformly controlled, the missing hypothesis appears before the main identity is attempted.

21.9 Big-O, little-o, and eventual domains

Asymptotic modules use filters rather than epsilon notation. A theorem of the form

$$f(x) = O(g(x)) \quad (x \to 0^+)$$

is represented through `IsBigO` at a within-neighborhood filter. The proof often begins with

```
filter_upwards [eventually_pos, eventually_small] with x hxpos hxsmall
```

and establishes a pointwise norm inequality.

Eventual positivity is particularly important for logarithms, divisions, and Lambert phases. The source proves threshold lemmas once and then includes them in filter-upwards blocks. Transitivity of Big-O and equivalence under asymptotically equal scales are used heavily to transfer results from the exact phase $\lambda(x)$ to $-\log x$.

21.10 Polynomial identities

Exact finite identities are proved at the polynomial level whenever possible. The usual techniques are:

- extensionality by coefficients;
- evaluation at an arbitrary point followed by `ring`;
- induction using polynomial addition and multiplication; and
- degree bounds plus a roots theorem.

For translation-invariant Thue–Morse sums, a degree bound and finite-difference theorem often give a cleaner proof than expanding every binomial coefficient.

21.11 Choosing automation deliberately

The development uses automation, but proofs are structured so each tactic sees an appropriate goal:

`simp`
: unfolds branch lemmas, finite-range membership, and standard algebraic identities.

`norm_num`
: closes concrete rational and natural computations, especially counterexamples and base cases.

`ring` / `ring_nf`
: handles polynomial identities after divisions and casts have been normalized.

`linarith` / `nlinarith`
: combines real inequalities once transcendental terms have been abstracted as variables with known bounds.

`positivity`
: proves positivity of products, powers, factorials, exponentials, and denominators.

`omega`
: handles Presburger arithmetic for indices, ranges, and exponents.

`field_simp`
: clears rational or field denominators after nonzero hypotheses are established.

`aesop`
: resolves local structural goals, but is not used as a substitute for choosing the main induction or analytic theorem.

21.12 Public wrappers and internal strength

Theorem-producing modules tend to prove results in a strong reusable form: unrestricted numerators, arbitrary candidate functions, general coefficient rings, or explicit error constants. Paper-facing wrappers then add notation and specialize hypotheses.

When extending the library, search for the strongest internal theorem before reproving a source-shaped special case. Names ending in `_all`, `_global`, `_scalar`, `_correct`, `_transfer`, or `_of_isFabius` often signal the reusable layer.

21.13 No placeholders and explicit corrections

The repository states that the development builds without `sorry`. More importantly, corrected claims are not represented by axioms that merely assume the missing fact. When a source formula is false, the project either adds the necessary hypothesis, changes the object to the signed extension, weakens the error term, or proves a counterexample.

This makes the dependency graph epistemically meaningful. A theorem about the Fabius function ultimately reduces to mathlib, Lean's kernel, explicit definitions, and proved lemmas—not to a paper citation embedded as an axiom.

22 How to read and extend the code

22.1 Five recommended reading paths

Different goals call for different entry points.

22.1.1 Path A: existence and uniqueness

Read, in order:

1. `Basic.lean`;
2. `Differential.lean`;
3. `DyadicClosedForm.lean`;
4. `DyadicAnalytic.lean`; and
5. `Existence.lean`.

This path explains the abstract specification, generic derivative consequences, dyadic rigidity, and fixed-point implementation.

22.1.2 Path B: exact evaluation

Read:

1. `Arithmetic.lean`;
2. `DyadicCorrectness.lean`;
3. `DyadicClosedForm.lean`;
4. `DyadicAnalytic.lean`; and
5. `GlobalDyadic.lean`.

Keep an editor evaluation buffer open and inspect the types of table, Horner, and wrapper correctness theorems.

22.1.3 Path C: the historical Rvachev theory

Read:

1. `GlobalExtension.lean`;
2. `FourierProduct.lean`;
3. `FourierAnalytic.lean`;
4. `OriginalUniqueness.lean`;
5. `ProbabilityRepresentation.lean`; and
6. `StepApproximationLimit.lean`.

This path emphasizes compact support, transform refinement, convolutions, and pointwise approximants.

22.1.4 Path D (post-snapshot): integer and fractional Volterra calculus

The fractional modules in this path were included in the compiled full-facade checkpoint `22f801337`. Read:

1. `NormalizedVolterra.lean`;
2. `FractionalVolterra.lean` and `FractionalVolterraSemigroup.lean`;
3. `FractionalVolterraCalculus.lean`;
4. `GlobalExtension.lean` and `FabiusAntiderivatives.lean`; and
5. `FabiusFractionalVolterra.lean`.

The first module is the reusable natural-order layer: arbitrary base points and oriented endpoints, real normed-space-valued kernels and algebra, exact affine covariance for every real scale, arbitrary base-point shifts, and the zero-local-tail Taylor jet. For continuous Banach-valued inputs it additionally identifies literal iteration with the direct kernel and supplies the semigroup's strict FTC, derivative cancellation, smoothness, and Taylor reconstruction. `FractionalVolterra.lean` and its semigroup companion give the Gamma-normalized real-order operator and positive-order composition on ordered intervals. `FractionalVolterraCalculus.lean` adds positive affine covariance for arbitrary real order and, for $\alpha > 0$, the exact integration-by-parts order-raising law with its left-endpoint boundary term. The two Fabius-specific modules are used only after this generic work: `FabiusAntiderivatives.lean` gives the signed-global and bounded natural-order ladders and finite monomial weights, while `FabiusFractionalVolterra.lean` gives the positive-real one-step fractional shifts for the signed extension on $x \geq 0$, the bounded function on $[0, 1]$, and the bridge from up based at -1 to F.

22.1.5 Path E: sharp asymptotics

Read:

1. `FabiusLogScale.lean`;
2. `FabiusLogSquaredAsymptotic.lean`;
3. `NegativeLaplace.lean`;
4. `PeriodicCorrection.lean`;
5. `MellinBose.lean`;

6. `FabiusLambertPhase.lean`;
7. `BromwichSaddle.lean`;
8. `FabiusSharpExactReduction.lean`;
9. `FabiusSharpAsymptotic.lean`; and
10. `FabiusFullAsymptoticExpansion.lean`.

Use the specialized central/tail and all-order modules as dependencies are encountered.

22.2 Adding a new exact identity

Suppose a new conjectured formula concerns $F(a/2^n)$. A productive workflow is:

1. state and test the identity entirely in $\mathbb{Q}$ using `fabiusDyadicValue`;
2. normalize representations and determine whether the bounded or global extension is intended;
3. prove the finite arithmetic identity, preferably through existing closed-form or q-binomial lemmas;
4. only then transport it to the analytic function using dyadic correctness; and
5. add a paper-facing wrapper if the notation comes from an external source.

This approach keeps exploratory computation cheap and avoids starting with real analysis when the content is combinatorial.

22.3 Adding a new moment theorem

For a new moment recurrence:

1. derive the analytic recurrence in a generic `IsFabius` context;
2. define an exact rational sequence only if computation or arithmetic consequences justify it;
3. prove equality by strong induction;
4. design a division-free numerator recurrence before attempting parity or divisibility; and
5. separate reduced-denominator conclusions into a later module.

The existing normalized moment files are templates for common-denominator induction.

22.4 Adding an asymptotic correction

A proposed correction term should be attached to the exact saddle reduction rather than inferred from numerical fitting. The robust route is:

1. identify which cumulant or periodic Fourier coefficient contributes at the desired order;
2. extend the formal coefficient recurrence;
3. prove a uniform central remainder at that order;
4. prove the tail is smaller than the target scale;
5. update the normalized mass expansion; and
6. transfer through logarithm and the exact phase.

If the desired final formula is in elementary logarithms, derive it as a corollary of the exact-phase theorem using `FabiusLambertAllOrderSmallArgument.lean`.

22.5 Testing a suspected source error

Formalization is most efficient when the claim is stress-tested early.

- Check endpoint and smallest-index cases by exact evaluation.
- Verify domains of every differentiated functional equation.
- Compare bounded and signed-global versions under translation.
- For asymptotic claims, sample subsequences of fixed dyadic phase.
- For uniform errors, examine knots and support boundaries.
- For q-dependent finite rows, test nondyadic locations before claiming parameter independence.

A small counterexample theorem is often more informative than a long stalled proof attempt.

22.6 Maintaining dependency hygiene

Before importing the umbrella module into a new file, ask which interface is actually needed. An exact lemma should normally depend on `Arithmetic.lean` or a narrow q-binomial module, not on `Existence.lean`. A generic analytic lemma should depend on `Basic.lean` and the relevant calculus modules, not on the canonical instance unless the theorem truly uses it.

A useful check is to run the import-graph tooling included through mathlib's dependencies. Unexpected paths from asymptotic modules into paper aggregators or from arithmetic modules into analysis are signs that a helper theorem belongs in a lower layer.

22.7 Naming and theorem granularity

The repository's long module names encode the proof stage: `Raw`, `Scalar`, `ExactReduction`, `Transfer`, `Central`, `Tail`, `AllOrders`, and so on. New declarations should preserve this vocabulary where possible.

A theorem should expose one conceptual transition. Good boundaries include:

- closed formula equals Horner program;
- analytic recurrence equals exact recurrence;
- central integral equals Gaussian main term plus error;
- exact phase equals truncated elementary expansion plus error; and
- source statement follows from the stronger internal theorem.

Large proofs become navigable when these transitions have names.

22.8 A minimal downstream example

A downstream file that only needs exact evaluation and the analytic correctness theorem can begin as follows:

```
import FabiusFunction.PaperStatements

open Fabius

example (n : Nat) (a : Int) :
    (fabiusDyadicValue n a : Real) =
      fabiusReal fabius ((a : Real) / (2 : Real) ^ n) := by
  exact fabiusDyadicValue_cast fabius fabius_spec n a
```

This statement accepts every signed numerator, so no reduced-fraction or unit-interval side condition remains. Importing `FabiusFunction.lean` would also work, but would hide which layer supplies the public wrapper.

22.9 What to trust when prose and code differ

The kernel-checked theorem type is the final authority. Module headers and README tables explain intent and source correspondence, while the proof term certifies the exact statement. When a paper formula differs from a Lean wrapper, inspect:

1. whether the paper silently assumes an index is positive;
2. whether a removable singularity has been totalized;
3. whether a global extension replaces a bounded function;
4. whether endpoint representatives differ only almost everywhere; and
5. whether an asymptotic periodic term has been omitted.

The repository usually records the reason in the aggregator header or a nearby correction theorem.

23 Concluding perspective

The ProveIt development is best understood not as a very long proof of a single theorem, but as a small mathematical library centered on one function. It has several independent foundations:

- an exact arithmetic language of moments, binary signs, products, and dyadic evaluators;
- an abstract analytic interface capturing the Fabius differential and symmetry laws;
- a Banach fixed-point construction of the bounded solution;
- a compact-support/Fourier theory of the Rvachev twin; and
- a transform-and-saddle theory for the flat endpoint.

The strongest parts of the architecture are the bridges. The dyadic bridge identifies executable rationals with analytic values. The moment bridge identifies finite recurrences with integrals. The Fourier bridge identifies a refinement equation with an infinite product. The saddle bridge identifies that product with a corrected sharp asymptotic. The Legendre bridge turns exact moments into optimal polynomial approximants.

Formalization also changes the mathematical product. It forces endpoint conventions, domains of delay equations, transform normalizations, and bounded-versus-global distinctions to be stated. It exposes false residual estimates and missing terms, while often supplying a corrected theorem stronger than the source statement. The result is not merely a verification of known facts: it is a coherent, reusable account of how the facts fit together.

For a reader entering the source, the single most useful habit is to ask which side of the project a declaration belongs to: exact arithmetic, generic analysis, canonical construction, or a bridge. Once that question is answered, the long file list resolves into a small number of repeated proof patterns and a remarkably systematic dependency graph.

A Detailed dependency map

A.1 Core spine

The shortest conceptual spine from definitions to the canonical function is:

$$\boxed{\texttt{Arithmetic.lean}} \longrightarrow \boxed{\texttt{Basic.lean}} \longrightarrow \boxed{\texttt{Differential.lean}}$$
$$\longrightarrow \boxed{\texttt{DyadicAnalytic.lean}} \longrightarrow \boxed{\texttt{Existence.lean}}$$
$$\longrightarrow \boxed{\texttt{PaperStatements.lean}}.$$

This display omits two important side inputs. `DyadicClosedForm.lean` supplies the exact finite polynomial matched by `DyadicAnalytic.lean`, while `DyadicCorrectness.lean` supplies evaluator correctness and representation invariance. `Existence.lean` proves unique existence; `PaperStatements.lean` makes the canonical choice and publishes `fabius_spec`. The resulting canonical function is then consumed by moment, Fourier, probability, global-extension, and asymptotic modules.

A.2 Arithmetic spine

The main arithmetic chain is:

$$\texttt{Arithmetic.lean} \longrightarrow \begin{cases} \texttt{NormalizedMoments.lean} \\ \texttt{NormalizedEvenMoments.lean} \\ \texttt{Parity.lean} \end{cases}$$
$$\longrightarrow \begin{cases} \texttt{HalfMomentDenominator.lean} \\ \texttt{TwoAdic.lean} \\ \texttt{DenominatorBound.lean} \end{cases} \longrightarrow \texttt{PaperStatements.lean}.$$

`AnalyticMoments.lean` and `ExactInversePower.lean` connect this chain to the canonical function before the source-facing wrappers are assembled.

A.3 Original-paper spine

The original Rvachev paper is represented by two parallel paths:

$$\text{differential/refinement law} \to \text{Fourier recurrence} \to \text{sinc product} \to \text{uniqueness},$$
$$\text{finite elementary laws} \to \text{convolutions} \to \text{weak convergence} \to \text{step approximants}.$$

Their principal modules are `OriginalUniqueness.lean`, `FourierProduct.lean`, `ProbabilityRepresentation.lean`, `WeakConvergence.lean`, and `StepApproximationLimit.lean`. `OriginalPaperSupplement.lean` adds the remaining numbered theorems and prose claims.

A.4 Activation spine

The activation development consists of one local Taylor branch and a small square-summable fork-and-join graph:

$$\texttt{HyperbolicActivation} \longrightarrow \texttt{ActivationTaylor},$$

$$\texttt{HyperbolicActivation} \longrightarrow \begin{cases} \texttt{ActivationSeries} \\ \texttt{ActivationAsymptotics} \end{cases}$$

$$\begin{cases} \texttt{ActivationSeries} \\ \texttt{ActivationAsymptotics} \end{cases} \longrightarrow \texttt{ActivationSeriesAsymptotics},$$

$$\texttt{ActivationSeries} \longrightarrow \texttt{GeometricActivationDimension},$$

$$\begin{cases} \texttt{ActivationSeriesAsymptotics} \\ \texttt{GeometricActivationDimension} \end{cases} \longrightarrow \texttt{GeometricActivationAsymptotics}.$$

The Taylor branch is a local analytic development and terminates on its own. Of the remaining two branches, the first contains global comparison and summability and the second contains the one-coordinate punctured limit. Tannery joins those two interfaces only after both are available. Geometric algebra stays in the structural module, so the final asymptotic specialization contains almost no repeated series computation.

A.5 Asymptotic spine

The sharp endpoint proof has the following high-level flow:

$$\texttt{AnalyticMoments.lean} \to \texttt{NegativeLaplace.lean} \to \begin{cases} \texttt{PeriodicCorrection.lean} \\ \texttt{MellinBose.lean} \end{cases}$$

$$\to \texttt{BromwichSaddle.lean} \to \texttt{FabiusLambertPhase.lean}$$

$$\to \begin{cases} \text{central modules} \\ \text{tail modules} \end{cases} \to \texttt{FabiusSharpExactReduction.lean}$$

$$\to \texttt{FabiusSharpAsymptotic.lean} \to \texttt{FabiusFullAsymptoticExpansion.lean}.$$

The actual source graph is finer because the all-order proof separates formal coefficient algebra, small-argument Lambert expansion, Gaussian contraction, central uniformity, tail flatness, and transfer lemmas.

B Paper theorem map

B.1 Arias de Reyna, arXiv:1702.05442

Source item	Lean declaration or family	Principal proof location
Theorem 1: structure and fold	`IsOriginalFabius`; `IsOriginalFabius.mk_of_derivative_law`; `IsFabius.isOriginalFabius_rvachevUp`	`OriginalCharacterization.lean`

Source item	Lean declaration or family	Principal proof location
Theorem 1: uniqueness and exact fold kernel	`isOriginalFabius_iff_eq_canonical`; `rvachevUp_eq_iff_eqOn_Iic_one`; `isFabius_iff_isOriginalFabius_rvachevUp_and_rightTail`; `isOriginalFabius_iff_existsUnique_isFabius`	`OriginalUniqueness.lean`
Lemma 1 and limiting measure	`finiteConvolutionProbability_tendsto`, bounded-continuous integral corollaries, and `support_rvachevMeasure`	`EarlyApproximants.lean` (finite convolution measures), `WeakConvergence.lean` (characteristic functions, Lévy convergence, and exact support)
Theorem 2	`stepApproximant_tendsto_rvachevUp` and endpoint-corrected variants	`StepApproximationLimit.lean`
Theorem 3	probability representation of the up-function	`ProbabilityRepresentation.lean`
Theorem 4	`original_theorem_four_a`, `original_theorem_four_b`, `original_theorem_four_c`	`OriginalPaperSupplement.lean`, Fourier and differential helpers
Nonanalyticity	`rvachev_not_analyticAt`	`OriginalPaperSupplement.lean`, strengthened to the exact locus in `NowhereAnalytic.lean`
Theorem 5	Poisson summation, real-axis Fourier rapid decay, `rvachev_poisson_support_specialization_unscaled_of_one_half_le`, and `rvachev_poisson_support_specialization_of_one_half_le`	`PoissonSummation.lean`
Theorem 6	`original_theorem_six`; moment formulas	`AnalyticMoments.lean`, `OriginalPaperSupplement.lean`
Theorem 7	`original_theorem_seven_global`; global rationality/dyadic statement	`GlobalDyadic.lean`, `OriginalPaperSupplement.lean`

B.2 Arias de Reyna, arXiv:1702.06487

Source item	Lean declaration or family	Principal proof location
Proposition 1	`proposition_one`	`PaperStatements.lean`; exact moment foundations
Proposition 2	`proposition_two`; `proposition_two_real`	`AnalyticMoments.lean`, `PaperStatements.lean`; complex and real removable-singularity forms
Proposition 3	`proposition_three`	`MomentPowerSeries.lean`; even-binomial transform of half moments

Source item	Lean declaration or family	Principal proof location
Proposition 4	`proposition_four`	`NormalizedMoments.lean`; natural normalization of half moments
Question 5	`reshetnikovQuestion`	recorded as a predicate/question in `PaperStatements.lean`
Proposition 6	`proposition_six`	exact arithmetic and dyadic modules
Theorem 7	`theorem_seven`	`DyadicClosedForm.lean`, `DyadicAnalytic.lean`
Proposition 8	`proposition_eight`	`PaperStatements.lean`; even-index Reshetnikov formula and power-of-two denominator bound
Theorem 9	`theorem_nine`, strengthened all-parameter form	`Paper06487Supplement.lean`, global/differential helpers
Lemma 1	`lemma_one`, with corrected order condition	`TaylorReduction.lean`, `PaperStatements.lean`
Proposition 10	`proposition_ten`	`TaylorReduction.lean`; recursive signed-extension reduction
Definition 12	`dyadicDenominator`	`Arithmetic.lean`
Theorem 13	`theorem_thirteen` and denominator bound	`DenominatorBound.lean`
Proposition 15	`proposition_fifteen`	`PaperStatements.lean`; inverse-power denominator divides the level dyadic denominator
Conjecture 16	`conjecture_sixteen`	recorded without asserting a proof
Theorem 17	`theorem_seventeen`	`PaperStatements.lean`; Lucas digit-product congruence
Proposition 18	`proposition_eighteen`	`Parity.lean`, `PaperStatements.lean`; count of odd binomial coefficients
Proposition 19	`proposition_nineteen`	`Parity.lean`; oddness of `momentNumerator`
Theorem 20	`theorem_twenty`	`TwoAdic.lean`; odd reduced numerator/denominator and valuation of $2d_n$; corrected proof-internal product $2d_kN_n$ in `Paper06487Supplement.lean`
Theorem 21	`theorem_twenty_one`	`TwoAdic.lean`; oddness of R_n and exact inverse-dyadic two-adic valuation
Proposition 22	`proposition_twenty_two_initial`; `proposition_twenty_two`	`BernoulliRecurrences.lean`, `PaperStatements.lean`

B.3 How to use the map

The declaration named after a source theorem is normally the best citation target in downstream formal work. The principal proof location is the best learning target. When the two differ, the wrapper is preserving source notation while the internal file proves a stronger or more reusable theorem.

C Mathematical object–Lean crosswalk

Mathematical object	Principal Lean name	Defining or identifying module
Half-base Gaussian root locus	`halfQBinomial_sum_rootMultiplicity_two_pow`, composed with `halfQBinomial_sum_eq_zero_iff` and `gaussianBinomial_half_eq_halfQBinomial`	Exhaustive zero-definition/one-theorem `HalfQBinomialRootSimplicity.lean` surface. Over $\mathbb{Q}$, every classified root 2^j, $j < n$, has multiplicity one; composition gives exactly the rational roots $1, 2, \ldots, 2^{n-1}$, no others, all simple, and makes `cor:halfbase-root-locus` Exact. The derivative proof deletes the unique vanishing finite-product factor. No arbitrary-characteristic simplicity or arbitrary-base root classification is claimed.
Leading Laurent coefficient of the reciprocal centered Rvachev MGF	Definition `rvachevCenteredMGF`; theorems `rvachevCenteredMGF_eq_rvachevFourierProduct`, `rvachevCenteredMGF_pi_mul_I_int`, `rvachevCenteredMGF_pi_mul_I_int_ne_zero_of_odd`, `tendsto_sub_pow_mul_inv_rvachevFourierProduct_int`, `tendsto_rvachevCenteredMGF_laurent_int`, `tendsto_rvachevCenteredMGF_laurent_two_pow_mul_odd`	Exhaustive one-definition/six-theorem `RvachevLaurentLeading.lean` surface. It normalizes $M(t)$ as the existing centered generating function at $2t$, proves $M(t) = \Phi(\mathrm{i}t/(2\pi))$, and transports the exact integer-zero cofactor limit. For $n = 2^v u$, u odd, the pole at $T_n = 2\pi\mathrm{i}n$ has leading coefficient $-T_n^{v+1}/M(\pi\mathrm{i}u)$, with nonzero denominator. All limits use $\mathcal{N}[\neq]$, because inverse is totalized at the pole. This makes `is:p2:thm:Laurent-leading` Exact but adds no lower Laurent coefficients or pole-shell/Appell asymptotics.

Mathematical object	Principal Lean name	Defining or identifying module
Finite dyadic Appell prefixes and exact recovery	Definitions `unitUniformRawMomentRat`, `centeredUnitUniformRawMomentRat`, `dyadicPrefixScaleRat`, `dyadicPrefixMomentRat`, `uncenteredDyadicPrefixMomentRat`, `centeredDyadicPrefixMomentRat`, `kabayaIriAppellPolynomialRat`, `uncenteredDyadicPrefixAppellPolynomialRat`, `centeredDyadicPrefixAppellPolynomialRat`, `uncenteredDyadicPrefixAppellScalePolynomialRat`, `centeredDyadicPrefixAppellScalePolynomialRat`; theorems `Appell.poly_binomialConv`, `Appell.binomialConv_dilate`, `Appell.dilate_dilate`, `dyadicPrefixMomentRat_zero`, `uncenteredDyadicPrefixMomentRat_zero`, `centeredDyadicPrefixMomentRat_zero`, `dyadicPrefixMomentRat_binomialConv_tail`, `binomialConv_uncenteredDyadicPrefixMomentRat_tail`, `binomialConv_centeredDyadicPrefixMomentRat_tail`, `uncenteredDyadicPrefixAppellPolynomialRat_eq_sum`, `centeredDyadicPrefixAppellPolynomialRat_eq_sum_even`, `uncenteredDyadicPrefixAppellPolynomialRat_eq_eval_scale`, `centeredDyadicPrefixAppellPolynomialRat_eq_eval_scale`, `natDegree_uncenteredDyadicPrefixAppellScalePolynomialRat`, `natDegree_centeredDyadicPrefixAppellScalePolynomialRat`, `kabayaIriAppellPolynomialRat_eq_sum_prefix`, `rvachevAppellPolynomialRat_eq_sum_prefix`	Exhaustive eleven-definition/seventeen-theorem `FinitePrefixAppellRecovery.lean` surface. Finite digit-moment convolution and reciprocal uniqueness give the exact uncentered 2^{-N} and centered even 4^{-N} expansions. Their outer degrees in $\mathrm{Polynomial}(\mathrm{Polynomial}\,\mathbb{Q})$ are n and $\lfloor n/2 \rfloor$; fixing the inner point can lower that degree, notably in centered odd degree at $x = 0$. Geometric Lagrange evaluation recovers the full uncentered polynomial from any $n+1$ consecutive prefixes and the centered polynomial from any $\lfloor n/2 \rfloor + 1$, without a limit. Thus `is:p2:thm:finite-prefix-expansion` and `is:p2:thm:exact-recovery` are Exact. This is rational coefficient algebra, not analytic MGF convergence.
Abstract bounded Fabius specification	`IsFabius`	`Basic.lean`
Canonical bounded function F	`fabius`, `fabius_spec`	chosen in `PaperStatements.lean`; existence and uniqueness proved in `Existence.lean`
Rvachev up-function up	`rvachevUp`	`Basic.lean`; analytic properties in `Differential.lean` and later shape modules
Signed global extension F_{gl}	`extendedFabius`	defined in `Basic.lean`; cell and smoothness theory in `GlobalExtension.lean`
Normalized Volterra operator $\mathsf{J}_{n,a}$	`normalizedVolterra`, `iteratedPrimitive`, `normalizedVolterra_affine`, `normalizedVolterra_comp_affine`, `normalizedVolterra_basepoint_shift`, `normalizedVolterra_succ_eq_taylor_of_eq_zero`	direct kernel, literal iterator, affine transport, and base-point Taylor jets in `NormalizedVolterra.lean`

Mathematical object	Principal Lean name	Defining or identifying module
Exact Fabius primitive ladders	`normalizedVolterra_extendedFabius`, `normalizedVolterra_fabiusReal_of_le_one`	`FabiusAntiderivatives.lean`
Fractional Volterra operator and order calculus	`fractionalVolterra`, `fractionalVolterra_add`, `fractionalVolterra_affine`, `fractionalVolterra_add_one_deriv`	`FractionalVolterra.lean`, `FractionalVolterraSemigroup.lean`, and `FractionalVolterraCalculus.lean`
Fractional Fabius–Rvachev shifts	`fractionalVolterra_add_one_extendedFabius_of_nonneg`, `fractionalVolterra_add_one_fabiusReal`, `fractionalVolterra_add_one_rvachevUp`	`FabiusFractionalVolterra.lean`
Pointwise analytic finite filters	`finiteAnalyticSeriesFilter`, `geometricAnalyticSeriesFilter`, `geometricLagrangeAnalyticSeriesFilter_shifted`	unconditional diagonal, exact-head/tail, and Gaussian-residual identities in `AnalyticSeriesFilter.lean`
Rvachev Appell deconvolution, arbitrary-phase reproduction, and parity-selected superconvergence	`integral_eval_rvachevDeconvolvedPolynomial_sub_mul_rvachev`, `integral_eval_rvachevAppellPolynomial_sub_mul_rvachev`, `integral_eval_rvachevAppellPolynomial_mul_rvachev_eq_zero`, `normalized_tsum_shifted_rvachevDeconvolvedPolynomial_mul_rvachevUp`, and the exact one-definition/eight-theorem `RvachevSuperconvergentSynthesis.lean` API	The first three belong to the exact six-definition/thirty-three-theorem surface of `RvachevMomentAppell.lean`; the fourth belongs to the zero-definition/five-theorem `RvachevPolynomialSynthesis.lean` and gives arbitrary-phase reproduction through degree $\nu_2(M)$ for $M \neq 0$. The superconvergent leaf defines `IsRvachevSuperconvergentPhase` and its eight theorems give selected-phase exactness through degree $\nu_2(M) + 1$. It is not a modulo-one phase classification and proves no maximality, positivity, or rationality theorem. The separate Hasse row below supplies finite Appell–Hasse formulas and reciprocal odd parity, but still no analytic reciprocal-MGF convergence or displayed low reciprocal coefficients. Together with the Hasse leaf and four unchanged downstream modules, the eight-module tranche has inventories $6/33$, $0/5, 1/8, 2/7, 1/14, 1/7, 6/7, 5/25$, hence exactly 128 public declarations.
Generic finite-node Lagrange–Rvachev dictionary	Definitions `lagrangeRvachevDecoder`, `lagrangeRvachevAtomCoefficient`; theorems `natDegree_lagrangeBasis_le_card_sub_one`, `natDegree_lagrangeInterpolate_le_card_sub_one`, `normalized_sum_Ioo_lagrangeRvachevDecoder_mul_shifted_rvachevUp`, `normalized_sum_Ioo_lagrangeRvachevDecoder_eval_node`, `lagrangeRvachevAtomCoefficient_eq_deconvolved_interpolate`, `sum_Ioo_lagrangeRvachevAtomCoefficient_mul_shifted_rvachevUp`, `sum_lagrangeRvachevDecoder_eq_one`	Exact two-definition/seven-theorem surface in `LagrangeRvachevSynthesis.lean`. It proves the generic finite-node cardinal synthesis, componentwise biorthogonality, linear data-to-atom transform, full interpolation loop, and unit row mass under the displayed hypotheses. The Hasse leaf below supplies the geometric Gaussian q-binomial/q-Pochhammer and elementary-symmetric decoder formula; the Matrix wrapper is separate, and no optimal/minimum-variation decoder is asserted.

Mathematical object	Principal Lean name	Defining or identifying module
Finite Appell–Hasse and geometric Gaussian decoder	Definition `Appell.polynomialTransform`; theorems `Appell.polynomialTransform_apply`, `Appell.polynomialTransform_monomial`, `Appell.polynomialTransform_eq_sum_hasseDeriv_of_natDegree_lt`, `Appell.polynomialTransform_eq_sum_hasseDeriv`, `rvachevReciprocalMomentRat_odd`, `rvachevDeconvolutionLinearMap_eq_appellPolynomialTransform`, `rvachevDeconvolvedPolynomial_eq_sum_even_hasseDeriv`, `eval_hasseDeriv_prod_X_sub_C_eq_elementarySymmetricEval`, `eval_rvachevDeconvolvedPolynomial_prod_X_sub_C`, `eval_rvachevDeconvolvedPolynomial_qFallingPower`, `lagrangeBasis_eq_nodalWeight_mul_prod_X_sub_C`, `lagrangeRvachevDecoder_eq_nodalWeight_mul_sum`, `geometric_nodalWeight_eq_geometricQPochhammer`, `geometric_lagrangeRvachevDecoder_eq`	Exhaustive one-definition/fourteen-theorem `RvachevAppellHasse.lean` surface. It gives the generic finite Appell–Hasse transform, odd reciprocal-moment vanishing, even-Hasse Rvachev deconvolution, root-product/elementary-symmetric formulas, q-falling specialization, and the full geometric decoder. The q-falling and Gaussian-decoder manuscript rows are exact only after composition with the separate deconvolution and Lagrange synthesis theorems that reconstruct atoms. Here γ is the formal `rvachevReciprocalMomentRat` sequence, with no analytic reciprocal-MGF convergence claim. Algebra is total at zero or colliding nodes; cardinal use assumes $c > 0$, $0 < q < 1$, and its mesh/interval/degree hypotheses. No larger right inverse or decoder optimality is claimed.
Nodes-only Rvachev–Appell amplitudes	Definition `rvachevDeconvolvedPolynomialRat`; theorems `map_rvachevDeconvolvedPolynomialRat`, `rvachevDeconvolvedPolynomial_eq_sum_appell`, `eval_rvachevDeconvolvedPolynomial_eq_sum_even_iterateDerivative`, `rvachevDeconvolvedPolynomial_prod_X_sub_C_eq_sum_appell`, `eval_rvachevDeconvolvedPolynomial_lagrangeBasis_eq_sum_even_iterateDerivative`, `eval_rvachevDeconvolvedPolynomial_lagrangeBasis_eq_nodalWeight_mul_sum_appell`, `lagrangeRvachevDecoder_eq_nodalWeight_mul_sum_appell`, `map_lagrangeBasis_ratCast`, `map_rvachevDeconvolvedPolynomialRat_lagrangeBasis`, `lagrangeRvachevDecoder_eq_ratCast`, `rvachevRawMomentRat_eq_centeredRvachevFullMoment`, `momentCumulant_rvachevRawMomentRat_eq_centeredRvachevFullCumulant`, `momentCumulant_rvachevRawMomentRat_even_eq_bernoulliMersenne`, `rvachevReciprocalMomentRat_eq_completeBellPolynomial_neg_centeredCumulant`	Exhaustive one-definition/fourteen-theorem `RvachevLagrangeNodesOnly.lean` surface. By composition, not a single wrapper, it makes `cor:lag-nodes-only` Exact/Complete. The derivative form assumes distinct nodes; rationality is only at rational evaluation points, including k/M; reconstruction separately retains $M \neq 0$, degree, and interval hypotheses. The Bell identity is finite formal coefficient algebra, not analytic reciprocal-MGF convergence, and no larger cardinal/right-inverse row is promoted.

Mathematical object	Principal Lean name	Defining or identifying module
Typed finite Lagrange–Rvachev Matrix/Markov layer	Definitions/abbreviation `rvachevAtomIndexSet`, `RvachevAtomIndex`, `lagrangeRvachevEncoderMatrix`, `lagrangeRvachevDecoderMatrix`; theorems `lagrangeRvachevEncoderMatrix_nonneg`, `sum_lagrangeRvachevEncoderMatrix_row_eq_one`, `sum_lagrangeRvachevDecoderMatrix_row_eq_one`, `lagrangeRvachevEncoderMatrix_mul_decoderMatrix`, `exists_neg_entry_of_rightInverse_of_row_overlap`, `exists_lagrangeRvachevDecoderMatrix_entry_neg_of_row_overlap`	Exhaustive four-definition/six-theorem surface of `LagrangeRvachevMatrix.lean`. On the exact atom block $\{-2M < k < 2M\}$, the normalized encoder is nonnegative and row-unital and the sampled decoder is row-unital under their exact hypotheses; distinct in-range nodes, $M \neq 0$, and $\lvert s\rvert - 1 \leq \nu_2(M)$ give $AD = I$. A shared strictly positive column forces a negative decoder entry under the stated conditional overlap. No DA range/kernel/rank/trace/characteristic-polynomial or Cauchy–Binet theorem or decoder-optimality result is asserted. Adding this leaf to the preceding eight-module 128-declaration subtotal gives nine modules and exactly 138 declarations.
Exact even moment c_n	`moment`, `momentIntegral`	`Arithmetic.lean`, `Basic.lean`, and `AnalyticMoments.lean`
Normalized half moment d_n	`halfMoment`, `halfMomentIntegral`	`Arithmetic.lean`, `Basic.lean`, and `AnalyticMoments.lean`
Canonical moment numerators F_n, G_n	`momentNumerator`, `halfMomentNumerator`	`Arithmetic.lean`, `NormalizedEvenMoments.lean`, and `NormalizedMoments.lean`
Inverse-power table $F(2^{-n})$	`fabiusInversePowTwoTable`, `halfMomentFabiusValue`	`Arithmetic.lean` and `ExactInversePower.lean`
Total dyadic evaluator	`fabiusDyadicValue`	`Arithmetic.lean`; analytic correctness wrapper in `PaperStatements.lean`
Dyadic common denominator D_n	`dyadicDenominator`	`Arithmetic.lean` and `DenominatorBound.lean`
Reshetnikov quantity R_n	`reshetnikov`	`Arithmetic.lean` and `PaperStatements.lean`
Prime-power Pascal-row valuation	`primePowerChoose_padicValNat_add`, `primePowerChoose_padicValNat`, `primePowerSubOneChoose_padicValNat`, `primePowerSubTwoChoose_padicValNat`, `twoPowChoose_padicValNat`, `twoPowSubTwoChoose_padicValNat`	Exhaustive zero-definition/six-theorem surface of `PrimePowerBinomialValuation.lean`; the row-p^m formulas include $j = p^m$ and $m = 0$, row $p^m - 1$ is a unit row for $j < p^m$, and the row-$p^m - 2$ formula and its dyadic wrapper assume exactly $0 < j < p^m$
Generic continuous-CDF and density extraction	`continuous_cdf_of_nullSingleton`, `cdf_reflection_sub`, `measure_eq_withDensity_of_cdf_hasDerivAt`	`ContinuousCDF.lean`
Affine independent-copy operator and uniqueness	`affineIndependentCopyLaw`, `affineIndependentCopyLaw_isProbabilityMeasure`, `affineIndependentCopyLaw_eq_map_prod`, `charFun_affineIndependentCopyLaw`, `charFun_eq_mul_charFun_of_affineIndependentCopy_fixedPoint`, `charFun_iterate_of_affineIndependentCopy_fixedPoint`, `eq_of_charFun_affine_recurrence`, `affineIndependentCopyLaw_fixedPoint_unique`, `affineIndependentCopy_map_fixedPoint_unique`	`AffineIndependentCopy.lean`; arbitrary measurable digit space and second-countable Borel real inner-product target, with completeness only for the three uniqueness declarations

Mathematical object	Principal Lean name	Defining or identifying module
Geometric uniform law μ_q	`geometricUniformDistribution`, `geometricUniformDistribution_selfSimilar`, `integral_id_geometricUniformDistribution_eq_one_half`, `geometricUniformDistribution_support_eq_Icc`, `eq_geometricUniformDistribution_of_selfSimilar`	The exhaustive `GeometricUniformLaw.lean` surface has 24 declarations and gives mean $1/2$ for every $\|q\| < 1$, including zero and negative ratios; `GeometricUniformUniqueness.lean` gives the characterization without support, density, or moment assumptions
Fixed half–quarter multisection	`evenCoordinates`, `oddCoordinates`, `geometricUniformSeries_one_half_multisection`, `geometricUniformDistribution_one_half_multisection`, `geometricUniformDistribution_one_half_conv_one_quarter`	`GeometricUniformMultisection.lean`; fixed normalized two-section only
General geometric-sinc Pochhammer factorization	`complexQPochhammerInf`, `complexQPochhammerInf_eq_qPochhammerInfIn`, `summable_norm_sineTerm_qpow_pair`, `geometricSincProduct_eq_tprod_pair`, `geometricSincProduct_eq_tprod_complexQPochhammerInf`, `rvachevFourierProduct_eq_tprod_complexQPochhammerInf`, `rvachevFourier_eq_tprod_complexQPochhammerInf`	`RvachevPochhammerFactorization.lean`; one definition and ten theorems, with the remaining defining and symbol-convergence theorems listed in the text above; the bridge to `qPochhammerInfIn` is an unconditional definitional equality, including the total-product junk-value regime
Fixed-nome entire Pochhammer symbol and simple zero lattice	`hasProdLocallyUniformly_complexQPochhammerInf`, `complexQPochhammerInf_differentiable`, `complexQPochhammerInf_eq_zero_iff`, `complexQPochhammerInf_eq_zero_iff_eq_inv_pow`, `analyticOrderAt_complexQPochhammerInf_of_eq_zero`	Exhaustive zero-definition/five-theorem surface of `QPochhammerEntire.lean`; for complex $\|q\| < 1$, locally uniform convergence in a, entireness, the $q = 0$-safe factor-zero equivalence, the reciprocal-power lattice under $q \neq 0$, and analytic order one under the historical complex-product name; the generic analytic-order theorem belongs to `QPochhammerInfinite.lean`
Normal convergence of the outer spectral Pochhammer product	`hasProdLocallyUniformly_geometricSincProduct_complexQPochhammerInf`, `hasProdLocallyUniformly_rvachevFourierProduct_complexQPochhammerInf`, `hasProdLocallyUniformly_rvachevFourier_complexQPochhammerInf`	Exhaustive zero-definition/three-theorem surface of `GeometricPochhammerNormalConvergence.lean`; locally uniform on the full complex plane for $\|q\| < 1$, including $q = 0$, with dyadic product and bounded-Fabius specializations
Finite q-Pochhammer residue-class dissection	`finiteQPochhammerIn_dissection`, `finiteQPochhammerIn_dissection_remainder`	Exhaustive zero-definition/two-theorem surface of `QPochhammerDissection.lean`; arbitrary commutative ring, all natural r, n, and remainder boundary $u \leq r$, with no hypothesis on q

Mathematical object	Principal Lean name	Defining or identifying module
Generic infinite q-Pochhammer product	`qPochhammerInfIn` and the twenty-nine theorems listed exhaustively in the current-tree discussion above	`QPochhammerInfinite.lean`; total topological-ring definition, complete-normed-ring convergence/concatenation/dissection, multiplicative-norm zero locus, locally compact field continuity, complex entireness, nonzero raw-factor derivatives, and analytic order one, including $q = 0$; distinct from the `complexQPochhammerInf` compatibility wrapper
Infinite q-binomial theorem and Euler expansions	`gaussianMajorant`, `hasSum_qBinomial_theorem`, `hasSum_euler_product`, `hasSum_euler_reciprocal`	`QBinomialTheoremInfinite.lean`; one definition and twenty-nine theorems, listed exhaustively above, including exact fixed-column convergence and error rates
Effective fixed-column Gaussian limit and rates	`norm_finiteQPochhammerIn_pow_sub_one_le`, `norm_finiteQPochhammerIn_self_mul_gaussianBinomial_sub_one_le`, `gaussianBinomial_fixedColumn_relativeError_isBigO`, `gaussianBinomial_shifted_fixedColumn_relativeError_isBigO`	`GaussianBinomialFixedColumnRate.lean`; zero definitions and nine theorems, listed exhaustively above. The generalized product bound and arbitrary-shift convergence live upstream in `QBinomialTheoremInfinite.lean`; together the two modules make `thm:fixed-column-limit` Exact, including $q = 0$
Greater-than-one Gaussian asymptotics	`gaussianBinomial_gt_one_fixedColumn_relativeError_isBigO`, `gaussianBinomial_gt_one_central_isEquivalent`	Exhaustive zero-definition/two-theorem `GaussianBinomialGreaterOneAsymptotics.lean` surface; real $q > 1$, exact fixed-column and central normalizations, and `cor:qgreaterone` Exact together with reciprocity
Thue–Morse GammaLog differential ladder	`hasDerivAt_mellin_mellinKernel_parameter`, `hasDerivAt_thueMorseGammaLog_succ`, `iteratedDeriv_thueMorseGammaLog`	Exhaustive zero-definition/three-theorem `ThueMorseGammaTowerDifferential.lean` surface. For $a > 0$ it gives the arbitrary-complex-spectral Mellin parameter shift, $L'_{r+1} = (r+1)L_r$, and the falling-factorial law for $k \le r$, making `p2:thm:gamma-tower` Exact for the chosen GammaLog coordinate; no principal-`Complex.log` or nonpositive-parameter claim
Jacobi triple product and pentagonal sums	`thetaExponent`, `pentagonalExponent`, `hasSum_jacobi_triple_product`, `hasSum_pentagonal`, `hasSum_pentagonal_pairs`	`JacobiTripleProduct.lean`; two definitions and twenty-five theorems, listed exhaustively above
Finite q-Pascal summations	`sum_gaussianBinomial_succ_mul`, `sum_gaussianBinomial_succ_mul'`, `Commute.gaussianBinomial_left`, `Commute.gaussianBinomial_right`	Exhaustive zero-definition/four-theorem surface of `QPascalSummation.lean`
Continuity and classical limit of Gaussian binomials	`continuous_gaussianBinomial`, `tendsto_gaussianBinomial_nhds_one`, `gaussianBinomial_eq_finiteQPochhammerIn_div`	Exhaustive zero-definition/three-theorem surface of `GaussianBinomialContinuity.lean`

Mathematical object	Principal Lean name	Defining or identifying module
Generic Gaussian-polynomial degree, palindromicity, mean, and linear coefficient	`reflect_add_of_natDegree_le`, `reflect_one'`, `gaussianBinomial_natDegree_le`, `gaussianBinomial_zero_left`, `gaussianBinomial_diag'`, `reflect_gaussianBinomial`, `coeff_gaussianBinomial_reflect`, `coeff_gaussianBinomial_zero`, `coeff_gaussianBinomial_top`, `gaussianBinomial_natDegree`, `gaussianBinomial_monic`, `two_mul_derivative_gaussianBinomial_eval_one`, `coeff_gaussianBinomial_one_of_pos_of_lt`, `coeff_gaussianBinomial_one`	Exhaustive zero-definition/fourteen-theorem surface of `GaussianBinomialPalindromic.lean`; total arbitrary-commutative-semiring classifier, one exactly for $0 < k < n$ and zero otherwise
Gaussian-binomial second derivative and cleared second moments	`eval_one_derivative_derivative_gaussianBinomial_X`, `twelve_mul_secondMoment_gaussianBinomial_eval_one`, `twelve_mul_varianceNumerator_gaussianBinomial_eval_one`	Three-theorem delta in the 2+24 `GaussianBinomialCumulants.lean` surface; the first uses a characteristic-zero field and $k \leq n$, while the latter two are division-free and total in n, k over every commutative semiring
Half-base q-binomial simple-root locus	`halfQBinomial_sum_rootMultiplicity_two_pow`, together with `halfQBinomial_sum_eq_zero_iff` and `gaussianBinomial_half_eq_halfQBinomial`	Exhaustive zero-definition/one-theorem surface of `HalfQBinomialRootSimplicity.lean`; every rational dyadic root 2^j, $j < n$, has multiplicity one, completing `cor:halfbase-root-locus` as Exact. No arbitrary-base or arbitrary-field simplicity claim; `cor:qbinom-inversion-law` remains Partial
Universal Gaussian-polynomial degree and palindromicity	`natDegree_gaussianBinomial_universal`, `gaussianBinomial_universal_monic`, `coeff_zero_gaussianBinomial_universal`, `gaussianBinomial_universal_reflect`, `coeff_gaussianBinomial_universal_symm`	Exhaustive zero-definition/five-theorem surface of `GaussianBinomialPolynomialStructure.lean`; the adjacent generic theorem supplies the linear-coefficient classifier
Central squared-base Gaussian reduction	`finiteQPochhammerIn_mul_neg`, `finiteQPochhammerIn_two_mul`, `finiteQPochhammerIn_map_ringHom`, `central_gaussianBinomial_sq_mul_int`, `central_gaussianBinomial_sq_mul`, `central_gaussianBinomial_sq_div`	Exhaustive zero-definition/six-theorem surface of `CentralQBinomialReduction.lean`
Cyclotomic q-factorizations	`div_add_div_le_div`, `div_le_div_add_div_add_one`, `mem_range_and_mem_divisors_iff`, `finiteQPochhammerIn_X_eq_prod_cyclotomic`, `finiteQPochhammerIn_X_eq_gaussianBinomial_mul`, `prod_cyclotomic_pow_div_extend`, `gaussianBinomial_X_eq_prod_cyclotomic`	Exhaustive zero-definition/seven-theorem surface of `CyclotomicFactorization.lean`
Primitive-root Gaussian block	`gaussianBinomial_isPrimitiveRoot_eq_zero`, `neg_one_pow_mul_pow_choose_two`, `finiteQPochhammerIn_isPrimitiveRoot`	Exhaustive zero-definition/three-theorem surface of `PrimitiveRootBlock.lean`; interior vanishing, top phase, and complete block in a commutative integral domain

Mathematical object	Principal Lean name	Defining or identifying module
The q-Lucas theorem	`add_mul_add_sub_one`, `choose_two_add`, `coeff_finiteQPochhammerIn_neg_X`, `finiteQPochhammerIn_neg_X_block`, `coeff_block_pow_mul`, `pow_choose_two_add_mul_eq`, `gaussianBinomial_q_lucas`	Exhaustive zero-definition/seven-theorem surface of `QLucas.lean`; positive primitive-root order and residue bounds remain explicit, and its local `two_mul_choose_two` is private
Terminating two-phi-one reversal	`twoPhiOneFinite`, `twoPhiOneReflection`; `choose_two_add_succ_choose_two`, `finiteQPochhammerIn_sub_eq`, `finiteQPochhammerIn_reversal_ne_zero`, `finiteQPochhammerIn_inv_pow_self`, `twoPhiOneReflection_involutive`, `twoPhiOneFinite_reversal`, `twoPhiOneFinite_reversal_twice`, `twoPhiOneFinite_eq_sum_twoPhiOneTerm`, `twoPhiOne_eq_twoPhiOneFinite_inv_pow`, `twoPhiOne_reversal`, `twoPhiOne_reversal_twice`, `twoPhiOne_one_eq_twoPhiOneFinite_zero`	Exhaustive two-definition/twelve-theorem surface of `TwoPhiOneReversal.lean`; the actual-tsum reversal is exact under its displayed nonvanishing hypotheses, reflection is involutive, and the separate zero-depth bridge includes $q = 0$
The two q-Chu–Vandermonde sums	`two_mul_choose_two`, `mul_sub_one_eq_mul_sub_add`, `finiteQPochhammerIn_div_eq_sum_chu`, `q_chu_vandermonde_first`, `finiteQPochhammerIn_div_eq_sum_chu_second`, `twoPhiOneFinite_mul_finiteQPochhammerIn_eq_chu_second`, `q_chu_vandermonde_second`, `q_chu_vandermonde_second_by_reversal`, `twoPhiOne_q_chu_vandermonde_first`, `twoPhiOne_q_chu_vandermonde_second`	Exhaustive zero-definition/ten-theorem surface of `QChuVandermonde.lean`; both displayed q-Chu formulas are exact on the full rational domain, while the specifically by-reversal route retains the additional $C \neq 0$ and $(A;q)_n \neq 0$ assumptions
Jacobi two-square count and Lambert forms	`sumSqRep_two_eq_four_mul_twoSquareDivisorSum`, `sumSqRep_two_eq_four_mul_prod`, `theta_sq_eq_chi4_lambert`, `theta_sq_eq_odd_lambert`	Exhaustive zero-definition/four-theorem `JacobiTwoSquareCount.lean` surface; the product retains its valuation hypothesis, while both Lambert identities are unconditional under $\|q\| < 1$
Cyclotomic carry criterion	`cyclotomic_exponent_eq_one_iff`, `cyclotomic_dvd_gaussianBinomial_iff`, `gaussianBinomial_mul_isPrimitiveRoot`	Exhaustive zero-definition/three-theorem surface of `CyclotomicDivisibility.lean`; rational-polynomial divisibility and integral-domain primitive-root value
MacMahon q-Catalan polynomial	`qCatalan`; `map_qInt`, `qInt_X_monic`, `qInt_X_natDegree`, `X_sub_one_mul_qInt`, `qInt_X_eq_prod_cyclotomic`, `qInt_X_dvd_gaussianBinomial_rat`, `qInt_X_dvd_gaussianBinomial_int`, `qInt_X_mul_qCatalan`, `qCatalan_natDegree`, `qCatalan_eval_one_mul`, `qCatalan_eval_one`	Exhaustive one-definition/eleven-theorem surface of `QCatalan.lean`; integral quotient, degree $n(n-1)$, and Catalan value at one

Mathematical object	Principal Lean name	Defining or identifying module
Newton interpolation and geometric divided differences	`newtonCoeff`, `nodeNewtonPoly`, `newtonInterpolant`; `newtonCoeff_eq`, `newtonCoeff_zero`, `newtonCoeff_mul_prod`, `nodeNewtonPoly_succ`, `eval_nodeNewtonPoly`, `degree_nodeNewtonPoly_lt`, `nodeNewtonPoly_eq_interpolate`, `eq_nodeNewtonPoly_of_eval_eq`, `coeff_nodeNewtonPoly_self`, `newtonCoeff_eq_sum`, `nodal_range_pow`, `prod_erase_pow_sub_pow`, `newtonCoeff_pow_eq_sum`, `newtonPoly_succ`, `eval_newtonPoly`, `degree_newtonPoly_lt`, `newtonPoly_eq_interpolate`, `eq_newtonPoly_of_eval_eq`, `coeff_newtonPoly_self`	Exhaustive three-definition/nineteen-theorem field-valued surface of `NewtonInterpolation.lean`
Jackson q-beta integral	`qBeta`; `qNumber_pos`, `qBeta_term_eq`, `qBeta_eq_prod`, `qBeta_eq_qGamma`, `qBeta_comm`, `qBeta_pos`, `qBeta_add_one_left`, `qBeta_add_one_right`	Exhaustive one-definition/eight-theorem surface of `QBetaIntegral.lean`; product and q-Gamma evaluations for $0 < q < 1$
Quantum binomial theorem	`quantumPlane_mul_pow`, `quantum_binomial`	Exhaustive zero-definition/two-theorem noncommutative surface of `QuantumBinomial.lean`
Rogers–Szegő polynomials	`rogersSzego`, `rogersSzego_succ`, `rogersSzego_three_term`, `hasSum_rogersSzego_generating`	`RogersSzegoPolynomial.lean`; one definition and nine theorems, listed exhaustively above
Geometric-uniform moment-polynomial algebra	`geometricUniformMomentPolynomial`; `geometricUniformMomentPolynomial_zero`, `geometricUniformMomentPolynomial_succ`, `geometricUniformMomentPolynomial_natDegree_le`, `geometricUniformMomentPolynomial_eval_zero`, `geometricUniformMomentPolynomial_one`, `geometricUniformMomentPolynomial_two`, `geometricUniformMomentPolynomial_three`, `geometricUniformMomentPolynomial_four`	Exhaustive one-definition/eight-theorem surface of `GeometricUniformMomentPolynomial.lean`; recursive polynomial algebra, bound, zero value, and P_0 through P_4
Real geometric-uniform moment bridge	`geometricUniformMomentPolynomial_eval₂_eq_mgf_taylorCoefficient`	Exhaustive zero-definition/one-theorem surface of `GeometricUniformMomentPolynomialBridge.lean`; exact `p7:eq:Pn-def` for real $\lvert q\rvert < 1$
Complex geometric-uniform moment product	`geometricUniformComplexMomentProduct`; `hasProdLocallyUniformly_geometricUniformComplexMomentProduct`, `differentiable_geometricUniformComplexMomentProduct`, `geometricUniformMomentPolynomial_eval₂_eq_complexMomentProduct_taylorCoefficient`	Exhaustive one-definition/three-theorem surface of `GeometricUniformComplexMomentProduct.lean`; actual locally uniform differentiable product and normalized Taylor-coefficient bridge for complex $\|q\| < 1$. This is an analytic analogue and does not turn the real probability identity into a complex one

Mathematical object	Principal Lean name	Defining or identifying module
Exterior complex geometric-uniform moment germ	`geometricUniformExteriorComplexMomentGerm`; `analyticAt_geometricUniformExteriorComplexMomentGerm`, `geometricUniformMomentPolynomial_eval₂_eq_exteriorComplexMomentGerm_taylorCoefficient`	Exhaustive one-definition/two-theorem surface of `GeometricUniformExteriorComplexMomentGerm.lean`; actual reciprocal germ, analyticity at zero, and normalized Taylor-coefficient bridge for complex $1 < \|q\|$; no unit-circle analytic value is asserted
Sharp geometric-uniform moment-polynomial degree	`coeff_geometricUniformMomentPolynomial_choose_two`, `coeff_geometricUniformMomentPolynomial_choose_two_sub_one`, `geometricUniformMomentPolynomial_natDegree_eq`	Exhaustive zero-definition/three-theorem surface of `GeometricUniformMomentPolynomialDegree.lean`; the top and subleading coefficients are $B_n(1)/n!$ and $-B_n(1)/n! + B_{n-1}(1)/(2(n-1)!)$, and the degree is $\binom{n}{2}$ exactly for even n or $n = 1$, one less otherwise. Thus `p7:thm:Pn` and `prop:qF-P-degree-sharp` are Exact
Common rational geometric-uniform moment coefficient	`geometricUniformMomentRatFunc`; `qFactorial_mul_geometricUniformMomentRatFunc`, `eval_geometricUniformMomentRatFunc_eq_complexMomentProduct_taylorCoefficient`, `eval_geometricUniformMomentRatFunc_eq_exteriorComplexMomentGerm_taylorCoefficient`, `eval_geometricUniformMomentRatFunc_one`	Exhaustive one-definition/four-theorem surface of `GeometricUniformMomentRatFunc.lean`; global $[n]_X!a_n = \mathcal{P}_n$, safe inner/exterior coefficient evaluation, and the removable $q = 1$ specialization. This makes `thm:qF-moment-polynomial` Exact without asserting analytic values at nontrivial unit roots
Geometric-uniform germ reciprocity	`geometricUniformComplexMomentGerm`; `geometricUniformComplexMomentGerm_of_norm_lt_one`, `geometricUniformComplexMomentGerm_of_one_lt_norm`, `analyticAt_geometricUniformComplexMomentGerm`, `geometricUniformComplexMomentGerm_reciprocity`, `geometricUniformComplexMomentGerm_moment_convolution`	Exhaustive one-definition/five-theorem surface of `GeometricUniformMomentReciprocity.lean`; for $q \neq 0$, $\|q\| \neq 1$, proves `EventuallyEq` reciprocity at zero and the exact derivative convolution, making `thm:qF-reciprocity` Exact; no global, unit-circle, maximal-disc, uniqueness, or pole-divisor claim
Geometric CDF F_q and density f_q	`geometricUniformCDF`, `geometricUniformDensity`, `geometricUniformDensity_zero`, `geometricUniformDensity_nonpos_of_neg`, `volume_withDensity_geometricUniformDensity_eq_zero_of_nonpos`, `geometricUniformDistribution_ne_withDensity_geometricUniformDensity_of_nonpos`, `geometricUniformCDF_hasDerivAt`, `geometricUniformDistribution_eq_withDensity`, `contDiff_geometricUniformCDF`, `contDiff_geometricUniformDensity`	`GeometricUniformCDF.lean`
Weighted-sum CDF and half-base bridges	`weightedSumCDF`, `weightedSumCDF_eq_geometricUniformCDF_one_half`, `geometricUniformCDF_one_half_eq_fabiusReal`, `geometricUniformDensity_one_half_eq_rvachevUp`	`ProbabilityRepresentation.lean`

Mathematical object	Principal Lean name	Defining or identifying module
Closed Rvachev tail and positive raw moments	`weightedSumDistribution_real_Ici_eq_rvachevUp_of_nonneg`, `integral_pow_weightedSumDistribution_eq_mul_intervalIntegral_rvachevUp`	Exactly two new theorems in the existing `ProbabilityLaplaceMoments.lean`: atomlessness gives the closed tail for every $t \geq 0$, and survival layer cake gives the full-law moment identity for every natural $n \geq 1$. With the existing global up/Fabius bridges they make `prop:up-tail` and `cor:up-moments` Exact
Totalized activation kernel p	`activationProbability`, `tanhDiv_eq_dslope`, `activationProbability_le_min_one_sq_div_three`, `activationProbability_mul_le_quadratic`, `tendsto_activationProbability_mul_div_sq`	`HyperbolicActivation.lean`, `ActivationSeries.lean`, and `ActivationAsymptotics.lean`
Finite activation Taylor jet	`tanh_sub_taylor_nine_isBigO`, `activationProbability_sub_taylor_eight_isBigO`, `activationProbability_sub_taylor_eight_isBigO_pow`	`ActivationTaylor.lean`; exact degree-nine tanh and degree-eight p polynomials with respective $O(\|x\|^{11})$ and $O(\|x\|^{10})$ remainders, plus the equivalent $O(x^{10})$ activation form
Square-summable activation series D_w	`summable_activationProbability_mul_of_summable_sq`, `tsum_activationProbability_mul_le`, `tendsto_tsum_activationProbability_mul_div_sq`	`ActivationSeries.lean` and `ActivationSeriesAsymptotics.lean`; D_w is guide notation, not a Lean definition
Geometric and dyadic activation dimensions	`geometricActivationDimension`, `dyadicEffectiveDimension`, `hasSum_normalizedGeometricWeight_sq`, `tendsto_geometricActivationDimension_div_sq`, `tendsto_dyadicEffectiveDimension_div_sq`	`GeometricActivationDimension.lean` and `GeometricActivationAsymptotics.lean`
Forward curvature profile	`deriv_deriv_fabiusReal`, `deriv_deriv_fabiusReal_eq_zero_iff`	`Convexity.lean`
Totalized inverse F^{-1}	`fabiusInv`, `fabiusOrderIso`, `fabiusInv_differentiableAt_iff`, `fabiusInv_contDiffAt_infty_iff`	`FabiusInverse.lean`
Inverse curvature	`deriv_deriv_fabiusInv`, `deriv_deriv_fabiusInv_half`, `deriv_deriv_fabiusInv_eq_zero_iff`, `strictConcaveOn_fabiusInv_firstHalf`, `strictConvexOn_fabiusInv_secondHalf`	`FabiusInverse.lean`
Exact inverse modulus	`fabiusIntervalMass`, `strictMonoOn_fabiusIntervalMass_firstHalf`, `fabiusIntervalMass_eq_fabiusReal_iff`, `fabiusInv_sub_eq_sub_iff_of_mem_Icc`, `abs_fabiusInv_sub_le`, `abs_fabiusInv_sub_eq_iff_of_mem_Icc`, `sSup_abs_fabiusInv_sub_eq`, `forall_abs_fabiusInv_sub_lt_iff`	`InverseModulus.lean`; strict and equality refinements are unit-interval qualified where clamping requires it
Effective dyadic inverse modulus	`inverseFabiusFactorialDenominator`, `inverseFabiusDeltaDenominator`, `fabiusInv_effectivelyUniformContinuous`	`FabiusInverseEffectiveContinuity.lean`; explicit primitive-recursive continuity modulus, not an inverse realizer

Mathematical object	Principal Lean name	Defining or identifying module
Logarithmic reciprocal inverse modulus	`inverseFabiusLogarithmicOrder`, `inverseFabiusLogarithmicFactorialDenominator`, `inverseFabiusLogarithmicDeltaDenominator`, `fabiusInv_effectivelyUniformContinuous_logarithmic`, `fabiusInv_effectivelyUniformContinuous_logarithmicDelta`	`FabiusInverseLogarithmicModulus.lean`; least binary-length selector and two primitive-recursive witnesses, not an inverse realizer
Sharp exact dyadic inverse modulus	`inverseFabiusExactDyadicDenominator`, `inverseFabiusExactDyadicDenominator_primrec`, `inverseFabiusExactDyadicDenominator_isLeast_strictModulus`, `inverseFabiusExactLogarithmicDenominator`, `inverseFabiusExactLogarithmicDenominator_primrec`, `abs_fabiusInv_sub_lt_inv_nat_of_lt_exactLogarithmicDenominator`	`FabiusInverseExactDyadicModulus.lean`; exact fixed-target ceiling and endpoint sharpness, primitive-recursive dyadic and logarithmic denominators, with $1/n$ only a witness and the zero input only a totalization convention
Fixed-depth effective monotone inversion	`SequentiallyComputableOn`, `unitClamp`, `unitClamp_sequentiallyComputable`, `tolerantDifference_error`, `tolerantDifference_safe_updates`, `tolerantDifference_inconclusive`, `tolerantBisection_correct`, `effectiveInversionOn_Icc`	`EffectiveMonotoneInverse.lean`; exact two-definition/six-theorem public surface, with a fixed p-step dyadic controller consuming a supplied positive computable inverse modulus
Positive-rational-gap effective inversion	`EffectivelyUniformContinuousOn`, `ComputablePositiveRationalSequence`, `ComputablePositiveRationalSequence.value`, `ComputablePositiveRationalSequence.reciprocalDenominator`, `ComputablePositiveRationalSequence.reciprocalDenominator_spec`, `inverseModulus_of_positiveRationalGap`, `effectiveInversionOn_Icc_of_computablePositiveRationalGap`, `clampedEffectiveInversion_of_computablePositiveRationalGap`	Exhaustive eight-declaration surface of `EffectiveGapInverse.lean`; derives the unit-interval modulus and continuity from supplied rational dyadic-gap bounds and certifies exactly the clamped total extension
Computable totalized Fabius inverse	`fabiusInv_isComputableRealFunction`	`FabiusInverseComputable.lean`; zero definitions and one theorem combining clamped tolerant-bisection sequential computability with logarithmic-Delta effective uniform continuity
Actual quarter inverse jet	`iteratedDeriv_centeredFabiusInv_quarter_eq_quadraticInverse`, `iteratedDeriv_fabiusInv_five_seventy_two_succ`	`FabiusInverseQuarterJet.lean`; equality of jets, not local analytic equality
Centered Thue–Morse mixed differences	`symmetricMixedDifference`, `symmetricMixedDifference_polynomial_eq_coeff_card`, `symmetricDyadicMixedDifference_eq_sum_thueMorseSign_smul`	`ThueMorseSymmetricDifference.lean`; finite algebraic identities, with no analytic limit claim

Mathematical object	Principal Lean name	Defining or identifying module
Local Thue–Morse corner integral	`centeredBoxIntegral`, `symmetricMixedDifference_range_eq_centeredBoxIntegral`, `symmetricMixedDifference_univ_eq_centeredBoxIntegral`	Exhaustive 1+4 `ThueMorseCornerIntegral.lean` surface; local open/order-connected `ContDiffOn` hypotheses on the full symmetric segment, zero widths and order zero included; makes `thm:TM-corner` Exact/Complete with the preceding algebra
Finite moment functional and affine Prouhet transport	`sum_weight_mul_eval₂_eq_sum_coeff_mul_moment`, `sum_weight_mul_eval₂_eq_eval₂_of_moments`, `sum_weight_mul_eval_affine_of_topCoeff_extractor`	Exhaustive 0+16 `FinitePolynomialFunctional.lean` surface; scalar-extension extraction and same-ring affine transport over commutative semirings, including zero scale; makes `cor:geometric-prouhet-affine` Exact with the half-base extractor
Central Rvachev–Legendre cancellation	`eval_legendrePolynomial_even_zero`, `eval_rvachevLegendreDeconvolutionPolynomial_even`, `rvachevLegendreCentralSum`	Exhaustive 0+3 `RvachevLegendreCentralSum.lean` surface; exact $M = 4^n$ finite identity, including $n = 0$, making `cor:leg-central-sum` Exact/Complete
Rvachev–Legendre analysis biorthogonality	`rvachevLegendreAnalysisKernel`, `rvachevLegendreBiorthogonality`	Exhaustive 1+1 `RvachevLegendreBiorthogonality.lean` surface. For $M \neq 0$ and synthesis degree $\ell \leq \nu_2(M)$, with arbitrary analysis degree m, it proves the normalized open-block delta identity and makes `def:leg-Lambda` and `thm:leg-biorthogonality` Exact. It does not prove the larger `thm:leg-Lambda` support/Fourier–Bessel claims, analysis-kernel rationality, or the reverse projector/rank/trace/Cauchy–Binet clauses, so `cor:leg-biorthogonal-matrices` remains Partial
Uniform spline S_p	`fabiusUniformSpline`	`FabiusUniformSpline.lean` and `FabiusComputability.lean`
Logarithmic profile	`fabiusLogProfile`	`FabiusLogScale.lean`
Forward real Lambert branch pairing	`principalLambertW_sub_lowerLambertW_pos`, `principalLambertW_eq_neg_gap_div`, `lowerLambertW_eq_neg_gap_mul_exp_div`, `lowerLambertW_eq_neg_gap_div_one_sub_exp_neg`, `eq_neg_gap_div_mul_exp`	Exhaustive zero-definition/seven-theorem surface of `LambertWBranchPairing.lean`; exact δ- and $t = e^{\delta}$-forms on $(-e^{-1}, 0)$ only
Lambert branch-gap coordinate and inverse	`gapPrincipal`, `gapLower`, `gapArg`, `branchGap`, `branchGap_invOn`, `branchGap_bijOn`, `gapArg_log`	Exhaustive four-definition/sixteen-theorem surface of `LambertWGapBijection.lean`; explicit bijection $(-e^{-1}, 0) \leftrightarrow (0, \infty)$, with the $t > 1$ inverse formulas
Symmetric real Lambert branch laws	`lowerLambertW_div_principalLambertW_eq_exp_branchGap`, `principalLambertW_add_lowerLambertW_eq_cosh_div_sinh_branchGap`, `principalLambertW_mul_lowerLambertW_eq_sinh_sq_branchGap`, `principalLambertW_add_lowerLambertW_lt_neg_two`, `principalLambertW_mul_lowerLambertW_mem_Ioo`	Exhaustive zero-definition/nine-theorem surface of `LambertWBranchSymmetry.lean`; ratio, sum, product, and strict open-domain bounds, with no endpoint or Bernoulli/higher-expansion claim

Mathematical object	Principal Lean name	Defining or identifying module
Bernoulli EGF and real branch-gap series	`summable_norm_bernoulli_mul_pow_div_factorial`, `summable_bernoulli_mul_pow_div_factorial_iff`, `hasSum_bernoulli_mul_pow_div_factorial`, `hasSum_bernoulli_mul_pow_div_factorial_complex_iff`, `principalLambertW_lowerLambertW_eq_bernoulliSeries`	Exhaustive zero-definition/five-theorem surface of `LambertWBranchGapBernoulli.lean`. The complex series has sum given by the inverse of `complexExpm1Div` at z exactly for $\lvert z\rvert < 2\pi$, hence diverges on and outside the boundary; `eq:pair-Bernoulli-general` and the canonical-removable reading of `eq:bernoulli-gen` are Exact. This is not the literal totalized quotient at zero and asserts no holomorphy; higher/full Puiseux claims remain open
Real Lambert-branch curvature	`deriv_deriv_principalLambertW`, `deriv_deriv_lowerLambertW`, the lower sign iff family, `strictConcaveOn_principalLambertW`, `strictConvexOn_lowerLambertW_left`, `strictConcaveOn_lowerLambertW_right`	`LambertWCurvature.lean`; exact regular-domain second derivatives, with shape theorems including the branch point and inflection while the lower domain stays open at zero
Real Lambert branch-point vertical geometry	`tendsto_deriv_principalLambertW_branchPoint_atTop`, `tendsto_deriv_lowerLambertW_branchPoint_atBot`, the two `secantSlope_branchPoint` limits, and the four `not_differentiable` endpoint theorems	`LambertWBranchPointGeometry.lean`; exactly eight theorems giving signed right-hand derivative/secant blow-up, failure of finite right derivatives, and failure of full differentiability
Real Lambert leading branch-point asymptotics	`lambertWBranchPointScale`, the two squared-ratio limits, the two squared equivalences, and `principalLambertW_add_one_isEquivalent_branchPoint`, `lowerLambertW_add_one_isEquivalent_branchPoint`	`LambertWBranchPointAsymptotics.lean`; one definition and eight theorems, with signed $+\sqrt{2e\Delta}$ and $-\sqrt{2e\Delta}$ laws only
Generic scaled power–exponential phases	`principalPowerExponentialPhase`, `lowerPowerExponentialPhase`, `powerExponentialLambertEpsilon`, `lowerPowerExponentialPhaseIntrinsicMain`, `deriv_principalPowerExponentialPhase_hasDerivAt`, `deriv_deriv_principalPowerExponentialPhase`, `deriv_lowerPowerExponentialPhase_hasDerivAt`, `deriv_deriv_lowerPowerExponentialPhase`	`PowerExponentialLambert.lean`, `PowerExponentialLambertCalculus.lean`, `PowerExponentialLambertInverse.lean`, `PowerExponentialLambertAsymptotics.lean`, and `PowerExponentialLambertCurvature.lean`; generic curvature sign/threshold/shape packages remain open
Classical principal/lower Lambert reference coordinates	`fabiusPrincipalLambertPhase`, `fabiusLambertPhase`	`PowerExponentialLambertFabius.lean` and `FabiusLambertSaddle.lean`
Fabius Lambert-phase curvature	`fabiusLambertInflectionInput`, `deriv_deriv_fabiusLambertPhase_eq_zero_iff`, `strictConvexOn_fabiusLambertPhase_left`, `strictConcaveOn_fabiusLambertPhase_right`, `strictConvexOn_fabiusPrincipalLambertPhase`	`PowerExponentialLambertFabiusCurvature.lean`; exact lower inflection and principal global-`Iic` convexity
Periodic correction Ψ	`negativeLaplacePsi`	`PeriodicCorrection.lean`
Even Legendre truncation	`rvachevLegendrePartialSumPolynomial`	`FabiusLegendreLeastSquares.lean`

Mathematical object	Principal Lean name	Defining or identifying module
Total squared zero-row integer Wigner datum	`legendreGauntAdmissible`, `legendreWignerThreeJZeroSqRat`, `legendreGauntRat_eq_two_mul_wignerThreeJZeroSqRat`	Exhaustive two-definition/twenty-five-theorem closed form, factorial formulas, sharp support, and positivity in `LegendreGauntClosedForm.lean`; no signed phase, half-integer/nonzero magnetic indices, orthogonality, or recoupling
Finite Rvachev/up-law Wigner-square Gram sums	`rvachevLegendreGramEntryRat_eq_two_mul_sum_wignerThreeJZeroSqRat`, `rvachevLegendreGramMatrixRat_apply_eq_two_mul_sum_wignerThreeJZeroSqRat`, `upLegendreGramMatrix_apply_eq_two_mul_sum_wignerThreeJZeroSqRat`	Exact zero-definition/three-theorem rational-entry, rational-matrix, and real-matrix surface of `FabiusLegendreGauntClosedForm.lean`
Formal arbitrary-power recurrence	`Fabius.coeff_recurrence_of_mul_derivative_eq`, `Fabius.mul_derivative_fallingSeries_subst_sub_one`, `Fabius.coeff_fallingSeries_subst_sub_one_recurrence`	Exhaustive 0+3 `UnitSeriesPowerRecurrence.lean` surface. Multiplying $FG' = \beta F'G$ by X, extracting the n-th Euler-derivative coefficient, and isolating the $j = 0$ term gives $na_0c_n = \sum_{j=1}^{n}((\beta+1)j - n)a_jc_{n-j}$ over every commutative ring. Formal substitution in the falling-factorial series gives the unit-constant power specialization over a rational algebra. Thus the power clause of `alg:merged-exp-log-power` is Exact, with no analytic branch claim. The 207-row coefficient-calculus concordance is 65 Lean, 35 Partial, and 107 None.
Weighted binomial translation and inversion	`Bell.binomialConv_unitSeq`, `Bell.binomialConv_four_swap`, `Appell.translate_zero`, `Appell.translate_translate`, `Appell.binomialConv_translate`, `Appell.translate_injective`, `Appell.translate_neg_translate`, `Appell.translate_translate_neg`, `Appell.translate_eq_iff`, `Appell.weighted_binomial_inversion_iff`, `Appell.binomialConv_translate_neg_translate`	Eleven-declaration extension of `AppellSequence.lean`. `thm:merged-weighted-binomial-translation` is Exact over commutative semirings, with left-cancellative addition needed only for injectivity; `cor:merged-weighted-binomial-inversion` is Exact over commutative rings. These are finite identities, not analytic EGF claims.

Mathematical object	Principal Lean name	Defining or identifying module
Unit-series Bell conversion, powers/logarithms, and exponential jets	`ordPartialBell_eq_factorialRatio_partialBell`, `factorial_mul_ordPartialBell_eq_factorial_mul_partialBell`, `coeff_fallingSeries_subst_eq_sum_ordPartialBell`, `coeff_fallingSeries_subst_eq_sum_ordPartialBell_of_pos`, `coeff_fallingSeries_subst_eq_sum_partialBell`, `coeff_negBinomSeries_subst_eq_sum_ordPartialBell`, `coeff_negBinomSeries_subst_eq_sum_ordPartialBell_of_pos`, `coeff_logOf_eq_sum_ordPartialBell`, `egfA_factorialDenormalize_coeff_eq`, `bellWeightSeries_factorialDenormalize_coeff_eq`, `coeff_logOf_eq_sum_partialBell`, `coeff_exp_subst_eq_completeBell`, `coeff_exp_subst_eq_partitionExpSum`, `coeff_exp_subst_eq_sum_weightedPartitions`, `coeff_exp_subst_eq_sum_div_weightedPartitions`, `coeff_exp_subst_recurrence`	Exhaustive 0+16 `UnitSeriesBellCoefficients.lean` surface. The labels `p0:lem:bell-conversion`, `p0:lem:power-log`, and `p0:cor:exp-log-jets` are Exact as formal power-series coefficient algebra; no analytic convergence or logarithm branch is asserted.
Dickson and Neumann well-basedness	`dickson_isPWO`, `dickson_antichain_finite`, `dickson_isPWO_pi`, `neumann_isPWO`, `neumann_finite_factorizations`, `neumann_isPWO_orderDual`, `neumann_finite_factorizations_orderDual`; generated `neumann_add_isPWO`, `neumann_finite_decompositions`	Exhaustive 0+7 written `TransseriesWellBased.lean` surface, plus the two `to_additive` names outside the lexical count. `q0:lem:dickson` remains Exact. `q0:lem:neumann` is Exact for the manuscript's totally ordered commutative monomial group after the reverse-well-order/strict-growth convention is represented by `Set.IsPWO` on `OrderDual`; the two named wrappers cover both clauses. They are proved more generally for a partially ordered cancel monoid, and do not construct Hahn series.
Elementary height comparison	`isLittleO_log_pow_rpow`, `isLittleO_log_pow_id`, `isLittleO_pow_mul_log_pow_exp`	Exhaustive 0+3 `TransseriesHeight.lean` surface. The two printed real `atTop` comparisons of `q0:prop:height` are Exact; no recursive height/depth type for arbitrary nested transmonomials is defined.
Sequence-indexed scale and coefficient uniqueness	`IsAsymptoticScale`, `poincarePartialSum`, `poincarePartialSum_zero`, `poincarePartialSum_succ`, `IsPoincareExpansion`, `IsPoincareExpansion.isLittleO_succ_remainder`, `IsPoincareExpansion.tendsto_coeff`, `IsPoincareExpansion.tendsto_coeff_div`, `IsPoincareExpansion.coeff_unique`	Exhaustive one-structure, two-definition, six-theorem `TransseriesScale.lean` surface. The sequence forms of `q0:def:scale`, `q0:def:poincare`, and `q0:prop:uniqueness` are Exact for coefficients in an arbitrary normed space over the scalar field; the quotient wrapper is scalar-valued. Uniqueness needs `[l.NeBot]`, and no convergence, flat-remainder recovery, unordered-set, or maximal-scale claim is made.

Mathematical object	Principal Lean name	Defining or identifying module
Power–logarithmic dominance	`plMonomial`, `tendsto_plMonomial_atTop_zero`, `plMonomial_div_eventuallyEq`, `tendsto_plMonomial_div_atTop_zero`, `tendsto_plMonomial_div_atTop_one`, `plMonomial_pos`, `tendsto_plMonomial_div_atTop`, `plMonomial_generators_dominance`	Exhaustive 1+7 `TransseriesScaleDominance.lean` surface. The analytic trichotomy and integer-generator rule of `plt:lem:mot-dominance` are Exact; the full unordered set/maximal package is not claimed.
Power–logarithmic scale sequences	`isLittleO_plMonomial`, `isAsymptoticScale_plMonomial`, `isAsymptoticScale_plMonomial_pow`, `isAsymptoticScale_plMonomial_log`	Exhaustive 0+4 `TransseriesPolyLogScale.lean` surface. Strictly decreasing exponent sequences and the two standard ladders are exact; the unordered-set and finite-maximal clauses of `plt:def:mot-scale` remain outside it.
Polynomial logarithmic-block antidifferentiation	`blockOperator`, `blockAntiderivative`, `resonantAntiderivative`; `sum_sub_sum_shift`, `blockOperator_zero`, `blockOperator_sub`, `blockOperator_blockAntiderivative`, `blockOperator_surjective`, `natDegree_C_mul_of_ne_zero`, `natDegree_blockOperator`, `blockOperator_injective`, `blockOperator_bijective`, `derivative_resonantAntiderivative`, `derivative_surjective`, `natDegree_resonantAntiderivative`	Exhaustive 3+12 `TransseriesBlockAntiderivative.lean` surface. The polynomial operator dichotomy of `plt:lem:mot-block-antiderivative` is Exact: explicit degree-preserving nonresonant inverse and degree-raising resonant primitive. It does not construct the full Laurent ring.
Abstract differential-block bridge	`Fabius.derivation_pow_t`, `Fabius.derivation_block`, `Fabius.derivation_zpow_block`, `Fabius.exists_zpow_block_primitive`, `Fabius.existsUnique_zpow_block_primitive`, `Fabius.exists_block_primitive`, `Fabius.derivation_block_zero`, `Fabius.exists_block_primitive_resonant`, `Fabius.derivation_val_inv`, `Fabius.derivation_pow_inv`, `Fabius.derivation_zpow_t`, `Fabius.derivation_block_zpow`	Exhaustive 0+12 `TransseriesDifferentialBlock.lean` surface. The natural and integer-exponent forms of `plt:eq:mot-block-derivative` are Exact. The source-shaped Laurent law and degree-preserving nonresonant primitive use an ambient field and $t \neq 0$; uniqueness also needs injective evaluation at L. The older integer form packages t as a unit. This closes exactly item (v) of `plt:thm:mot-smallest-differential-algebra`; that compound theorem remains Partial because the minimal-ring, growth, algebraic-independence, and concrete Laurent-ring clauses are absent.
Catalan quadratic core	`quadHalf`, `halfBinom`, `quadCoef`; `catalan_two_step`, `quadHalf_zero`, `quadHalf_antidiagonal`, `halfBinom_step`, `quadHalf_rat`, `quadCoef_rat`, `quadCoef_zero`, `quadCoef_rec`	Exhaustive 3+8 `QuadraticCoreCatalan.lean` surface. `p6:prop:quadratic-core-catalan` is Exact; `p6:lem:quadratic-core` is Partial because only the coefficient family and coefficientwise recursion are proved; and `p6:thm:deepest-pole` is Absent because no Gamma/Barnes identification is formalized.

Mathematical object	Principal Lean name	Defining or identifying module
Derangement nearest integer	`Fabius.subfactorialDefect`, `Fabius.subfactorialDefect_zero`, `Fabius.subfactorialDefect_succ`, `Fabius.subfactorialDefect_pos`, `Fabius.subfactorialDefect_lt`, `Fabius.numDerangements_sub_eq`, `Fabius.abs_numDerangements_sub_lt_half`, `Fabius.round_factorial_mul_exp_neg_one`	Exhaustive 1+7 `DerangementNearestInteger.lean` surface. The integer identity, bounds, and nearest-integer conclusion of `p8:cor:nearest-integer` are Exact. The compound printed result is Partial at arbitrary real $x > -1$, while `p8:thm:branch-splitting` and its branch family are Absent.
Linear–logarithmic core inversion	`Fabius.linLogCoreArg`, `Fabius.linLogCoreRoot`, `Fabius.linLogCoreThreshold`, `Fabius.linLogCoreRootLower`, `Fabius.linLogCore_eq_iff`, `Fabius.principalLambertW_linLogCoreArg_pos`, `Fabius.linLogCoreRoot_pos`, `Fabius.linLogCore_linLogCoreRoot`, `Fabius.strictMonoOn_linLogCore`, `Fabius.linLogCoreRoot_unique`, `Fabius.hasDerivAt_linLogCore`, `Fabius.linLogCore_slope_eq`, `Fabius.linLogCore_critical`, `Fabius.linLogCoreArg_mem_Ioo_iff`, `Fabius.principalLambertW_linLogCoreArg_neg`, `Fabius.linLogCoreRoot_pos_of_neg`, `Fabius.linLogCoreRootLower_pos`, `Fabius.linLogCore_linLogCoreRoot_of_neg`, `Fabius.linLogCore_linLogCoreRootLower`, `Fabius.linLogCoreRoot_lt_critical`, `Fabius.critical_lt_linLogCoreRootLower`, `Fabius.linLogCoreRoot_ne_linLogCoreRootLower`	Exhaustive 4+18 `LinLogCoreInversion.lean` surface. Parts (2)–(4) of `p0:thm:lambert-core` are Exact over the reals, including both signs of b. The compound theorem remains Partial because the asymptotic clause and complex general-W_k reading are absent.
Ordinary partial Bell normalization	`Fabius.ordinarySeries`, `Fabius.ordinaryPartialBell`, `Fabius.ordinaryPartialBell_pow`, `Fabius.bellWeightSeries_eq_ordinarySeries`, `Fabius.factorial_mul_ordinaryPartialBell`, `Fabius.ordinaryPartialBell_eq_zero_of_lt`	Exhaustive 2+4 `OrdinaryPartialBell.lean` surface. The generating characterization, normalization bridge, and vanishing clause of `plt:lem:bell-normalizations` are Exact. The compound printed lemma is Partial at its displayed multinomial-sum definition and separate $C_{n,k}$ identification, which this module does not prove.
Power–logarithmic core inversion	`Fabius.powerLogCore`, `Fabius.powerLogCoreArg`, `Fabius.powerLogCoreRoot`, `Fabius.powerLogCore_exp`, `Fabius.log_powerLogCoreRoot_sub`, `Fabius.powerLogCore_of_lambert`, `Fabius.powerLogCore_powerLogCoreRoot`, `Fabius.hasDerivAt_powerLogCore`, `Fabius.hasDerivAt_powerLogCore_root`	Exhaustive 3+6 `PowerLogCoreInversion.lean` surface. The real $r = 1$ inversion and slope clauses of `p6:lem:core` are Exact. The compound lemma is Partial because general r, its root determination, and the complex general-W_k reading are absent.

Mathematical object	Principal Lean name	Defining or identifying module
Remainder transport	`Fabius.lipschitzOn_of_abs_deriv_le`, `Fabius.transport_bound_mul`, `Fabius.transport_bound`	Exhaustive 0+3 `RemainderTransport.lean` surface. Part (1), `p0:eq:transport-bound`, of `p0:thm:remainder-transport` is Exact. The compound theorem is Partial because part (2), the first-order law needing a second-order Taylor estimate, is absent.
Staircase inversion	`Fabius.isLeast_ceil`, `Fabius.staircase_ceil`, `Fabius.staircase_separation`, `Fabius.staircase_separation_fails`, `Fabius.staircase_round`, `Fabius.isLeast_residue_class`, `Fabius.exists_half_error_of_jump`	Exhaustive 0+7 `StaircaseInversion.lean` surface. All five clauses of `p0:thm:staircase` are Exact at the strict-monotonicity and inverse equation interface, including the genuine separation failure. Construction of an interpolation and the periodic Fourier expansion are absent.
Flatness and invisible remainders	`Fabius.IsFlat`, `Fabius.flatSubmodule`, `Fabius.AbsorbsScale`, `Fabius.powScale`, `Fabius.isFlat_zero`, `Fabius.IsFlat.add`, `Fabius.IsFlat.neg`, `Fabius.IsFlat.sub`, `Fabius.IsFlat.const_smul`, `Fabius.isFlat_exp_neg_rpow_atTop`, `Fabius.IsPoincareExpansion.add_flat`, `Fabius.IsPoincareExpansion.sub_same_coeff_isFlat`, `Fabius.IsPoincareExpansion.iff_sub_isFlat`, `Fabius.IsFlat.smul_of_scale_absorption`, `Fabius.IsFlat.smul_of_isBigO_inv_pow`, `Fabius.mem_flatSubmodule_iff`, `Fabius.IsFlat.mul_absorbsScale`, `Fabius.absorbsScale_const`, `Fabius.IsPoincareExpansion.add_isFlat`, `Fabius.isFlat_sub_of_isPoincareExpansion`, `Fabius.isPoincareExpansion_iff_isFlat_sub`, `Fabius.isPoincareExpansion_zero_iff`, `Fabius.powScale_eq_rpow`, `Fabius.absorbsScale_of_isBigO_pow`, `Fabius.isFlat_exp_neg`, `Fabius.isPoincareExpansion_add_exp_neg`	Exhaustive 4+22 `TransseriesFlat.lean` surface. `q0:def:flat` and all three clauses of `q0:prop:invisible` are Exact for vector-valued functions and coefficients on a scalar scale, with the same-coefficient statement strengthened to an equivalence. Variable scalar multiplication requires scale absorption; the named submodule and compatibility wrappers remain scalar-valued. No convergence or recovery beyond flat remainders is asserted.
Harmonic-increment leading order	`Fabius.tendsto_div_atTop_of_tendsto_sub`, `Fabius.tendsto_div_atTop_of_harmonic_increment`	Exhaustive 0+2 `TransseriesHarmonicIncrement.lean` surface. The leading $w_n/n \to c_0$ clause of `plt:lem:mot-harmonic` is Exact. The compound lemma is Partial because the logarithmic correction, constant, and $o(1)$ remainder are absent.

Mathematical object	Principal Lean name	Defining or identifying module
Stirling-series coefficients	`Fabius.stirlingKernelCoeff`, `Fabius.stirlingKernel`, `Fabius.stirlingCoeff`, `Fabius.coeff_stirlingKernel`, `Fabius.constantCoeff_stirlingKernel`, `Fabius.stirlingCoeff_recurrence`, `Fabius.bernoulli_three`, `Fabius.bernoulli_four`, `Fabius.stirlingKernelCoeff_one`, `Fabius.stirlingKernelCoeff_two`, `Fabius.stirlingKernelCoeff_three`, `Fabius.stirlingCoeff_zero`, `Fabius.stirlingCoeff_one`, `Fabius.stirlingCoeff_two`, `Fabius.stirlingCoeff_three`	Exhaustive 3+12 `StirlingSeriesCoefficients.lean` surface. The formal definition, recurrence, and four displayed values in `q2:eq:stirling-cj` are Exact. The surrounding Fubini-sector corollary remains Partial because the analytic sector expansion is not proved by this leaf.
Wright omega basics	`Fabius.wrightOmega`, `Fabius.analyticAt_wrightOmega`, `Fabius.wrightOmega_pos`, `Fabius.wrightOmega_add_log`, `Fabius.principalLambertW_eq_wrightOmega_log`, `Fabius.wrightOmega_leftInverse`, `Fabius.wrightOmega_strictMono`, `Fabius.wrightOmega_one`, `Fabius.one_le_wrightOmega`, `Fabius.wrightOmega_le_self`, `Fabius.sub_log_le_wrightOmega`, `Fabius.wrightOmega_envelope`, `Fabius.add_one_div_two_le_wrightOmega`, `Fabius.tendsto_wrightOmega_atTop`	Exhaustive 1+13 `WrightOmega.lean` surface. The complete real proposition `plt:prop:mot-omega-basic` is Exact: its equation, inverse, positivity, strict monotonicity, Lambert bridge, envelope, divergence, and real-analyticity clauses are formalized. The analyticity theorem is `AnalyticAt` over $\mathbb{R}$ at every real input; it makes no complex holomorphic-continuation or branch claim.
First two Wright-omega orders	`Fabius.wrightOmega_lt_self`, `Fabius.tendsto_log_wrightOmega_div_atTop_zero`, `Fabius.tendsto_wrightOmega_div_atTop_one`, `Fabius.tendsto_log_wrightOmega_sub_log_atTop_zero`, `Fabius.tendsto_log_wrightOmega_div_log_atTop_one`, `Fabius.self_sub_wrightOmega_isEquivalent_log`, `Fabius.wrightOmega_sub_self_isEquivalent_neg_log`, `Fabius.wrightOmega_residual_isEquivalent`	Exhaustive 0+8 `WrightOmegaTwoOrders.lean` surface. The two concluding real equivalences of `plt:prop:mot-two-orders` are Exact at `atTop`. The proposition remains Partial: its four-term expansion and explicit quantitative envelope are not formalized here.

The table maps notation to declarations; it does not assert that every informal formula in either source paper is a proved theorem. In particular, `reshetnikovQuestion` and `conjecture_sixteen` record open assertions, the lower-Lambert coordinate is not claimed to solve the full residual phase's exact critical equation, the periodic correction is not constant, finite complex-q approximants need not be translation independent, and the literal k-fold polygonal limit is formally refuted.

D Annotated module guide

The following guide groups the principal files by role. The development contains many fine-grained helper modules; the descriptions emphasize the transition each module contributes rather than every

local lemma.

Module	Role in the development
Core interfaces and construction	
`Arithmetic.lean`	Exact natural/rational sequences, binary weight, Thue–Morse signs, product normalizations, inverse-power values, and executable dyadic evaluators.
`Basic.lean`	The unit-interval codomain, bounded-function type, abstract IsFabius specification, up-function, signed extension names, transforms, moments, and generating functions.
`Differential.lean`	Derivative identities for any IsFabius candidate, including iterated scale formulas and reflected branches.
`Existence.lean`	Banach fixed-point construction of `Existence.boundedCandidate`, calculus bootstrap, unique existence, and generic equality of all `IsFabius` candidates. The canonical choice itself is made in `PaperStatements.lean`.
`ScaleTranslation.lean`	Reusable chain-rule formulas for affine rescaling and translation of Fabius and up-functions.
`GlobalExtension.lean`	Local finiteness, smoothness, unit-interval agreement, and translation laws of the signed global extension.
`NormalizedVolterra.lean`	Base-point and endpoint-order independent repeated integration. Over arbitrary real normed targets it provides direct normalized kernels, division-free affine covariance for every scale (and an inverse form at nonzero scale), arbitrary base-point shifts into a local tail plus finite jet, the zero-local-tail specialization, and finite Hasse-polynomial and monomial commutators. Completeness is confined to the continuous literal-iterator/strict-FTC bridge, derivative cancellation and smoothness, and Taylor reconstruction.
`FractionalVolterra.lean`	Total Gamma-normalized real-order Volterra operator, unconditional endpoint/locality/scalar laws, the order-one and positive-natural-order bridges, and positive-order kernel integrability and input additivity on ordered intervals.
`FractionalVolterraSemigroup.lean`	Shifted beta-kernel integrability and evaluation, and addition of two positive real orders for continuous Banach-valued inputs on ordered compact intervals.
`FractionalVolterraCalculus.lean`	Positive-scale affine covariance at arbitrary real order on ordered endpoints, its inverse-smul form, and the positive-order integration-by-parts law raising the order of a right interior derivative, including the exact boundary-free specialization.
`GlobalDyadic.lean`	Exact dyadic identities for the signed extension with unrestricted and nonreduced representatives.
`DyadicDerivativeFiltration.lean`	Zero definitions and six theorems: support vanishing, the above-depth zero and critical-depth signed value, exact denominator-depth detection, and the all-real and dyadic below-depth formulas through `extendedFabius`.
Global shape, inversion, and computability	

Module	Role in the development
`ScalingRecurrence.lean`	Mathlib-only chain rules for arbitrary positive dilations, in vector and scalar forms, with caller-normalized outgoing weight; inert-parameter variants and dyadic wrappers used by negative-Laplace derivatives.
`Monotonicity.lean`	Strict increase of F on $[0, 1]$, endpoint range, and order consequences.
`Regularity.lean`	Optimal global Lipschitz constants for F and up.
`Convexity.lean`	Exact second-derivative formula and sign/zero loci, convexity before the midpoint, and concavity afterward.
`GlobalBounds.lean`	Sharp value and iterated-derivative bounds for the signed global extension and up, including attainment and derivative-support control.
`BoundedDerivatives.lean`	Exact global sup-norm bounds for every derivative, including attainment.
`FabiusAntiderivatives.lean`	Complete natural-order primitive ladder for the signed extension at every real endpoint, its bounded specialization on $x \leq 1$, and finite natural-monomial formulas for both.
`FabiusFractionalVolterra.lean`	Positive-real one-step fractional order shift for the signed extension on $x \geq 0$, its bounded specialization on $[0, 1]$, and the affine bridge from up based at -1 to the bounded Fabius function for $x \geq -1$.
`EffectiveFlatness.lean`	Explicit power bounds at the flat endpoint.
`SharpFlatness.lean`	Factorial-strengthened endpoint power bounds for F and up.
`NowhereAnalytic.lean`	Exact real-analytic loci: off the closed transition supports and nowhere on them.
`FabiusInverse.lean`	Totalized inverse, order isomorphism on the unit interval, exact positive reciprocal derivative, exact global smoothness locus (continuity everywhere, but differentiability and every positive finite or C^∞ order exactly off the clamping endpoints), reciprocal-cubic second derivative, midpoint jet and interior curvature loci, strict closed-half curvature, and endpoint quotient/derivative blow-up.
`InverseModulus.lean`	Exhaustive one-definition/thirty-one-theorem structural inverse-modulus layer: global weak and maximal strict fixed-length increment shape, exact endpoint minimizers, constrained-superadditivity equality, unit-interval inverse-gap rigidity, global sharp modulus bounds and attained suprema, and effective injectivity. The inverse equality loci are deliberately not global because of the clamped tails.
`FabiusInverseEffectiveContinuity.lean`	Exhaustive two-definition/twelve-theorem effective inverse-continuity layer: primitive-recursive factorial and report denominators, recurrence and endpoint-mass lower bounds, strict and closed global inverse moduli, and `EffectivelyUniformContinuous`; the inverse remains noncomputable and no sequential realizer or combined computable-function theorem is supplied.

Module	Role in the development
`FabiusInverseLogarithmicModulus.lean`	Three definitions and fifteen theorems: the primitive-recursive least binary-length order, its exact `Nat.log2` formula and minimality, factorial and Delta denominators at that order, strict/closed-input reciprocal moduli, and an effective-uniform-continuity theorem for either witness; no exact endpoint-mass ceiling, tolerant-bisection realizer, sequential computability, combined computable-real-function theorem, or input-bit asymptotic.
`FabiusInverseExactDyadicModulus.lean`	Exactly two definitions and ten theorems: the positive primitive-recursive exact endpoint-mass ceiling denominator, its rational bound and arithmetic leastness, its strict inverse modulus, the endpoint counterexample for every smaller positive denominator, fixed-dyadic strict-modulus leastness, and the primitive-recursive logarithmic $1/n$ witness with $d(0) = 1$. Leastness is only for the fixed dyadic target; no leastness for $1/n$, zero-input modulus conclusion, or input-bit asymptotic is asserted.
`EffectiveMonotoneInverse.lean`	Exactly two public definitions and six theorems: sequential computability on a set, the computable unit clamp, the difference certificate and its safe-update and inconclusive-success consequences, fixed-depth dyadic tolerant-bisection correctness, and the abstract unit-interval inversion theorem. The abstract theorem consumes a positive computable inverse modulus.
`EffectiveGapInverse.lean`	Exactly eight public declarations: effective uniform continuity on a set; a computable positive rational sequence, its value and reciprocal denominator with specification; the rational-gap inverse modulus; unit-interval sequential computability plus effective continuity; and the total computability of the clamped extension.
`FabiusInverseComputable.lean`	Zero public definitions and exactly one theorem, `fabiusInv_isComputableRealFunction`: the total inverse preserves computable real sequences by clamping and tolerant bisection and has the logarithmic-Delta effective continuity modulus.
`FabiusInverseQuarterJet.lean`	Full smooth derivative jet of the centered inverse at $5/72$, identified with the factorial-scaled coefficients of the quadratic compositional inverse; this is not a local analytic identity.
`ImplicitPowerSeries.lean`	Nonmonic complete-adic lifting and residue-class uniqueness, with arbitrary-base and zero-constant formal implicit-root specializations over power-series rings; it does not construct or analytically realize a Fabius inverse germ.
`QuarterCatalanGerm.lean`	The explicit scaled quarter Catalan formal germ, its functional equation and uniqueness, and the forward/reverse rescaling bridge from `dyadicGermTwo` to `QuadraticInverse.inverse`.

Module	Role in the development
`InverseBranch.lean`	Analytic inverse theorem, analytic-density preservation for continuous right-inverse branches, and the complement-interior analyticity condition and conditional Lambert-branch criterion for `IsElementaryOrInverse`.
`AlgebraicBranch.lean`	Algebraic-function branch interface used in non-elementarity arguments.
`ElementaryFunction.lean`	Formal expression language and semantics for elementary real functions.
`NotElementary.lean`	Non-elementarity theorem for the bounded Fabius function on its transition interval.
`InverseNotElementary.lean`	Interior noncriticality derived from the positive derivative API in `FabiusInverse.lean`, the exact analytic locus of `fabiusInv`, and local analytic-density/non-elementarity obstructions.
`FabiusComputability.lean`	Global uniform spline error $\leq 2^{-p}$, moving-input stability and diagonal convergence, an effective uniform-continuity modulus, and the computable-real-function bridge.
`FabiusComputableSpline.lean`	Primitive-recursive bounded and signed-global dyadic spline evaluators, each with an explicit error certificate, and their computable-real interfaces.
Dyadic formulas and verified computation	
`DyadicClosedForm.lean`	Closed finite Taylor formulas, highest-bit decompositions, and equivalence among exact dyadic expressions.
`DyadicCorrectness.lean`	Termination-facing invariants, table-prefix stability, Horner correctness, refinement invariance, and evaluator representation independence.
`DyadicAnalytic.lean`	Finite Taylor recurrence for an abstract IsFabius function and proof that analytic dyadic values equal the exact evaluator.
`ExactInversePower.lean`	Links inverse-power Fabius values with exact moments and table entries.
`TaylorReduction.lean`	Finite Taylor-reduction identities used to move between dyadic cells.
`FabiusDyadicLogBounds.lean`	Explicit logarithmic bounds for inverse-dyadic values, used in endpoint asymptotics.
`DyadicSharpConditional.lean`	Conditional sharp statements at dyadic points under specified recurrence estimates.
`DyadicSharpDecomposition.lean`	Exact decomposition of dyadic logarithms into main and correction pieces.
`FabiusDyadicSharpCumulant.lean`	Cumulant organization specialized to inverse-dyadic asymptotics.
`FabiusDyadicSharpAsymptotic.lean`	Sharp asymptotic statements along the inverse-power sequence.
Moments, arithmetic, and denominators	
`AnalyticMoments.lean`	Compact-support integration, total mass, moment and half-moment recurrences, identification with exact rational sequences, and the named even Fourier modes `rvachevFourierMomentTerm` with their zero term, scaling law, and genuine `HasSum` to the Rvachev Fourier transform.

Module	Role in the development
`MomentPowerSeries.lean`	Exact `PowerSeries` algebra over $\mathbb{Q}$, including $M(4X) = M(X)S(X)$; analytic evaluation is deferred to later modules.
`FinitePrefixAppellRecovery.lean`	Exhaustive 11+17 rational finite-prefix Appell API named in the crosswalk: constructive digit-moment convolutions, exact uncentered and centered-even expansions, exact symbolic outer degrees, and finite geometric-Lagrange recovery. Specializing the inner variable can lower the centered outer degree; no analytic MGF convergence is asserted.
`NormalizedMoments.lean`	Division-free normalization theorem for half moments using factorial and Mersenne products.
`NormalizedEvenMoments.lean`	Natural numerator and common-denominator formula for even centered moments.
`Parity.lean`	Lucas-theorem parity analysis and oddness of natural moment numerators.
`TwoAdic.lean`	Reduced numerator/denominator parity and exact two-adic valuation consequences.
`PrimePowerBinomialValuation.lean`	No definitions and six exhaustive theorems for prime-power Pascal rows: additive/subtraction forms for $0 < j \le p^m$, the unit row $p^m - 1$, the exact companion $\nu_p\binom{p^m-2}{j-1} = \nu_p(j)$ for $0 < j < p^m$, and both dyadic wrappers; no histogram result.
`DenominatorBound.lean`	Common-denominator divisibility and explicit denominator bounds for source-facing arithmetic results.
`HalfMomentDenominator.lean`	Denominator control specialized to half moments and inverse-power values.
`BernoulliRecurrences.lean`	Translation of moment recurrences into Bernoulli-number identities used by the arithmetic paper.
`LaplaceMoments.lean`	Moment identities in the Laplace-transform normalization.
`LaplaceMomentBounds.lean`	Uniform factorial/exponential bounds needed for transform series and differentiation.
`ProbabilityLaplaceMoments.lean`	Moment and Laplace identities connecting the weighted-sum probability model with the analytic generating function; its two newest theorems, `weightedSumDistribution_real_Ici_eq_rvachevUp_of_nonneg` and `integral_pow_weightedSumDistribution_eq_mul_intervalIntegral_rvachevUp`, identify the atomless closed tail with up on the nonnegative ray and give the full-law positive-natural-degree tail-moment formula, making `prop:up-tail` and `cor:up-moments` Exact for $t \ge 0$ and natural $n \ge 1$.
`FabiusRecurrenceSequence.lean`	Auxiliary exact recurrence sequence used in asymptotic and dyadic estimates.
Fourier, probability, and the first paper	
`FourierProduct.lean`	Entire sinc, pointwise multipliability, finite-product recurrence, and equality of the product with the analytic Fourier transform.

Module	Role in the development
`RvachevLaurentLeading.lean`	Exhaustive 1+6 centered-MGF/Fourier rotation and integer-cofactor limit API, including the manuscript's exact odd-core leading Laurent coefficient. Limits are punctured because inversion is totalized at the pole; no lower Laurent coefficients are supplied.
`GeneralizedRvachevIdentifiability.lean`	Zero definitions and six theorems recovering an admissible exponent sequence from dyadic analytic orders, with the exact generalized-product equality criterion.
`FourierAnalytic.lean`	Entire Fourier transform of up, Fourier inversion consequences, and its imaginary-axis relation to the half-moment generating function.
`OriginalCharacterization.lean`	Rvachev's original compact-support predicate, derived scale positivity, the forced scale and normalization, and the general fold from bounded Fabius solutions.
`OriginalUniqueness.lean`	Fourier-refinement uniqueness, the exact kernel of the fold, and the equivalences with fixed candidates or unique bounded Fabius witnesses.
`WeightedUniformSeries.lean`	Absolutely summable uniform-coordinate series, their canonical pushforward laws, pointwise and law-level head–tail splitting, reflection, and carried-interval results.
`WeightedUniformDistribution.lean`	Exact zero-definition/eleven-theorem compatibility API for weighted laws, including the generic half-total-weight barycenter, scalar transport, affine recursion, sharp absolute continuity from any nonzero real coefficient, and null singletons.
`WeightedUniformSupport.lean`	Support equal to the continuous series range, exact signed-real min/max support intervals, and nonnegative unit-mass specializations.
`ContinuousCDF.lean`	Generic continuity of atomless probability CDFs, reflection symmetry from measure invariance, and the theorem identifying any everywhere pointwise CDF derivative as the exact Lebesgue density without prior regularity or absolute continuity assumptions.
`AffineIndependentCopy.lean`	The affine independent-copy convolution operator, its probability and product-map forms, characteristic-function factorization and finite fixed-point iteration, and contractive uniqueness. The digit space is arbitrary measurable, the target is second-countable Borel real inner-product, and completeness is confined to the three uniqueness declarations; $\lvert q\rvert < 1$ includes zero and negative scales without support, density, or moment hypotheses.
`GeometricUniformLaw.lean`	Exhaustive 24-declaration API for the named weights $(1-q)q^n$, their series and probability law, exact mean $1/2$ for $\lvert q\rvert < 1$ including zero and negative ratios, affine self-similarity, reflection, absolute continuity, null singletons, and exact range/support statements.
`GeometricUniformMultisection.lean`	Fixed normalized even/odd split of the half-ratio coordinate series into two independent quarter-ratio copies, with pointwise, product-map, and convolution forms.

Module	Role in the development
`GeometricUniformUniqueness.lean`	Specializes the generic product-map theorem to prove `eq_geometricUniformDistribution_of_selfSimilar`: every probability solution of the geometric affine head–tail equation is μ_q when $\lvert q\rvert < 1$, including $q = 0$ and negative q, with no support, density, or moment assumption.
`GeometricSincFactorization.lean`	Closed characteristic function of the scaled uniform digit, exact closed-factor and phase-collapsed finite residual formulas for $\lvert q\rvert < 1$, pointwise convergence of the phase-bearing sinc prefixes, and the two dyadic weighted-sum wrappers.
`GeometricReciprocalGamma.lean`	Locally uniform convergence, pointwise multipliability and `HasProd`, and entireness of the geometric sinc product, together with its reciprocal-Gamma reflection factorization.
`GeometricSincCharacteristicFunction.lean`	The four-theorem, zero-definition surface: two exact phase-bearing bridges from the geometric-uniform characteristic function to the rescaled geometric sinc product and paired reciprocal-Gamma factors, plus locally uniform convergence of the full phase-bearing prefixes on the real frequency line and uniform convergence on every compact real frequency set, for every $\lvert q\rvert < 1$, including zero and negative ratios.
`RvachevPochhammerFactorization.lean`	One definition and ten theorems: the total complex infinite Pochhammer product, its unconditional definitional equality with `qPochhammerInfIn`, its strict-contraction convergence API, absolute summability and paired scale–zero product, the global $S_q(z) = \prod_{k\geq 0}(z^2/(k+1)^2; q^2)_\infty$ factorization for complex $\lVert q\rVert < 1$, and its two dyadic Rvachev/Fabius specializations.
`QPochhammerEntire.lean`	Exactly five compatibility theorems and no public definitions: for each fixed complex $\lVert q\rVert < 1$, locally uniform convergence in a, entireness of $a \mapsto (a; q)_\infty$, the total factor-zero criterion, the reciprocal-power lattice when $q \neq 0$, and analytic order one at every zero under the historical complex-product name. The generic analytic-order theorem is owned by `QPochhammerInfinite.lean`; the $q = 0$-safe statements remain division-free, and no joint holomorphy or centered/MGF wrapper is asserted.
`GeometricPochhammerNormalConvergence.lean`	Exactly three theorems and no public definitions: locally uniform convergence on the whole complex plane of the outer geometric-sinc Pochhammer product for every strict contraction $\lVert q\rVert < 1$, including $q = 0$, and its dyadic standalone-product and bounded-Fabius specializations.
`QPochhammerDissection.lean`	Exactly two theorems and no public definitions: finite exact residue-class dissection over every commutative ring, and its remainder form under $u \leq r$, including all empty and $u = r$ boundaries.

Module	Role in the development
`QPochhammerInfinite.lean`	One definition and exactly twenty-nine theorems: the total generic q-Pochhammer `tprod`; strict-contraction summability, multipliability, finite-prefix convergence, concatenation, factor removal and positive-modulus dissection in complete normed commutative rings; the exact factor-zero locus under a multiplicative norm; locally uniform convergence and continuity over complete locally compact normed fields; complex entireness, nonzero derivatives at raw factor zeros (including $q = 0$), reciprocal-power derivative formulas for $q \neq 0$, and analytic order one at every zero.
`QBinomialTheoremInfinite.lean`	One definition and exactly twenty-nine theorems: dominated passage from finite Gaussian coefficients, positivity and nonvanishing controls, effective fixed-column convergence with arbitrary fixed upper-index shifts and geometric error rates, the infinite q-binomial theorem, and Euler product and reciprocal expansions.
`GaussianBinomialFixedColumnRate.lean`	No definitions and exactly nine theorems: two finite-product defect bounds, the denominator-free relative Gaussian bound, fixed and shifted nonasymptotic additive bounds, and four relative/additive geometric `IsBigO` laws. The generalized product bound and arbitrary-shift convergence are in `QBinomialTheoremInfinite.lean`. The ring layer assumes a normalized multiplicative norm and $\|q\| \leq 1$; the field layer assumes only $\|q\| < 1$, without completeness or $q \neq 0$.
`GaussianBinomialGreaterOneAsymptotics.lean`	No definitions and exactly two theorems, named in the crosswalk: reciprocity transfers the printed fixed-column rate and central equivalence to every real $q > 1$, making `cor:qgreaterone` Exact; no shifted-central or wider-domain claim.
`JacobiTripleProduct.lean`	Two definitions and exactly twenty-five theorems: theta and pentagonal exponents, finite and infinite Jacobi triple products, and Euler pentagonal sums.
`QPascalSummation.lean`	Exactly four theorems and no public definitions: two finite Gaussian-binomial summations and the left and right commuting q-binomial identities.
`GaussianBinomialContinuity.lean`	Exactly three theorems and no public definitions: continuity, the classical $q \to 1$ limit, and the finite-Pochhammer quotient formula.
`GaussianBinomialPalindromic.lean`	Exactly fourteen theorems and no public definitions: general reflection helpers; generic degree bound, boundary values, reflection, constant/top coefficients, coefficient symmetry, mean identity, and the interior/total linear-coefficient theorems; exact degree and monicity over a nontrivial commutative semiring.

Module	Role in the development
`GaussianBinomialCumulants.lean`	Exactly two definitions and twenty-four theorems. Its three newest theorems are `eval_one_derivative_derivative_gaussianBinomial_X` over a characteristic-zero field under $k \leq n$, and the all-n, k, arbitrary-commutative-semiring, division-free `twelve_mul_secondMoment_gaussianBinomial_eval_one` and `twelve_mul_varianceNumerator_gaussianBinomial_eval_one`; the latter include the zero, diagonal, and above-row boundaries.
`GaussianBinomialPolynomialStructure.lean`	Exactly five theorems and no public definitions: universal natural degree, monicity, constant coefficient one, exact reflection, and bounded-index coefficient symmetry.
`CentralQBinomialReduction.lean`	Exactly six theorems and no definitions: sign pairing, even–odd dissection, naturality, integral and commutative-ring division-free central reduction, and a field quotient under two nonvanishing hypotheses.
`CyclotomicFactorization.lean`	Exactly seven theorems and no definitions: quotient bounds, divisor reindexing, finite-product and factorial identities over commutative rings, interval-product extension, and Gaussian cyclotomic factorization over an integral domain.
`PrimitiveRootBlock.lean`	Exactly three theorems and no definitions: primitive-root interior Gaussian vanishing, the top phase, and the complete finite q-Pochhammer block.
`QLucas.lean`	Exactly seven theorems and no definitions: two natural quadratic identities, signed-product coefficient and block formulas, the primitive-root phase, and q-Lucas. Its local `two_mul_choose_two` is private; the public theorem is in `QChuVandermonde.lean`.
`TwoPhiOneReversal.lean`	Exactly two definitions and twelve theorems: the finite normalization and reflected parameter map, finite-to-tsum bridge, involution, terminating reversal, and one- and two-step actual-tsum reversal laws.
`QChuVandermonde.lean`	Exactly ten theorems and no definitions: the public `two_mul_choose_two`, both denominator-cleared sums and quotient evaluations, actual-tsum wrappers, and the separately named by-reversal route; only that route retains the two auxiliary nonvanishing assumptions.
`JacobiTwoSquareCount.lean`	Exactly four theorems and no definitions: the nonzero two-square divisor count, its conditional prime product, and the unconditional chi-four and alternating-odd Lambert identities over complete normed fields under $\|q\| < 1$.
`CyclotomicDivisibility.lean`	Exactly three theorems and no definitions: the carry characterization of exponent one, the rational-polynomial divisibility iff, and the multiple-index primitive-root value.
`QCatalan.lean`	Exactly one definition and eleven theorems: q-integer naturality, degree and cyclotomic identities, rational and integral divisibility, and the integral q-Catalan quotient with its degree and value at one.

Module	Role in the development
`NewtonInterpolation.lean`	Exactly three definitions and nineteen theorems: triangular Newton coefficients, the node polynomial, interpolation and uniqueness, divided differences, the geometric-power-grid formulas, and the definitionally identical `newtonInterpolant` compatibility API over a field.
`QBetaIntegral.lean`	Exactly one definition and eight theorems: the Jackson q-beta integral, its term/product/q-Gamma evaluations, symmetry, positivity, and two recurrences.
`QuantumBinomial.lean`	Exactly two theorems and no public definitions: the quantum-plane power lemma and the noncommutative quantum binomial theorem.
`RogersSzegoPolynomial.lean`	One definition and exactly nine theorems: the Rogers–Szegő polynomial, its initial values, dilation and three-term recurrences, and its generating series.
`GeometricUniformCDF.lean`	The named F_q and explicit f_q, conditioning and interval-integral equations, pointwise derivative and exact positive-ratio `withDensity` law, continuity, reflection, exterior zeros, ordinary/topological support containments, compact support, and C^∞ regularity under their exact parameter hypotheses. It also proves that the selected total formula is zero at $q = 0$, nonpositive for $q < 0$, yields the zero clamped measure for $q \leq 0$, and cannot represent μ_q on the nonpositive contraction range.
`HyperbolicActivation.lean`	Four definitions and 58 theorems forming the totalized real hyperbolic-sinc, `tanhDiv`, activation-odds, and activation-probability dictionary, with continuity, parity, positivity, exact odds bridges, global elementary bounds, and the canonical identification `tanhDiv_eq_dslope`.
`ActivationTaylor.lean`	Three theorems: the exact degree-nine Taylor polynomial of $\tanh$ with $O(\|x\|^{11})$ remainder, the induced degree-eight activation-probability polynomial with $O(\|x\|^{10})$ remainder, and its equivalent $O(x^{10})$ form.
`ActivationSeries.lean`	Three theorems: the rescaled pointwise quadratic budget, genuine summability for arbitrary-index square-summable signed real weights, and the corresponding global `tsum` budget.
`ActivationAsymptotics.lean`	Two punctured-neighborhood theorems: the sharp coefficient $1/3$ proved by one l'Hopital step, and its all-real scaled form $a^2/3$, including $a = 0$.
`ActivationSeriesAsymptotics.lean`	One arbitrary-index Tannery theorem identifying the exact quadratic coefficient of every square-summable activation series; the global majorant equals the limiting ℓ^2 coefficient, so no Taylor remainder is used.
`GeometricActivationDimension.lean`	Two definitions and 30 theorems: geometric/dyadic activation sums, genuine summability, global budgets, positivity and parity, refinement and finite-prefix/tail certificates, and the exact `HasSum` of the squared normalized geometric weights.
`GeometricActivationAsymptotics.lean`	Two thin specializations of the generic Tannery theorem, giving coefficient $(1 - q)/(3(1 + q))$ for $\|q\| < 1$ and the dyadic value $1/9$.

Module	Role in the development
`ProbabilityRepresentation.lean`	Product measure on $[0,1]^{\mathbb{N}_0}$, weighted coordinate sum, self-similarity, and exact CDF/up identifications. Its dyadic CDF smoothing, continuity, exterior-value, and reflection proofs are half-base specializations of `GeometricUniformCDF.lean`; it also identifies $F_{1/2}$ with every bounded Fabius function and $f_{1/2}(x)$ with $2\,\mathrm{up}(2x-1)$.
`WeakConvergence.lean`	Characteristic-function convergence, Lévy continuity, bounded-continuous test convergence, positivity on every open set meeting $(-1,1)$, and exact limiting-measure support $[-1,1]$.
`StepMeasureBridge.lean`	Relates discrete/step approximants to their associated finite measures.
`EarlyMeasureBridge.lean`	Foundational bridge between early convolution approximants and measure formulations.
`StepApproximationLimit.lean`	Coefficient monotonicity, endpoint-corrected step functions, and pointwise convergence by an interval-average squeeze.
`EarlyApproximants.lean`	Definitions and elementary properties of the finite approximants used before the limiting theorem.
`PoissonSummation.lean`	Poisson summation, rapid decay of every real-axis Fourier derivative, and upper-bound-free support specializations for $a \geq \frac{1}{2}$.
`OriginalPaperSupplement.lean`	Remaining numbered and prose results of the first paper, including exact Taylor formulas and nonanalyticity.
`Paper05442.lean`	Aggregator and coverage audit for arXiv:1702.05442.
The arithmetic paper	
`PaperStatements.lean`	Source-numbered propositions, theorems, question, definition, and conjecture for arXiv:1702.06487.
`Paper06487Supplement.lean`	Prose-level and strengthened arithmetic-paper consequences, with corrected hypotheses and bounded/global distinctions.
`Paper06487.lean`	Compact aggregator for the complete arithmetic-paper formalization.
Finite q-binomial and Thue–Morse algebra	
`HalfQBinomial.lean`	Exact q-binomial coefficients at q=1/2, q-Pascal recurrence, Mersenne normalization, and a finite q-binomial theorem.
`HalfQBinomialRootSimplicity.lean`	Exhaustive zero-definition/one-theorem surface: `halfQBinomial_sum_rootMultiplicity_two_pow` proves multiplicity one at every dyadic root 2^j, $j < n$, of the exact rational half-base polynomial. With the complete root classification this makes `cor:halfbase-root-locus` Exact; `cor:qbinom-inversion-law` remains Partial.
`QBinomialCauchy.lean`	One q-Bernstein definition and five denominator-free finite q-Cauchy/q-Bernstein theorems over arbitrary commutative rings, including the compatibility alias `finiteQPochhammerIn_mul_eq_sum_gaussianBinomial`.

Module	Role in the development
`GaussianBinomialContinuity.lean`	Zero definitions and exactly three theorems: `continuous_gaussianBinomial`, `tendsto_gaussianBinomial_nhds_one`, and `gaussianBinomial_eq_finiteQPochhammerIn_div`. The first two hold for all n, k in every topological semiring with continuous operations. The quotient identity needs only a field, $k \le n$, and $(q;q)_k \neq 0$, with no topology.
`JacobiTripleProduct.lean`	Two definitions, `thetaExponent` and `pentagonalExponent`, and exactly twenty-five theorems: `two_dvd_mul_sub_one`, `mul_sub_one_nonneg`, `two_mul_thetaExponent`, `thetaExponent_natCast`, `thetaExponent_neg_natCast`, `choose_two_add_choose_two_succ_eq`, `sum_mul_sum_eq_sum_antidiagonal`, `thetaCoeff_pair_eq`, `sum_antidiagonal_thetaCoeff`, `finite_triple_product_poly`, `pow_mul_finiteQPochhammerIn_div`, `neg_one_zpow_natCast_sub`, `finite_triple_product`, `norm_gaussianBinomialInt_le`, `tendsto_gaussianBinomialInt_central`, `summable_pow_thetaExponent_mul_zpow`, `hasSum_jacobi_triple_product`, `hasSum_jacobi_triple_product'`, `mul_three_mul_sub_one_nonneg`, `two_dvd_mul_three_mul_sub_one`, `two_mul_pentagonalExponent`, `pentagonalExponent_eq`, `pentagonalExponent_pos`, `hasSum_pentagonal`, and `hasSum_pentagonal_pairs`. Exponent arithmetic is unconditional; the finite polynomial block is over commutative rings, while the Laurent block is over fields and its divided-product formulas assume $z \neq 0$. The norm bound needs a normed field and $\|q\| < 1$; the central limit, Jacobi, and pentagonal identities add completeness, with $z \neq 0$ exactly for Jacobi. The pentagonal results include $q = 0$; the real theta majorant assumes $0 \le r < 1$ and $0 < s$.

Module	Role in the development
`QBinomialTheoremInfinite.lean`	One definition, `gaussianMajorant`, and exactly twenty-nine theorems: `hasSum_of_tendsto_of_dominated`, `one_le_qPochhammerInfIn_neg`, `finiteQPochhammerIn_neg_le_qPochhammerInfIn`, `qPochhammerInfIn_nonneg`, `qPochhammerInfIn_le_finiteQPochhammerIn`, `qPochhammerInfIn_pos_of_lt_one`, `qPochhammerInfIn_zero_left`, `norm_finiteQPochhammerIn_le_neg_norm`, `finiteQPochhammerIn_norm_le_norm`, `norm_finiteQPochhammerIn_le`, `qPochhammerInfIn_norm_le_norm_finiteQPochhammerIn`, `finiteQPochhammerIn_ne_zero_of_norm_lt_one`, `gaussianBinomial_eq_div`, `gaussianMajorant_nonneg`, `norm_gaussianBinomial_le`, `tendsto_finiteQPochhammerIn_pow_atTop`, `norm_finiteQPochhammerIn_pow_sub_one_le_exp`, `norm_finiteQPochhammerIn_pow_sub_one_le_exp_of_norm_le_one`, `isBigO_finiteQPochhammerIn_pow_sub_one`, `tendsto_gaussianBinomial_atTop`, `tendsto_gaussianBinomial_add_atTop`, `tendsto_gaussianBinomial_add_const_atTop`, `isBigO_gaussianBinomial_sub_inv`, `isBigO_gaussianBinomial_add_sub_inv`, `summable_pow_choose_two_mul_pow`, `hasSum_euler_product`, `qPochhammerInfIn_ne_zero_of_norm_lt_one`, `hasSum_qBinomial_theorem`, and `hasSum_euler_reciprocal`. The Tannery theorem is Banach-valued with its common summable bound, row limits, row `HasSum`s, and sum limit. The real comparison block uses exactly $0 \le r < 1$ and its displayed hypotheses on x; the infinite zero-left theorem is unconditional over a topological commutative ring. Norm bounds and fixed-column limits need a normed field and their displayed contraction/unit-ball hypotheses. Over a complete normed field Euler's product needs only $\|q\| < 1$ and arbitrary z, while the infinite q-binomial and reciprocal identities add $\|z\| < 1$, with arbitrary a and no $q \neq 0$. The finite zero-left theorem belongs to `GaussianBinomialAtOne.lean`.

Module	Role in the development
`QPascalSummation.lean`	Zero definitions and exactly four theorems: `sum_gaussianBinomial_succ_mul`, `sum_gaussianBinomial_succ_mul'`, `Commute.gaussianBinomial_left`, and `Commute.gaussianBinomial_right`. The first summation and both commutation laws hold in every semiring, with only the indicated `Commute` hypothesis for the latter; the second summation assumes a commutative semiring. There is no division, order, topology, or restriction on q, n, or the weight sequence.
`QuantumBinomial.lean`	Zero definitions and exactly two theorems, `quantumPlane_mul_pow` and `quantum_binomial`, over an arbitrary possibly noncommutative semiring. The first assumes that q commutes with X and $YX = q(XY)$; the second additionally assumes that q commutes with Y. No division or topology is used.
`RogersSzegoPolynomial.lean`	One definition, `rogersSzego`, and exactly nine theorems: `rogersSzego_zero`, `rogersSzego_one`, `rogersSzego_succ`, `one_sub_pow_succ_mul_gaussianBinomial_succ_succ`, `rogersSzego_dilation`, `rogersSzego_three_term`, `summable_norm_pow_div_finiteQPochhammerIn_self`, `sum_antidiagonal_euler_mul_euler`, and `hasSum_rogersSzego_generating`. The definition and first three theorems use a commutative semiring; the cross, dilation, and three-term laws use a commutative ring and are total in their natural indices. The summability theorem needs a normed field, $\|q\| < 1$, and $\|z\| < 1$; the antidiagonal identity needs only the contraction. The generating function adds completeness and assumes exactly $\|q\| < 1$, $\|t\| < 1$, and $\|zt\| < 1$.
`PolynomialQDerivative.lean`	Two definitions, `qInt` and `qDerivative`, and exactly seventeen theorems: `qInt_zero`, `qInt_one`, `qInt_succ`, `qInt_succ'`, `qInt_add`, `qInt_one_left`, `one_sub_mul_qInt`, `qDerivative_apply`, `qDerivative_monomial`, `qDerivative_C`, `qDerivative_X_pow`, `qDerivative_X`, `qDerivative_C_mul_X_pow`, `eval_qDerivative_mul`, `qDerivative_comp_C_mul_X`, `qDerivative_mul`, and `qDerivative_mul'`. The q-integer layer is semiring-valued; its geometric identity and the division-free evaluation law use a commutative ring; the remaining polynomial derivative API uses a commutative semiring. Arbitrary q, including zero and one; no analytic or limiting derivative.

Module	Role in the development
`PolynomialQLeibniz.lean`	Zero definitions and exactly four theorems: `comp_C_mul_X_comp_C_mul_X`, `qDerivative_C_mul`, `qDerivative_iterate_comp_C_mul_X`, and `qDerivative_iterate_mul`. All hold over a commutative semiring for arbitrary q and every natural order, with no division, nonvanishing, topology, or convergence hypothesis.
`QPochhammerDerivative.lean`	Zero definitions and exactly three theorems: `hasDerivAt_finiteQPochhammerIn`, `hasDerivAt_finiteQPochhammerIn_of_ne_zero`, and `hasDerivAt_finiteQPochhammerIn_comp`. Arbitrary q, a, n over a nontrivially normed field; unconditional cofactor derivative, logarithmic form under exact factorwise nonvanishing, and chain rule under the supplied pointwise derivative. No infinite-product derivative.
`LambertSeriesLog.lean`	Zero definitions and exactly four theorems: `one_le_norm_natCast_add_one`, `summable_lambert_series`, `hasSum_lambert_log_complex`, and `exp_neg_tsum_lambert_eq_qPochhammerInfIn`. Apart from the unconditional norm helper, exactly complex $\|a\| < 1$, $\|q\| < 1$; principal factor logs and a branch-free exponential identity, with no norm-one boundary, product-log branch identity, continuation, or differentiated series.
`QGamma.lean`	Two total real definitions, `qGamma` and `qNumber`, and exactly ten theorems: `norm_lt_one_of_pos_of_lt_one`, `qPochhammerInfIn_rpow_pos`, `qPochhammerInfIn_self_pos`, `qGamma_pos`, `qGamma_one`, `qGamma_add_one`, `qGamma_nat_succ`, `qNumber_natCast`, `qGamma_mul_qGamma_one_sub`, and `hasSum_theta_qGamma_reflection`. Positive recurrence API under $0 < q < 1$ and displayed $x > 0$; natural bridge under $q \neq 1$; reflection under only $q < 1$ and arbitrary real x; theta form under $0 < q < 1$, $0 < x < 1$. No complex continuation, classical limit, pole, log-convexity, uniqueness, or digamma theorem.
`ThueMorseMixedDifference.lean`	Order-independent translation differences, Boolean-cube expansion, polynomial coefficient extraction, and dyadic Thue–Morse specialization.
`ThueMorseSymmetricDifference.lean`	Centered half-step recurrence and Boolean-cube expansion, sign-free sharp polynomial extraction, strict-degree cancellation, and centered/increasing-grid dyadic Thue–Morse block formulas.
`ThueMorseCornerIntegral.lean`	Exhaustive one-definition/four-theorem surface giving the recursive centered box integral and the exact range/`FinN` local corner identities under open, order-connected, segment-local `ContDiffOn` hypotheses.

Module	Role in the development
`FinitePolynomialFunctional.lean`	Exhaustive zero-definition/sixteen-theorem surface giving scalar-extended moment expansion, reproduction, coefficient extraction, cancellation and congruence, same-ring forms, and affine top-coefficient transport including zero scale.
`FabiusRawQBinomialFormula.lean`	Raw coefficient-valued finite q-binomial identity before scalar or analytic specialization.
`FabiusRawQBinomialScalar.lean`	Scalar evaluation of the raw finite identity.
`FabiusQBinomialFormula.lean`	Principal exact Thue–Morse/q-binomial formula for unrestricted natural parameters.
`FabiusQBinomialScalarFormula.lean`	Scalar-valued form of the principal q-binomial theorem.
`FabiusDyadicQBinomialScalar.lean`	Specialization using exact dyadic Fabius values.
`FabiusQBinomialTaylor.lean`	Constancy in the common translation indeterminate and identification, after normalization, with `fabiusReductionSum`.
`FabiusGlobalQBinomialSeries.lean`	Real and complex all-real q-series for the signed extension, with pointwise q-independent summands and zero-series behavior on the nonpositive side.
`FabiusBinaryReductionSeries.lean`	Exact binary telescope, the scale-zero term distinguishing even and odd unit blocks, a global 2^{1-N} residual bound, uniform convergence on $\mathbb{R}$, and all-real sum identities.
`FabiusParityPowerSeries.lean`	All-real parity-power form of the signed extension, including the integer-valued scale-zero exponent, termwise vanishing on the nonpositive side, and the restricted older $m = 1$ form.
Discrete-limit family	
`FabiusUniformSpline.lean`	Centered finite Thue–Morse splines; empty-prefix support, block translation, finite-CDF identification, monotonicity/range bounds, and pointwise convergence.
`FabiusComplexShiftSpline.lean`	Taylor control of complex shifts of finite splines.
`FabiusDiscreteLimitToeplitz.lean`	Toeplitz-type averaging principle for normalized finite rows.
`FabiusDiscreteLimitComplexShift.lean`	Vanishing of complex-shift corrections under the limit scaling.
`FabiusDiscreteLimitIntegration.lean`	Exact outer q-binomial reindexing, Toeplitz passage for complex-shift splines, identification with the binary/literal global series, and all-real extension by empty-prefix vanishing.
Coarse endpoint asymptotics	
`FabiusLogScale.lean`	Defines the dyadic logarithmic coordinate and proves the domain-correct delay identity for the logarithm.
`FabiusLogSquaredAsymptotic.lean`	Quadratic logarithmic leading term, floor/ceiling transfer to real scale, and an explicit O(t log t) remainder.
`FabiusSmallArgumentScale.lean`	Filter and scale conversions between x tending to zero and the large logarithmic variable.
`FabiusFlatness.lean`	Proof that the Fabius function is smaller than every positive power at zero.

Module	Role in the development
`FabiusDecayComparison.lean`	Comparison with exponential-flat functions such as exp(-c/x).
`FabiusLogMainDefect.lean`	Definition and first bounds for the defect after subtracting the quadratic logarithmic main term.
Negative Laplace, Mellin, and periodicity	
`NegativeLaplace.lean`	Infinite product and absolutely convergent logarithm, exact dilation equation, quadratic decomposition, periodic correction, exponential tail, and strict signs of the factors, logarithm, forward tail, and positive tail error.
`NegativeLaplaceVerticalAllOrderBound.lean`	Uniform positive-order vertical log-product bounds at radius 2^b: $C(b+1)$ for $b \geq 0$, with the legacy Cb corollary for $b \geq 1$.
`NegativeLaplaceVerticalOrdinaryJets.lean`	Exact zero-axis vertical logarithmic jets in terms of ordinary negative-Laplace jets, including the exceptional order-zero normalization.
`PeriodicCorrection.lean`	Continuity of the multiplicatively periodic correction by locally uniform convergence.
`PeriodicMean.lean`	Mean value and normalization of the period-one logarithmic correction.
`PeriodicRegularity.lean`	Differentiability and higher regularity of the periodic correction.
`PeriodicFourier.lean`	Fourier coefficients and nonconstancy of the periodic correction.
`MellinBose.lean`	Mellin transform of the Bose kernel and Gamma/zeta factorization.
`MellinFinitePart.lean`	Finite-part continuation after subtraction of singular Laurent terms.
`BoseFinitePartIntegral.lean`	Convergent integral representation for the regularized Bose/Mellin contribution.
`StieltjesConstant.lean`	Sign-fixed first Stieltjes constant, exact real and complex derivatives at one, and the local zeta Taylor/Laurent interface.
`GammaSecondOrder.lean`	Exact $\Gamma''(1)$, analytic cubic factorization of the quadratic Taylor remainder, and the sharp complex $O(z^3)$ estimate at zero.
`LaplaceCumulantAsymptotics.lean`	Bell-polynomial inversion of normalized Laplace moments through order four and the conditional corrected $O(1/n)$ endpoint comparison.
`LaplacePeriodicSecondOrder.lean`	Second-order transform asymptotics retaining the periodic correction.
Bromwich and Lambert saddle analysis	
`FabiusBromwichInput.lean`	Analytic and decay hypotheses needed to apply Bromwich inversion to the Fabius transform.
`BromwichSaddle.lean`	Generic Fourier–Bromwich inversion and exact change to dimensionless saddle coordinates, without contour shifting.
`FabiusLambertPhase.lean`	Exact lower-Lambert reference coordinate and its domain/inversion identities.
`FabiusLambertRates.lean`	Growth, comparison, and eventual positivity for the exact phase, plus the unconditional reciprocal logarithmic identity and filter-generic Big-O scale conversion.

Module	Role in the development
`FabiusLambertSaddle.lean`	Exact algebraic identities at the lower-Lambert reference radius, without claiming it solves the full residual phase's critical equation.
`FabiusLambertDerivativeBounds.lean`	Uniform bounds for derivatives/cumulants at the saddle scale.
`FabiusLambertMinorArc.lean`	Modulus loss away from the central saddle window.
`FabiusLambertTailFlat.lean`	Exponential tail bounds upgraded to every inverse-power scale.
`FabiusSharpConstant.lean`	Exact normalization constant in the sharp asymptotic formula.
`FabiusSharpLambertMain.lean`	Assembly of the explicit Lambert-phase main term.
`FabiusSharpLambertTransfer.lean`	Transfer of estimates stated in saddle variables to the small-x filter.
`FabiusSharpExactReduction.lean`	Exact decomposition into explicit main term, normalized saddle mass, and forward-tail error.
`FabiusExplicitSharpTransfer.lean`	Conversion between exact sharp expressions and source-facing elementary forms.
`FabiusSharpAsymptoticTransfer.lean`	General transfer lemmas for logarithmic and exponential asymptotic equivalence.
`FabiusSharpAsymptotic.lean`	Correct sharp formula with periodic phase, first error bound, and asymptotic equivalence.
Post-snapshot algebra and phase-extraction machinery	
`ExponentialPartition.lean`	Weighted partitions and their reciprocal-factorial coefficient sum over arbitrary commutative $\mathbb{Q}$-algebras, including the marked-part and normalized successor recurrences and the field quotient bridge.
`ExponentialBell.lean`	Recurrence-uniqueness bridge identifying the finite weighted-partition family with `SaddleExpansion.expCoeff`.
`MomentCumulantAlgebra.lean`	Factorial normalization, the complete Bell transform and inverse moment–cumulant transform, the classical Bell recurrence, normalized inverse laws, and functoriality over arbitrary commutative $\mathbb{Q}$-algebras.
`EndpointTransferPolynomials.lean`	Universal endpoint-transfer polynomials over $\mathbb{Q}[X]$: recurrence, weighted-partition formula, exact formal exponential series, finite truncation, evaluation naturality, and the coefficients through order three.
`PolynomialExpectationCumulant.lean`	Finite polynomial integrals as coefficient-weighted raw moments and complete Bell transforms of formal cumulants, including the exact zeroth-moment correction and the probability-measure specialization.
`CompleteHomogeneous.lean`	Evaluation and adjoining-variable recurrences for complete homogeneous symmetric polynomials, together with their univariate quotient polynomial, over arbitrary commutative semirings or rings as appropriate.

Module	Role in the development
`CompleteHomogeneousGenerating.lean`	Finite formal generating series for complete homogeneous evaluations: coefficient and adjoining-variable recurrences over commutative semirings, and one-factor/full-product denominator clearing plus the canonical `PowerSeries.invOfUnit` reciprocal over commutative rings. No analytic evaluation, convergence, or Bell/generalized-harmonic conversion is asserted.
`CompleteHomogeneousBell.lean`	Finite power sums and factorially weighted Bell input; the division-free complete-homogeneous/complete-Bell identity over every commutative semiring and its factorially normalized pointwise report form over commutative $\mathbb{Q}$-algebras. No analytic convergence is asserted.
`GeometricCompleteHomogeneous.lean`	Denominator-free principal specialization $h_n(1, q, \ldots, q^p) = \begin{bmatrix} n+p \\ p \end{bmatrix}_q$, its arbitrary common-scale form over every commutative semiring, and the range-indexed bridge used by interpolation rows.
`LagrangeResidualMoments.lean`	Every higher residual moment of a polynomially exact finite row, with a representation-independent commutative-ring theorem and the distinct-node Lagrange specialization.
`GeometricResidualMoments.lean`	Exhaustive zero-definition/nine-theorem surface: every positive residual moment on $c, cq, \ldots, cq^p$ over a commutative ring, assuming exactness directly on those scaled nodes, together with distinct-node Lagrange, arbitrary-dilation, shifted-block, and polynomial-exactness field corollaries. The theorem `sum_geometricLagrangeWeight_mul_eval_scaled_geometric` assumes finite power-node injectivity and allows arbitrary c, making `cor:scaled-geometric-moments` Exact by composition with the positive-moment theorem.
`AnalyticSeriesFilter.lean`	Unconditional pointwise evaluation of finite coefficient filters: diagonalization under weighted sampled-series summability, exact finite heads and tails, and Gaussian residual tails for geometric and geometric-Lagrange rows, including shifted scales. This generic module does not supply conditional boundary summation, uniform estimates, asymptotic rates, or a sinc-product instantiation.
`RvachevQBinomialFilter.lean`	Exact specialization of the Gaussian filter to the actual infinite Rvachev product: the named even modes sum to the product and scale by powers of c^2; the generic complex theorem assumes exactly finite-power injectivity, the $c = 1/2$, $q = 1/4$ theorem is assumption-free, and the model-generic wrapper applies to every bounded `IsFabius` solution. Finite-prefix quotients and analytic acceleration estimates are not asserted.
`QBinomialVandermonde.lean`	Both zero-extended q-Vandermonde orientations, full-range and exact-$\min$-support central forms, the total positive shifted-central identity, and two negative-shift forms under exactly $k \leq N$, plus the total integer-index shifted-central identity and its finitely supported `finsum` form built from `gaussianBinomialInt`, all denominator-free over arbitrary commutative semirings.

Module	Role in the development
`GeometricRichardson.lean`	Generic geometric root polynomials and normalized forward Richardson polynomials, including their evaluation and coefficient interfaces.
`GeometricLagrangeWeights.lean`	Identification of the geometric Lagrange row with the Richardson coefficient polynomial and its complete-homogeneous all-order moments.
`GeometricLagrangeQBinomial.lean`	Field-valued q-Pochhammer weight formulas and report-facing rational q-binomial and quarter-base coefficient formulas.
`GeometricLagrangeQMoments.lean`	Closed forms for every rational geometric moment under nonzero-base and nonvanishing finite-denominator hypotheses, with exact residual sign and finite total variation for $0 < q < 1$ and dedicated quarter-base corollaries.
`GeometricRichardsonGenerating.lean`	Exhaustive three-definition/seven-theorem surface. The definitions are `geometricRichardsonKernel`, `qPochhammerNormalizedDataSeries`, and `geometricRichardsonTransform`; the theorems are `coeff_rescale_qPochhammerSeries_eq_geometricRichardsonKernel`, `coeff_qPochhammerNormalizedDataSeries`, `geometricRichardsonTransform_generating`, `geometricRichardsonTransform_eq_sum_lagrange`, `geometricLagrangeRichardson_generating`, `hasSum_geometricRichardsonTransform_mul_pow`, and `hasSum_geometricLagrangeRichardson_mul_pow`. The first generating identity is formal over a commutative ring with totalized inverses; the exact geometric-Lagrange form assumes a field and nonzero nome; the two analytic `HasSum` forms additionally require completeness, strict nome contraction, and absolute summability of the normalized data.
`GeometricUniformMomentPolynomial.lean`	Exhaustive one-definition/eight-theorem algebraic surface: recursive rational polynomial family, residual-product recurrence, triangular `natDegree` bound, reciprocal-factorial value at zero, and P_0 through P_4.
`GeometricUniformMomentPolynomialBridge.lean`	Exhaustive zero-definition/one-theorem surface: `geometricUniformMomentPolynomial_eval₂_eq_mgf_taylorCoefficient` identifies that polynomial with the normalized Taylor coefficient of the actual geometric-uniform MGF for every real $\lvert q\rvert < 1$, making `p7:eq:Pn-def` Exact in that regime. It supplies no complex-q product or sharp degree theorem by itself; the later product, exterior-germ, and degree leaves supply those clauses.
`RvachevLagrangeNodesOnly.lean`	Exhaustive one-definition/fourteen-theorem surface named in the crosswalk: ordinary-derivative and raw omitted-node Appell forms, rational descent and rational lattice samples, and formal complete-Bell/Bernoulli–Mersenne coefficients. It makes `cor:lag-nodes-only` Exact only by composition; the rational-point, synthesis-hypothesis, and non-analytic-MGF caveats remain explicit.

Module	Role in the development
`ThueMorseGammaTowerDifferential.lean`	No definitions and exactly three theorems, named in the crosswalk: positive-parameter Mellin differentiation and the full finite GammaLog derivative ladder. This makes `p2:thm:gamma-tower` Exact under its chosen-coordinate convention, not as a principal-log identity.
`GeometricUniformComplexMomentProduct.lean`	Exhaustive one-definition/three-theorem surface: `geometricUniformComplexMomentProduct` is the actual complex product, `hasProdLocallyUniformly_geometricUniformComplexMomentProduct` proves its locally uniform convergence for $\|q\| < 1$, `differentiable_geometricUniformComplexMomentProduct` proves complex differentiability on the whole plane, and `geometricUniformMomentPolynomial_eval₂_eq_complexMomentProduct_taylorCoefficient` gives the normalized Taylor-coefficient bridge. It is an analytic analogue of the real identity, not a complex probability statement.
`GeometricUniformExteriorComplexMomentGerm.lean`	Exhaustive one-definition/two-theorem surface: `geometricUniformExteriorComplexMomentGerm` is the actual $\mathcal{A}_{q^{-1}}(-z)^{-1}$ reciprocal germ, `analyticAt_geometricUniformExteriorComplexMomentGerm` proves analyticity at zero for $1 < \|q\|$, and `geometricUniformMomentPolynomial_eval₂_eq_exteriorComplexMomentGerm_taylorCoefficient` gives its normalized Taylor-coefficient bridge; no unit-circle analytic value is asserted.
`GeometricUniformMomentPolynomialDegree.lean`	Exhaustive zero-definition/three-theorem surface: `coeff_geometricUniformMomentPolynomial_choose_two` and `coeff_geometricUniformMomentPolynomial_choose_two_sub_one` give the top and subleading Bernoulli coefficients, while `geometricUniformMomentPolynomial_natDegree_eq` gives the exact parity-sensitive degree. Together with the preceding leaves this makes `p7:thm:Pn` Exact and independently makes `prop:qF-P-degree-sharp` Exact.
`GeometricUniformMomentRatFunc.lean`	Exhaustive one-definition/four-theorem surface: `geometricUniformMomentRatFunc`; `qFactorial_mul_geometricUniformMomentRatFunc`; `eval_geometricUniformMomentRatFunc_eq_complexMomentProduct_taylorCoefficient`; `eval_geometricUniformMomentRatFunc_eq_exteriorComplexMomentGerm_taylorCoefficient`; and `eval_geometricUniformMomentRatFunc_one`. It packages one rational a_n, proves its global q-factorial clearing, identifies its safe inner/exterior values, and handles $q = 1$ by polynomial continuation. Thus `thm:qF-moment-polynomial` is Exact; genuine poles and unit-circle analytic values are not totalized.

Module	Role in the development
`GeometricUniformMomentReciprocity.lean`	Exhaustive one-definition/five-theorem surface: `geometricUniformComplexMomentGerm`; `geometricUniformComplexMomentGerm_of_norm_lt_one`; `geometricUniformComplexMomentGerm_of_one_lt_norm`; `analyticAt_geometricUniformComplexMomentGerm`; `geometricUniformComplexMomentGerm_reciprocity`; and `geometricUniformComplexMomentGerm_momentconvolution`. Under exactly $q \neq 0$ and $\|q\| \neq 1$, it proves local `EventuallyEq` reciprocity and the derivative convolution, making `thm:qF-reciprocity` Exact; no global or unit-circle identity, maximal disc, uniqueness, or pole divisor is asserted.
`LambertPhaseLockedRichardson.lean`	Exact lower-Lambert phase locking, reciprocal-grid Lagrange weights, cancellation, and every complete-homogeneous residual moment.
`LambertPhaseLockedBell.lean`	Generalized harmonic power sums and Bell input for the shifted reciprocal alphabet, together with exact complete-homogeneous and reciprocal-Lagrange moment rewrites in factorially normalized complete-Bell form over characteristic-zero fields (with the direct rewrite using the rational-algebra structure). Total field inversion requires no positivity or nonvanishing hypothesis on the shift.
`FabiusLambertPhaseLockedPullback.lean`	Pullback of the arbitrary-order logarithmic Fabius expansion to every fixed phase-locked Lambert node, with the periodic correction and saddle coefficients frozen at the base phase.
`CompleteHomogeneousAsymptotics.lean`	Coordinatewise Big-O closure for fixed finite complete-homogeneous evaluations, with the comparison scale raised to the homogeneous degree and degree zero included.
`LambertReciprocalAsymptotics.lean`	Theta and Big-O comparison of shifted reciprocal powers, fixed-row coefficient and total-variation growth, and the power-balancing estimate for weighted remainders.
`FabiusLambertPhaseExtraction.lean`	The reduced phase-locked sample, reciprocal-Lagrange periodic estimator, exact residual terms, fixed-order Poincaré extraction with arbitrary residual depth, the first-omitted improvement, and integer-ray convergence.
`FabiusLambertPhaseExtractionBell.lean`	Exact Bell-polynomial rewrites of one phase-locked residual term and its finite partial sum. This leaf adds no definition, convergence theorem, sign claim, or asymptotic estimate.
Central integral and all-order machinery	
`FabiusSaddleReduction.lean`	Algebraic reduction of the inversion integral to a normalized central weight.
`FabiusSaddleCentral.lean`	Leading Gaussian approximation in the central window.
`FabiusSaddleCentralLambert.lean`	Central estimate expressed at the exact Lambert saddle.
`FabiusSaddleCentralRadiusAsymptotics.lean`	Growth and compatibility conditions for the chosen central radius.
`FabiusSaddleTail.lean`	Coarse tail bound outside the central window.

Module	Role in the development
`FabiusSaddleReferenceWeight.lean`	Normalized Gaussian reference integrals and identities.
`FabiusSaddleReferenceTail.lean`	Gaussian reference tail estimates.
`FabiusFirstSaddleCorrection.lean`	Explicit first non-Gaussian correction from cubic and quartic cumulants.
`FabiusSaddleCoefficientRecurrence.lean`	Recursive definition and correctness of all-order expansion coefficients.
`FabiusSaddlePolynomialCoefficients.lean`	Polynomial coefficient extraction for the rescaled central exponent.
`FabiusSaddleExpansionCoefficients.lean`	Final scalar coefficients after Gaussian contraction.
`FabiusSaddleExponentAllOrders.lean`	Arbitrary-order expansion of the local exponent with uniform remainder.
`FabiusSaddleCentralAllOrders.lean`	Uniform central-integral expansion to every inverse phase order.
`FabiusSaddleTailAllOrders.lean`	Tail smaller than every requested inverse phase power.
`FabiusSaddleMassAllOrders.lean`	All-order expansion of the normalized saddle mass.
`SaddleExpansionFlat.lean`	Flat-remainder calculus and characterization of equality of full coefficient families modulo $O(\mathrm{scale}^N)$ for every N.
`SaddleExpansionSmallArgument.lean`	Bidirectional transfer of full expansions between the logarithmic coordinate and $x \to 0^+$, for arbitrary normed scalar and vector codomains.
`FabiusSaddleJetClosedForm.lean`	Closed finite formula for periodic saddle jets in derivatives of the periodic correction.
`FabiusSaddleExponentClosedForm.lean`	Closed decomposition of exponent coefficients into a jet term and universal tail.
`FabiusSaddleJetStirling.lean`	Identification of jet weights with shifted signed first-kind Stirling numbers.
`FabiusSaddleLeadingCoefficient.lean`	Exact leading coefficient of the Fabius-specific saddle polynomial.
`FabiusSecondSaddleCorrection.lean`	Closed period-one coefficient at order λ^{-2}.
`FabiusSecondSaddleExpansion.lean`	Public expansion through the second saddle correction with $O(\lambda^{-3})$ remainder.
`GaussianPolynomialContraction.lean`	Linear functional sending even monomials to Gaussian moments and odd monomials to zero.
`GaussianPolynomialWholeIntegral.lean`	Exact whole-line Gaussian integrals of coefficient polynomials.
`GaussianPolynomialTail.lean`	Tail bounds for fixed Gaussian polynomials.
`GaussianPolynomialTailAllOrders.lean`	Uniform tail bounds compatible with arbitrary expansion order.

Lambert formal algebra and final expansion

Module	Role in the development
`SaddleLogExpansionAlgebra.lean`	Recursive formal logarithm and mutually inverse exponential/logarithmic coefficient transforms, including assumption-free positive-index forms.
`SaddleLogExpansionPowerSeries.lean`	Identification of the recursive logarithm with Mathlib's `PowerSeries.logOf` for unit-constant series.
`FabiusLambertFormalLog.lean`	Formal logarithm/exponential algebra for truncated saddle series.
`FabiusLambertHigherExpansion.lean`	Higher explicit approximants to the exact Lambert phase.
`FabiusLambertAllOrderAlgebra.lean`	Coefficient identities for arbitrary-order Lambert substitution.
`FabiusLambertAllOrderRemainder.lean`	Analytic control of the remainder left by formal Lambert algebra.
`FabiusLambertAllOrderSmallArgument.lean`	Expansion of the exact phase in log(1/x) and log log(1/x) to arbitrary fixed order.
`FabiusFullAsymptoticExpansion.lean`	All-order expansion of log F at the exact phase and weakened elementary-log corollaries.
`FabiusWikipediaMain.lean`	Concise corrected main asymptotic in a public-summary format.
`FabiusWikipediaExpansion.lean`	Corrected higher expansion suitable for comparison with encyclopedic formulas.
`FabiusWikipediaObstruction.lean`	Proof that omission of the nonconstant periodic term invalidates a vanishing residual.
`FabiusQuotientExponentialMismatch.lean`	Rigorous mismatch between Fabius decay and a proposed quotient-of-exponentials fit.
`PaperFabiusAsymptotic.lean`	Aggregator and audit for the coarse, sharp, corrected, and all-order asymptotic program.
Legendre and approximation	
`LegendrePolynomial.lean`	Rodrigues formula, parity, differential equation, monomial coefficients, norms, and endpoint data for Legendre polynomials.
`LegendreSeriesConvergence.lean`	Generic orthogonal-series convergence and coefficient-decay infrastructure for smooth functions.
`FabiusLegendreCoefficients.lean`	Exact even Legendre coefficients expressed through moments and inverse-dyadic values; odd coefficients vanish.
`FabiusLegendreSeries.lean`	Orthogonality, absolute/uniform convergence, and pointwise identification of the up-function series.
`FabiusTranslatedLegendreSeries.lean`	Affine translation to the signed Fabius function and monomial expansion of the translated series.
`FabiusLegendreLeastSquares.lean`	Pythagorean identity and unique least-squares optimality of finite Legendre truncations.
`RvachevLegendreCentralSum.lean`	Exhaustive zero-definition/three-theorem surface: the central value of P_{2n}, evenness of the deconvolved even mode, and the exact finite central-binomial cancellation at mesh 4^n, including $n = 0$.

Module	Role in the development
`RvachevLegendreBiorthogonality.lean`	Exhaustive one-definition/one-theorem surface: `rvachevLegendreAnalysisKernel` is the normalized translated Legendre analysis kernel and `rvachevLegendreBiorthogonality` is its exact finite open-block pairing with the Rvachev deconvolution polynomial for every nonzero natural mesh with $\ell \le v_2(M)$ and arbitrary analysis degree. It closes `def:leg-Lambda` and `thm:leg-biorthogonality`; the larger `thm:leg-Lambda` Fourier–Bessel claims and matrix/projector corollary remain Partial.
`FabiusLegendreEnergy.lean`	Polynomial-form Legendre blocks, their complete orthogonality, full coefficient Parseval identity, exact shifted coefficient-energy tails in `HasSum` and `tsum` form, and the real-variable Legendre series for $\int_0^1 F(t)^2\,dt$. Finite up-translate realization, Fourier/sinc energy formulas, and rationality of partial sums remain outside the module.
`LegendreGaunt.lean`	Four definitions and twelve theorems giving executable rational Lebesgue moments and Gaunt coefficients, real-cast and moment-functional bridges, two index swaps, finite real/rational Legendre product linearizations, and the proved parity and triangle-support zeros; see section 19.13.
`LegendreGauntClosedForm.lean`	Two definitions and twenty-five theorems giving the total rational integer-index zero-row Wigner-square datum, pairwise-coordinate central-binomial and factorial forms, the half-sum factorial form, exact global rational and real Gaunt identities, sharp support, positivity, nonnegativity, degenerate and zero-index boundary values, and product-coefficient laws; no signed phase, half-integer or nonzero magnetic indices, Wigner orthogonality, or recoupling; see section 19.13.1.
`FabiusLegendreGaunt.lean`	One definition and eight theorems extending the rational Rvachev–Legendre coefficients by odd zeros and expressing rational and genuine up-law Gram entries as exact finite full/even Gaunt sums; see section 19.13.
`FabiusLegendreGauntClosedForm.lean`	Zero public definitions and three theorems expressing the finite rational entry, rational matrix, and real up-law matrix as twice finite canonical-coefficient sums against the total rational integer-index zero-row Wigner-square datum; no signed phase, half-integer or nonzero magnetic indices, broader Wigner-symbol theory, orthogonality, or recoupling; see section 19.13.1.
K-fold Thue–Morse audit and auxiliary asymptotics	
`ThueMorseApproximation.lean`	Exact identification of corrected grid samples with moving-center step approximants and pointwise convergence, including the endpoint.
`ThueMorsePrefix.lean`	Iterated prefix recurrences, Prouhet cancellation, exact terminal dyadic zero runs and boundary values, and full signed pre-run reciprocity with its reflected zero-locus equivalence.

Module	Role in the development
`ThueMorseGenerating.lean`	Formal power-series identities, shifted grid recurrence, and exact endpoint refutation of the literal grid and polygonal limits.
`ThueMorseBinomialLog.lean`	Exact integer signed-binomial sum and rational binomial–log formula for the zero–one bit, including the power-of-two log argument and ring-general bridge to the signed convention.
`DraftCounterexamples.lean`	Exact and asymptotic counterexamples to false literal claims in the informal draft.
`StirlingAsymptotics.lean`	Factorial and binomial asymptotics needed for the k-fold scale.
`LowerLambertW.lean`	Correctly defined lower Lambert branch, endpoint-inclusive continuity on $[-\exp(-1), 0)$, smooth-interior calculus, divergence at zero, and intrinsic two-term small-argument expansion.
`PrincipalLambertW.lean`	Principal real branch, endpoint-inclusive continuity on $[-\exp(-1), \infty)$, smooth-interior calculus, unit derivative at zero, and $W_0(z) \sim z$.
`LambertWBranchGapBernoulli.lean`	Exactly five theorems: real absolute convergence inside $\|z\| < 2\pi$, complex summability iff $\|z\| < 2\pi$, the actual real nonzero-argument sum, the actual complex sum with its canonical removable origin, and the paired compact branch-gap series. Thus the radius, boundary divergence, `eq:pair-Bernoulli-general`, and the canonical-removable reading of `eq:bernoulli-gen` are Exact; the literal totalized quotient and holomorphy are not claimed, and higher/full Puiseux claims remain open.
`LambertWCurvature.lean`	Nonsingular raw-branch second derivatives; principal strict concavity on its closed natural domain; and the lower branch's exact inflection, left strict convexity, and right strict concavity.
`LambertWBranchPointGeometry.lean`	Exactly eight theorems: signed right-hand derivative blow-up, signed endpoint-secant blow-up, failure of finite right differentiability, and failure of full differentiability for both totalized real branches.
`LambertWBranchPointAsymptotics.lean`	One noncomputable scale definition and eight theorems: positivity and square of $\sqrt{2\exp(1)(z + \exp(-1))}$, two squared-ratio limits, two squared equivalences, and the signed principal/lower leading square-root equivalents.
`PowerExponentialLambert.lean`	Definitions and exact solve laws for the principal and lower inverses of $A\lambda^m e^{-\beta\lambda}$.
`PowerExponentialLambertCalculus.lean`	Interior derivative and order laws, plus full-domain continuity of both scaled phases through their common finite branch point.
`PowerExponentialLambertCurvature.lean`	Four regular-interior derivative-of-derivative and exact second-derivative declarations for the two generic scaled phases; no generic sign, threshold, or shape package.
`PowerExponentialLambertInverse.lean`	Exact images and `InvOn` laws, classification of every nonnegative root below or at the peak, and strict-interior separation of the two roots.

Module	Role in the development
`PowerExponentialLambertAsymptotics.lean`	Principal root equivalence, lower-branch divergence, and the intrinsic two-term ϵ-expansion only; no cleaned $L = \log(A/x)$ form or full series.
`PowerExponentialLambertFabius.lean`	Unit-rate generalized-coordinate bridge and the classical Fabius principal/lower specialization, including full-domain continuity, exact root dichotomy, interior separation, and the principal x-equivalence.
`PowerExponentialLambertFabiusCurvature.lean`	One definition and eighteen theorems: the lower phase's exact inflection and two shape intervals, plus principal calculus and strict convexity on the whole `Iic` peak domain.
`PaperKFoldThueMorse.lean`	Aggregator separating exact results, repaired statements, and formal counterexamples.
Top-level packaging	
`FabiusFunction.lean`	Umbrella import exposing the two paper audits, generic Volterra calculus and exact Fabius primitive ladders, shape and inverse/non-elementarity theory, computability, negative-Laplace and all-order/closed-form saddle APIs, raw Lambert curvature and branch-point geometry/asymptotics, scaled Lambert curvature, k-fold audit, and Legendre developments.

Remark D.1. Some helper files are intentionally absent from this table, and a few families may acquire additional leaves as the repository evolves. The umbrella module and the actual import graph remain the definitive inventory. Duplicate-looking module names usually mark distinct proof stages—for example, raw algebra versus scalar specialization, or a central estimate versus its all-order strengthening.

The two Volterra rows headed by `NormalizedVolterra.lean` and `FabiusAntiderivatives.lean`, together with the corresponding reading path and focused-import example, originated in a post-snapshot addendum checked at code checkpoint `3da579387`. The four current declarations `normalizedVolterra_affine`, `normalizedVolterra_comp_affine`, `normalizedVolterra_basepoint_shift`, and `normalizedVolterra_succ_eq_taylor_of_eq_zero` are later post-snapshot additions checked at code checkpoint `e109088ed`. The sixteen rows from `ExponentialPartition.lean` through `LambertPhaseLockedRichardson.lean` are a post-snapshot addendum. The four immediately following rows, from `FabiusLambertPhaseLockedPullback.lean` through `FabiusLambertPhaseExtraction.lean`, form the later current-tree phase-extraction tranche. The three separate Bell rows `CompleteHomogeneousBell.lean`, `LambertPhaseLockedBell.lean`, and `FabiusLambertPhaseExtractionBell.lean` are a still later current-tree tranche inspected on 30 August 2026. They are recorded declaration-by-declaration without fabricating an aggregate commit hash; they close finite conversions and exact finite rewrites only, adding no convergence, sign, or asymptotic estimate. The first two of those sixteen were checked at code checkpoint `e119db9032ab2641955d47e1da1338a8386aac4d`; the third reflects mainline checkpoint `3989cd9d3`; the endpoint-transfer polynomials were checked at checkpoint `a86aee8e203812816fa5a0e97b9e077a0a6f974e`; and the finite polynomial-integral bridge is present at checkpoint `6e9148873264978eef43181d6bd214c36cdc405a`. The two interpolation rows reflect checkpoints `deefddd335b55bd74b7ac13d782a0b58fb7e2e93` and `cf6cd332660db235675fe3214d5bf757cb424086`. The `CompleteHomogeneousGenerating.lean` row is a later current-tree formal-series addition with the formal/analytic boundary stated explicitly. The two geometric specialization rows, including the sixteen exact declarations named in the post-snapshot

discussion above, were checked at checkpoint `1e761ed77583e9870ceeaeb2c74ac824094f0f3f`, all on 27 August 2026. The subsequent five geometric and Lambert rows reflect source checkpoint `c45276994d9411a45bef409ad6bc481771613113`, also on 27 August 2026. The `ImplicitPowerSeries.lean` row and the generic formal-root discussion are a post-snapshot foundation checked at source checkpoint `cd9086a80cbecefecb9bc1e00378d151061595e0` on 28 August 2026. The seven non-curvature Lambert rows from `LowerLambertW.lean` through `PowerExponentialLambertFabius.lean` record the current-tree endpoint-continuity, exact-root, and intrinsic-asymptotic tranche originating at source checkpoint `925ca34a698cbd08703ec5d8abc65515730e30f6` and described in Section 20; their stated open boundaries are part of the crosswalk. The three added rows `LambertWCurvature.lean`, `PowerExponentialLambertCurvature.lean`, and `PowerExponentialLambertFabiusCurvature.lean` are the current-tree curvature tranche, compiled 30 Aug 2026. They are recorded declaration-by-declaration without a fabricated checkpoint or aggregate hash. This tranche closes raw-branch curvature, the four generic second-derivative formulas, and the nineteen-declaration Fabius leaf. The two additional rows `LambertWBranchPointGeometry.lean` and `LambertWBranchPointAsymptotics.lean` are the later current-tree branch-point tranche, also compiled 30 Aug 2026 and likewise recorded without a fabricated checkpoint or aggregate hash. Their exact public inventory is eight theorems and, respectively, one definition plus eight theorems. They close the raw signed derivative/secant limits, non-differentiability, squared-ratio limits, and leading signed square-root equivalences. They do not close $O(\Delta)$ remainders, higher or convergent/full Puiseux expansions, generic/Fabius endpoint wrappers, or the generic $m \pm \sqrt{m}$ threshold and shape layer. The later row `LambertWBranchGapBernoulli.lean`, compiled in a focused build on 4 September 2026, has the exhaustive zero-definition/five-theorem surface listed above. It makes the Lambert guide's compact `eq:pair-Bernoulli-general` Exact and also makes the complex radius-2π, boundary-divergence, and canonical-removable `eq:bernoulli-gen` clauses Exact. The literal totalized quotient at zero and holomorphy of a named sum function are not claimed, and all higher Puiseux claims remain open. The `FabiusInverseLogarithmicModulus.lean` row is a still later current-tree addition inspected declaration-by-declaration on 30 August 2026. Its three definitions and fifteen theorems close the least logarithmic scale selector and both primitive-recursive reciprocal continuity witnesses only; the exact endpoint-mass ceiling, tolerant bisection, sequential computability, combined computable-real-function theorem, and input-bit asymptotics remain outside that leaf. No aggregate checkpoint is fabricated for this merged surface. The subsequent `FabiusInverseExactDyadicModulus.lean` source-only row has exactly two definitions and ten theorems. It closes the exact fixed-dyadic ceiling, endpoint counterexample, and target-specific strict leastness, and composes them with the logarithmic selector to obtain a reciprocal $1/n$ witness, and proves both denominator maps primitive recursive. It does not claim leastness for $1/n$ or assert a modulus at its convention-only zero input. When this row first landed, the preceding walkthrough PDF predated it; the row is included in this rendered publication, whose current receipt is kept externally. The still later compiled current-tree tranche `EffectiveMonotoneInverse.lean` and `FabiusInverseComputable.lean` has exactly the two-definition, six-theorem and zero-definition, one-theorem surfaces inventoried above. It closes fixed-depth tolerant bisection, sequential computability of the clamped inverse, and the combined computable-real-function theorem. The generic theorem consumes its inverse modulus. The later four-definition/four-theorem `EffectiveGapInverse.lean` module derives such a modulus and effective continuity from a supplied computable positive rational dyadic-gap sequence, and proves total computability precisely for the clamped extension; no input-bit asymptotic is added. The seven activation rows and the finite-Taylor and square-summable discussions in Section 11 record source checkpoint `a345425d21d90e680bf15e34093af42c69c08a83`, published 31 August 2026. Serialized `LAKE_JOBS=1` focused builds of `ActivationSeries`, `ActivationAsymptotics`, `GeometricActivationDimension`, `ActivationSeriesAsymptotics`, and `GeometricActivationAsymptotics` all exited zero without diagnostics. The final `lakebuild+FabiusFunction.ActivationTaylor` target also rebuilt the modified hyperbolic-activation dependency and exited zero. Direct Lean audits of the

previously recorded new or refactored declarations, `tanhDiv_eq_dslope`, and all three finite-Taylor theorems reported exactly `propext`, `Classical.choice`, and `Quot.sound` for each. At that upstream checkpoint, static inspection found all 610 modules below `FabiusFunction/` reachable from the facade and every one of the 105 activation declarations documented. A later historical current-tree audit scanned 925 modules and 11,619 public declarations, with no missing module headers or declaration comments. The later 934/11,709 checkpoint and both branches' subsequent inventories are historical: the local branch reached 961/11,974 and then 970/12,056, while the incoming branch reached 952/11,884, 967/12,001, 969/12,048, and 970/12,051. The later incoming audit scanned 985 modules and 12,199 explicit public declarations, with zero documentation gaps. These are historical figures; the current census is pinned in `docs/doc_audit_baseline.json`. The Legendre-biorthogonality 925/11,619 census remains an explicit historical checkpoint. At the retained historical mainline checkpoint before the latest six-module q-series tranche was 623 modules and 8,476 declarations. Its immediately preceding census, after the Rvachev Appell and original four-theorem `QPochhammerEntire.lean` tranche but before the generic q-Pochhammer modules, was 621 modules and 8,446 declarations. Relative to the earlier 610/8,318 checkpoint, the union adds the two raw Lambert branch-point leaves (seventeen declarations), the two Legendre–Gaunt leaves (twenty-five), `GeometricUniformMultisection.lean` (two coordinate definitions and three half–quarter laws), and the four-declaration `GaussianBinomialAtNegOneDerivative.lean` leaf. It also adds the two-definition/seven-theorem `LagrangeRvachevSynthesis.lean` leaf, the four-definition/six-theorem `LagrangeRvachevMatrix.lean` leaf, the generic and geometric barycenter theorems `integral_id_weightedUniformDistribution` and `integral_id_geometricUniformDistribution_eq_one_half`, the historical three-theorem surface of `PrimePowerBinomialValuation.lean` (now expanded to six), the two-definition/twenty-five-theorem `LegendreGauntClosedForm.lean` core, and its zero-definition/three-theorem `FabiusLegendreGauntClosedForm.lean` wrapper. Existing modules add the generalized spectral q-Pochhammer APIs, four geometric-density diagnostics, two geometric-sinc phase-prefix convergence theorems, eleven inverse-modulus refinements, three signed-power Rvachev moment theorems, and the general iterated shift–refinement product law; the merged `PolynomialCombExactness.lean` surface also contains `integral_polynomial_mul_rvachevUp_eq_dyadic_tsum`, the exact normalized physical-coordinate self-sampling quadrature. The final increment at that historical stage, from 620/8,438 to 621/8,446, consisted exactly of the three centered-convolution and mean-zero theorems added to `RvachevMomentAppell.lean`, the arbitrary-phase synthesis theorem added to `RvachevPolynomialSynthesis.lean`, and the zero-definition/four-theorem `QPochhammerEntire.lean` leaf documented above. After that prior Appell/entire tranche, the increment from 621/8,446 to 623/8,476 is exactly `QPochhammerDissection.lean`, with zero definitions and two theorems, and `QPochhammerInfinite.lean`, with one definition and twenty-seven theorems: two modules and thirty public declarations. That historical merge then adds `complexQPochhammerInf_eq_zero_iff_eq_inv_pow`, the fifth theorem of `QPochhammerEntire.lean`, plus the six modules inventoried above: `QBinomialTheoremInfinite.lean` (one definition, twenty-two theorems), `JacobiTripleProduct.lean` (two definitions, twenty-five theorems), `QPascalSummation.lean` (four theorems), `GaussianBinomialContinuity.lean` (three theorems), `QuantumBinomial.lean` (two theorems), and `RogersSzegoPolynomial.lean` (one definition, nine theorems). Those six modules contribute sixty-nine declarations; together with the added reciprocal-lattice theorem, they explain the exact 623/8,476 to 629/8,546 q-series increment. The subsequent zero-definition/six-theorem `GeneralizedRvachevIdentifiability.lean` leaf raises that census by one module and six declarations to the historical 630/8,552 checkpoint. Subsequent integration adds the zero-definition/three-theorem `GeometricPochhammerNormalConvergence.lean` leaf, expands `PrimePowerBinomialValuation.lean` from three to six theorems, adds the two-definition/six-theorem `EffectiveMonotoneInverse.lean` module and the one-theorem `FabiusInverseComputable.lean` module, and adds two theorems to `QPochhammerInfinite.lean`. The first six q-calculus modules contribute another 36 declarations.

Four subsequent modules contribute 38 more, the next four contribute twenty, and the final two q leaves contribute thirteen. Two later linear-coefficient theorems enlarge `GaussianBinomialPalindromic.lean` to 0+14, while `EffectiveGapInverse.lean` contributes one module and eight declarations. The one-definition/eight-theorem `RvachevSuperconvergentSynthesis.lean` leaf contributes nine; the four root-of-unity/q-Catalan modules contribute twenty-five; the original q-beta/Newton pair contributes twenty-four; the integer/complex upper Gaussian and q-Pfaff–Saalschütz tail contributes twenty; and the noncommutative q-multinomial leaf contributes five. Finally, `GaussianBinomialBounds.lean` contributes six theorems and the Newton compatibility union contributes seven declarations. That source intermediate census was therefore 672/8,875. The subsequent 5+14 `BinaryWordInversions.lean`, 2+8 `BoxPartitions.lean`, and 0+5 `TelescopingCertificate.lean` leaves add three modules and thirty-four declarations, giving the historical 675/8,909 checkpoint. The subsequent source union, including the exact 0+7, 4+16, and 0+9 Lambert branch-pairing, gap-bijection, and symmetry surfaces, was followed by the exact inverse, Jacobi two-square, Lagrange–Rvachev Matrix, weighted/geometric barycenter, twelve-declaration terminating-basic-hypergeometric, and 3+7 `GeometricRichardsonGenerating.lean` tranches. Three subsequent theorems in the existing 2+24 `GaussianBinomialCumulants.lean` module add the explicit second derivative at one and the division-free raw second moment and variance numerator. The later 0+5 `LambertWBranchGapBernoulli.lean` leaf adds `summable_bernoulli_mul_pow_div_factorial_iff` and `hasSum_bernoulli_mul_pow_div_factorial_complex_iff`; together with the three finite branch-coordinate modules it gives four definitions and thirty-seven theorems, forty-one declarations in all. The initial four-theorem exact-radius tranche formed the historical 903/11,447 checkpoint, and the complex `HasSum` equivalence raised that checkpoint to 903/11,448. Its complex Bernoulli series converges exactly on the open radius-2π disk and there has the canonical removable `complexExpm1Div` inverse as its sum; this is not the literal totalized quotient at zero and asserts no holomorphy. The real quotient evaluation retains exactly $z \neq 0$, the branch theorem excludes both endpoints, and no remainder, higher-Puiseux, or Guide nearest-zero proof route is claimed. The two new `iteratedDeriv_rvachevUp_eq_extendedFabius` and `iteratedDeriv_rvachevUp_dyadic_below` theorems also remain in the exhaustive zero-definition/six-theorem `DyadicDerivativeFiltration.lean` surface. On the independently retained local chronology, the branch-pairing, gap-bijection, and symmetry surfaces together with twelve later terminating basic-hypergeometric declarations gave the 901/11,430 checkpoint. The 3+7 `GeometricRichardsonGenerating.lean` module then gave 902/11,440; three additions to the existing `GaussianBinomialCumulants.lean` surface gave 902/11,443; and the successive three-, four-, and five-theorem Lambert–Bernoulli receipts were 903/11,446, 903/11,447, and 903/11,448. The local five-theorem effective fixed-column extension of `QBinomialTheoremInfinite.lean` then gave 903/11,453. Independently, the one-definition/eight-theorem `GeometricUniformMomentPolynomial.lean` branch gave 904/11,457 from the Lambert checkpoint, and its first union with the fixed-column branch gave the historical 904/11,462 receipt. The later 1+17 `GeometricUniformRealization.lean` and 2+1 `RegularCentralQBinomialSum.lean` leaves, followed on the incoming source branch by the 0+10 `GaussianBinomialFixedColumnRate.lean` leaf and the 1+14 `RvachevAppellHasse.lean` leaf, brought that checkpoint to 910/11,525. The subsequent 1+14 `RvachevLagrangeNodesOnly.lean` and 0+2 `GaussianBinomialGreaterOneAsymptotics.lean` leaves brought it to 912/11,542. Finally, the 1+8 `GeometricUniformMomentPolynomial.lean` leaf brings that census to 913/11,551 and supplies the algebraic portion of `p7:thm:Pn`; the analytic normalization and sharp leading, subleading, and exact-degree claims remain absent. The final 0+3 `ThueMorseGammaTowerDifferential.lean` leaf raises the historical census to 914/11,554 and makes `p2:thm:gamma-tower` Exact on $a > 0$ for the chosen GammaLog coordinate. The subsequent polynomial-exactness theorem in `GeometricResidualMoments.lean` raises that declaration count to 11,555 and makes `cor:scaled-geometric-moments` Exact by composition. The exhaustive zero-definition/one-theorem `GeometricUniformMomentPolynomialBridge.lean` companion then raises that checkpoint to 915/11,556; its sole theorem, `geometricUniformMomentPolyno`

mial_eval$_2$_eq_mgf_taylorCoefficient, supplies the real-$|q| < 1$ analytic normalization and makes p7:eq:Pn-def Exact in that regime. The leading-coefficient and strict odd-degree clauses are supplied below by the sharp-degree leaf. The final exact-closure tranche adds the 1+4 ThueMorseCornerIntegral.lean and 0+3 RvachevLegendreCentralSum.lean leaves, while the existing zero-definition FinitePolynomialFunctional.lean module grows to sixteen theorems. The subsequent then-public $1 + 2$ GeometricUniformComplexMomentProduct.lean leaf constructs the actual product for complex $\|q\| < 1$, proves its locally uniform convergence, and identifies its normalized Taylor coefficient with the same polynomial. On its source branch it formed the historical 906/11,461 checkpoint; in the merged chronology it formed the historical 918/11,568 checkpoint. The exhaustive $0 + 1$ HalfQBinomialRootSimplicity.lean leaf then completed the half-base simple-root locus and raised the actual merged-main checkpoint to 919/11,569. The subsequent exhaustive $1 + 2$ GeometricUniformExteriorComplexMomentGerm.lean leaf defines the actual reciprocal germ for complex $1 < \|q\|$, proves it analytic at zero, and identifies its normalized Taylor coefficient with the same polynomial, raising the census to 920/11,572. Neither analytic leaf alone supplies the global rational RatFunc; the later rational-coefficient leaf provides that assembly. The exhaustive zero-definition/three-theorem GeometricUniformMomentPolynomialDegree.lean leaf then proves the top and subleading Bernoulli coefficients and the exact parity-sensitive degree, making p7:thm:Pn and prop:qF-P-degree-sharp Exact and raising the census to the historical 921/11,575 checkpoint. The exhaustive $1 + 6$ RvachevLaurentLeading.lean leaf then makes is:p2:thm:Laurent-leading Exact and gives 922/11,582. The exhaustive $11 + 17$ FinitePrefixAppellRecovery.lean leaf makes is:p2:thm:finite-prefix-expansion and is:p2:thm:exact-recovery Exact and gives the historical 923/11,610 checkpoint. The separately inventoried five-module polynomial q-calculus increment—PolynomialQDerivative.lean (2+17), PolynomialQLeibniz.lean (0+4), QPochhammerDerivative.lean (0+3), LambertSeriesLog.lean (0+4), and QGamma.lean (2+10)—is retained in full: four definitions and thirty-eight theorems, forty-two public declarations, with the exact hypotheses and boundaries stated in its rows above. The exhaustive $1 + 4$ GeometricUniformMomentRatFunc.lean leaf then packages the common rational coefficient, its global q-factorial clearing, both safe analytic evaluations, and the removable $q = 1$ value. It makes thm:qF-moment-polynomial Exact and gives the historical 924/11,615 checkpoint. The two new theorems in the existing ProbabilityLaplaceMoments.lean module then give 924/11,617, and the exhaustive one-definition/one-theorem RvachevLegendreBiorthogonality.lean leaf gives the historical 925/11,619 census. Subsequent merged-main additions give the pre-reciprocity 930/11,678 checkpoint. Publicizing differentiable_geometricUniformComplexMomentProduct gives 930/11,679, and the exhaustive $1 + 5$ GeometricUniformMomentReciprocity.lean leaf gives the historical reciprocity checkpoint 931/11,685. The subsequently merged upstream DyadicBoundaryIdentity.lean and FinitePrefixThueMorseCollapse.lean modules add two facade modules and ten public declarations, giving the historical dyadic/finite-prefix checkpoint 933/11,695. The incoming union adds one module and fourteen public declarations: the new zero-definition/six-theorem ProuhetBaseTwoBridge.lean module, one theorem added to DyadicBoundaryIdentity.lean, and seven theorems added to ThueMorseNewmanSelfSimilarity.lean. This gives the historical 934/11,709 checkpoint. The local branch's subsequent transseries-algebra integration gives the historical 943/11,791 checkpoint. The exhaustive (1+11) TransseriesFlat.lean leaf and the three new integer-Laurent theorems in TransseriesDifferentialBlock.lean give the historical 944/11,806 checkpoint.

Against that checkpoint, the preceding integration added exactly sixteen modules: BackwardErrorExistence.lean (1+6), BellLeibnizTower.lean (1+5), CayleyTreeFunction.lean (1+8), DerangementNearestInteger.lean (1+7), LambertCorrectionEquation.lean (2+9), LambertShiftConcavity.lean (0+5), LeastTermIndex.lean (1+6), LinLogCoreInversion.lean (4+18), OrdinaryPartialBell.lean (2+4), PowerLogCoreInversion.lean (3+6), RemainderTransport.lean (0+3), StaircaseInversion.lean (0+7), To

uchardEulerOperator.lean (2+8), TransseriesDifferentialClosure.lean (2+9), TransseriesHarmonicIncrement.lean (0+2), and WrightOmega.lean (1+12), totaling 21 definitions and 115 theorems. Existing surfaces supply the remaining 24 declarations: Transser iesFlat.lean grows by (3+11) to (4+22), TransseriesDifferentialBlock.lean gains four theorems to (0+12), QBinomialTheoremInfinite.lean gains five theorems to (1+27), and RvachevPochhammerFactorization.lean retains one additional public bridge. The modified LaplaceMomentBoundsSharp.lean is excluded from the new-module count. Hence that historical census is 960/11,966, with zero missing module headers and zero missing declaration comments.

The retained source-only StirlingCompleteHomogeneous.lean leaf has zero definitions and eight public theorems, taking that 960/11,966 checkpoint to the historical local 961/11,974 source inventory. Its principal crosswalk entry points are Fabius.stirlingSecond_add_eq_complet eHomogeneousEvalOn and Fabius.stirlingSecond_add_eq_sum_finsuppAntidiag; the companion declarations expose generating-series, symmetric-polynomial, and factorial-normalization interfaces. The focused leaf compiled warning-free; these source counts assert neither a successful aggregate build nor PDF parity.

On the incoming branch, the nine-module series/transseries overlay carried 72 explicit public declarations, with two additional Neumann names generated by to_additive outside the lexical count. Six concurrent declarations gave the historical 943/11,787 checkpoint; nine further modules with 90 explicit declarations and four integer-exponent theorems gave 952/11,881. Two written OrderDual Neumann wrappers and the real-analytic Wright omega theorem then gave the incoming historical checkpoint 952/11,884. These branch inventories overlap and are not additive.

The preceding local merge audit contained 970 facade-reachable modules and 12,056 explicit public declarations, with zero missing module headers, missing declaration comments, or duplicate public names. Relative to the local 961/11,974 checkpoint, the nine new modules are CayleyKernel.lean, CayleyLocalCoordinate.lean, DivisorTransform.lean, ExpSeriesRecurrence.lean, FabiusEndpointTwoTerm.lean, StirlingSeriesCoefficients.lean, TransseriesBlock Classes.lean, TransseriesMonomialUniqueness.lean, and WrightOmegaTwoOrders.l ean. At that checkpoint they contributed eight definitions and 60 theorems; existing-module changes contributed a net 14 declarations. The retained Stirling complete-homogeneous leaf still has eight theorems. The later incoming branch recorded 967/12,001 after fifteen modules and 117 declarations, then 969/12,048 after two modules and 47 declarations; the three-theorem power-recurrence leaf gave its historical 970/12,051 checkpoint. These counts overlap the local inventories and are not additive.

The preceding merged audit was 977 modules and 12,133 explicit public declarations, with zero documentation gaps. Relative to 970/12,056, seven new leaves contribute 80 declarations, while three existing declarations move from NorlundDiagonal.lean to ExponentialRescaling.lean, for a net increase of 77. The eight-theorem Stirling leaf is retained. The two certificate leaves added nine public theorems at the historical 979/12,142 checkpoint. The incoming branch recorded a historical 985/12,199 inventory after six further leaves added 57 declarations. The authoritative current census is computed by scripts/doc_audit.py and pinned in docs/doc_audit_baseline.json. No new aggregate-build or PDF-parity claim is made.

The crosswalk remains conservative: the abstract Faà di Bruno result, the ordinary Bell normalization, and the Touchard definition and displayed Euler-operator equation are Exact. The real basic Wright omega row plt:prop:mot-omega-basic is also Exact, including real analyticity, without asserting complex holomorphy. The broader Wright omega claims, differential closure, harmonic increments, Cayley, derangement, Lambert correction and its bracket bounds, linear-logarithmic inversion, power-logarithmic inversion, remainder transport, and staircase inversion remain Partial; the least-term-index row is Neighboring. Every module is reachable from the facade. No aggregate facade build is claimed for this current-tree census. The title-page pin remains historical; current publication receipts are external. The present union has 1004 modules and 12,500 public declarations, with zero module-header or declaration-comment gaps. Its QBinomialTheoremInfinite surface is 1+29 and GaussianBinomialFixedColumnRate is 0+9, retaining compatibility names for the stronger

upstream product and shifted-limit APIs. The exact rational-moment and reciprocity closures are retained; the stronger MGF row remains Partial. The q forward ledger is 181/79/14/8; the q-Lucas root-evaluation theorem does not yet close the polynomial-congruence row.

The exhaustive public surface of `FinitePrefixThueMorseCollapse.lean` is zero definitions and eight theorems: `Appell.sum_thueMorseSign_mul_eval_poly`, `Fabius.sum_thueMorseSign_mul_uncenteredDyadicPrefixAppellPolynomialRat`, `Fabius.sum_thueMorseSign_mul_uncenteredDyadicPrefixAppellPolynomialRat_of_lt`, `Fabius.sum_thueMorseSign_mul_uncenteredDyadicPrefixAppellPolynomialRat_self`, `Fabius.sum_thueMorseSign_mul_centeredDyadicPrefixAppellPolynomialRat`, `Fabius.sum_thueMorseSign_mul_centeredDyadicPrefixAppellPolynomialRat_succ`, `Fabius.sum_thueMorseSign_mul_centeredDyadicPrefixAppellPolynomialRat_of_lt`, and `Fabius.sum_thueMorseSign_mul_centeredDyadicPrefixAppellPolynomialRat_self`. Writing $A^{\mathrm{unc}}_{N,n}$ and $A^{\mathrm{cen}}_{N,n}$ for the two rational prefix Appell polynomials and $s_N = 1 - 2^{-N}$, their complete signed responses are

$$\sum_{k<2^N} \varepsilon_k A^{\mathrm{unc}}_{N,n}(x + k/2^N) = (-1)^N 2^{-\binom{N+1}{2}} n^{\underline{N}} x^{n-N}$$

and

$$\sum_{k<2^N} \varepsilon_k A^{\mathrm{cen}}_{N,n}(x + s_N - 2k/2^N) = 2^{-\binom{N}{2}} n^{\underline{N}} x^{n-N}.$$

Thus the uncentered response has the exact sign and $2^{-\binom{N+1}{2}}$ normalization, while the centered response is sign-free with $2^{-\binom{N}{2}}$. For positive depth $N = m + 1$, the successor theorem uses the literal manuscript grid $x + s_{m+1} - k/2^m$. The two `of_lt` theorems give cancellation below the block depth, and the two `self` theorems give the first nonzero constants. These results make `is:p2:thm:TM-uncentered`, `is:p2:cor:Prouhet-canonical`, and `is:p2:thm:TM-centered` Exact by composition. The total main theorems include $N = 0$, strengthening the positive-depth formulas. The entire leaf is rational coefficient algebra: it introduces no random variable or `HasLaw`, proves no analytic MGF, and makes no Barnes-function identification. The three promotions move the 194-row inverse concordance to 57 Lean-proved, 88 human-proved, 10 conjectural, 15 open, and 24 nonassertoric rows.

Those six counts are `QPochhammerInfiniteBounds.lean` (0+5), `HeineTransformation.lean` (2+5), `QGaussSummation.lean` (0+2), `QPochhammerComplexOrder.lean` (1+4), `BasicHypergeometricSeries.lean` (2+5), and `QMultinomial.lean` (1+9). The four subsequent counts are `GaussianBinomialPalindromic.lean` (0+14), `JacksonIntegral.lean` (1+7), `QExponential.lean` (3+8), and `ThetaQuasiPeriodicity.lean` (1+6). They add Gaussian-polynomial degree, palindromicity, and the total linear-coefficient classifier, q-exponential and q-derivative laws, Jackson integration, and bilateral-theta quasi-periodicity and zeros under the displayed analytic hypotheses. The four newest counts are `GaussianBinomialPolynomialStructure.lean` (0+5), `JacobiCubic.lean` (0+2), `QPochhammerLogDerivative.lean` (0+10), and `QPochhammerOrderDerivative.lean` (0+3). Their twenty theorems add universal Gaussian polynomial structure, Jacobi's cubic identity, the q-Pochhammer logarithmic derivative and Lambert-series form, and differentiation with respect to complex order under the displayed hypotheses. The final two counts are `CentralQBinomialReduction.lean` (0+6) and `CyclotomicFactorization.lean` (0+7). Their thirteen theorems add finite-symbol sign pairing and even–odd dissection, the central Gaussian reduction, and cyclotomic factorizations. The quotient and integral-domain hypotheses remain explicit. The four root-of-unity and q-Catalan counts are `CyclotomicDivisibility.lean` (0+3), `PrimitiveRootBlock.lean` (0+3), `QCatalan.lean` (1+11), and `QLucas.lean` (0+7). Their one definition and twenty-four theorems add the cyclotomic carry criterion, primitive-root blocks, q-Lucas, and the integral q-Catalan polynomial with exact degree and specialization at one. The newest two counts are `QBetaIntegral.lean` (1+8) and `NewtonInterpolation.lean` (3+19). Their four definitions

and twenty-seven theorems evaluate the Jackson q-beta integral in product and q-gamma forms and develop triangular Newton interpolation, divided differences, the explicit geometric-grid specialization, and the seven-name `newtonInterpolant` compatibility surface. The q-beta theorems retain $0 < q < 1$ and positivity of both arguments. The latest three counts are `GaussianBinomialInteger.lean` (1+10), `GaussianBinomialComplexOrder.lean` (1+5), and `QPfaffSaalschutz.lean` (0+3). Their two definitions and eighteen theorems extend Gaussian coefficients to integer and principal-branch complex upper indices, prove the two integer q-Pascal laws and negative-index reflection, give the reciprocal finite and generalized complex-order q-binomial series, and prove the terminating balanced ${}_3\phi_2$ q-Pfaff–Saalschütz sum. All nonzero, strict-contraction, and denominator hypotheses are retained explicitly. The terminating reversal pair contributes `TwoPhiOneReversal.lean` (2+12) and `QChuVandermonde.lean` (0+10), with the public `two_mul_choose_two` in the latter module. The actual-tsum reversal and both displayed q-Chu formulas are exact; the separately named by-reversal proof of the second formula retains its two auxiliary nonvanishing hypotheses. The subsequent `JacobiTwoSquareCount.lean` leaf is 0+4 and closes the nonzero two-square count and both Lambert forms; its prime-product valuation hypothesis and the analytic condition $\|q\| < 1$ remain explicit. The final zero-definition/five-theorem `QuantumMultinomial.lean` module decomposes tuple antidiagonals, transports Gaussian symmetry to arbitrary semirings, proves q-multinomial coefficient commutation, and expands powers of a finite sum into ordered monomials under the pairwise relations $x_j x_i = q(x_i x_j)$ and commutation of the nome with every variable. It is finite and division-free. The final `GaussianBinomialBounds.lean` count is 0+6: `gaussianBinomial_inv`, `one_le_gaussianBinomial`, `finiteQPochhammerIn_pow_le_one`, `gaussianBinomial_le_inv_qPochhammerInfIn`, `pow_le_gaussianBinomial_of_one_lt`, and `gaussianBinomial_le_pow_div_of_one_lt`. These are the reciprocal uniform bounds for $0 \le q < 1$ and dimension-dominant bounds for $Q > 1$, with all index and order hypotheses explicit. The proof reuses the stronger generic `finiteQPochhammerIn_self_pos` theorem from `GeneralQConditionNumber.lean` rather than exporting a duplicate. The final terminating-basic-hypergeometric pair is `TwoPhiOneReversal.lean` (2+12) and `QChuVandermonde.lean` (0+10). The two definitions in the first are `twoPhiOneFinite` and `twoPhiOneReflection`; its twelve theorems are `choose_two_add_succ_choose_two`, `finiteQPochhammerIn_sub_eq`, `finiteQPochhammerIn_reversal_ne_zero`, `finiteQPochhammerIn_inv_pow_self`, `twoPhiOneReflection_involutive`, `twoPhiOneFinite_reversal`, `twoPhiOneFinite_reversal_twice`, `twoPhiOneFinite_eq_sum_twoPhiOneTerm`, `twoPhiOne_eq_twoPhiOneFinite_inv_pow`, `twoPhiOne_reversal`, `twoPhiOne_reversal_twice`, and `twoPhiOne_one_eq_twoPhiOneFinite_zero`. Thus it connects the finite terminating sum to the actual `twoPhiOne` tsum and proves that the parameter reflection and the full reversal are involutive.

The ten theorems of `QChuVandermonde.lean` are `two_mul_choose_two`, `mul_sub_one_eq_mul_sub_add`, `finiteQPochhammerIn_div_eq_sum_chu`, `q_chu_vandermonde_first`, `finiteQPochhammerIn_div_eq_sum_chu_second`, `twoPhiOneFinite_mul_finiteQPochhammerIn_eq_chu_second`, `q_chu_vandermonde_second`, `q_chu_vandermonde_second_by_reversal`, `twoPhiOne_q_chu_vandermonde_first`, and `twoPhiOne_q_chu_vandermonde_second`. They prove both q-Chu–Vandermonde evaluations, their actual-tsum wrappers, a denominator-cleared second identity, and a separately named reversal route. Thus the two displayed q-Chu evaluations and the reversal lemma are exact. The claim that reversal alone yields the second identity over its full displayed domain remains partial: the compiled by-reversal theorem assumes $C \neq 0$ and $(A;q)_n \neq 0$, while the unrestricted denominator-defined theorem is obtained directly from finite q-Cauchy. Rational continuation and a commutative-ring cleared extension are not claimed.

The three later additions to `GaussianBinomialCumulants.lean`, now 2+24, are `eval_one_derivative_derivative_gaussianBinomial_X`, `twelve_mul_secondMoment_gaussianBinomial_eval_one`, and `twelve_mul_varianceNumerator_gaussianBinomial_eval_one`. For $P = [n,k]_X$, $D = k(n-k)$, and $C = \binom{n}{k}$, the first assumes a characteristic-zero field and exactly $k \le n$, and proves $P''(1) = D(3D+n-5)C/12$. The other two hold for every n, k over an arbitrary

commutative semiring and state

$$12\big(P''(1)+P'(1)\big)=D(3D+n+1)C,\qquad 12C\big(P''(1)+P'(1)\big)=12P'(1)^2+D(n+1)C^2.$$

They use no division, nonvanishing, or characteristic hypothesis, are total above the Gaussian row by zero extension, and include the zero and diagonal rows. The first cleared identity is the raw second moment; the second is the variance numerator.

E Corrections and critical distinctions

Topic	Informal hazard	Formal treatment
Bounded versus global F	A translation or dyadic formula is written for a clamped function on all of $\mathbb{R}$.	Use `extendedFabius` for the signed global statement; prove agreement with the bounded function on the unit interval.
Original finite convolution	A limiting convolution object is used where a finite-stage identity is required.	State the convolution theorem for each finite row, then pass to the limit through weak convergence.
Step endpoints	Closed interval indicators count a shared endpoint twice.	Define a half-weight endpoint representative when the theorem is pointwise; retain ordinary indicators for almost-everywhere/integral statements.
Fourier equation factor	A transform variable or scaling factor is dropped during differentiation/dilation.	Derive the identity from mathlib's Fourier derivative and affine-map theorems under a fixed convention.
Poisson equations (25) and (32)	Equation (25) omits the translation variable from the exponential, equation (32) retains a factor $1/a$ after the equation has been scaled by a, and the printed specialization carries an unused upper bound $a \le 1$.	Use the proved normalized identity with $e^{2\pi \mathrm{i} k t/u}$; after multiplying both sides by the spacing, cancel the reciprocal factor. The sharp support specialization needs only $a \ge \frac{1}{2}$.
Positive index	A denominator, derivative order, or truncated product is used at $n=0$ although the formula assumes $n>0$.	Add the explicit order hypothesis and prove the zero case separately when meaningful.
Removable singularity	A quotient such as $(e^z-1)/z$ is treated as globally defined.	Define a total function with the limiting value at zero and prove a global power-series theorem.
Dyadic representation	A formula is proved only for a reduced numerator even though later sums produce unreduced fractions.	Prove refinement invariance and formulate global correctness for arbitrary representatives.

Topic	Informal hazard	Formal treatment
Taylor series	Exact finite Taylor truncation is mistaken for local analyticity.	Use a finite-order Taylor theorem with vanishing remainder; separately prove the analytic-at predicate fails.
Logarithmic delay equation	A branch-dependent derivative law is differentiated without a domain.	State the transformed identity only for $t \geq 1$ and use eventual filters thereafter.
Periodic asymptotic term	A bounded log-periodic correction is absorbed into an $o(1)$ or inverse-log error.	Prove the correction is nonconstant and exhibit phase subsequences with different residual limits.
Binary-reduction series	The outer sum begins after the endpoint contribution.	Start at $m = 0$ or include the missing boundary term explicitly so finite telescoping is exact.
Finite q dependence	Independence of the limiting value is promoted to independence of every approximant.	Preserve the complex q parameter at finite depth; prove its effect tends to zero through complex-shift estimates.
Theorem 20 naturality	A proof-internal sentence asserts that $d_k N_n$ is natural.	The corrected integral quantity is $2d_k N_n$; `two_mul_halfMoment_mul_reshetnikovMultiplier_isNatural` and its oddness strengthening retain the missing factor 2.
Saddle expansion composition	A truncated Lambert expansion is substituted to all orders without a composition remainder proof.	State the primary all-order theorem at the exact phase and prove elementary-log expansions separately with implicit-function error bounds.
Quotient exponential fit	Numerical agreement over a finite range is treated as asymptotic equivalence.	Compare logarithmic scales and prove the proposed quotient has the wrong endpoint decay.
K-fold normalization	A draft normalization or global error is assumed from plots or low-depth data.	Evaluate exact small cases and formalize counterexamples before proving a repaired statement.
K-fold polygon	Convergence of corrected grid samples is restated as convergence of the literal interpolating polygon.	The literal polygon is refuted at $x = 1$; only the normalized grid samples are identified with moving-center step approximants and proved convergent.
K-fold local error	The proposed $4h$ error bound is inferred from a picture.	At $k = 2, j = 1$, the proposed slope is -2 while $F'(1/4) = 1$, giving error $3 > 4h = 1$.
K-fold equation (7)	An unbounded proxy sequence is assigned a finite maximum.	For every $x > 0$, its successive-ratio analysis gives growth like $x2^{n+1}/(n+1)$, so no displayed maximum exists.

Topic	Informal hazard	Formal treatment
K-fold equation (10)	A linear residual is hidden inside $O(\log n)$.	Substituting the printed stationary equation leaves an explicit $+n$ term, which cannot be absorbed into a logarithmic error.

E.1 What these corrections teach

The corrections fall into four recurring classes. Domain errors arise when a local functional equation is used globally. Representation errors arise when a bounded object is substituted for a signed extension or an almost-everywhere representative is substituted in a pointwise claim. Normalization errors arise from Fourier, convolution, and dyadic scaling conventions. Asymptotic errors arise when a bounded oscillation is mistaken for a small remainder or when a formal series is composed without analytic control.

Lean detects each class differently. A domain error appears as an unprovable interval side condition. A representation error appears as a concrete endpoint contradiction. A normalization error leaves an unmatched scalar factor. An asymptotic error usually survives local algebra but fails at the limit theorem; the repository then proves a negative result by constructing a subsequence or comparing scales.

F Lean and mathematical glossary

Term or identifier	Meaning in this development
`UnitInterval`	The subtype $\{x : \mathbb{R} \mid 0 \le x \le 1\}$, used as the codomain of the bounded function.
`BoundedFabius`	A real-input function whose values carry the unit-interval bound in their type.
`fabiusReal`	Real-valued projection of a subtype-valued bounded function, suitable for calculus and integration.
`IsFabius`	Abstract specification combining tail values, smoothness, symmetry, and the left-half derivative law.
`fabius` / `fabius_spec`	The canonical constructed object and the theorem that it satisfies the specification.
`rvachevUp`	Compactly supported even twin used in Fourier and probability arguments.
`extendedFabius`	Signed globally defined extension assembled from Thue–Morse-weighted translates of the up-function.
`HasDerivAt`	Pointwise derivative proposition carrying differentiability and the derivative value together.
`ContDiff`	Smoothness predicate to finite or infinite order.
`tsum`	Infinite sum over a type; translate sums are proved locally finite before this is manipulated.
`HasSum`	Proposition that a series has a specified sum; often more convenient than rewriting a `tsum`.
`Finset.rangen`	Finite set $\{0, \dots, n-1\}$, the dominant indexing object for recurrences and finite products.
`Nat.strong_induction_on`	Well-founded induction giving hypotheses for every smaller natural index.

Term or identifier	Meaning in this development
`IsBigO`, `IsLittleO`	Filter-based asymptotic bounds and negligibility.
`atTop`	Filter describing a variable tending to positive infinity.
`nhdsWithin0(Set.Ioi0)`	Right-hand approach to zero, the endpoint filter used for $x \downarrow 0$.
`eventually`	A property holding on some set belonging to a filter; used to encode large-variable thresholds.
`Summable`	Unconditional summability in the relevant normed additive group. In `AnalyticSeriesFilter.lean` the generic hypothesis is on each weighted sampled series; specialized geometric statements require an unweighted sample only when its weight is nonzero.
`intervalIntegral`	Oriented integral over real endpoints, useful for FTC and integration by parts.
`MeasureTheory.Integrable`	Whole-space integrability; usually proved from compact support and continuity.
`SchwartzMap`	Smooth rapidly decreasing function type used for Fourier injectivity and Poisson summation.
`fabiusDyadicValue`	Executable exact rational evaluator for a dyadic input representation.
`fabiusDyadicUnitAux`	Well-founded highest-bit recursion at the core of unit-interval evaluation.
`halfMoment` / `moment`	Exact rational recurrence sequences later identified with analytic integrals.
`halfMomentNumerator` / `momentNumerator`	Division-free natural recurrences used for parity and denominator proofs.
`binaryWeight`	Number of one-bits of a natural number.
`thueMorseSign`	Sign $(-1)^{\mathrm{wt}_2(n)}$.
`halfQBinomial`	Exact q-binomial coefficient specialized to $q = 1/2$.
`field_simp`	Tactic for clearing field denominators after nonvanishing hypotheses have been supplied.
`norm_cast` / `push_cast`	Tactics normalizing canonical casts across algebraic operations.
`ring_nf`	Polynomial normalizer used after divisions, casts, and finite sums are brought to algebraic normal form.
`filter_upwards`	Tactic for collecting eventual hypotheses and proving a pointwise eventual conclusion.

G Example exploration snippets

The following snippets are intentionally schematic where theorem argument order may change. They show the style of interaction rather than replacing editor inspection of the current declarations.

G.1 Inspect the public objects

```
import FabiusFunction

open Fabius

#check UnitInterval
#check BoundedFabius
#check IsFabius
```

```
#check fabius
#check fabius_spec
#check rvachevUp
#check extendedFabius
```

G.2 Evaluate exact dyadic data

```
import FabiusFunction.Arithmetic

open Fabius

#eval fabiusInversePowTwoTable 8
#eval fabiusDyadicValue 1 1
#eval fabiusDyadicValue 5 4
#eval extendedFabiusDyadicValue 13 5
```

The returned objects are exact rationals. To evaluate a rational input through the recognition layer, inspect the declarations near `IsDyadicRational`, `dyadicExponent?`, and the evaluator wrappers in `Arithmetic.lean`.

G.3 Work generically under the specification

```
import FabiusFunction.Differential

open Fabius

variable (F : BoundedFabius) (hF : IsFabius F)

#check hF.symmetry_all
-- Search within Differential.lean for the iterated derivative
-- theorem whose interval hypotheses fit the desired dyadic cell.
```

The generic style is useful for proving a new consequence once and applying it both to the canonical function and to any future equivalent construction.

G.4 Use the normalized Volterra foundation (post-snapshot)

```
import FabiusFunction.NormalizedVolterra

open Fabius

#check iteratedPrimitive_add
#check iteratedPrimitive_succ_hasStrictDerivAt
#check iteratedPrimitive_eq_normalizedVolterra
#check normalizedVolterra_affine
#check normalizedVolterra_comp_affine
#check normalizedVolterra_basepoint_shift
#check normalizedVolterra_succ_eq_taylor_of_eq_zero
#check normalizedVolterra_succ_hasStrictDerivAt
#check iteratedDeriv_normalizedVolterra_add
#check contDiff_normalizedVolterra
#check normalizedVolterra_add
#check normalizedVolterra_succ_iteratedDeriv_eq_sub_taylor
#check normalizedVolterra_polynomial
```

```
#check normalizedVolterra_monomial
```

This focused import is independent of the Fabius specification. Its operators have arbitrary base points and oriented endpoints. The division-free affine law has no integrability hypothesis and includes negative and zero scales; its inverse form asks only that the scale be nonzero. The base-point shift assumes interval integrability on its two oriented pieces, while the positive-order zero-tail form asks for integrability only up to the new base point and vanishing on the open interval beyond it. All four work in a real normed space. Only the continuous literal-iterator/strict-FTC bridge, derivative cancellation and smoothness, and Taylor reconstruction need completeness. Specialize only when the Fabius dilation law is actually needed:

```
import FabiusFunction.FabiusAntiderivatives

open Fabius

variable (F : BoundedFabius) (hF : IsFabius F)

#check normalizedVolterra_extendedFabius F hF
#check normalizedVolterra_fabiusReal_of_le_one F hF
#check normalizedVolterra_pow_mul_extendedFabius F hF
#check normalizedVolterra_pow_mul_fabiusReal_of_le_one F hF
```

The signed-extension theorems have unrestricted real endpoints; the bounded theorems use the exact hypothesis $x \leq 1$. The generic affine and base-point API does not yet give named Fabius-specific declarations for the up ladder based at -1, the reflected $1 - F$ ladder, arbitrary affine copies of F, or the full piecewise polynomial tail beyond the support of up; those remain separate frontier targets rather than additional `#check` entries here.

G.5 Find source-facing arithmetic theorems

```
import FabiusFunction.Paper06487

open Fabius

#check proposition_one
#check theorem_seven
#check theorem_twenty
#check theorem_twenty_one
#check conjecture_sixteen
```

A declaration representing a conjecture is a definition or proposition describing the assertion, not a proof of it.

G.6 Inspect asymptotic interfaces

```
import FabiusFunction.PaperFabiusAsymptotic

open Fabius

-- Use #check on the named main, obstruction, and full-expansion
-- theorems, then follow their exact-phase helper declarations.
#check Fabius.log_fabius_sub_correctedWikipediaMain_isBigO
#check Fabius.log_fabius_sub_WikipediaElementaryMain_not_isBigO
#check Fabius.log_fabius_dyadicReal_sub_sharpLambertExpansion_isBigO
```

Because filter statements can be long, editor hover information is often more useful than printing the entire type in a terminal.

G.7 Search tactics

Within an editor, useful text searches include:

```
IsFabius -> HasDerivAt
fabiusDyadicValue -> Real
rvachevUp -> Fourier
negativeLaplaceLog
HasAsymptoticExpansion
Legendre -> HasSum
```

Searching by the transition one wants—rather than by a source theorem number—usually finds the strong internal lemma.

H Reproduction and trust checks

H.1 Pin the inspected tree

The original walkthrough baseline was checked against the full base commit printed on the title page, not against an unspecified moving branch. This merged edition preserves that recipe as a historical baseline check; it is not a validation recipe for the later merged closures. To reproduce the original baseline, begin with:

```
git rev-parse HEAD
source_pin=eaea1b9afe2b7ff0b7dd880bd1710e303dec6d80
test "$(git rev-parse "$source_pin^{commit}")" = "$source_pin"
git diff --exit-code "$source_pin" -- \
  Analysis/FabiusFunction/Lean
git show -s --format=%cs "$source_pin"
lake env lean --version
rg --files Analysis/FabiusFunction/Lean/FabiusFunction \
  -g '*.lean' | wc -l
```

At the pinned snapshot the last command reports 189 leaf modules. A different count is not automatically an error, but it means this guide's module inventory must be compared with the newer umbrella import. Do not run `lake update`: the committed manifest is part of the audited environment.

H.2 Build and inspect trust

The release-level library check is

```
LAKE_JOBS=1 lake build +FabiusFunction
```

The repository policy requires one prefixed module target per Lake invocation, with `LAKE_JOBS=1`, including during debugging, and forbids setting `LEAN_NUM_THREADS=0`. It also requires a source audit for declarations that bypass ordinary proof checking:

```
rg -n '\b(sorry|admit)\b' \
  Analysis/FabiusFunction/Lean -g '*.lean'
rg -n '\b(axiom|opaque)\b' \
  Analysis/FabiusFunction/Lean -g '*.lean'
```

The first search is empty at the pinned snapshot. The second intentionally shows two comment-only uses of the word *opaque*; inspect the raw matches to confirm that neither is a declaration. For representative public theorems, a small scratch file may additionally request:

```
import FabiusFunction

#print axioms Fabius.fabiusDyadicValue_cast
#print axioms Fabius.log_fabius_sub_correctedWikipediaMain_isBigO
```

Any reported axioms must be drawn from the repository's permitted set `propext`, `Classical.choice`, and `Quot.sound`; an individual theorem may use only a subset.

H.3 Rebuild this document

From this document's directory, use exactly three passes:

```
pdflatex -interaction=nonstopmode -halt-on-error \
  fabius_lean_walkthrough.tex
pdflatex -interaction=nonstopmode -halt-on-error \
  fabius_lean_walkthrough.tex
pdflatex -interaction=nonstopmode -halt-on-error \
  fabius_lean_walkthrough.tex
rm -f fabius_lean_walkthrough.aux \
  fabius_lean_walkthrough.log \
  fabius_lean_walkthrough.out \
  fabius_lean_walkthrough.toc
```

The source and generated PDF belong in the same commit. A successful TeX build checks syntax and references, but it does not validate mathematical prose; that is why this walkthrough also cites exact declaration names and records its nonclaims.

Bibliographic and repository notes

[1] Vladimir Reshetnikov, *ProveIt: Analysis/FabiusFunction*, GitHub repository, source tree and README inspected 25 August 2026. https://github.com/VladimirReshetnikov/ProveIt/tree/eaea1b9afe2b7ff0b7dd880bd1710e303dec6d80/Analysis/FabiusFunction

[2] Juan Arias de Reyna, *An infinitely differentiable function with compact support: definition and properties*, arXiv:1702.05442 (2017). https://arxiv.org/abs/1702.05442

[3] Juan Arias de Reyna, *Arithmetic of the Fabius function*, arXiv:1702.06487, version 3 (2017). https://arxiv.org/abs/1702.06487

[4] Leonardo de Moura and Sebastian Ullrich, *The Lean 4 Theorem Prover and Programming Language*, Automated Deduction – CADE 28, 2021.

[5] The mathlib community, *The Lean mathematical library*, CPP 2020 and continuously maintained Lean 4 library. https://github.com/leanprover-community/mathlib4

Document metadata

Title	The Fabius Function in Lean: A Guided Walkthrough of the ProveIt Formalization
Edition provenance	merged edition, 4 September 2026; historical baseline commit `eaea1b9afe2b7ff0b7dd880bd1710e303dec6d80` on `codex/fabius-both-papers`, inspected 25 August 2026; later closures retain the per-addendum checkpoints recorded above
Toolchain recorded by source	Lean 4.32.0; repository-pinned mathlib revision
Primary path	`Analysis/FabiusFunction/Lean/`
Umbrella module	`FabiusFunction.lean`
Document purpose	Human-readable structural and proof-oriented guide to the Lean development